



Mühendislik Fakültesi

Bilgisayar Mühendisliği Bölümü

Veri Tabanı Sistemleri



Öğr. Gör. Muhammed Tekin

mutekin16@gmail.com

muhammed.tekin@yalova.edu.tr



<http://tiny.cc/mutekin16d>



<http://tiny.cc/mutekin16b>

Kaynak Kitaplar & Değerlendirme

➤ Kaynak Kitaplar

- Raghu Ramakrishnan, Database Management Systems, McGraw Hill, 3rd edition
- Ramez Elmasri, Shamkant B. Navathe, Fundamentals of Database Systems, Pearson, 7th edition
- A. Silberschatz, H.F. Korth, S. Sudarshan, Database System Concepts, McGraw Hill, 7th edition,
- Coşkun Telciler, Veri Tabanı Kavramı ve MS–SQL Uygulamaları, Pusula Yayıncılık, 2013
- Dr. Kerem Köseoğlu Veri Tabanı Mantığı, Pusula Yayıncılık, Nisan 2013
- Prof. Dr. Mithat Uysal, SQL Veri Tabanı Sorgulama Dili, Nirvana Yayınları, Şubat 2012
- Dr. Ali Nizam, Veri Tabanı Tasarımı: İlişkisel Veri Modeli ve Uygulamaları, Papatya Yayıncılık, 2011
- Yaşar Daşdemir, Veri Tabanı Yönetim Sistemleri, Türkmen Kitabevi, 2004

➤ Değerlendirme

- Lab : %30
- Proje : %30
- Final : %40

İÇERİK

1. Giriş

- Veri Tabanı Sistemleri
- Veri Tabanı vs. Dosya
- Neden Veri tabanı?
- Veri Bütünlüğünün Sigortası – ACID
- DBMS Mimari Yapısı
- Veri Tabanı Uygulamaları & DBMS
- DBMS Kullanıcıları
- Veri Tabanı Uygulama Mimarileri

2. Varlık – İlişki Modeli (Entity-Relationship Model)

- Veri Tabanı Tasarımı
- Veri Tabanı Modeli
- Varlık – İlişki Modeli
- İlişkilerin Şemaya Dönüştürülmesi

3. Normalizasyon

4. İlişkisel Cebir

- i. Seçim (Selection)
- ii. İzdüşüm (Projection)
- iii. Çarpım (Cross-Product)
- iv. Bağlantı (Join)
- v. Birleşim (Union)
- vi. Kesişim (Intersection)
- vii. Fark (Difference)
- viii. Yeniden İsimlendirme (Rename)
- ix. Bölme (Division)

5. Structured Query Language (SQL)

1. Bölüm

GİRİŞ

Veri Tabanı Sistemleri

- **Veri (Data) & Bilgi (Information)**
 - **Veri**
 - Henüz anlamı çıkarılmamış ham gerçekler, olgular
 - **Bilgi**
 - Karar verme süreçlerinde anlamlı olacak şekilde işlenmiş ve yorumlanabilir hâle getirilmiş veridir
- Bir veri tabanı;
 - Paylaşımlı kullanım için tasarlanmış
 - Bütünleşik ve kalıcı verilerden oluşan bir koleksiyondur
- Bir veri tabanı içerisinde;
 - Son kullanıcı verileri (tablolar içerisinde)
 - Metadata (veriyi tanımlayan veri; tablo yapıları, sütunlar, veri tipleri, kısıtlar ve ilişkiler gibi bilgiler) bulunur

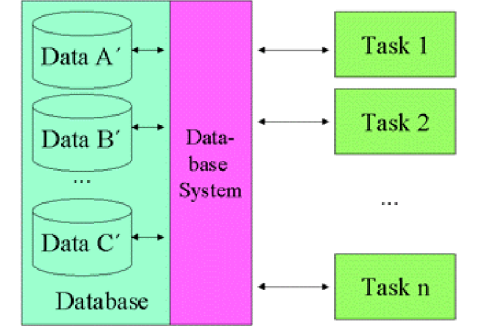
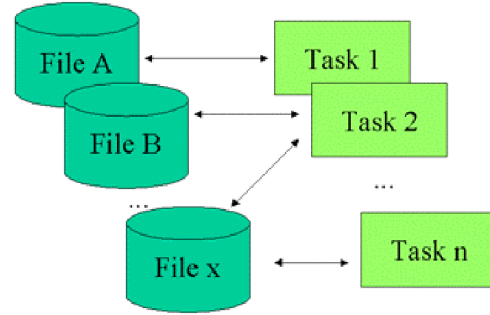
Veri Tabanı Sistemleri

- **Veri Tabanı Yönetim Sistemleri (Database Management Systems, DBMS)**
 - Kullanıcıların ve uygulamaların verileri sistematik bir şekilde depolamasına, düzenlemesine, yönetmesine ve bu verilere erişmesine olanak tanıyan bir yazılım katmanıdır
 - Basit bir ifadeyle DBMS, veri tabanı ile son kullanıcı arasında bir köprü görevi görür.
 - Verilerin karmaşık dosya sistemleri yerine disiplinli bir yapıda tutulmasını sağlayarak veri tekrarını önler ve güvenliği artırır
- **DBMS'in Temel İşlevleri:** Bir DBMS genellikle şu dört ana işlemi (CRUD) yönetir
 - Oluşturma (Create)
 - Veri tabanı yapısını ve verileri tanımlama
 - Okuma (Read)
 - İhtiyaç duyulan veriyi sorgulama ve getirme
 - Güncelleme (Update)
 - Mevcut verileri değiştirme.
 - Silme (Delete)
 - Gereksiz verileri kaldırma.

Veri Tabanı vs. Dosya

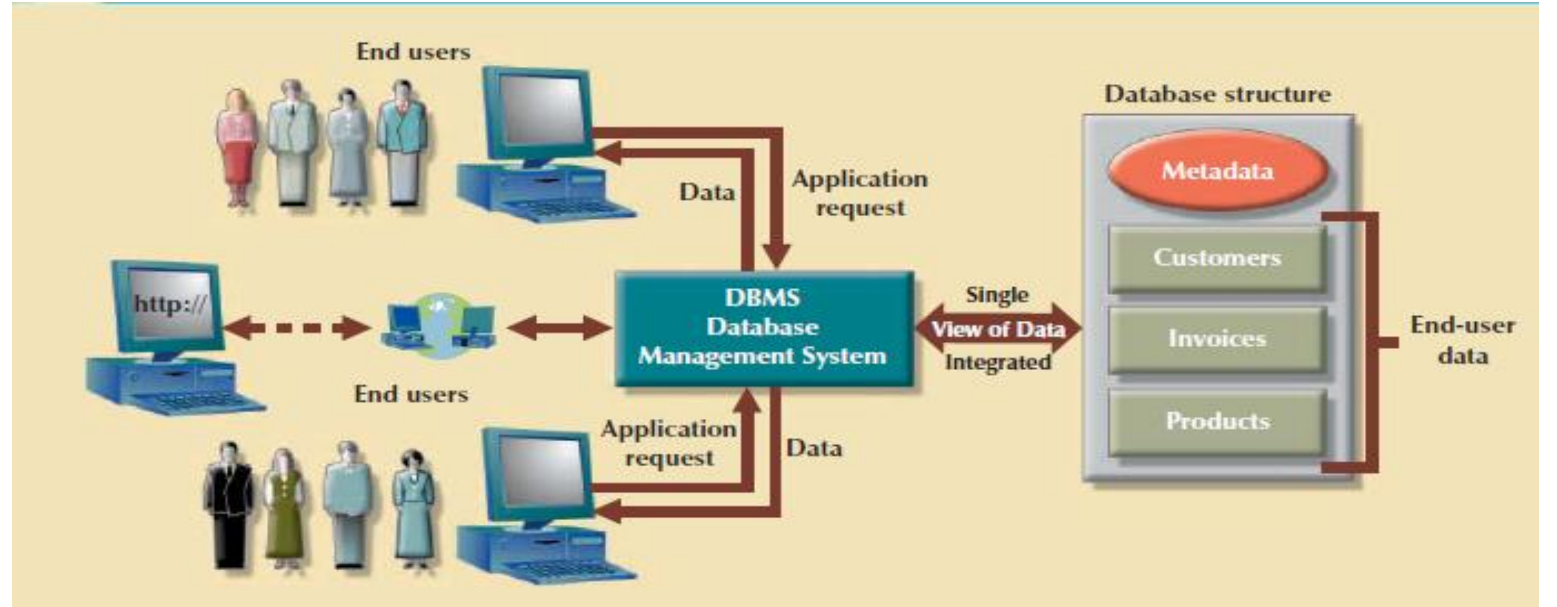
- **Dosya**

- Her bir iş (task) bir ya da birden fazla dosya (file) kullanır.
- Her bir dosyada farklı veri yapısı olabilir ve dosyaya erişecek programın veri yapısını bilmesi gerekir.
 - Program–Veri bağımlılığı yüksektir.
- Veriye erişim direk olarak gerçekleşir



- **Veri Tabanı**

- İşlemler veri dosyalarına direk erişemezler.
- Taleplerini veri tabanı sistemine iletirler.
- O da yetki uygunluğuna göre gerekli dönüşleri yapar.



Neden Veri Tabanı?

- Veri tekrarının kontrol altına alınması
 - Veri tabanları, veri tekrarını tamamen ortadan kaldırmayı değil, kontrollü hâle getirmeyi amaçlar.
- Çoklu Erişim
 - Birden fazla kullanıcının aynı anda veriye erişmesine ve işlem yapmasına izin verir (Concurrency Control).
- Veri Bağımsızlığı
 - Fiziksel Veri Bağımsızlığı
 - Disk üzerindeki kayıt yapısı değişse bile (örn: dosyalar taşınsa, indekslense) uygulamanın bozulmaması.
 - Mantıksal Veri Bağımsızlığı
 - Tabloya yeni sütun eklense bile, o sütunu kullanmayan eski sorguların çalışmaya devam etmesi.

Neden Veri Tabanı?

- Güvenlik
 - Kimin hangi veriyi görebileceğini veya değiştirebileceğini belirleyen yetkilendirme mekanizmaları sunar.
- Yedekleme ve Kurtarma
 - Sistem çökmelerine karşı veriyi korur ve geri yükleme imkanı sağlar.
- Tutarsızlıklar belirli ölçülerde engellenebilir
- İstenen standartlar uygulanabilir
- Çoğu durumda hızlı veri erişimi sunar
- Verinin farklı bir çok görüntüsüne erişilebilir

Veri Bütünlüğünün Sigortası – ACID

- Bir DBMS'in (özellikle SQL Server gibi ilişkisel sistemlerin) bir işlemi (transaction) başarıyla tamamladığını söyleyebilmemiz için şu dört kurala uyması gerekir:
- Atomiklik (Atomicity)
 - "Ya hep ya hiç"
 - Bir işlem birden fazla adımdan oluşuyorsa, bu adımlardan biri bile başarısız olursa tüm işlem iptal edilir ve veri tabanı eski haline döner.
- Tutarlılık (Consistency)
 - İşlem öncesinde ve sonrasında veri tabanı tanımlanmış tüm kurallara (constraint) uygun olmalıdır.
- İzolasyon (Isolation)
 - Eşzamanlı işlemlerin birbirlerinin ara (geçici) sonuçlarını görmemesini ve sonuçların seri çalışmaya eşdeğer olmasını sağlar.
- Dayanıklılık (Durability)
 - İşlem onaylandığı (commit) andan itibaren, sistem çökse veya elektrik kesilse bile veriler kalıcı olarak kaydedilir.

DBMS Mimari Yapısı

- DBMS genellikle üç katmanlı bir mimari (3-Schema Architecture) kullanır.
- Bu yapı, kullanıcının verinin fiziksel depolama detaylarından bağımsız kalmasını sağlar
- **Dış Katman (External Level)**
 - Kullanıcıların gördüğü kısım (Görünümler/Views).
 - Her kullanıcı sadece yetkisi dahilindeki veriyi görür.
- **Kavramsal Katman (Conceptual Level)**
 - Veri tabanının mantıksal yapısıdır.
 - Hangi tablolar var?
 - Tablolar arası ilişkiler neler? (Örn: Öğrenci tablosu ile Ders tablosu arasındaki ilişki).
- **İç Katman (Internal Level)**
 - Verinin diskte fiziksel olarak nasıl saklandığı (indeksler, dosya organizasyonları) ile ilgilenen katmandır.

Veri Tabanı Uygulamaları & DBMS

- Bankalar, rezervasyon sistemleri, e-ticaret, iletişim, çoklu ortam, coğrafik bilgi sistemleri, veri ambarları vb.
- Günümüzde İlişkisel (Relational DBMS, RDBMS) ve NoSQL veri tabanı sistemleri birlikte kullanılabilmektedir.
- **RDBMS**
 - Microsoft SQL Server
 - Oracle
 - PostgreSQL
 - MySQL
- **NoSQL**
 - MongoDB
 - Cassandra
 - Redis

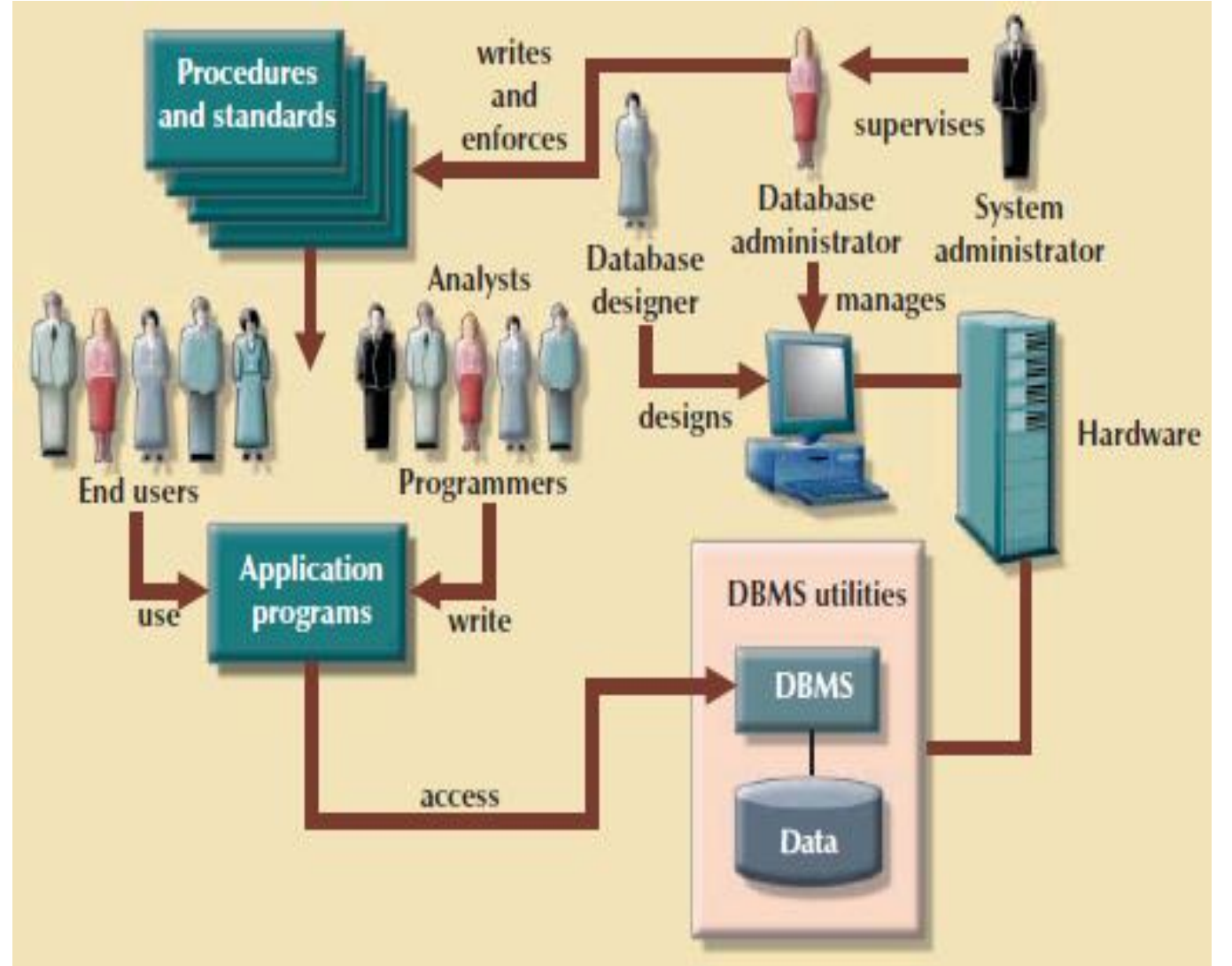
DBMS Kullanıcıları

- **Database Administrator**

- Veri tabanı yöneticisidir
- Erişim yetkilerinin belirlenmesinden, kullanımların izlenmesinden, yazılım ve donanım kaynaklarının oluşturulmasından, ve verimli kullanım yapılmasından sorumludur.

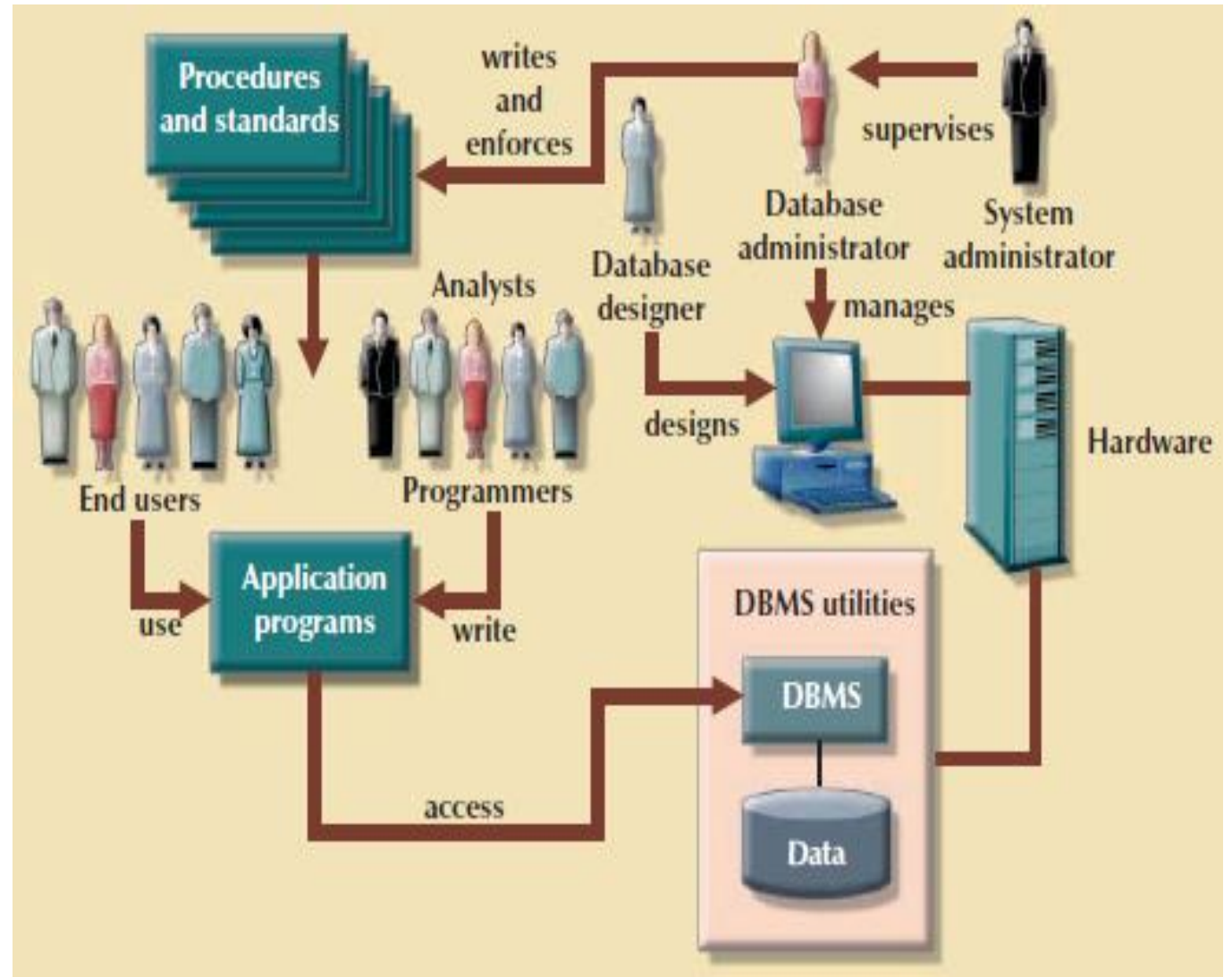
- **Database Designers**

- Veri tabanı tasarımcısıdır
- Veri tabanındaki yapıların, fonksiyonların ve işlemlerin içeriklerinin tanımlanmasından sorumludur
- Son kullanıcılarla iletişim halinde olmalı ve ihtiyaçlarını iyi anlamalıdır.



DBMS Kullanıcıları

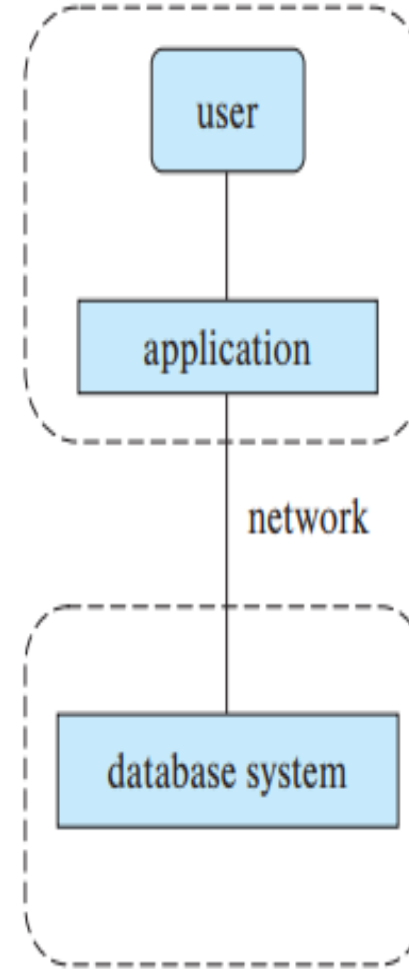
- **Son kullanıcılar (End users)**
 - Gündelik, geçici (casual)
 - Yöneticiler gibi belirli zaman aralıklarında veri tabanına erişip sorgulamalar yapan kullanıcılardır
 - Vasat (naïve)
 - Son kullanıcıların çoğunluğunu oluştururlar
 - Banka görevlileri gibi belirli birkaç özelliği sürekli kullanan kullanıcılardır
 - Bilgili (sophisticated)
 - İş analistleri, bilim adamları, mühendisler ve diğer sisteme genel manada aşina kullanıcılardan oluşur



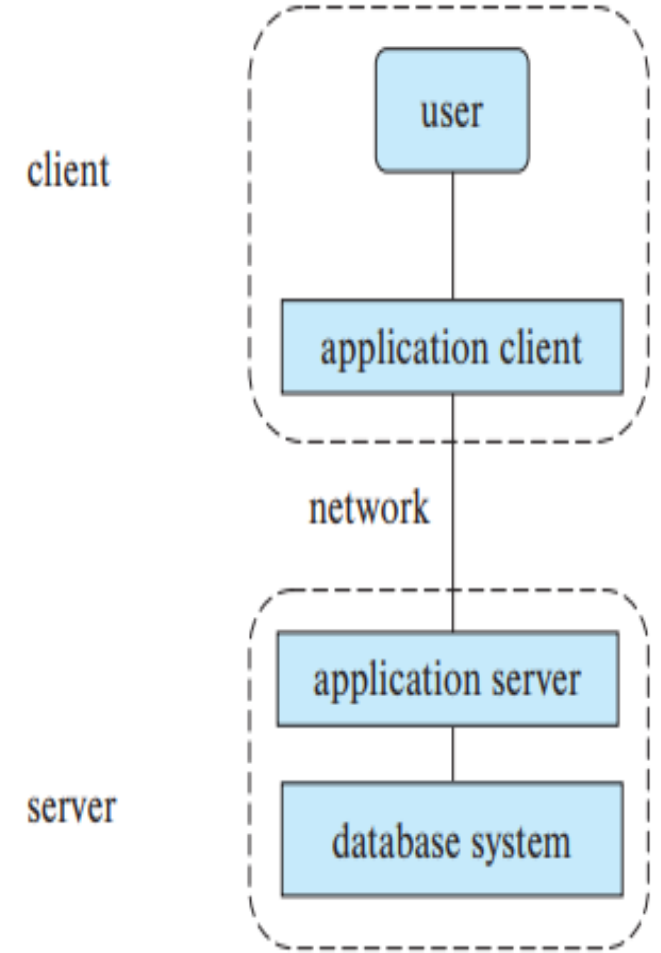
Veri Tabanı Uygulama Mimarileri

- **İki Katmanlı Mimari (Two-Tier Architecture)**

- Önceki nesil veri tabanı uygulamalarında iki katmanlı bir mimari kullanılıyordu.
- Bu yapıda uygulama istemci (client) bilgisayarda çalışır, veri tabanı işlemleri ise sunucuya (server) gönderilen sorgu dili komutları aracılığıyla gerçekleştirilirdi.



(a) Two-tier architecture

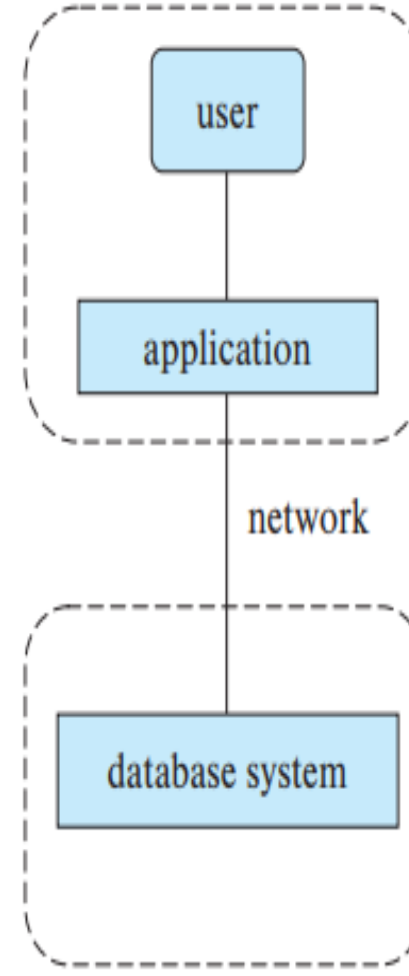


(b) Three-tier architecture

Veri Tabanı Uygulama Mimarileri

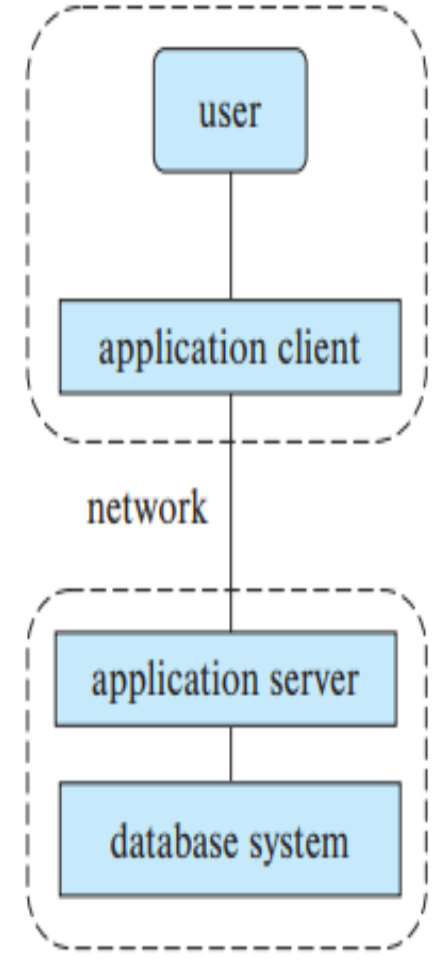
- **Üç Katmanlı Mimari (Three-Tier Architecture)**

- Günümüzün modern uygulamaları (web tarayıcıları, mobil uygulamalar vb.) üç katmanlı bir mimari kullanır.
- Burada istemci bilgisayarları yalnızca bir ön yüz (front end) görevi görür ve doğrudan veri tabanı çağrıları içermez.
- Ön yüz bir uygulama sunucusu ile iletişim kurar
- Uygulama sunucusu da verilere erişmek için veri tabanı sistemi ile iletişim kurar.
- Uygulamanın iş mantığı (hangi koşullarda hangi işlemlerin gerçekleştirileceğini belirleyen kısım), birden fazla istemciye dağıtılmak yerine uygulama sunucusunda yer alır.
- Üç katmanlı uygulamalar, uygun tasarlandığında iki katmanlı uygulamalara kıyasla hem daha iyi güvenlik hem de daha iyi performans sunar.



(a) Two-tier architecture

client



server

(b) Three-tier architecture

2. Bölüm

Varlık – İlişki Modeli Entity-Relationship Model

Veri Tabanı Tasarımı

1. Gereksinim Analizi (Requirements Analysis)

- Kullanıcılar veri tabanından ne yapmasını istiyorlar?

2. Kavramsal Veri Tabanı Tasarımı (Conceptual Database Design)

- E-R ya da benzeri yüksek seviyeli veri modelinin oluşturulması.

3. Mantıksal Veri Tabanı Tasarımı (Logical Database Design)

- Veri tabanı tasarımının uygulanabileceği bir DBMS seçilmesi ve şemaların oluşturulması

4. Şema Düzeltme (Schema Refinement)

- İlişkilerin (tabloların) normalizasyonu.

5. Fiziksel Veri Tabanı Tasarımı (Physical Database Design)

- İndekslerin oluşturulması vb.

6. Güvenlik Tasarımı (Security Design)

- Farklı kullanıcı türleri için gruplar ve rollerin belirlenmesi.
- Belirlenen gruplar ve roller için yetkilendirmelerin yapılması

Veri Modeli

- **Veri Modeli**

- Veriyi tanımlayacak kavramlardan oluşur

- **İlişkisel Veri Modeli**

- Gerçek dünyadaki varlık ve ilişkilerin, matematiksel temellere dayalı tablo (**ilişki, relation**) yapıları ile temsil edilmesini sağlar.
- İlişkiler birbirlerine yabancı anahtar (**Foreign Key**) denen özel sütunlarla bağlanırlar. Dolayısıyla bir Foreign Key başka bir ilişkiyi gösteren işaretçi (pointer) gibi düşünülebilir (Dikkat! Foreign Key, pointer değildir; sadece mantıksal referans bütünlüğü sağlayan bir alan olarak düşünülebilir).

- **Şema (Schema)**

- Verinin, kullanılan modele göre tanımını belirtir.
- Bir ilişki için; adını, her bir alanın (field, özellik(attribute), sütun(column)) adını ve tipini içerir.

Veri Modeli

- Şema (schema)

- Örneğin bir üniversite sisteminde öğrenci bilgileri aşağıdaki şemaya bağlı bir relation da saklanabilir.

students(sid:string, name:string, login:string, age:integer, gpa:real)

FIELDS (ATTRIBUTES, COLUMNS)

<i>sid</i>	<i>name</i>	<i>login</i>	<i>age</i>	<i>gpa</i>
50000	Dave	dave@cs	19	3.3
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2
53650	Smith	smith@math	19	3.8
53831	Madayan	madayan@music	11	1.8
53832	Guldu	guldu@music	12	2.0

Veri Modeli

- **Kavramsal Şema (Conceptual Schema)**

- Kullanıcı gereksinimlerinin toplanması ve analizinden sonra oluşturulur.
- Kullanıcıların veri gereksinimlerinin kısa bir açıklamasıdır.
- Varlık türlerinin, ilişkilerinin ve kısıtlamalarının ayrıntılı açıklamalarını içerir.
- Bunlar, yüksek seviyeli (high-level) veri modeli tarafından sağlanan kavramlar kullanılarak ifade edilir.

- **Mantıksal Şema (Logical Schema)**

- Kavramsal şema yüksek seviyeli veri modelinden uygulama veri modeline dönüştürülür.
- Veri modeli eşlemesi, veri tabanı tasarım araçları içinde genellikle otomatik veya yarı otomatik olarak gerçekleştirilir.

- **Fiziksel Şema (Physical/Internal Schema)**

- Kavramsal şemadaki ilişkilerin fiziksel diskler üzerinde nasıl saklanacağını tanımlar
- Diğer bir deyişle veri tabanı dosyaları için dahili depolama yapıları, dosya organizasyonları, dizinler (indexes), erişim yolları ve fiziksel tasarım parametreleri belirlenir.

Veri Modeli

- **Harici şema (external schema)**
 - Veri erişim yetkilerinin kullanıcı ya da gruplar seviyesinde belirlenmesini sağlar
- **İlişki örneği (relation instance)**
 - Her bir kaydın (**record/tuple**) bir satırda (**row**) olduğu ve her bir satırında aynı sayıda ve tipte alanlara (**fields**) sahip olduğu bir tablodur (**table**).

Veri Modeli - Örnek

- Bir üniversitenin öğrenci, ders ve not verilerini saklayacağı veri tabanı aşağıdaki gibi olsun :
 - **STUDENT** : Her bir öğrencinin kaydedileceği ilişki (tablo)
 - **COURSE** : Her bir derse ait bilgilerin olduğu tablo
 - **SECTION** : Her bir dersin her bir yıl ve dönemde veren hocaya ait bilgilerin olduğu tablo
 - **GRADE_REPORT** : Her bir öğrencinin aldığı her bir dersten notlarının olduğu tablo
 - **PREREQUISITE** : Bir dersin alınmasından önce alınması gereken öncelikli dersleri içeren tablo

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Veri Modeli - Örnek

- Şemalar :

student(sid:string, name:string, login:string, age:integer, gpa:real)

course(course_name:string, course_number:string, credit_hours:int, department:string)

section(section_identifier:int, course_number:string, semester:string, year:string, instructor:string)

grade_report(student_number:int, section_identifier:int, grade:string)

prerequisite(course_number:string, prerequisite_number:string)

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

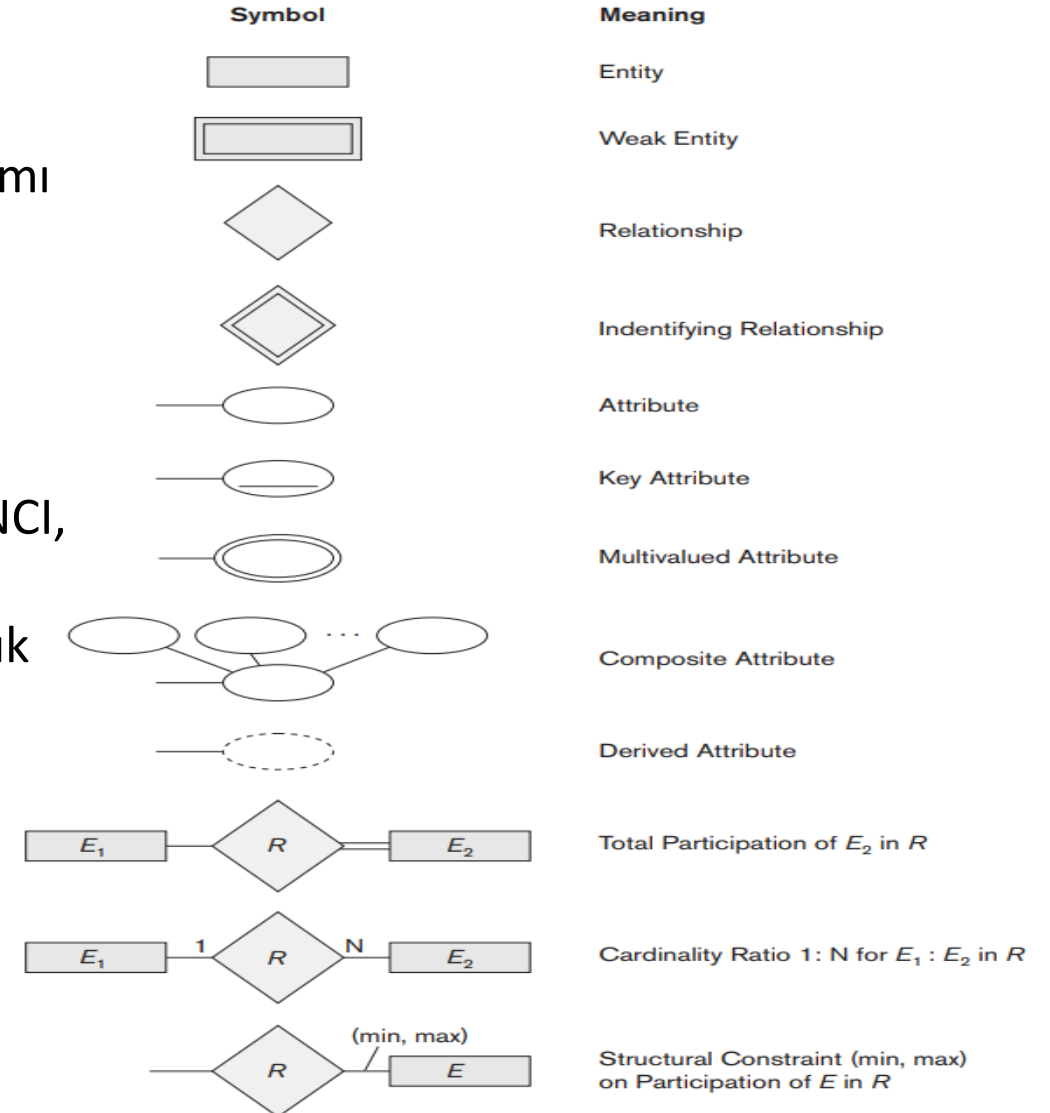
Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Varlık-İlişki (Entity-Relationship) Model

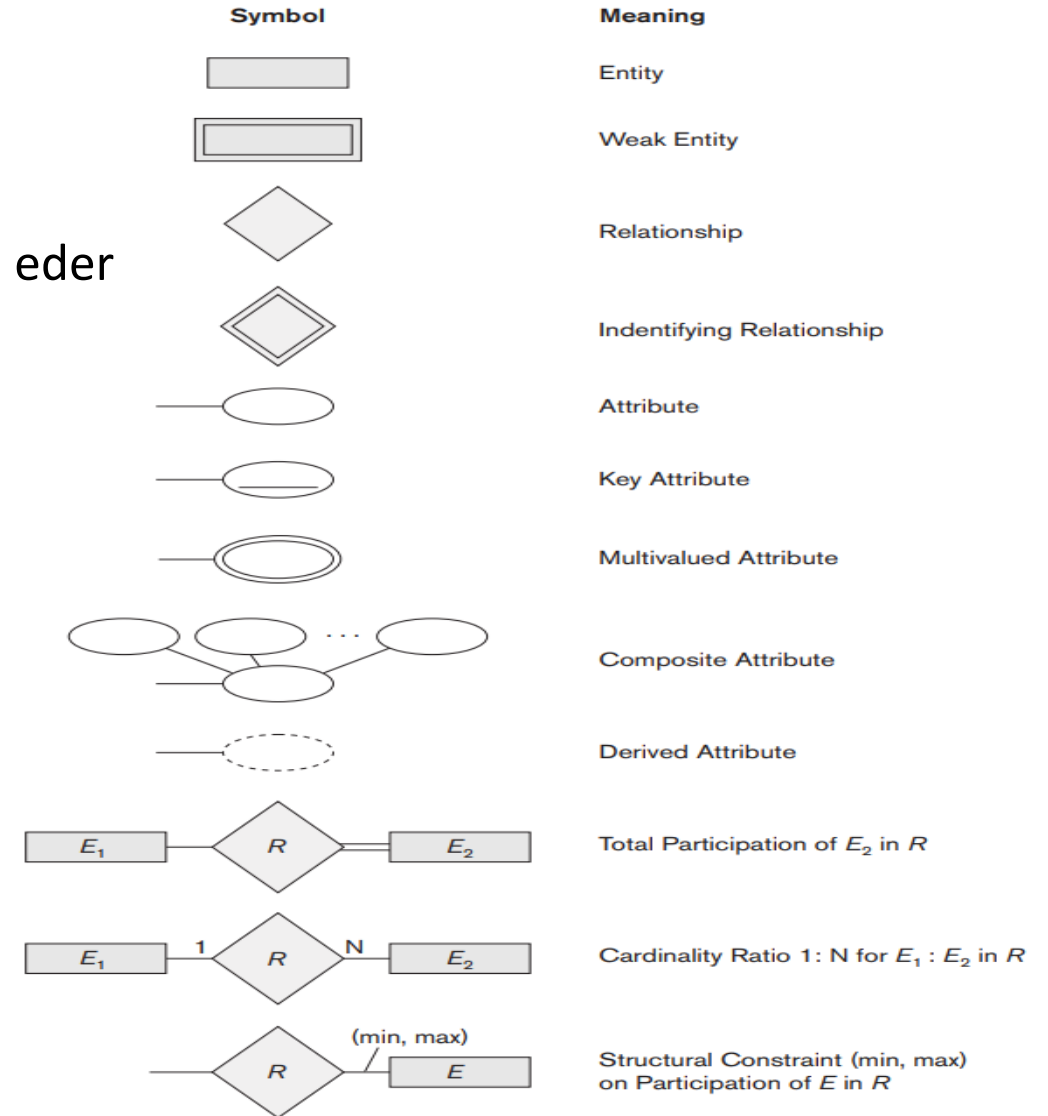
- İlişkisel modelin grafiksel gösterimini sağlar.
- Varlık (Entity)**
 - Gerçek dünyadaki ayırt edilebilir bir nesneyi veya kavramı temsil eder.
 - Örneğin personel, firma, olay ya da proje gibi
 - Dikdörtgen ile gösterilir
 - Varlığın ismi dikdörtgenin içine yazılır
 - Genelde büyük harflerle ve tekil biçimde yazılır (OGRENCI, PERSONEL).
 - Varlık kümesinde (entity set, tablo) her bir satır bir varlık örneği (entity instance) olarak ifade edilir
- Özellik (Attribute)**
 - Varlıklara ait karakteristik özelliklerdir
 - Diğer bir deyişle tablonun sütunlarıdır
 - Örneğin personel adı, soyadı, öğrenci notu vb.
 - Varlığa bağlı olan oval şekil ile gösterilir



Varlık-İlişki (Entity-Relationship) Model

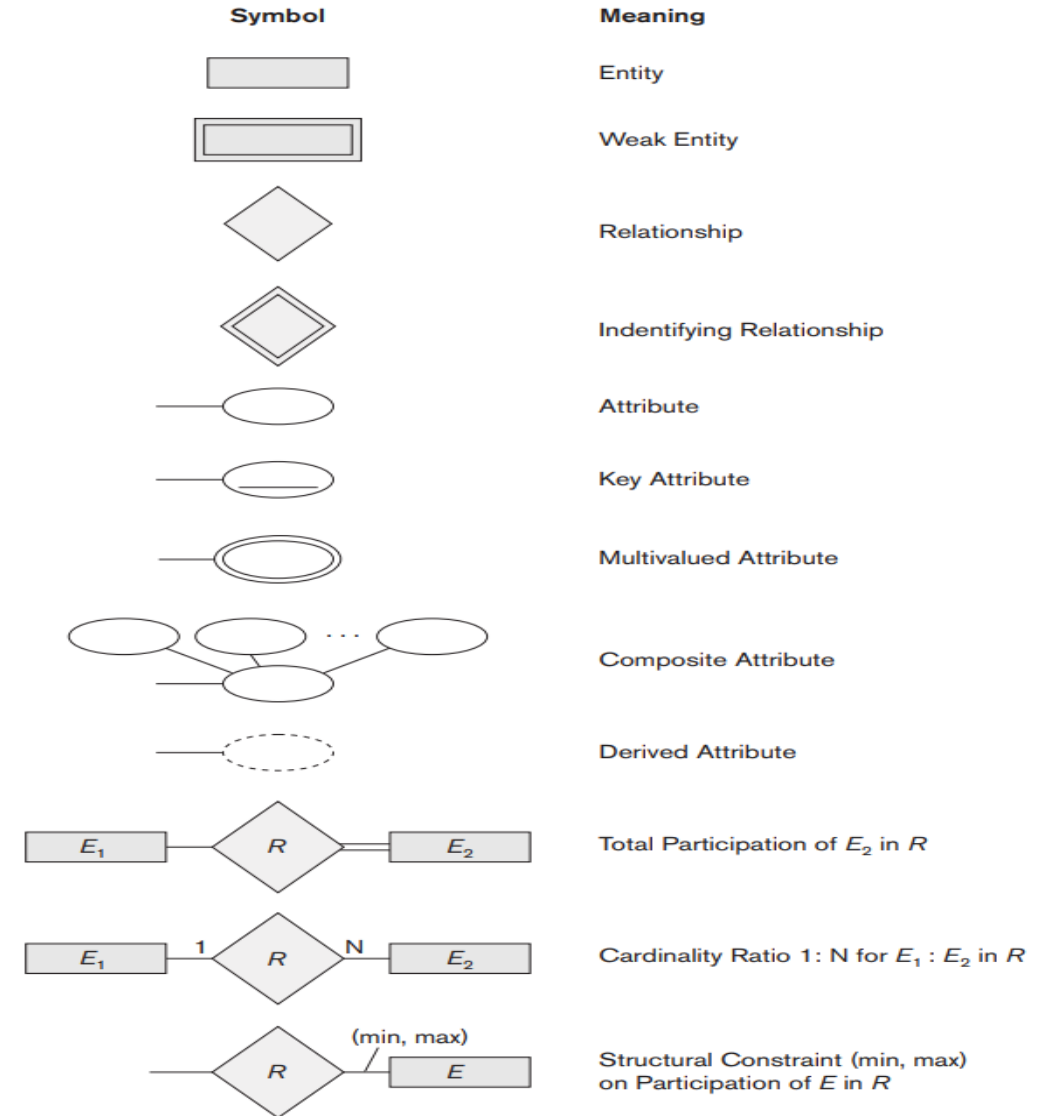
- İlişki (Relationship, R)

- Varlıklar arası bağlantıları ifade eder
- Baklava (elmas, diamond) şekli ile gösterilir
- İlişki ismi şekil içine yazılır ve genelde fiil / eylem ifade eder (YAPAR, OLUR, SAHİPTİR vb.)
- Üç tip ilişki vardır
 - 1-M : Bir'e çok ilişki
 - M-N : Çok'a çok ilişki
 - 1-1 : Bir'e bir ilişki
- Bazı durumlarda ilişkilerin de özellikleri olabilir
 - M-N ilişkilerde daha çok görülür
- Yinelemeli İlişki (Recursive Relationship)
 - Bir varlığın kendine dönen ilişkisine denir
 - Ör. Personel-Yönetici gibi



Varlık-İlişki (Entity-Relationship) Model

- **Anahtar Özellik (Key Attribute, PRIMARY KEY, PK)**
 - Bir varlığı, içinde bulunduğu varlık kümesindeki (tablo) diğer varlıklardan ayıracak benzersiz (unique) özellik değeri.
 - İsminin altı çizilerek yazılır.
 - Bir ya da daha fazla özellik beraber PRIMARY KEY'i oluşturabilirler.
 - Bu yapıya **Bileşik (Composite) Primary Key** denir.
 - PRIMARY KEY **NULL** değer içeremez



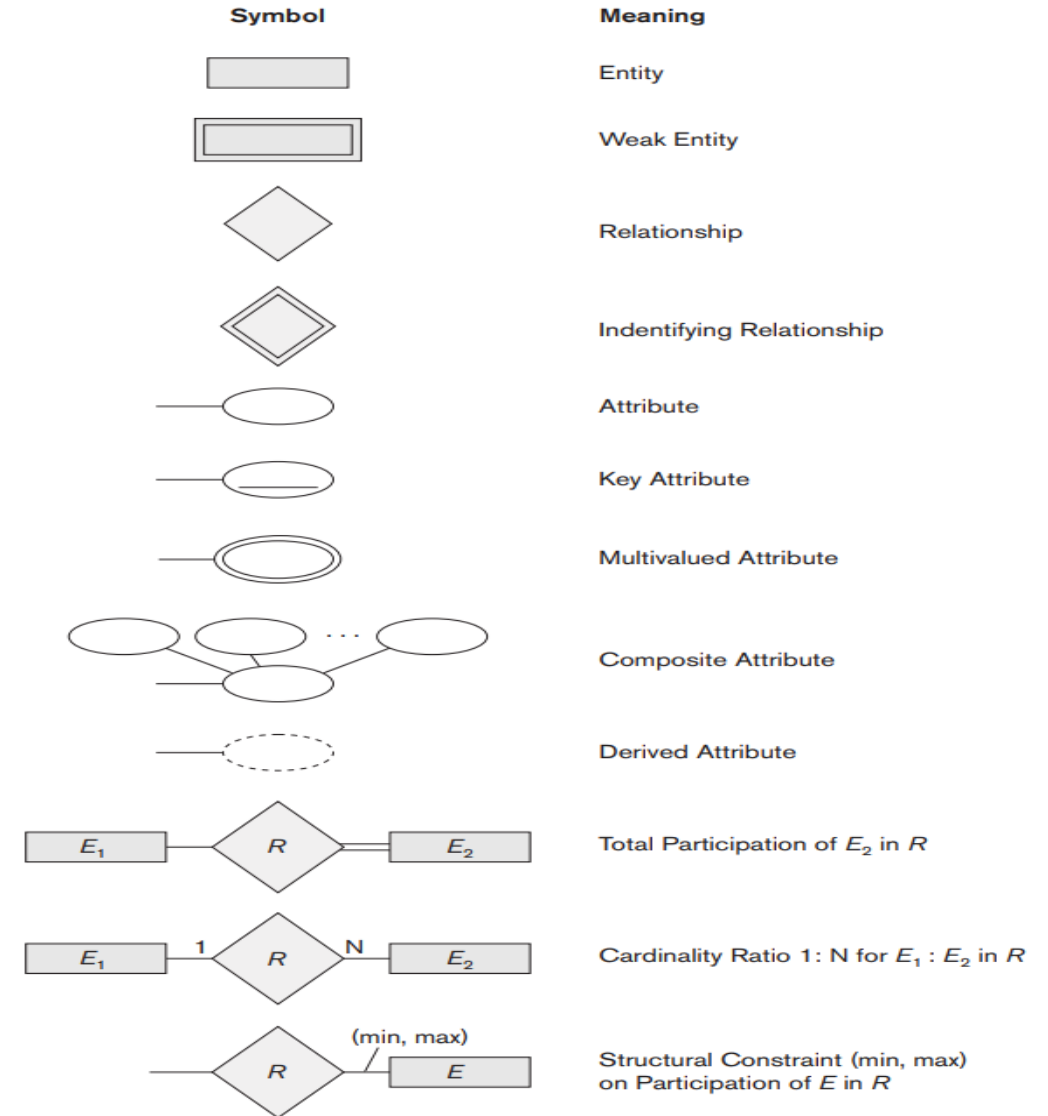
Varlık-İlişki (Entity-Relationship) Model

- **Çok Değerli Özellik (Multivalued Attribute)**

- Normalde özellikler tekil değer (single value) olurlar ama eğer bir özelliğin birden çok değer alma ihtimali varsa o zaman çok değerli özellik olarak tanımlanır.
- Örneğin bir kişinin birden çok telefon numarası olabilir veya bir çalışan birden çok yabancı dil biliyor olabilir.

- **Bileşik Özellik (Composite Attribute)**

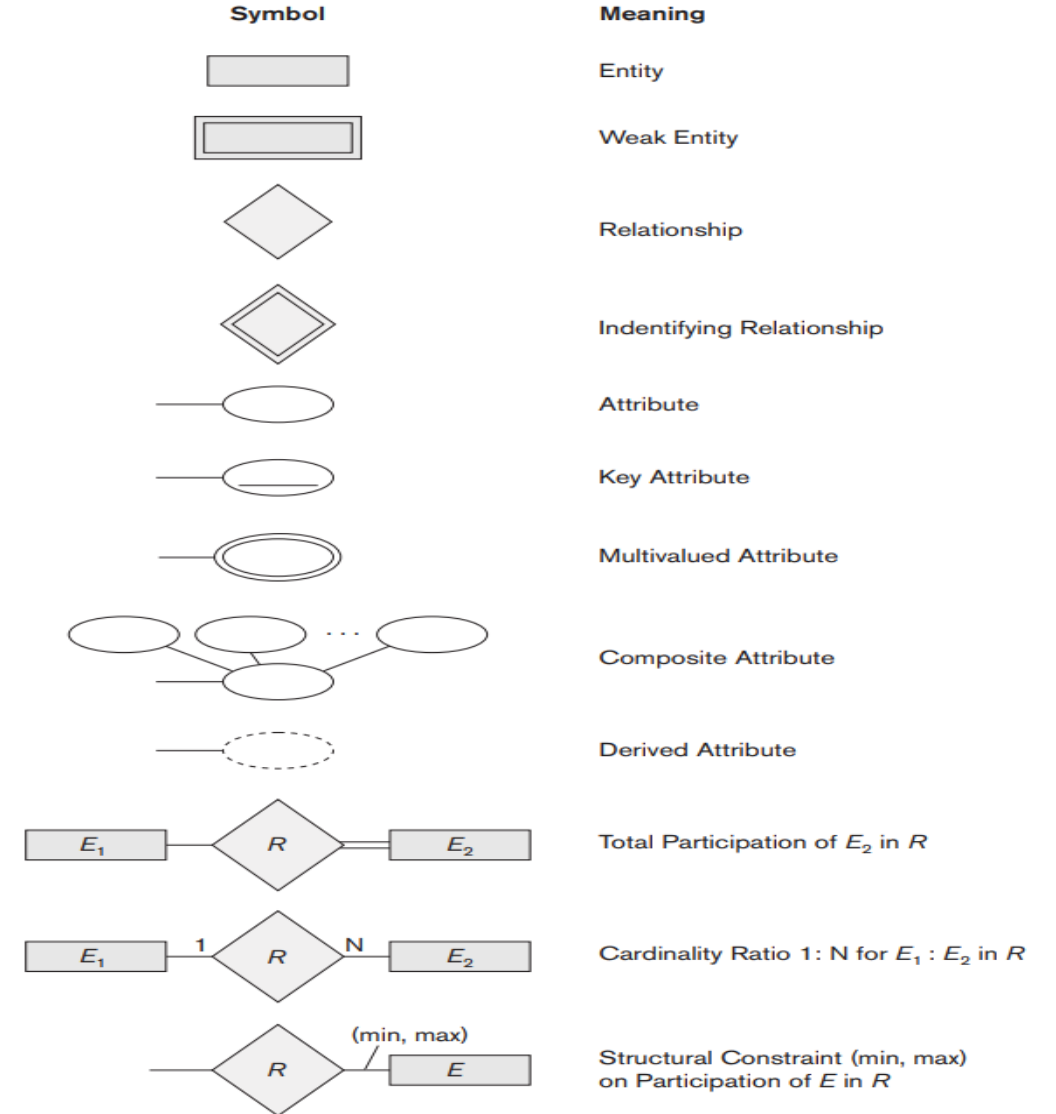
- Birden fazla özelliğin bir araya gelmesinden oluşan özellikler.
- Anlamlı daha küçük parçalara ayrılabilirler.
- Örneğin bir personele ait adres alanı il, ilçe, mahalle ve sokak alanlarının birleşiminden elde edilir



Varlık-İlişki (Entity-Relationship) Model

- **Türetilmiş Özellik (Derived Attribute)**

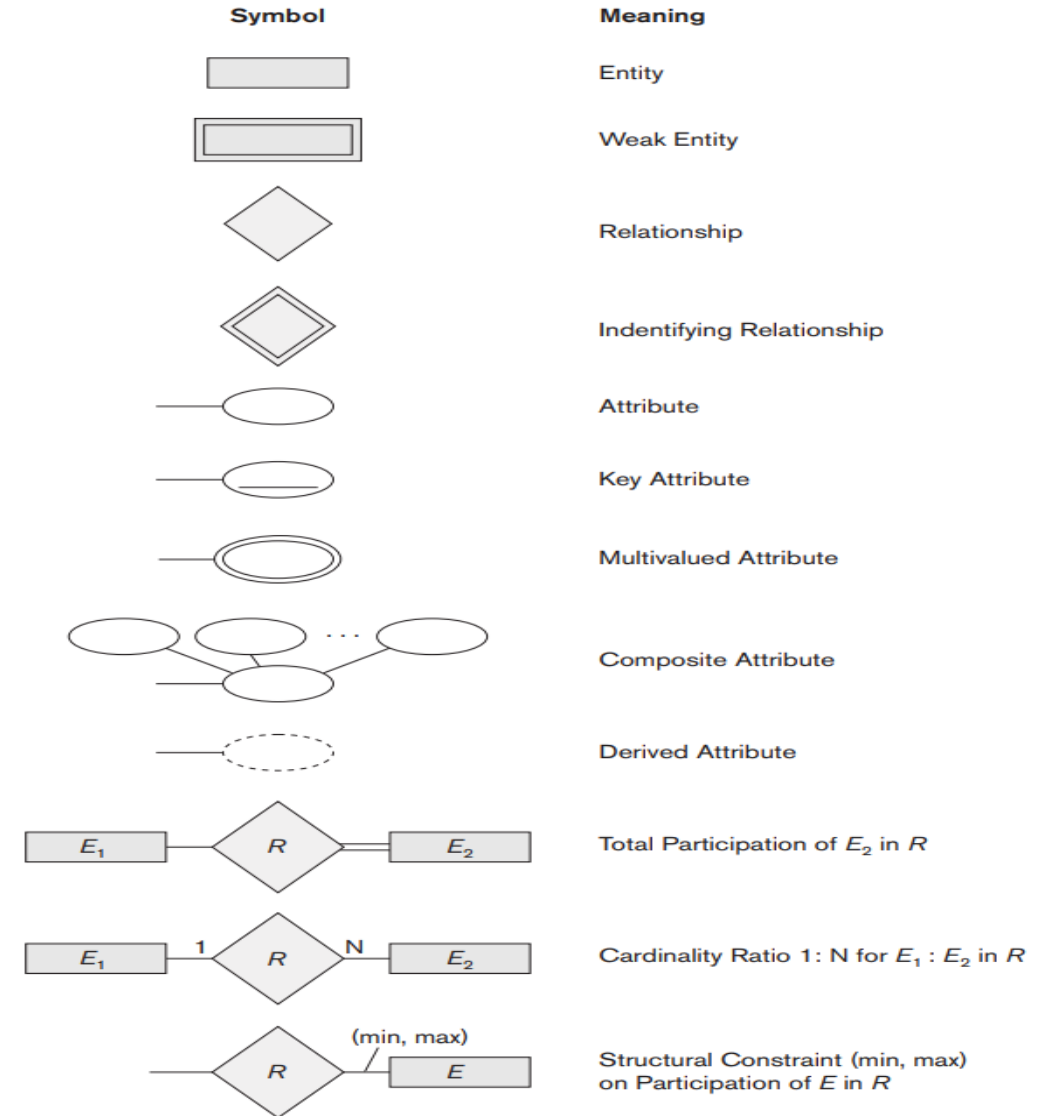
- Varlık kümesindeki diğer özelliklerden yola çıkılarak hesaplanan veya türetilen özelliklerdir.
- Kesik çizgili oval ile gösterilir
- Örneğin yaş özelliği doğum tarihi özelliğinden türetilir



Varlık-İlişki (Entity-Relationship) Model

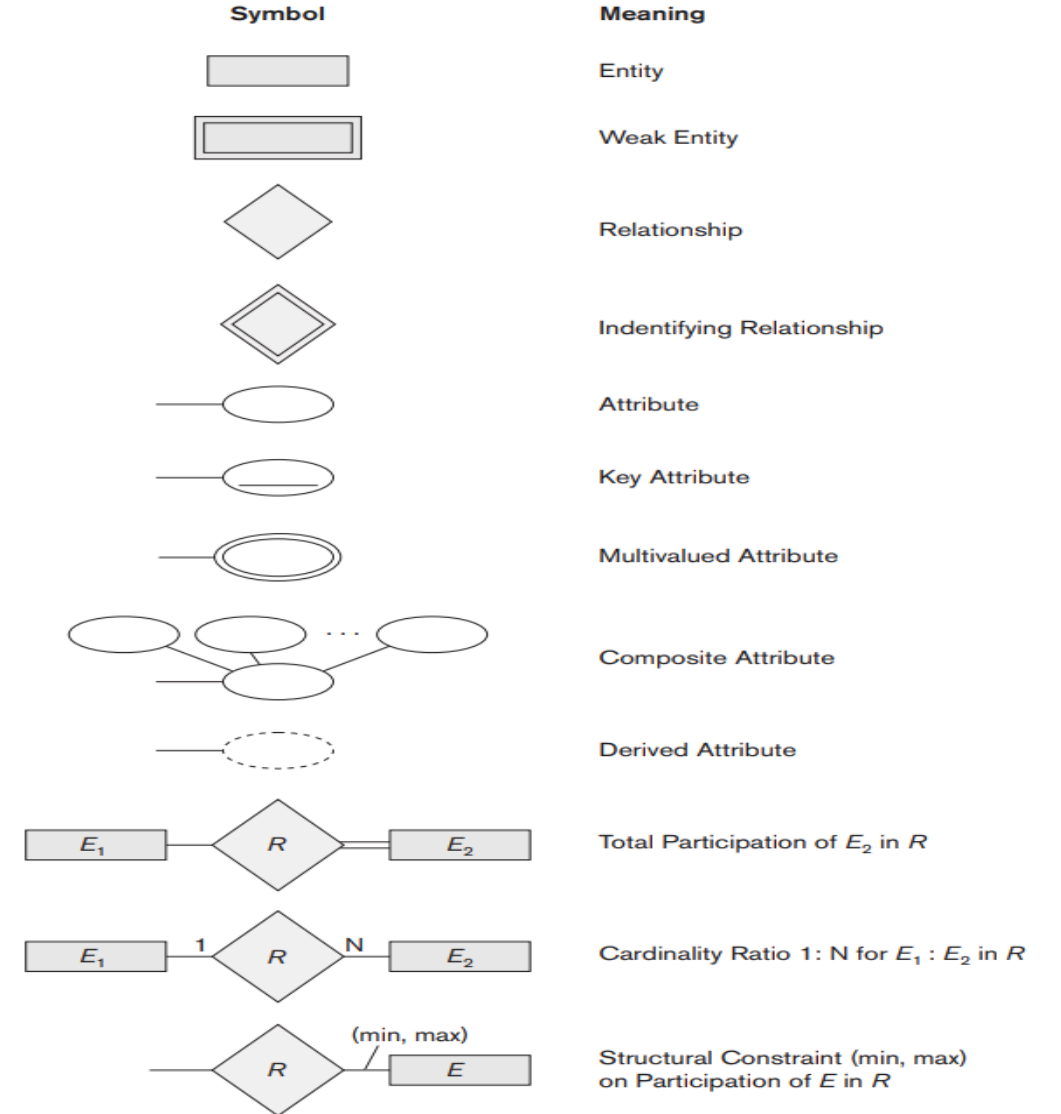
- **Zayıf Varlık (Weak Entity)**

- Kendi başına tanımlanamayan ve var olabilmek için başka bir varlığa (güçlü / sahip varlık) ihtiyaç duyan varlık türüdür.
 - Örneğin çalışanın bakmakla yükümlü olduğu kişiler varlığı çalışan olmadan bir anlam ifade etmez.
 - Benzer şekilde bir dönemde verilen dersler listesi de asıl ders listesi olmadan anlamsızdır
- PK'leri: Güçlü varlığın PK + Kendi kısmi anahtarı (partial key/discriminator, altı kesik çizgili gösterilir)
 - Ör. Çalışan_Sicil_No + Aile_bireyi_ismi
 - Zayıf varlığın kendi anahtarı tek başına benzersiz değildir.
- Çift çizgili dikdörtgen ile gösterilir.



Varlık-İlişki (Entity-Relationship) Model

- **Tanımlayıcı İlişki (Identifying Relationship)**
 - Zayıf varlıkların asıl varlıkla olan ilişkisidir.
 - Örneğin ders varlığı ile bir dönemde verilen dersler arasında olan ilişki
 - Çift çizgili baklava (double diamond) şekli ile gösterilir



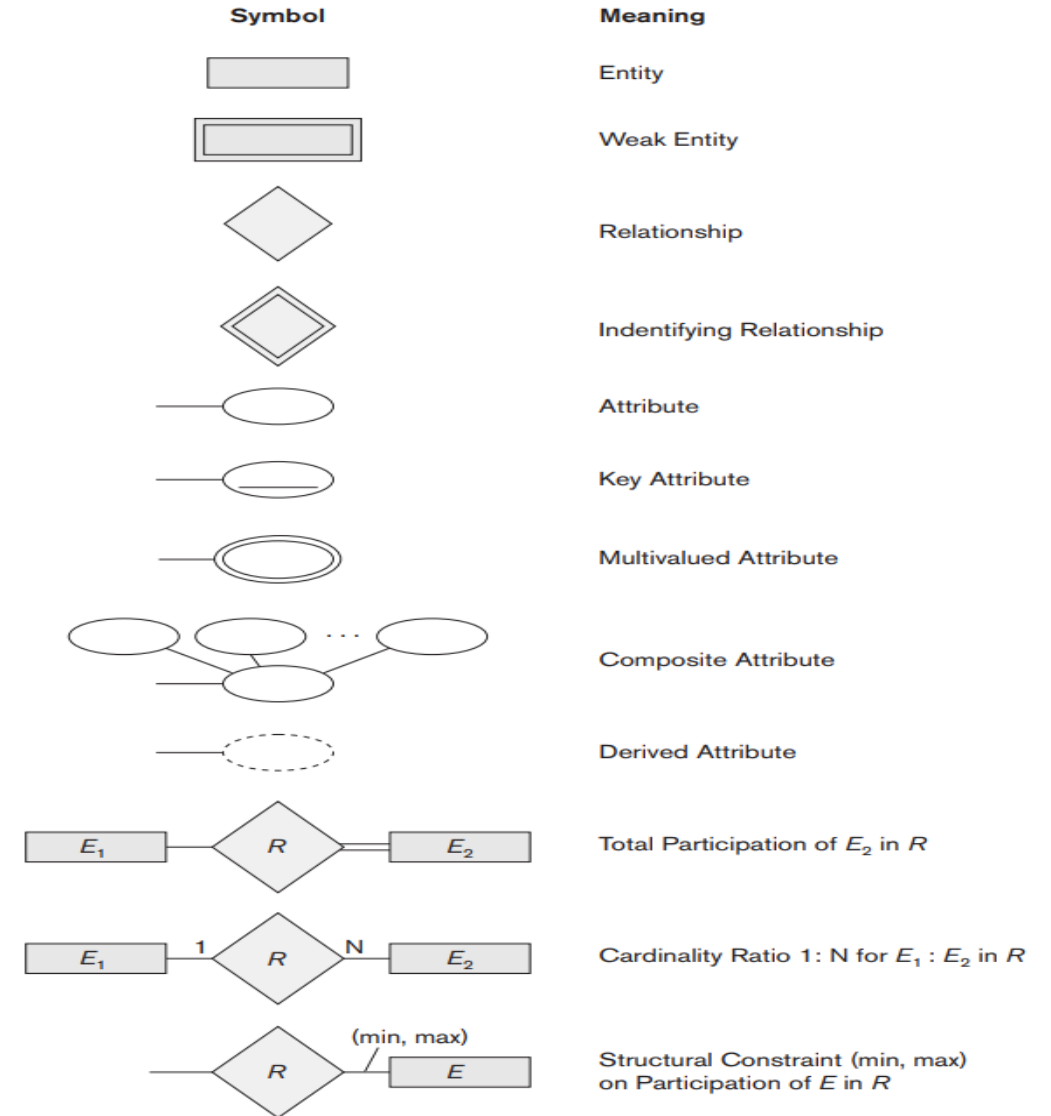
Varlık-İlişki (Entity-Relationship) Model

- **Tam Katılım (Total Participation)**

- İlişkiye giren varlıklardan birinin, mutlaka o ilişkide yer almak zorunda olması durumudur.
 - Zayıf varlıkların güçlü/sahip varlık ile yaptıkları ilişki mutlaka tam katılımda olmak zorundadır.
 - Diğer bir deyişle "bu varlık, diğeri olmadan var olamaz veya sisteme kaydedilemez" anlamına gelir.
- Çift ya da kalın çizgi ile gösterilir.
- Örneğin, yandaki örnekte E2 varlığının tüm elemanları E1 varlığına gitmelidir

- **Alt-Üst Değer (min,max)**

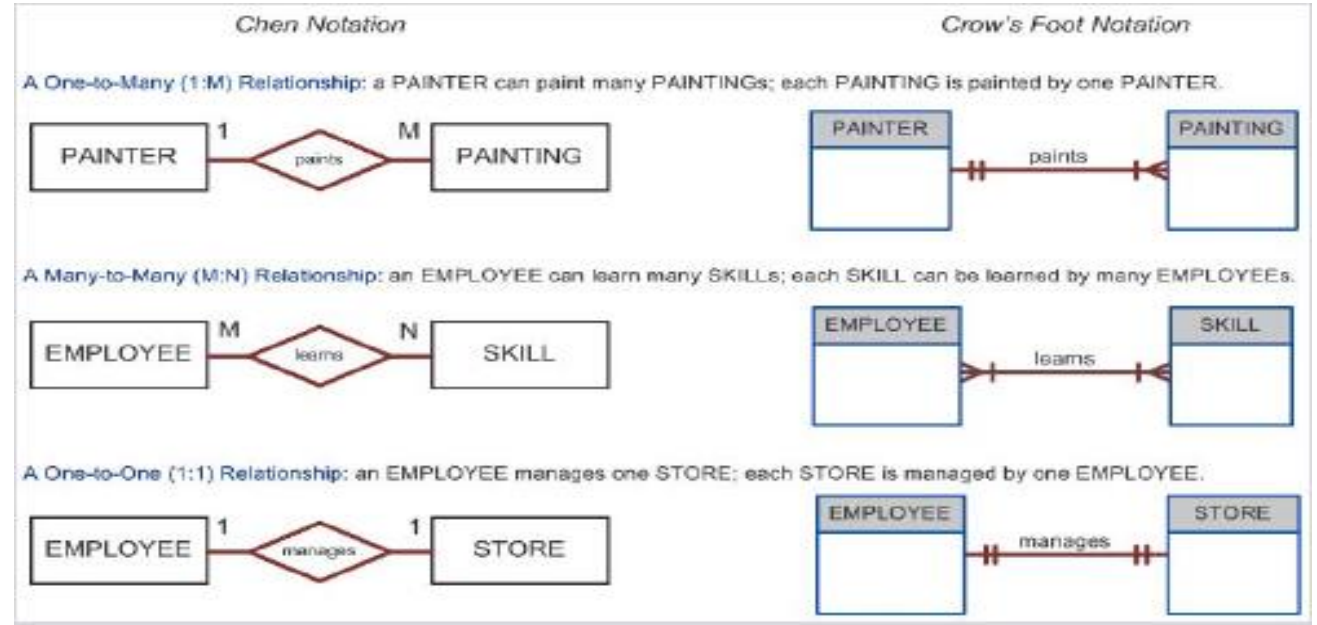
- İlişkinin alt ve üst sınırları belirlenmesi gerekiyorsa kullanılır.



Varlık-İlişki (Entity-Relationship) Model

- **Notasyon**

- En çok kullanılan iki notasyon türü CHEN ve CROW's FOOT notasyonlarıdır
- Farklı notasyonlar (Chen, Crow's Foot) aynı kavramsal modeli temsil eder; aralarındaki fark görseldir.



CROW'S FOOT SYMBOL	CARDINALITY	COMMENT
0..N	(0,N)	Zero or many. Many side is optional.
1..N	(1,N)	One or many. Many side is mandatory.
1..1	(1,1)	One and only one. 1 side is mandatory.
0..1	(0,1)	Zero or one. 1 side is optional.

Varlık-İlişki (Entity-Relationship) Model

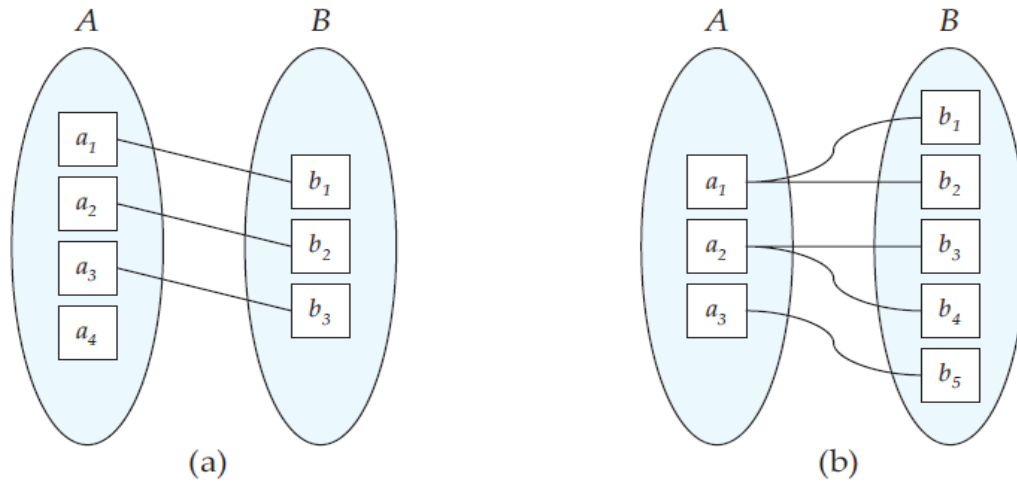


Figure 7.5 Mapping cardinalities. (a) One-to-one. (b) One-to-many.

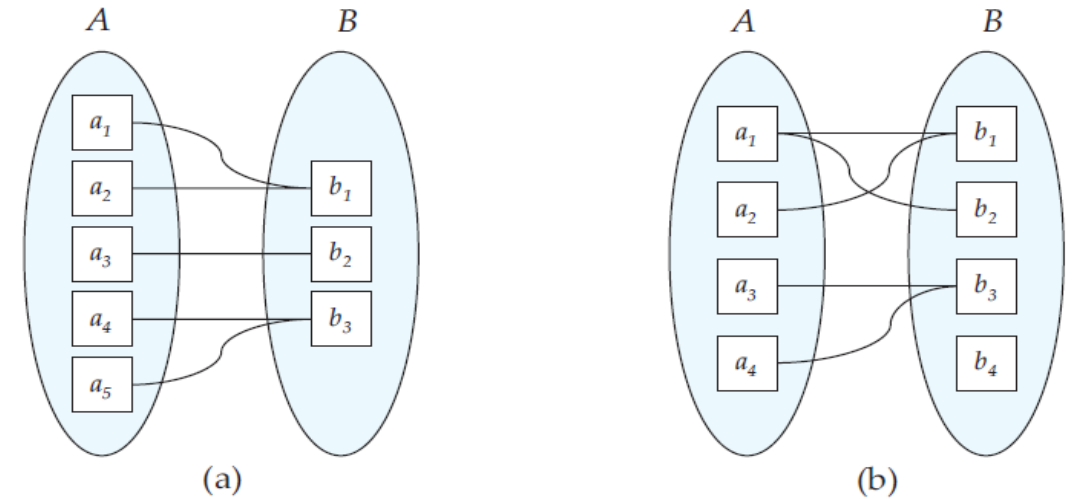
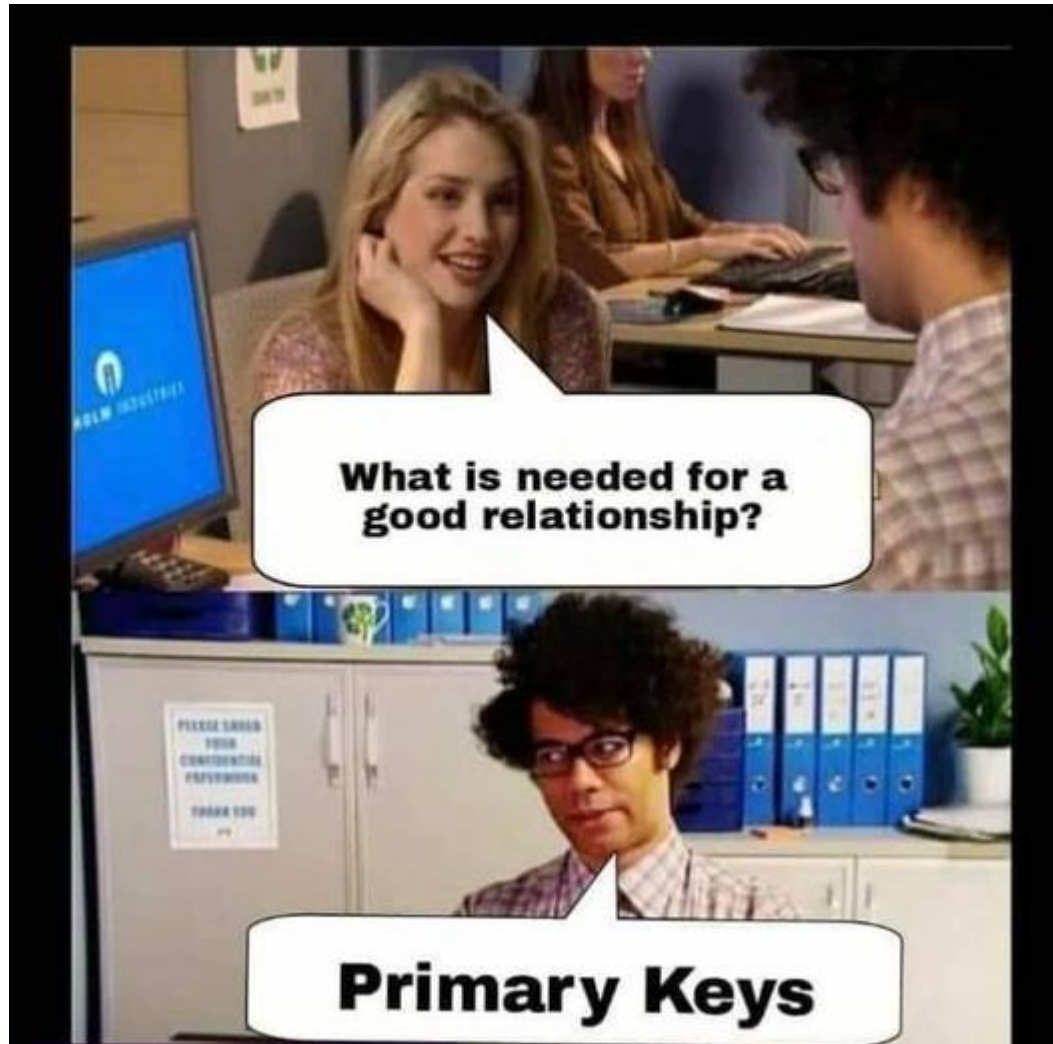


Figure 7.6 Mapping cardinalities. (a) Many-to-one. (b) Many-to-many.

Varlık-İlişki (Entity-Relationship) Model



How normal people see relationships

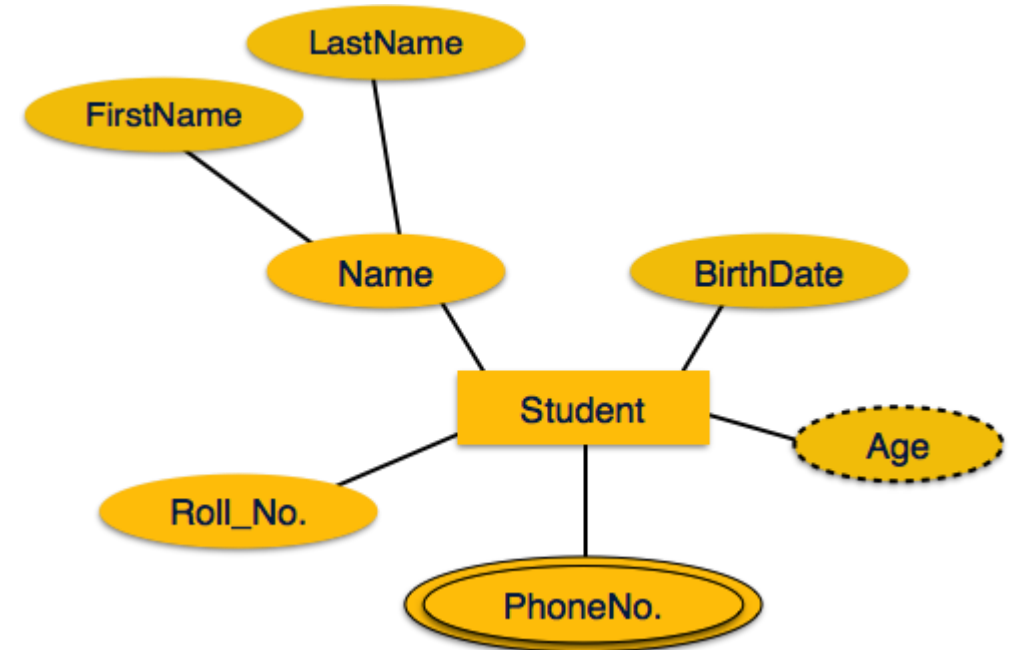


How database engineers see relationships



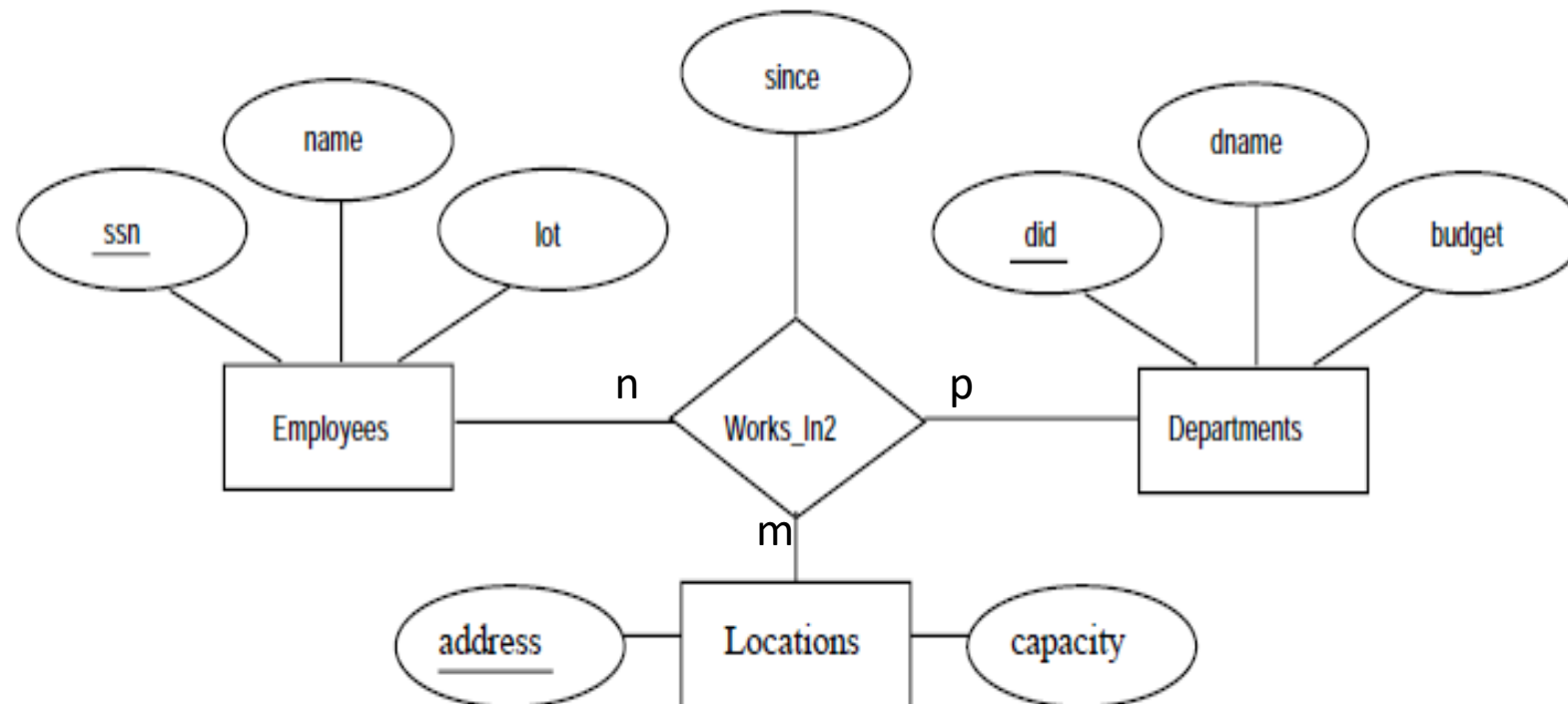
Varlık-İlişki (Entity-Relationship) Model

- **Entity:** Student
- **Attributes:** BirthDate, Roll_No
- **Composite Attribute:** Name
- **Multivalued Attributes:** PhoneNo
- **Derived Attributes:** Age



Varlık-İlişki (Entity-Relationship) Model

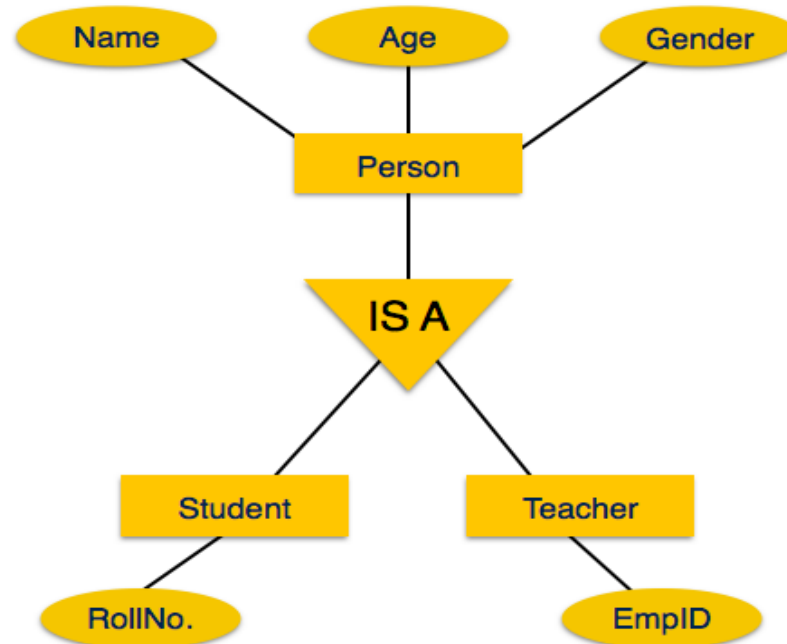
- **Entity:** Employees, Departments, Locations
- **Attributes:** ssn, name, lot, since, did, dname, budget, address, capacity



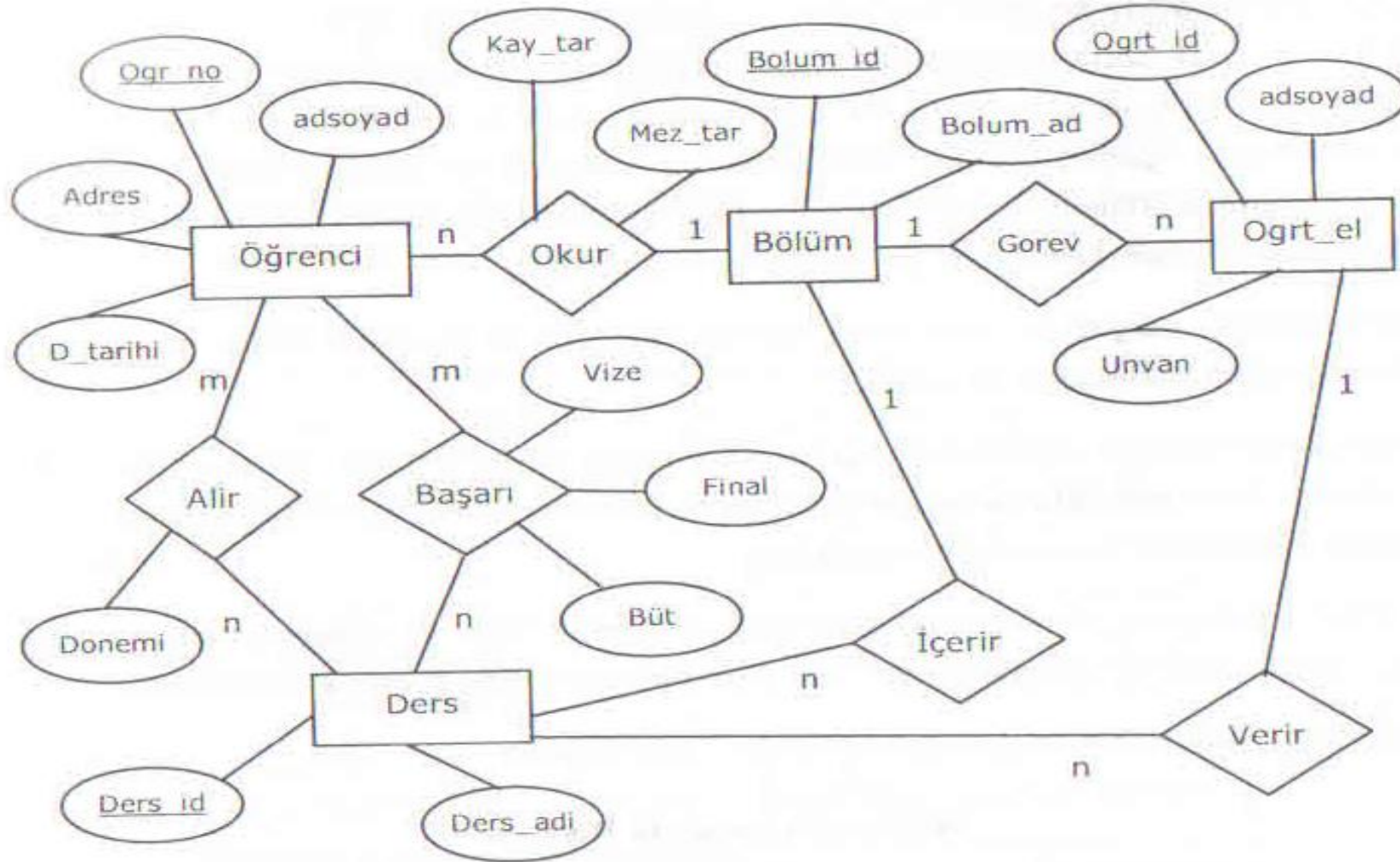
Varlık-İlişki (Entity-Relationship) Model

- **Kalıtım (Inheritance)**

- Kişi varlığına ait isim, yaş ve cinsiyet özellikleri vardır.
- Aynı zamanda her bir kişi ya öğrencidir (ve öğrenci numarası vardır) ya da öğretmendir (çalışan numarası vardır).
- Diğer bir deyişle öğrenci ya da öğretmen kalıtım yoluyla kişi varlığına ait özellikleri de kullanabilmektedirler.



Varlık-İlişki (Entity-Relationship) Model



Bire-Bir (1-1) İlişkilerin Şemaya Dönüştürülmesi

- Varlık kümeleri şemalara dönüştürülür.
- Özellikler şemaların sütunlarına dönüştürülür.
- İlişkideki varlıklardan birisinin PRIMARY KEY alanı diğer varlığa ait şemaya FOREIGN KEY olarak eklenir.
- Eğer ilişkiye katılım bir tarafta Zorunlu (Total) diğer tarafta Opsiyonel (Partial) ise, FK Zorunlu olan tarafa konulması tercih edilir.
 - Aksi takdirde FK sütununda çok fazla NULL değer oluşur.
- Zorunlu–zorunlu (total–total) katılım durumunda Foreign Key’in hangi tarafa konulacağı tasarıma bağlıdır.

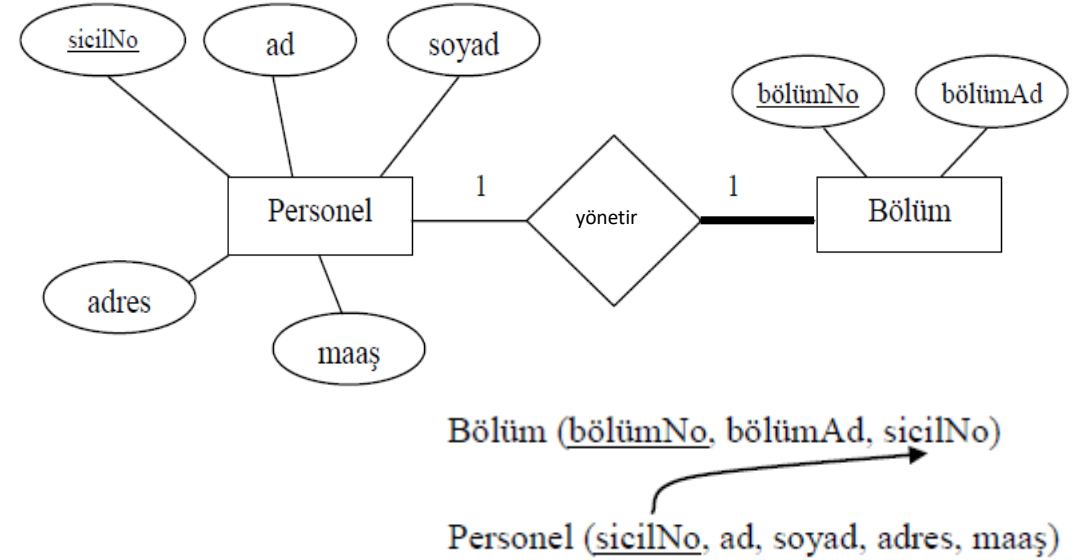
Bire-Bir (1-1) İlişkilerin Şemaya Dönüştürülmesi

Personel				
sicilNo	ad	soyad	adres	maaş
1	Mustafa	Taş	Yalova	5000
2	Sevinç	Acar	Bursa	6000
3	Hüseyin	Koç	İstanbul	2500
4	Ayşe	Kaya	Ankara	8000

Bölüm	
bölümNo	bölümAd
1	Muhasebe
2	Satış
3	Depo



Bölüm		
bölümNo	bölümAd	sicilNo
1	Muhasebe	3
2	Satış	2
3	Depo	1

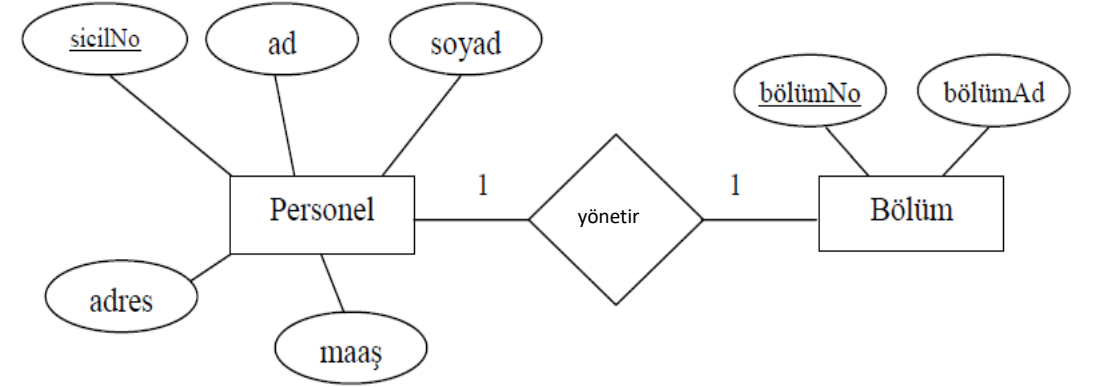


Bire-Bir (1-1) İlişkilerin Şemaya Dönüştürülmesi

- İlişkiye katılım zorunlu olmadığı için NULL değerler görülebilir

Personel				
sicilNo	ad	soyad	adres	maaş
1	Mustafa	Taş	Yalova	5000
2	Sevinç	Acar	Bursa	6000
3	Hüseyin	Koç	İstanbul	2500
4	Ayşe	Kaya	Ankara	8000

Bölüm	
bölümNo	bölümAd
1	Muhasebe
2	Satış
3	Depo



Bölüm		
bölümNo	bölümAd	sicilNo
1	Muhasebe	3
2	Satış	2
3	Depo	NULL

Bölüm (bölümNo, bölümAd, sicilNo)

Personel (sicilNo, ad, soyad, adres, maaş)

VEYA

Bölüm (bölümNo, bölümAd)

Personel (sicilNo, ad, soyad, adres, maaş, bölümNo)

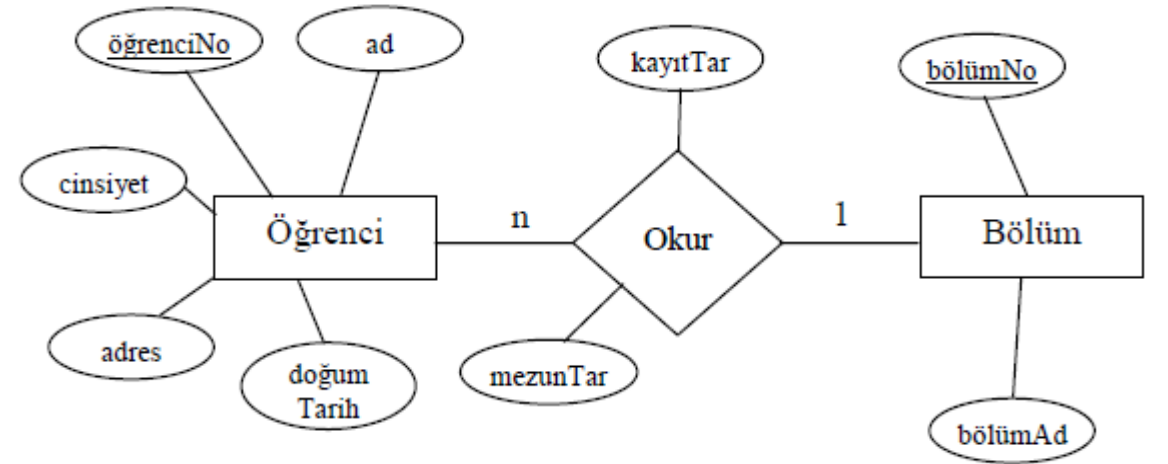
Personel					
sicilNo	ad	soyad	adres	maaş	bölümNo
1	Mustafa	Taş	Yalova	5000	3
2	Sevinç	Acar	Bursa	6000	2
3	Hüseyin	Koç	İstanbul	2500	1
4	Ayşe	Kaya	Ankara	8000	NULL

Bire-Çok (1-N) İlişkilerin Şemaya Dönüştürülmesi

- Varlık kümeleri şemalara dönüştürülür.
- Özellikler şemaların sütunlarına dönüştürülür.
- İlişkinin 1 tarafındaki varlığa ait PRIMARY KEY değeri diğer (n tarafındaki) varlığın şemasına FOREIGN KEY olarak eklenir.
- İlişkilerde özellik bulunuyorsa bu özellikler ilişkinin n tarafındaki varlığa ait şemaya sütun olarak eklenir.

Bölüm	
bölümNo	bölümAd
1	Bilgisayar Mühendisliği
2	Endüstri Mühendisliği
3	İşletme

Öğrenci				
öğrenciNo	ad	cinsiyet	adres	doğum tarihi
1	Mustafa	E	Yalova	01.01.2001
2	Sevinç	K	Bursa	31.12.2001
3	Ali	E	Ankara	30.06.2001



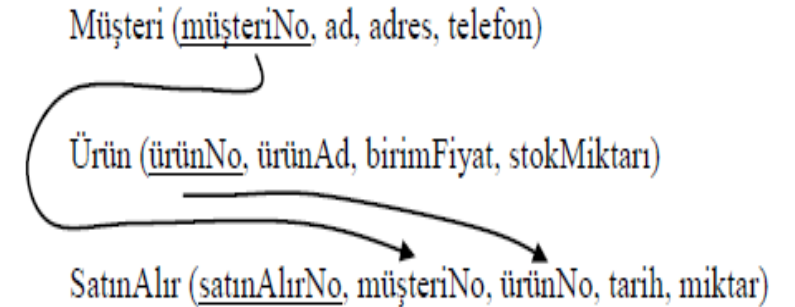
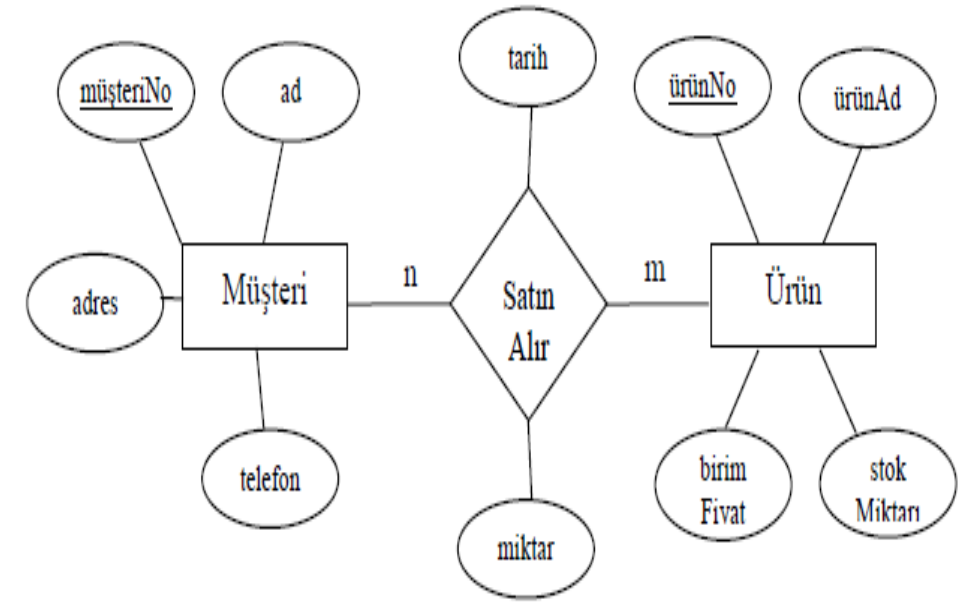
Öğrenci(öğrenciNo, ad, cinsiyet, adres, doğumTarih, bölümNo, kayıtTar, mezunTar)

Bölüm(bölümNo, bölümAd)

Öğrenci							
öğrenciNo	ad	cinsiyet	adres	doğum tarihi	bölümNo	kayıtTar	mezunTar
1	Mustafa	E	Yalova	01.01.2001	3	27.08.2018	15.06.2022
2	Sevinç	K	Bursa	31.12.2001	1	29.08.2020	NULL
3	Ali	E	Ankara	30.06.2001	2	01.09.2021	NULL

Çoka-Çok (N-M) İlişkilerin Şemaya Dönüştürülmesi

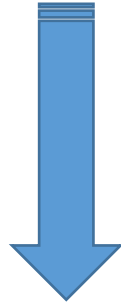
- Varlık kümeleri şemalara dönüştürülür.
- İlişki isminde bir şema (junction table) oluşturulur.
- Özellikler şemaların sütunlarına dönüştürülür.
- İlişki üzerindeki özellikler ise ilişkiden oluşturulan şemaya sütun olarak eklenir.
- İlişkiden oluşturulan yeni şemaya, ilişkiye dahil olan tüm varlıklara ait PRIMARY KEY değerleri FOREIGN KEY olarak eklenir.
- İlişkiden oluşturulan yeni şemanın PRIMARY KEY değeri:
 - Yeni bir sütun olabilir
 - FOREIGN KEY alanlarının birleşiminden de oluşabilir (Composite PRIMARY KEY)



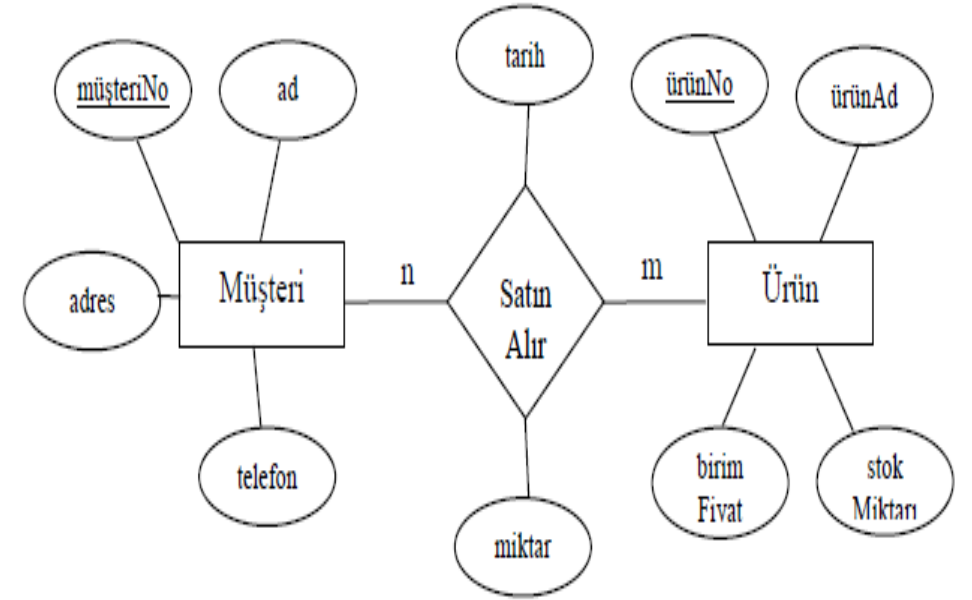
Çoka-Çok (N-M) İlişkilerin Şemaya Dönüştürülmesi

Müşteri			
<u>müşteriNo</u>	ad	telefon	adres
1	Mustafa	1234567	Yalova
2	Sevinç	7654321	Bursa
3	Ali	1112233	Ankara

Ürün			
<u>ürünNo</u>	ürünAd	birimFiyat	StokMiktarı
1	deterjan	15	15
2	pirinç	10	25
3	patates	5	35



SatınAlır				
<u>satınAlırNo</u>	<u>müşteriNo</u>	<u>ürünNo</u>	tarih	miktar
1	1	1	01.01.2026	1
2	1	3	01.01.2026	10
3	2	2	03.01.2026	8
4	1	1	01.02.2026	5



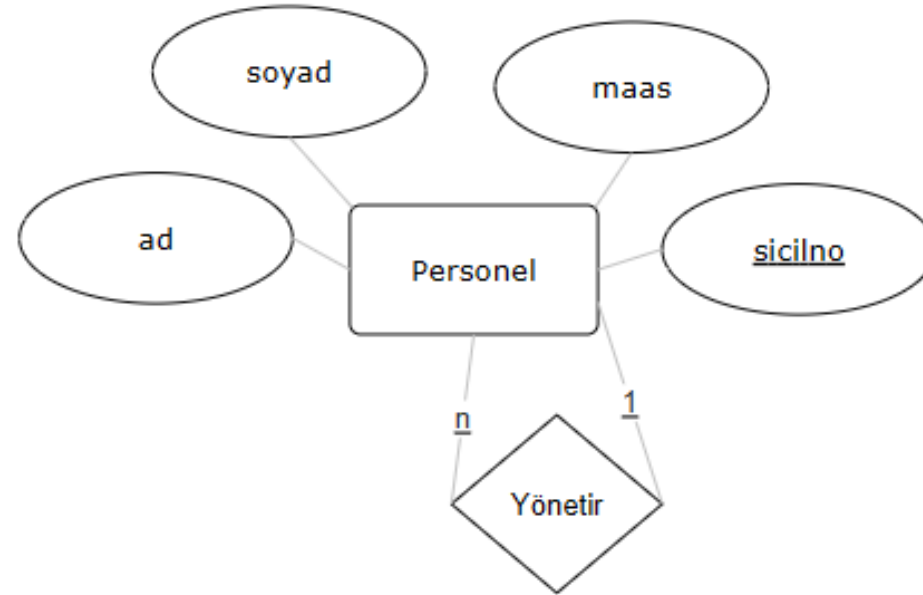
Müşteri (müşteriNo, ad, adres, telefon)

Ürün (ürünNo, ürünAd, birimFiyat, stokMiktarı)

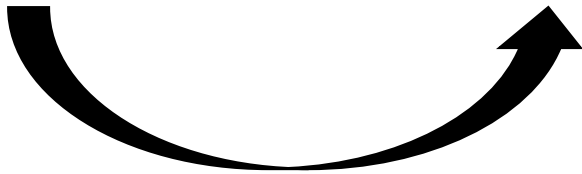
SatınAlır (satınAlırNo, müşteriNo, ürünNo, tarih, miktar)

Yinelemeli İlişkilerin Şemaya Dönüştürülmesi

- Tablonun PRIMARY KEY değeri, aynı tabloya farklı bir isimle FOREIGN KEY (self-referencing foreign key) olarak eklenir.

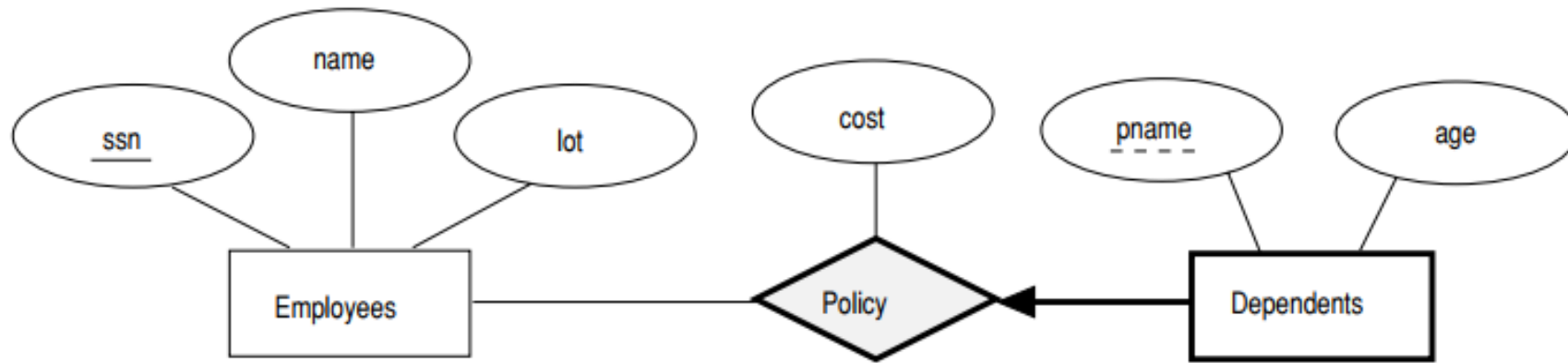


personel(sicilno, ad, soyad, maas, yönetici_id)



Zayıf Varlık Kümelerinin Şemaya Dönüştürülmesi

- Zayıf varlığın PRIMARY KEY'i hem kendi anahtar değerinden hem de ana varlığın PRIMARY KEY değerinden oluşur

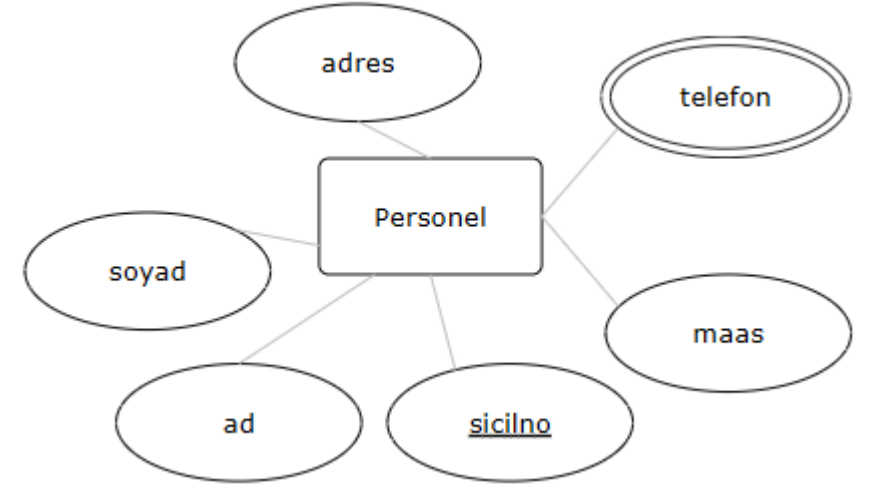


Employees(ssn, name, lot)

Dependents(ssn, pname, age)

Çok Değerli Özelliklerin Şemaya Dönüştürülmesi

- Çok değerli özellik için yeni bir şema oluşturulur.
- Ana varlığın PK'i, bu yeni tabloya FK olarak eklenir.
- Çok değerli özelliğin kendisi (örneğin telefon numarası) bu yeni tabloya bir özellik (sütun) olarak eklenir.
- Genellikle (FK + Özellik Sütunu) ikilisi, bu yeni tablonun PK'i olarak belirlenir.

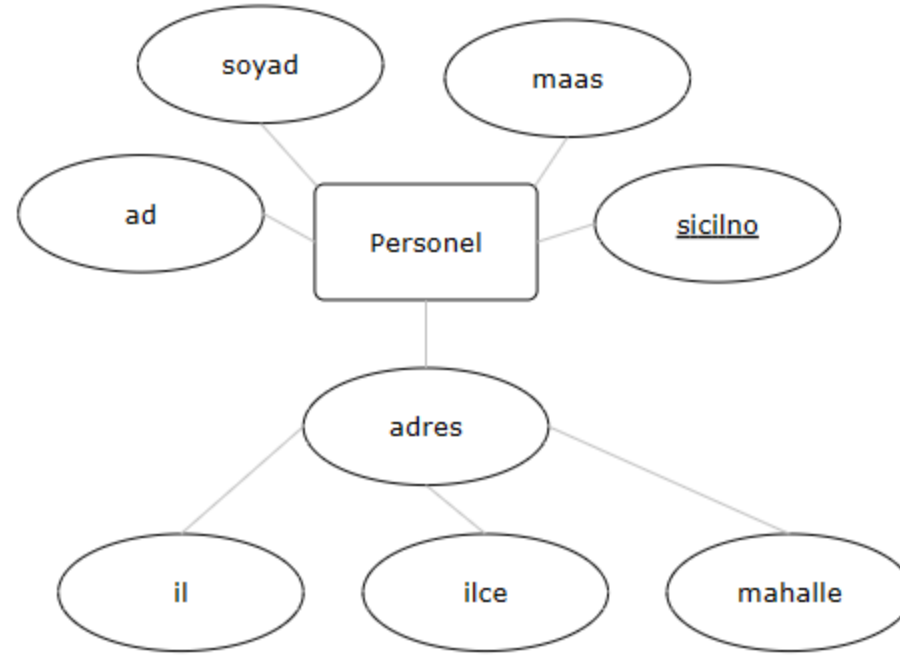


personel(sicilno, ad, soyad, adres, maas)

telefon(telefon, sicilno)

Bileşik Özelliklerin Şemaya Dönüştürülmesi

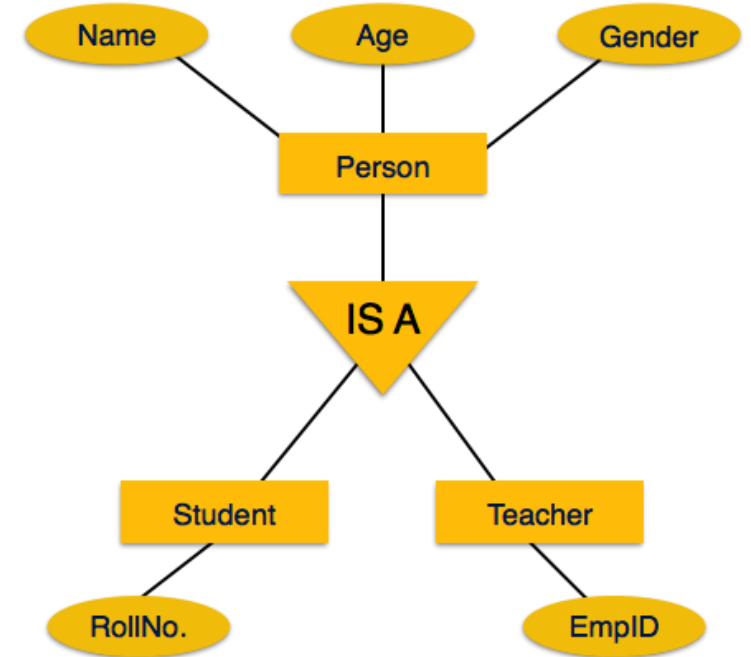
- Alt parçalarına ayrılarak ana tabloya sütun olarak eklenir.



personel(sicilno, ad, soyad, maas, il, ilce, mahalle)

Hiyerarşik İlişkilerin Şemaya Dönüştürülmesi

- Üç temel yöntem vardır
 - **Table per Type:** Her sınıf için ayrı tablo.
Person(ID, name, age, gender)
*Student(ID, RollNo, **PerID**)*
*Teacher(ID, EmpID, **PerID**)*
 - **Single Table Strategy:** Tüm hiyerarşiyi tek bir dev tabloya gömüp Type sütunu ile ayırmak.
Person(ID, type, name, age, gender, rollno, empid)
 - Genellikle performanslıdır; ancak çok sayıda NULL değer içerebilir
 - Eğer alt sınıfların (Student, Teacher) kendilerine has çok fazla sütunu yoksa tercih edilebilir
 - **Table per Concrete Class:** Sadece alt sınıflar için tablo oluşturmak.
Student(RollNo, name, age, gender)
Teacher(EmpID, name, age, gender)



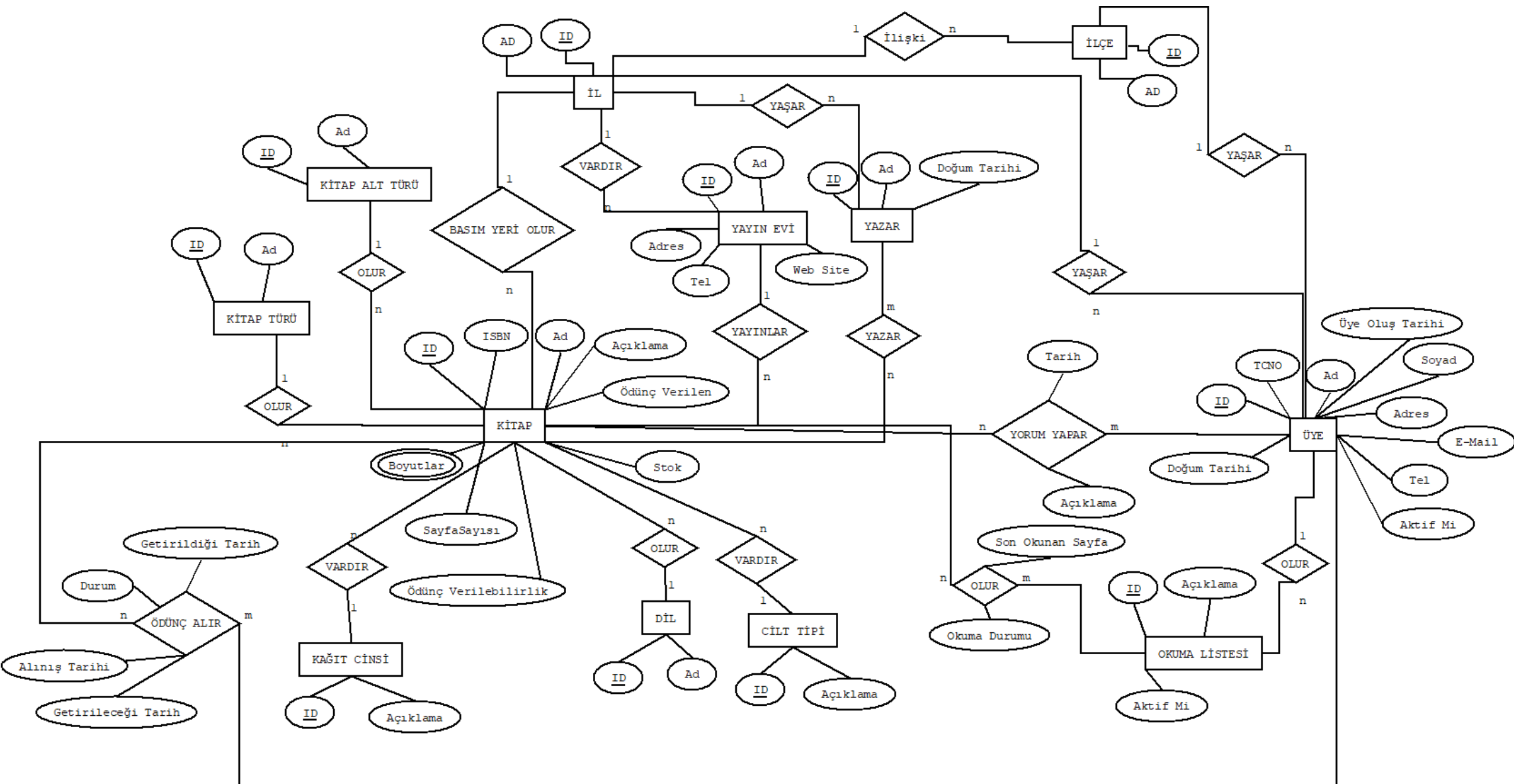
E-R Örnek – Kütüphane DB

Bir kütüphane otomasyon sistemi için gereksinimler aşağıda verilmiştir. Buna göre E-R diyagramını ve şemalarını oluşturunuz.

- Kütüphane de bulunan her bir kitap için aşağıdaki bilgilerin kayıt edilmesi gerekmektedir
 - ISBN numarası
 - Adı
 - Yayın evi (bir kitap bir yayın evinden çıkar ancak bir yayın evinden bir çok kitap çıkmıştır)
 - Yazarı (bir kitabın birden fazla yazarı olabileceği gibi bir yazarında birden çok kitabı olabilir)
 - Kitabın türü (Başvuru, bilgisayar, edebiyat, çocuk) ve alt türü (bilgisayar için programlama gibi)
 - Basım yeri
 - Ödünç verilebilir olup olmadığı
 - Kitabın kütüphanedeki toplam stoğu ve geçerli ödünç verilen miktarı
 - Boyutları (en , boy, yükseklik(kalınlık))
 - Sayfa sayısı
 - Kısa açıklama bilgisi
 - Kitabın dili (Türkçe, İngilizce, vb.)
 - Cilt tipi (Karton kapak, ciltli, vb.)
 - Kağıt cinsi (1. Hamur, 2. Hamur vb.)
 - Kitap hakkında üyelerin yorumları

E-R Örnek – Kütüphane DB

- Kütüphane üyelik sistemi ile çalışmaktadır her üye için aşağıdaki bilgi ve eylemler bulunmaktadır
 - TCNO
 - Ad
 - Soyad
 - Adres
 - İl
 - İlçe
 - E-Mail
 - Tel
 - Doğum tarihi
 - Üye oluş tarihi
 - Aktif Mi
 - Her bir üye kendisi için okuma listesi oluşturabilir. Buraya okumak istediği, okuyup bitirdiği ve halen okumakta olduğu kitapları kayıt edebilir. İsteddiği listeyi aktif ya da pasif yapabilir.
 - Her bir üye istediği zaman ödünç kitap alabilir. Ödünç kitap alabilmesi için kitabın ödünç alınabilir bir kitap olması ve üyenin ödünç alıp getirmediği en çok 3 kitap olması gereklidir. Varsayılan olarak başlangıç ödünç alma süresi 7 gündür. Bu süre dolmadan süre istenildiği kadar uzatılabilir. Kitabın ödünç alındığı tarih, geri getirilmesi gereken tarih, geri getirildiği tarih, geri getirildiğinde ki durumu (sorunsuz, hasarlı, kayıp) kaydedilmelidir.



E-R Örnek – Kütüphane DB - Şemalar

- il(id, ad)
- ilce(id, ad, il_id)
- kitap(id, ISBN, ad, aciklama, en, boy, yükseklik, odunc_verilen, odunc_verilebilirlik, sayfa_sayisi, stok, basim_il_id, ...)
- yazar(id, ad, dtarihi)
- kitap_yazari(kitap_id, yazar_id) → *(PK için ayrı bir id sütunu da oluşturulabilir)*
- uye(id, tcno, ad, soyad, dtarihi, tel, e_mail, aktif_mi, adres, ilce_id, il_id)
- uye_kitap_yorum(id, uye_id, kitap_id, tarih)
-
-
-

3. Bölüm

Normalizasyon

Normalizasyon

- Normalizasyon,
 - İlişkisel veri tabanlarında tekrarları azaltmak
 - Veri bütünlüğünü sağlamak
 - Ekleme, silme ve güncelleme anomalilerini önlemek amacıyla kullanılan bir tasarım sürecidir.
- Normalizasyon Seviyeleri (*BCNF ve üzeri normal formlar bu ders kapsamında ele alınmayacaktır*)
 - Normal Olmayan Form (Unnormalized Form, UNF)
 - Birinci Normal Form (First Normal Form, 1NF)
 - İkinci Normal Form (Second Normal Form, 2NF)
 - Üçüncü Normal Form (Third Normal Form, 3NF)
 - BCNF (Boyce-Codd Normal Form)
 - Dördüncü Normal Form (Fourth Normal Form, 4NF)
 - Beşinci Normal Form (Fifth Normal Form, 5NF)

Normal Olmayan Form (Unnormalized Form, UNF)

- İlişkisel veri tabanı modeline göre bir özelliğin (sütunun) bir satırda/tuple (hücrede) aldığı değerler atomik olmalıdır.
 - Daha küçük parçalara bölünemez
 - Tek bir değer içerir
- Bir tabloda, bir sütunun birden fazla değer içermesi veya tekrarlı gruplar bulunması durumunda tablo Normal Olmayan Form (UNF) olarak adlandırılır.
- Aşağıdaki tabloda **Dersler** alanı atomik olma kuralına uymadığı için (çok değerli/tekrar eden grup) bu tablo normal olmayan formdadır.

OGRENCI_DERS_UNF					
OgrNo	OgrAd	BolumKodu	BolumAd	BolumTel	Dersler (tekrar eden grup)
1001	Ayşe Yılmaz	CENG	Bilgisayar Müh.	0212-555-10-10	(DB101, Veri Tabanı, 4, 70), (PRG101, Programlama, 5, 85)
1002	Mehmet Kaya	CENG	Bilgisayar Müh.	0212-555-10-10	(DB101, Veri Tabanı, 4, 60)
1003	Elif Demir	IENG	Endüstri Müh.	0212-555-20-20	(MAT101, Matematik, 6, 90), (DB101, Veri Tabanı, 4, 75)

Birinci Normal Form (First Normal Form, 1NF)

- **Odak Noktası:** Atomik Değerler
 - Bir tablonun tüm sütunları atomik değerler içeriyorsa o tablo Birinci Normal Form'dadır (1NF).
 - 1NF, veri tekrarını ortadan kaldırmaz; yalnızca atomikliği garanti eder.
- Aşağıdaki OGRENCI_DERS_1NF tablonun her bir satırın her bir sütununda sadece bir değer vardır (1NF)
- **OgrNo** ve **DersKodu** beraber benzersiz (unique) özellik gösterdiğinden tablonun PRIMARY KEY'ini oluşturmaktadırlar (bir öğrenci aynı dersi bir kez alıyor varsayımıyla).

OGRENCI_DERS_1NF								
<u>OgrNo</u>	OgrAd	BolumKodu	BolumAd	BolumTel	<u>DersKodu</u>	DersAd	AKTS	Not
1001	Ayşe Yılmaz	CENG	Bilgisayar Müh.	0212-555-10-10	DB101	Veri Tabanı	4	70
1001	Ayşe Yılmaz	CENG	Bilgisayar Müh.	0212-555-10-10	PRG101	Programlama	5	85
1002	Mehmet Kaya	CENG	Bilgisayar Müh.	0212-555-10-10	DB101	Veri Tabanı	4	60
1003	Elif Demir	IENG	Endüstri Müh.	0212-555-20-20	MAT101	Matematik	6	90
1003	Elif Demir	IENG	Endüstri Müh.	0212-555-20-20	DB101	Veri Tabanı	4	75

Birinci Normal Form (First Normal Form, 1NF)

- Ancak aşağıdaki sorunlar / anomaliler engellenememektedir:
 - EKLEME:** Yeni bir ders tanımlamak için öğrenci ve not bilgisi de girilmesi gerekmektedir
 - SİLME:** Bir dersi sistemden silmek istediğimizde eğer dersin bağlı olduğu bölümde başka bir ders yoksa bölüm de sistemden silinmiş olur
 - GÜNCELLEME:** Ders adı değişikliği yapılması durumunda ilgili dersin olduğu tüm satırların güncellenmesi gerekecektir.

OGRENCI_DERS_1NF								
<u>OgrNo</u>	OgrAd	BolumKodu	BolumAd	BolumTel	<u>DersKodu</u>	DersAd	AKTS	Not
1001	Ayşe Yılmaz	CENG	Bilgisayar Müh.	0212-555-10-10	DB101	Veri Tabanı	4	70
1001	Ayşe Yılmaz	CENG	Bilgisayar Müh.	0212-555-10-10	PRG101	Programlama	5	85
1002	Mehmet Kaya	CENG	Bilgisayar Müh.	0212-555-10-10	DB101	Veri Tabanı	4	60
1003	Elif Demir	IENG	Endüstri Müh.	0212-555-20-20	MAT101	Matematik	6	90
1003	Elif Demir	IENG	Endüstri Müh.	0212-555-20-20	DB101	Veri Tabanı	4	75

Bağımlılıklar

1. İşlevsel Bağımlılık (Functional Dependency)

- Bir tablodaki bir sütunun (veya sütun grubunun, composite PK) değerinin, başka bir sütunun değerini benzersiz bir şekilde belirlemesi durumudur.
- Matematikteki fonksiyonlara benzetilebilir yani $f(x)=y$ için eğer x 'i biliyorsanız y 'yi de kesin olarak bilirsiniz.
- A sütunundaki her benzersiz değer için, B sütununda her zaman aynı değer bulunmalıdır.
- $A \rightarrow B$
 - A, B'yi belirler veya B, A'ya fonksiyonel olarak tam bağımlıdır
- Örneğin aşağıdaki tabloda Öğrenci No (101) bilindiğinde, Öğrenci Adı da (Ahmet) kesinlikle bilinir.

Öğrenci No (A)	Öğrenci Adı (B)
101	Ahmet
102	Ayşe
103	Mehmet

Bağımlılıklar

1. İşlevsel Bağımlılık (Functional Dependency)

- Kısmi İşlevsel Bağımlılık (Partial Functional Dependency)

- Bir tablonun anahtar olmayan bir sütunun, tablo anahtarının (composite PK) sadece bir parçasına bağlı olması durumu.
- Örnek
 - $ogrenci_no + ders_kodu \rightarrow not$ (tam bağımlılık)
 - $ders_adi$ (Matematik) sadece ve sadece $ders_koduna$ (MAT101) bağlıdır.
 - ❑ $ogrenci_no$ ile hiçbir ilgisi yoktur.
 - ❑ Dolayısıyla kısmi bağımlılık oluşur ($ders_kodu \rightarrow ders_adi$)

<u>ogrenci_no</u>	<u>ders_kodu</u>	not	ders_adi
101	MAT101	85	Matematik
102	MAT101	40	Matematik
101	PHY101	70	Fizik

Bağımlılıklar

2. Geçişli Bağımlılık (Transitive Dependency)

- Fonksiyonel bağımlılığın dolaylı bir zincirleme halidir.
- Eğer A sütunu B'yi belirliyorsa ve B sütunu da C'yi belirliyorsa (ancak C, A'yı belirlemiyorsa), o zaman A, C'yi geçişli olarak belirler.
- $A \rightarrow B$ ve $B \rightarrow C$ ise, dolaylı olarak $A \rightarrow C$ olur.
- Örneğin aşağıdaki tabloda:
 - $A \rightarrow B$: Sipariş No bilindiğinde Müşteri ID bulunur çünkü her sipariş bir müşteriye aittir.
 - $B \rightarrow C$: Müşteri ID bilindiğinde Müşteri Şehir bulunur. (Müşteri "M20" her zaman "İstanbul"dadır).
 - $A \rightarrow C$: Sipariş No, Müşteri Şehir bilgisini dolaylı yoldan belirler.

Sipariş No (A)	Müşteri ID (B)	Müşteri Şehir (C)
5001	M20	İstanbul
5002	M35	Ankara
5003	M20	İstanbul

Birinci Normal Form (First Normal Form, 1NF)

- **Bağımlılıklar**
 - (OgrNo, DersKodu) → Not (Tam İşlevsel Bağımlılık)
 - OgrNo → OgrAd, BolumKodu, BolumAd, BolumTel (Kısmi İşlevsel Bağımlılık)
 - DersKodu → DersAd, AKTS (Kısmi İşlevsel Bağımlılık)
- Bu yüzden 1NF tabloyu parçalarız

OGRENCI_DERS_1NF								
<u>OgrNo</u>	OgrAd	BolumKodu	BolumAd	BolumTel	<u>DersKodu</u>	DersAd	AKTS	Not
1001	Ayşe Yılmaz	CENG	Bilgisayar Müh.	0212-555-10-10	DB101	Veri Tabanı	4	70
1001	Ayşe Yılmaz	CENG	Bilgisayar Müh.	0212-555-10-10	PRG101	Programlama	5	85
1002	Mehmet Kaya	CENG	Bilgisayar Müh.	0212-555-10-10	DB101	Veri Tabanı	4	60
1003	Elif Demir	IENG	Endüstri Müh.	0212-555-20-20	MAT101	Matematik	6	90
1003	Elif Demir	IENG	Endüstri Müh.	0212-555-20-20	DB101	Veri Tabanı	4	75

İkinci Normal Form (Second Normal Form, 2NF)

- **Odak Noktası:** Kısmi İşlevsel Bağımlılıklar
 - Tablo 1NF'de olmalıdır
 - Anahtar olmayan ve bileşik PK'in sadece bir kısmına bağlı her sütun ayrılmalıdır.
 - Bu yüzden tek sütunlu PK'e sahip her 1NF tablo, doğal olarak 2NF'dedir.
- **KAYIT** isimli tablonun birinci normal formunda (OGRENCI_DERS_1NF) yaşanan sorunlar/anomaliler ikinci normal forma dönüştürme ile ortadan kalkmıştır
- Ama hâlâ 3NF sorunu mevcuttur
 - **OGRENCI** tablosunda geçişli bağımlılık bulunmaktadır

OGRENCI				
<u>OgrNo</u>	OgrAd	BolumKodu	BolumAd	BolumTel
1001	Ayşe Yılmaz	CENG	Bilgisayar Müh.	0212-555-10-10
1002	Mehmet Kaya	CENG	Bilgisayar Müh.	0212-555-10-10
1003	Elif Demir	IE	Endüstri Müh.	0212-555-20-20

DERS		
<u>DersKodu</u>	DersAd	AKTS
DB101	Veri Tabanı	4
PRG101	Programlama	5
MAT101	Matematik	6

KAYIT		
<u>OgrNo</u>	<u>DersKodu</u>	Not
1001	DB101	70
1001	PRG101	85
1002	DB101	60
1003	MAT101	90
1003	DB101	75

İkinci Normal Form (Second Normal Form, 2NF)

- **OGRENCI** tablosu için yaşanabilecek sorunlar / anomaliler:
 - **EKLEME**: Yeni bir bölüm tanımlamak için öğrenci bilgisi de girilmesi gerekmektedir
 - **SİLME**: Bir öğrenciyi sistemden silmek istediğimizde eğer ait olduğu bölümde başka bir öğrenci yoksa bölüm de sistemden silinmiş olacaktır
 - **GÜNCELLEME**: Bir bölümün adı ve/veya telefonu güncellenmek istendiği takdirde ilgili bölümdeki öğrenci sayısı kadar güncelleme işlemi yapılması gerekecektir

OGRENCI				
<u>OgrNo</u>	OgrAd	BolumKodu	BolumAd	BolumTel
1001	Ayşe Yılmaz	CENG	Bilgisayar Müh.	0212-555-10-10
1002	Mehmet Kaya	CENG	Bilgisayar Müh.	0212-555-10-10
1003	Elif Demir	IE	Endüstri Müh.	0212-555-20-20

İkinci Normal Form (Second Normal Form, 2NF)

- **Bağımlılıklar**
 - $OgrNo \rightarrow BolumKodu$ (Tam İşlevsel Bağımlılık)
 - $BolumKodu \rightarrow BolumAd, BolumTel$ (Tam İşlevsel Bağımlılık)
 - $OgrNo \rightarrow BolumAd, BolumTel$ (Geçişli Bağımlılık)
- Bu yüzden 2NF tabloyu parçalarız

OGRENCİ				
<u>OgrNo</u>	OgrAd	BolumKodu	BolumAd	BolumTel
1001	Ayşe Yılmaz	CENG	Bilgisayar Müh.	0212-555-10-10
1002	Mehmet Kaya	CENG	Bilgisayar Müh.	0212-555-10-10
1003	Elif Demir	IE	Endüstri Müh.	0212-555-20-20

Üçüncü Normal Form (Third Normal Form, 3NF)

- **Odak Noktası:** Geçişli Bağımlılıklar
 - Tablo 2NF'de olmalıdır
 - Anahtar olmayan sütunlar arasındaki bağımlılıklar ($A \rightarrow B \rightarrow C$) koparılmalıdır
 - Diğer bir ifadeyle anahtar olmayan sütunların yalnızca PRIMARY KEY'e doğrudan bağlı olması sağlanmalıdır
- **OGRENCI** isimli tablonun ikinci normal formunda yaşanan sorunlar/anomaliler üçüncü normal forma dönüştürme ile ortadan kalkmıştır
- Tüm tablolar 3NF'e getirilmiştir
- 3NF, veri tutarlılığını artırırken JOIN sayısını da arttırabilir.

OGRENCI		
<u>OgrNo</u>	OgrAd	BolumKodu
1001	Ayşe Yılmaz	CENG
1002	Mehmet Kaya	CENG
1003	Elif Demir	IE

KAYIT		
<u>OgrNo</u>	<u>DersKodu</u>	Not
1001	DB101	70
1001	PRG101	85
1002	DB101	60
1003	MAT101	90
1003	DB101	75

DERS		
<u>DersKodu</u>	DersAd	AKTS
DB101	Veri Tabanı	4
PRG101	Programlama	5
MAT101	Matematik	6

BOLUM		
<u>BolumKodu</u>	BolumAd	BolumTel
CENG	Bilgisayar Müh.	0212-555-10-10
IE	Endüstri Müh.	0212-555-20-20

Denormalizasyon

- Normalizasyon veri tekrarını azaltır ancak sorguların karmaşıklığını (JOIN sayısını) artırır.
- Çok fazla JOIN işlemi, veri tabanı performansını düşürebilir (sorgular yavaşlar).
- Denormalizasyon, performansı artırmak amacıyla, bilinçli olarak veri tekrarına izin verme ve normalizasyon kurallarını ihlal etme işlemidir.
- Ne Zaman Kullanılır?
 - Veri okuma hızının (SELECT), veri yazma/güncelleme hızından çok daha önemli olduğu raporlama sistemlerinde.
 - Çok büyük verilerin olduğu Veri Ambarı (Data Warehouse) uygulamalarında.
- Veri tutarlılığını sağlamak artık veri tabanı motorunun değil, yazılımcının sorumluluğundadır
 - Update anomalilerine dikkat edilmelidir.

4. Bölüm

İLİŞKİSEL CEBİR

İlişkisel Cebir – Relational Algebra

- İlişkisel veri modelinde tanımlı tablolar (relations) üzerinde çalışan, matematiksel temelli ve işlemsel (procedural) bir sorgulama dilidir.
- Her ifade bir ilişki (relation) üretir
- Sorgulamalar, tek ya da çift girdi alan operatörler ile gerçekleştirilir
- Sorgu Dilleri != Programlama Dilleri
 - Karmaşık hesaplama yapmak için geliştirilmemiştir
 - Turing-complete değildir
 - Büyük veri kümelerini kolay ve hızlı bir şekilde erişmeyi sağlar

İşlem	Amaç	Notasyon
SELECTION (Seçim)	Verilen şarta uyan tüm satırları seçer	$\sigma_{\langle \text{SELECTION CONDITION} \rangle}(R)$
PROJECTION (İzdüşüm)	Bir tablonun sadece belirli sütunlarını seçerek yeni bir tablo oluşturma işlemidir, tekrarlar silinir	$\pi_{\langle \text{ATTRIBUTE LIST} \rangle}(R)$
CROSS-PRODUCT (Çarpım)	İki farklı tablonun satırlarının mümkün olan bütün kombinasyonlarının birleştirilerek yeni bir tablo oluşturulması	$R_1 \times R_2$
UNION (Birleşim)	Şemaları aynı olan iki tabloda bulunan satırların tümü	$R_1 \cup R_2$
INTERSECTION (Kesişim)	Şemaları aynı olan iki tabloda da bulunan satırların sadece ortak olanları	$R_1 \cap R_2$
DIFFERENCE (Fark)	Şemaları aynı olan iki tablodan birincisinde olup ta ikincisinde olmayan satırların tümü	$R_1 - R_2$
JOIN (Bağlantı)	İki tablonun satırlarının verilen bir şarta uygun bütün kombinasyonlarının birleştirilerek yeni bir tablo oluşturulması	$R_1 \bowtie_{\langle \text{JOIN CONDITION} \rangle} R_2$

İlişkisel Cebir – Seçim (Selection) , σ (sigma)

$\sigma_{Yas>25}(Musteri)$

Ad	Soyad	Yas	Meslek
Şadi Evren	Şeker	30	Bilgisayar Müh.
Ali	Baba	50	Tüccar
Veli	Demir	20	Öğrenci
Cem	Yıldız	40	Programcı



Ad	Soyad	Yas	Meslek
Şadi Evren	Şeker	30	Bilgisayar Müh.
Ali	Baba	50	Tüccar
Cem	Yıldız	40	Programcı

$\wedge$: Ve

$\vee$: Veya

$\neg$: Değil

$=$: Eşit

$\neq$: Eşit değil

$>$: Büyük

$\geq$: Büyük eşit

$<$: Küçük

$\leq$: Küçük eşit

$\sigma_{Yas<30 \vee Ad='Ali'}(Musteri)$

Ad	Soyad	Yas	Meslek
Şadi Evren	Şeker	30	Bilgisayar Müh.
Ali	Baba	50	Tüccar
Veli	Demir	20	Öğrenci
Cem	Yıldız	40	Programcı



Ad	Soyad	Yas	Meslek
Ali	Baba	50	Tüccar
Veli	Demir	20	Öğrenci

İlişkisel Cebir – İzdüşüm (Projection) , $\pi(p_i)$

$\pi_{Ad}(Musteri)$

Ad	Soyad	Yas	Meslek
Şadi Evren	Şeker	30	Bilgisayar Müh.
Ali	Baba	50	Tüccar
Veli	Demir	20	Öğrenci
Cem	Yıldız	40	Programcı



Ad
Şadi Evren
Ali
Veli
Cem

$\pi_{Soyad,Meslek}(Musteri)$

Ad	Soyad	Yas	Meslek
Şadi Evren	Şeker	30	Bilgisayar Müh.
Ali	Baba	50	Tüccar
Veli	Demir	20	Öğrenci
Sedat	Şeker	34	Bilgisayar Müh.
Cem	Yıldız	40	Programcı



Soyad	Meslek
Şeker	Bilgisayar Müh.
Baba	Tüccar
Demir	Öğrenci
Yıldız	Programcı

- Birden fazla sütun elde edilebilir.
- İşlem sonunda satır tekrarı olmaz.



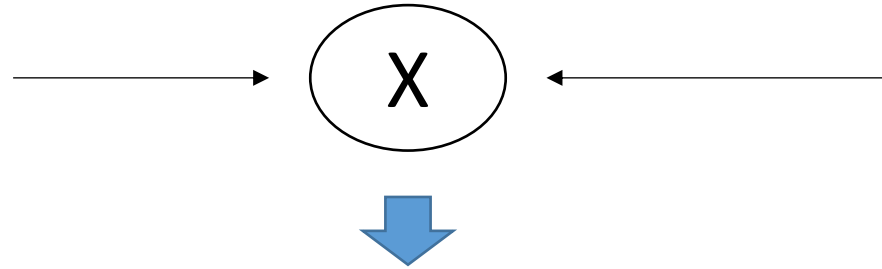
İlişkisel Cebir – Çarpım (Cross-Product)

Öğrenci

ÖğrenciNo	İsim	GPA	Yaş
1	A	2.35	21
2	B	3.43	20
3	C	2.99	22

DersSecimi

ÖğrenciNo	DersID
1	CS201203
3	CS201212



ÖğrenciNo	İsim	GPA	Yaş	ÖğrenciNo	DersID
1	A	2.35	21	1	CS201203
2	B	3.43	20	1	CS201203
3	C	2.99	22	1	CS201203
1	A	2.35	21	3	CS201212
2	B	3.43	20	3	CS201212
3	C	2.99	22	3	CS201212

İlişkisel Cebir – Çarpım (Cross-Product)

Satışlar

SatışID	SatışUrunID	Adet
4053	547	3
547	3211	12
3211	234	7

Ürünler

UrunID	Isim
234	Saat
547	Kalem
3211	Klavye

X

UrunID	Isim	SatışID	SatışUrunID	Adet
234	Saat	4053	547	3
234	Saat	547	3211	12
234	Saat	3211	234	7
547	Kalem	4053	547	3
547	Kalem	547	3211	12
547	Kalem	3211	234	7
3211	Klavye	4053	547	3
3211	Klavye	547	3211	12
3211	Klavye	3211	234	7

$\sigma_{UrunID = SatışUrunID} (Satışlar \times Ürünler)$

UrunID	Isim	SatışID	SatışUrunID	Adet
234	Saat	3211	234	7
547	Kalem	4053	547	3
3211	Klavye	547	3211	12

İlişkisel Cebir – Bağlantı (Join), ⋈

- Join işleminde verilen koşul PK ve FK bağlantılarını yapmak zorunda değildir ama bu durumda olası tüm eşleşme kombinasyonları oluşur.
- **Teta Join (θ)**
 - Çarpım işlemi ile seçim işleminin birlikte kullanılmasıdır
- **Equi-Join**
 - Verilen koşul eşitlik ise Equi-Join oluşur
- **Natural Join**
 - Her iki tablodaki birleşim sütun isimleri aynı ise koşul belirtilmeden yazılabilir
 - "Aynı isimli sütunlar aynı şeyi temsil eder" varsayımıyla hareket eder
 - Dolayısıyla aynı isimli fakat anlamsal olarak ilgisiz sütunlar varsa yanlış eşleşmelere yol açabilir; bu nedenle de dikkatli kullanılmalıdır.

İlişkisel Cebir – Bağlantı (Join), ⋈

- **Outer Join**

- Genelde Join işleminde yapılan işlem **Inner Join** olarak adlandırılır.
 - Yani sadece verilen koşulu sağlayan satırlar gösterilir.
- Ancak bazı durumlarda soldaki, sağdaki ya da her iki tablodaki koşulu sağlamayan satırların da görüntülenmesi gerekebilir.
- Bu durumlarda sırasıyla **Left Outer Join** ($\bowtie$), **Right Outer Join** ($\bowtie$) ve **Full Outer Join** ($\bowtie$) işlemleri kullanılır.
- Eşleşme olmayan satırların diğer tablodaki sütunları için **NULL** yani belirsiz ifadesi gelir.

İlişkisel Cebir – Bağlantı (Join), ⋈

Satışlar

SatışID	SatışUrunID	Adet
4053	547	3
547	3211	12
3211	234	7

Ürünler ⋈_{UrunID=SatışUrunID} Satışlar

UrunID	Isim	SatışID	SatışUrunID	Adet
234	Saat	3211	234	7
547	Kalem	4053	547	3
3211	Klavye	547	3211	12

Ürünler

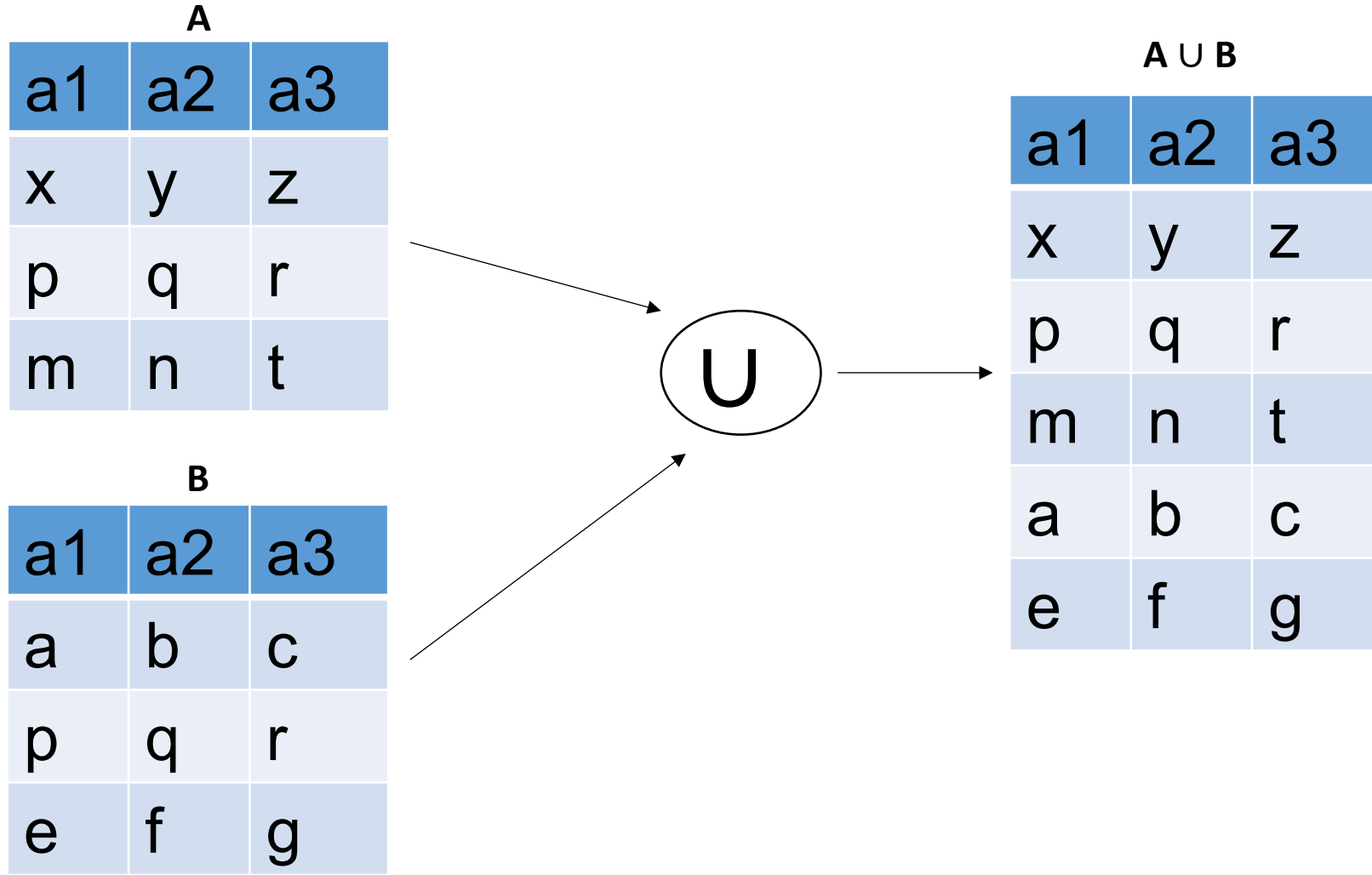
UrunID	Isim
234	Saat
547	Kalem
3211	Klavye
5887	Silgi

Ürünler ⋈_{UrunID=SatışUrunID} Satışlar

UrunID	Isim	SatışID	SatışUrunID	Adet
234	Saat	3211	234	7
547	Kalem	4053	547	3
3211	Klavye	547	3211	12
5887	Silgi	NULL	NULL	NULL

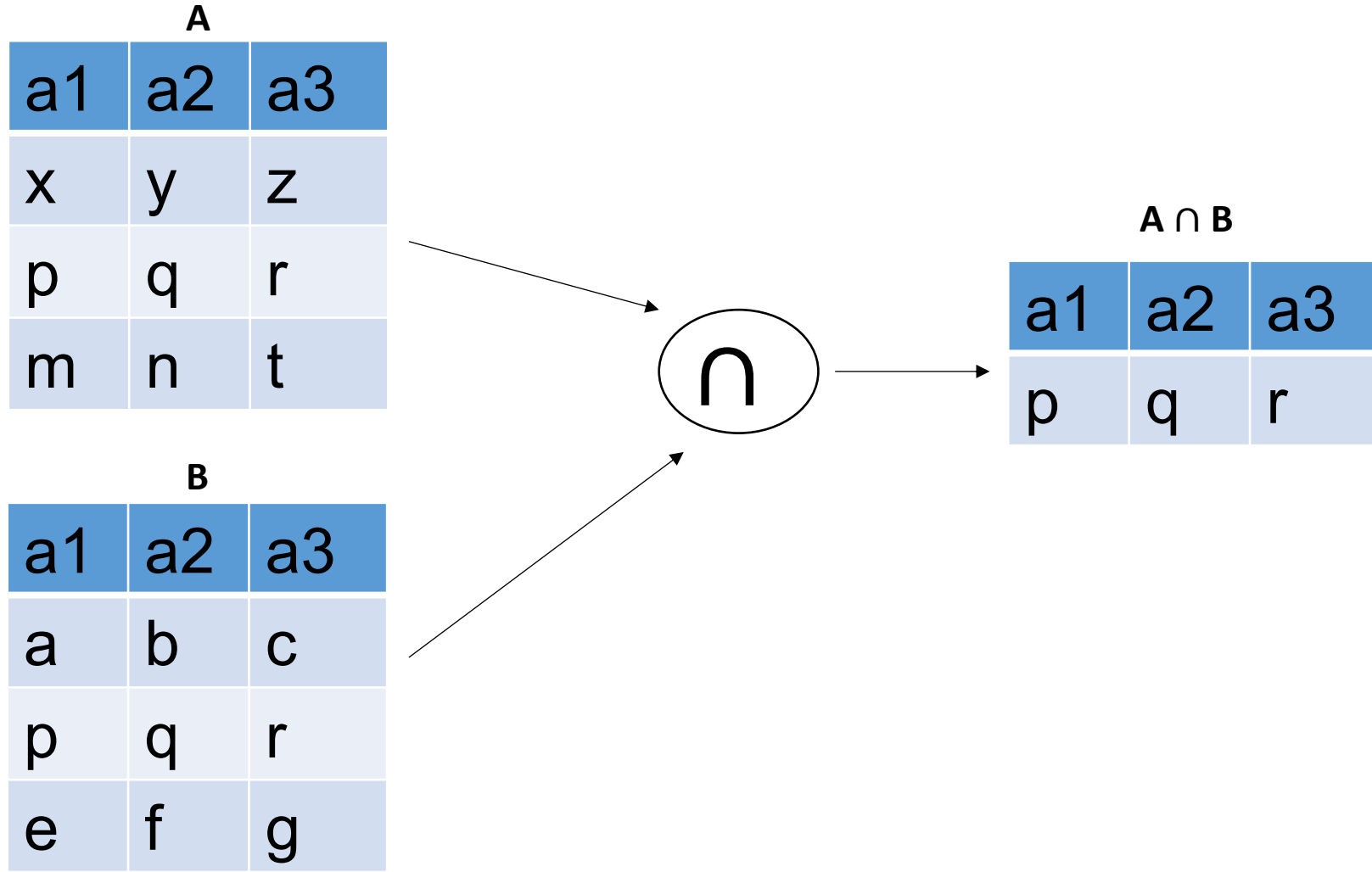
İlişkisel Cebir – Birleşim (Union), $\cup$

- Bu işlemin yapılabilmesi için, verilen iki tablonun şemalarının birebir aynı olması gerekir.
- İki tablodaki tüm satırlar birleştirilerek yeni bir tablo oluşturulur ve tekrar eden satırlar varsa elenir.



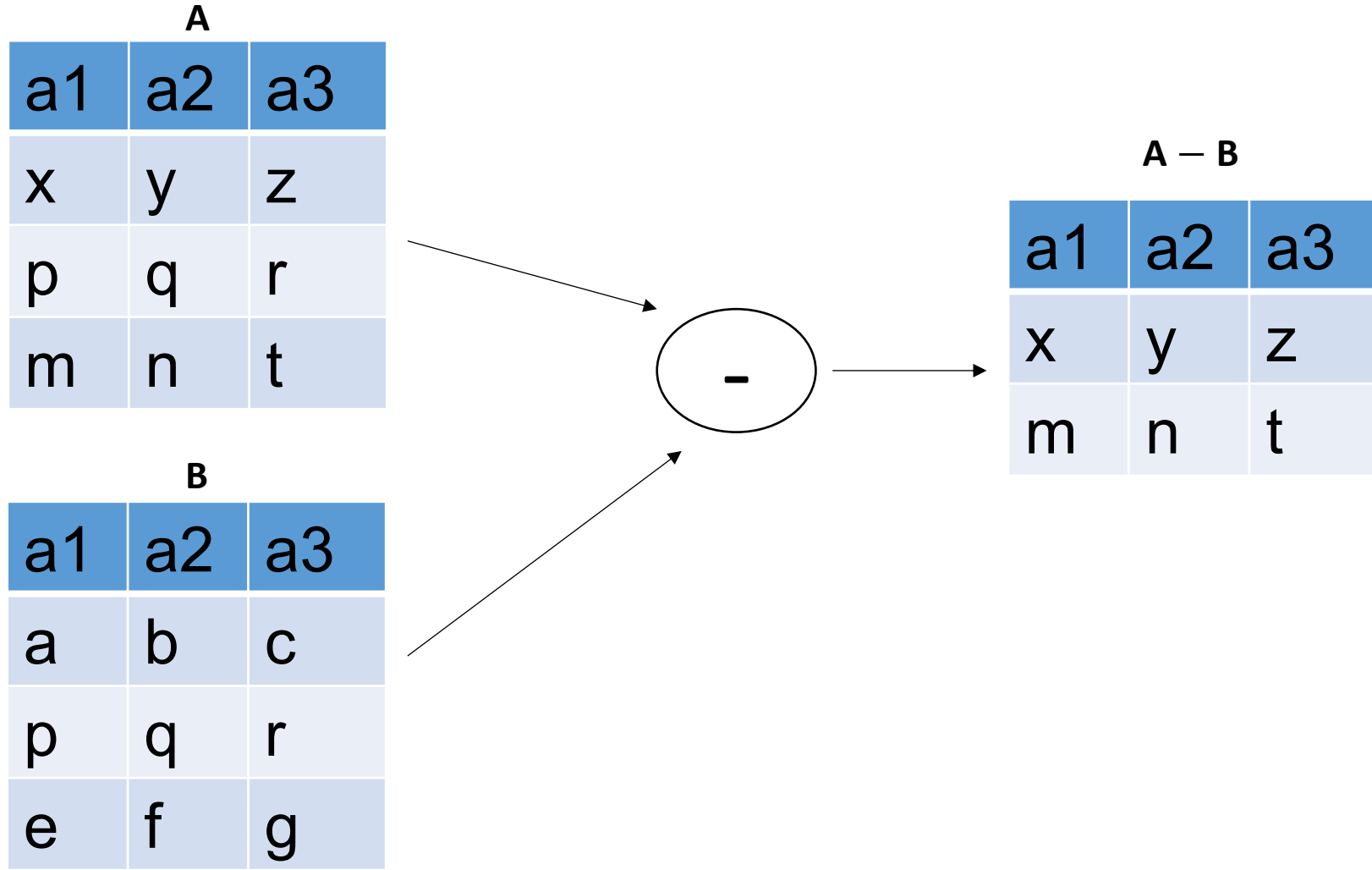
İlişkisel Cebir – Kesişim (Intersection), $\cap$

- Bu işlemin yapılabilmesi için, verilen iki tablonun şemalarının birebir aynı olması gerekir.
- İki tabloda da var olan satırlar kalır diğerleri elenir.



İlişkisel Cebir – Fark (Difference)

- Bu işlemin yapılabilmesi için, verilen iki tablonun şemalarının birebir aynı olması gerekir.
- A tablosunda olup da B tablosunda olmayan satırları ihtiva eder.



İlişkisel Cebir – Yeniden İsimlendirme (Rename), $\rho(\text{ro})$

R

A_1	A_2	A_3	...	A_n

$\rho_{\text{Tablom}}(R)$



Tablom

A_1	A_2	A_3	...	A_n

$\rho_{\text{Tablom}}(T_1, T_2, \dots, T_n)(R)$



Tablom

T_1	T_2	T_3	...	T_n

$\rho(T_1, T_2, \dots, T_n)(R)$



R

T_1	T_2	T_3	...	T_n

İlişkisel Cebir – Bölme (Division), $\div$ *yada* |

- Bölme (Division, $\div$) işlemi "her X için tümünü sağlayan" mantığını temsil eder
- Örneğin:
 - Tüm dersleri alan öğrenciler
 - Tüm ürünleri alan müşteriler
 - Bütün projelerde çalışan personeller
 - Her şehirde şubesi olan firmalar
 - X kategorisindeki tüm kitapları alan üyeler

gibi
- SQL'de doğrudan karşılığı yoktur
- Genellikle NOT EXISTS veya GROUP BY + HAVING ile gerçekleştirilir

Teaching

Lecturer	Module
Brown	Compilers
Brown	Databases
Green	Prolog
Green	Databases
Lewis	Prolog
Smith	Databases

Lecture

Module
Database

Teaching $\div$ Lecture
ifadesi, tüm dersleri
("Databases") dersini
veren "Lecturer" listesini
verir

Lecturer
Brown
Green
Smith

İlişkisel Cebir – Bölme (Division), $\div$ *yada* |

Teaching

Lecturer	Module
Brown	Compilers
Brown	Databases
Green	Prolog
Green	Databases
Lewis	Prolog
Smith	Databases

Lecture

Module
Prolog
Database

Teaching $\div$ Lecture $\rightarrow$ Prolog ve Databases derslerini veren
Lecturer listesi

Lecturer
Green

İlişkisel Cebir – Bölme (Division), $\div$ *yada* |

Teaching

Lecturer	Module
Brown	Compilers
Brown	Databases
Green	Prolog
Green	Databases
Lewis	Prolog
Smith	Databases

Lecture

Module
Prolog
Compilers

Teaching $\div$ Lecture $\rightarrow$ Prolog ve Compilers derslerini birlikte veren
Lecturer listesi

Lecturer

İlişkisel Cebir – Örnek 1

Öğrenci

OgrNo	Isim	DogTar
321	Ali Can	1991
342	Ayşe Acar	1989
357	Savaş Barış	1990

Ders

DersKodu	Ad	Kredi
COM213	Veri Tabanı	5
COM284	Nesnel Programlama	4
COM1010	Algoritma	3

Katılım

OgrNo	DersKodu	Not
321	COM213	84
342	COM1010	43
357	COM213	47
321	COM284	76
321	COM1010	79
342	COM213	91

Veri Tabanı dersinden 50 üzeri alan öğrencilerin isim listesini veren ilişkisel cebir ifadesini yazınız.



İlişkisel Cebir – Örnek 1

İlk önce üç tabloyu da içerecek **NATURAL JOIN** işlemini uygulayalım. Sonraki adımda kolaylık olması için oluşan yeni tabloya bir isim de verelim.

$$\rho_{\text{gecici_tablo}}(\text{Öğrenci} \bowtie \text{Ders} \bowtie \text{Katılım})$$

gecici_tablo

OgrNo	Isim	DogTar	DersKodu	Ad	Kredi	Not
321	Ali Can	1991	COM213	Veri Tabanı	5	84
342	Ayşe Acar	1989	COM1010	Algoritma	3	43
357	Savaş Barış	1990	COM213	Veri Tabanı	5	47
321	Ali Can	1991	COM284	Nesnel Programlama	4	76
321	Ali Can	1991	COM1010	Algoritma	3	79
342	Ayşe Acar	1989	COM213	Veri Tabanı	5	91



İlişkisel Cebir – Örnek 1

Şimdi oluşturulan yeni tablodan **SELECTION** ve **PROJECTION** yapılabilir

$$\pi_{isim}(\sigma_{Ad='Veri Tabanı' \wedge Not > 50}(gecici_tablo))$$

Isim
Ali Can
Ayşe Acar

İlişkisel Cebir – Örnek 2

Öğrenci

OgrNo	Isim	DogTar	GPA
321	Ali Can	1991	2.35
342	Ayşe Acar	1989	3.42
357	Savaş Barış	1990	2.99

Aşağıda verilen ifadenin sonucu nedir? Neyi göstermektedir?

$$\pi_{OgrNo}(\text{Öğrenci}) - \pi_{T1.OgrNo}(\rho_{T1}(\text{Öğrenci}) \bowtie_{T1.GPA < T2.GPA} \rho_{T2}(\text{Öğrenci}))$$

İlişkisel Cebir – Örnek 2

$$\rho_{T1}(\text{Öğrenci}) \bowtie_{T1.GPA < T2.GPA} \rho_{T2}(\text{Öğrenci})$$

T1

OgrNo	Isim	DogTar	GPA
321	Ali Can	1991	2.35
342	Ayşe Acar	1989	3.42
357	Savaş Barış	1990	2.99

T2

OgrNo	Isim	DogTar	GPA
321	Ali Can	1991	2.35
342	Ayşe Acar	1989	3.42
357	Savaş Barış	1990	2.99

OgrNo	Isim	DogTar	GPA	OgrNo	Isim	DogTar	GPA
321	Ali Can	1991	2.35	342	Ayşe Acar	1989	3.42
321	Ali Can	1991	2.35	357	Savaş Barış	1990	2.99
357	Savaş Barış	1990	2.99	342	Ayşe Acar	1989	3.42

İlişkisel Cebir – Örnek 2

$$\pi_{T1.OgrNo}(\rho_{T1}(\text{Öğrenci}) \bowtie_{T1.GPA < T2.GPA} \rho_{T2}(\text{Öğrenci}))$$

OgrNo
321
357

$$\pi_{OgrNo}(\text{Öğrenci}) - \pi_{T1.OgrNo}(\rho_{T1}(\text{Öğrenci}) \bowtie_{T1.GPA < T2.GPA} \rho_{T2}(\text{Öğrenci}))$$

OgrNo
321
342
357

—

OgrNo
321
357

=

OgrNo
342

GPA'ı en yüksek öğrenci
gösterilmiş oldu

İlişkisel Cebir – Örnek 3

Öğrenci

OgrNo	Isim	DogTar	GPA
321	Ali Can	1991	2.35
342	Ayşe Acar	1989	3.42
357	Savaş Barış	1990	2.99

Ders

DersKodu	Ad	Kredi
COM213	Veri Tabanı	5
COM284	Nesnel Programlama	4
COM1010	Algoritma	3

Katılım

OgrNo	DersKodu	Not
321	COM213	84
342	COM1010	43
357	COM213	47
321	COM284	76
321	COM1010	79
342	COM213	91

Algoritma dersinden en yüksek notu alan öğrencinin doğum tarihini belirleyen ilişkisel cebir ifadesini yazınız.



İlişkisel Cebir – Örnek 3

İlk önce üç tabloyu da içerecek **NATURAL JOIN** işlemini uygulayalım. Sonraki adımda kolaylık olması için oluşan yeni tabloya bir isim de verelim.

$$\rho_{\text{gecici_tablo}}(\text{Öğrenci} \bowtie \text{Ders} \bowtie \text{Katılım})$$

gecici_tablo

OgrNo	Isim	DogTar	GPA	DersKodu	Ad	Kredi	Not
321	Ali Can	1991	2.35	COM213	Veri Tabanı	5	84
342	Ayşe Acar	1989	3.42	COM1010	Algoritma	3	43
357	Savaş Barış	1990	2.99	COM213	Veri Tabanı	5	47
321	Ali Can	1991	2.35	COM284	Nesnel Programlama	4	76
321	Ali Can	1991	2.35	COM1010	Algoritma	3	79
342	Ayşe Acar	1989	3.42	COM213	Veri Tabanı	5	91



İlişkisel Cebir – Örnek 3

Şimdi oluşturulan yeni tabloyu da T1 ve T2 diye isimlendirerek tüm sorguyu oluşturalım

$$\pi_{DogTar}(\text{Öğrenci}) - \pi_{T1.DogTar}(\rho_{T1}(\text{gecici_tablo}) \bowtie_{T1.Not < T2.Not \wedge Ad = 'Algoritma'} \rho_{T2}(\text{gecici_tablo}))$$

Sonuç : 1991

Not: Bu örnek için aynı doğum tarihine sahip iki kişi olmadığından sorun olmayacaktır

5. Bölüm

Yapısal Sorgu Dili Structured Query Language

Yapısal Sorgu Dili (Structured Query Language, SQL)

- İlişkisel veri tabanlarında veri tabanı oluşturma, veri ekleme, güncelleme, silme ve sorgulama işlemlerini gerçekleştirmek için kullanılan standart bir dildir.
- 1970'li yıllarda IBM tarafından geliştirilen deneysel ilişkisel veri tabanı yönetim sistemi System R projesi kapsamında (ilk adıyla SEQUEL olarak) ortaya çıkmış ortaya çıkmış, daha sonra ANSI ve ISO tarafından standartlaştırılmıştır.
- ISO SQL standardına göre dört ana alt dilden oluşur:
 - **Veri İşleme Dili (Data Manipulation Language – DML)**
 - Tablolar üzerinde veri ekleme (**INSERT**), güncelleme (**UPDATE**), silme (**DELETE**) ve sorgulama (**SELECT**) işlemlerini gerçekleştirmek için kullanılan komutları kapsar.
 - **Veri Tanımlama Dili (Data Definition Language – DDL)**
 - Veri tabanı nesnelerini oluşturmak (**CREATE**), değiştirmek (**ALTER**) ve silmek (**DROP, TRUNCATE**) için kullanılan komutlardan oluşur.
 - **Veri Kontrol Dili (Data Control Language – DCL)**
 - Veri tabanı üzerindeki erişim yetkilerini düzenlemek amacıyla kullanılan **GRANT, DENY** ve **REVOKE** gibi komutları kapsar.

Yapısal Sorgu Dili (Structured Query Language, SQL)

- **İşlem Kontrol Dili (Transaction Control Language – TCL)**

- SQL'de gerçekleştirilen işlemlerin (transaction) kalıcı hâle getirilmesi veya geri alınmasını sağlayan **COMMIT**, **ROLLBACK** ve **SAVEPOINT** gibi komutları kapsar.
 - ❑ **Transaction**: Tek bir mantıksal bütün oluşturan, birlikte başarılı olması veya birlikte başarısız olması gereken işlemler grubudur.
- TCL, DML komutlarıyla yapılan değişikliklerin kalıcılığını ve geri alınmasını kontrol eder.
 - ❑ TCL, doğrudan veri değiştirmez

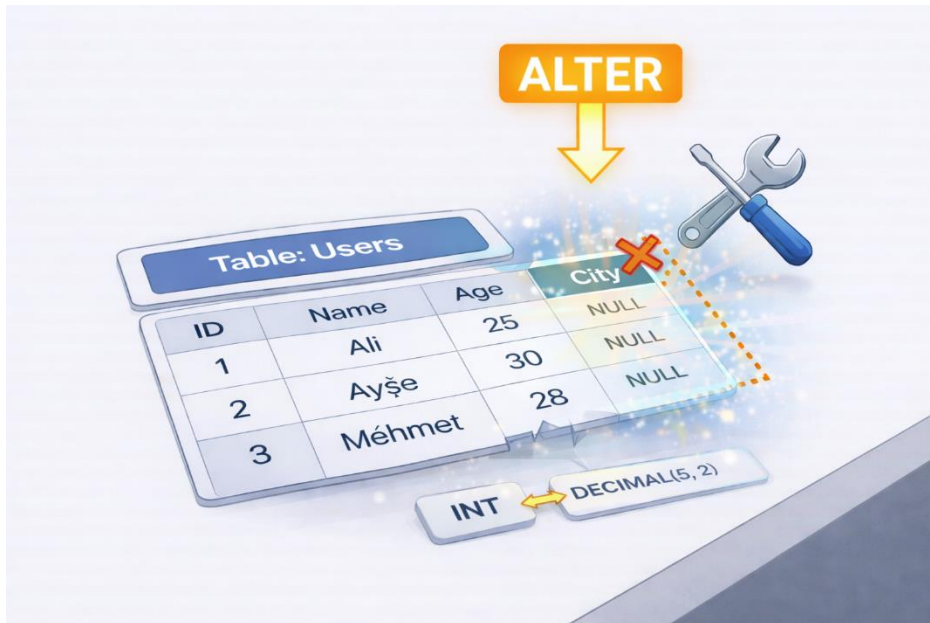
Microsoft SQL Server İsimlendirme ve Sözdizimi Kuralları

- Varsayılan ayarlarda büyük-küçük harf ayrımı yapılmaz.
- Nesne isimlerinin harf veya alt çizgi (_) ile başlaması önerilir.
- İsimlerde boşluk ve Türkçe karakterlerin kullanılması önerilmez. Ancak zorunlu durumlarda bu tür isimler köşeli parantez ([]) içinde yazılmalıdır.
- Metin içeren ifadeler tek tırnak işareti ile yazılmalıdır.
- Açıklama (yorum) satırları aşağıdaki şekilde belirtilir:
 - Tek satır için yan yana iki tire işareti (--)
 - Birden fazla satır için /* */

Veri Tabanı Yönetim Araçları (Database Management Tools)

- Microsoft SQL Server Management Studio (SSMS)
- Alternatifler
 - Microsoft Azure
 - Toad for SQL Server
 - erwin Data Modeler
 - Dbeaver
 - dbForge Studio for SQL Server
 - Navicat for SQL Server
 - SQLPro Studio
 - HeidiSQL
 - Beekeeper Studio
- Browser üzerinden çalışan sorgulama araçları
 - <https://onecompiler.com/sqlserver>
 - <https://www.programiz.com/sql/online-compiler>
 - <https://sqlfiddle.com/sql-server/online-compiler>
 - <https://www.codechef.com/sql-online-compiler>
 - <https://www.mycompiler.io/new/sql>

Veri Tanımlama Dili (Data Definition Language – DDL)



CREATE DATABASE

- **CREATE**, veri tabanı nesnelerini oluşturmak için kullanılan bir DDL komutudur.
- Veri tabanı oluşturma işlemi ise **CREATE DATABASE** komutu ile gerçekleştirilir.

CREATE DATABASE *a*

[ON PRIMARY (NAME = *b*, FILENAME = *c*, SIZE = *d*, MAXSIZE = *e*, FILEGROWTH = *f*)]

[LOG ON (NAME = *g*, FILENAME = *h*, SIZE = *i*, MAXSIZE = *j*, FILEGROWTH = *k*)]

Burada;

a : Veri tabanının ismi. En az bildirilmesi gereken parametredir. Örneğin veritabani1

b : Veri tabanı dosyasının SQL içerisindeki ismi. Örneğin 'vt1_data'

c : Veri tabanı dosyasının diskteki ismi. İstenirse dosya yolu da belirtilebilir. Örneğin 'C:\vt\vt1_data.mdf'

d : Veri tabanının başlangıç boyutu. Örneğin 100MB

e : Veri tabanının ulaşabileceği en üst boyut. Eğer verilmezse disk dolana kadar büyüme devam eder.

Örneğin 1GB

f : Veri tabanının her dolduğunda ne kadar büyütüleceğini belirten parametre. Örneğin 30%

g : Log veri tabanı ismi.

h : Log veri tabanı dosya ismi.

i : Log veri tabanı başlangıç boyutu.

j : Log veri tabanının ulaşabileceği en üst boyut.

k : Log veri tabanı büyüme parametresi

CREATE TABLE

- **CREATE TABLE** komutu; veri tabanında yeni bir tablo oluşturmak, bu tablonun sütunlarını, veri tiplerini ve üzerindeki kısıtlamaları (constraints) tanımlamak için kullanılan bir DDL komutudur.

CREATE TABLE *tablo_adi* (*alan1 tip1*, *alan2 tip2*, ... , *alan_n tip_n*)

- Burada;
 - *tablo_adi* belirtilirken öncesinde veri tabanı ve şema adı da belirtilebilir. Belirtilmezse o anda çalışılan veri tabanı altında ve bağlı kullanıcının şeması ile oluşturulacaktır.
 - Veri tipi belirtildikten sonra istenirse alana ait şu özellik ve kısıtlamalarda belirtilebilir
 - [**CONSTRAINT** *kisit_ismi*]
 - ❑ **NULL / NOT NULL**
 - ❖ Sütunun NULL değer içerip içermemesini belirler. Varsayılanı NULL'dur.
 - ❑ **DEFAULT** *sabit_ifade*
 - ❖ Kayıt esnasında değer belirtilmediğinde (sütun isminin yazılmaması durumu) veya varsayılan değer atanması istendiğinde, veri tabanı tarafından ilgili sütuna otomatik olarak atanacak değer veya fonksiyon sonucu burada tanımlanır.
 - ❑ **PRIMARY KEY**
 - ❖ Sütunu **PRIMARY KEY** yapar.
 - ❖ Her tabloda yalnız bir tane PRIMARY KEY olur ancak tek bir sütundan oluşması gerekmez
 - ❖ Birden çok sütundan oluşan PK yapmak için sütun tanımında değil ayrı olarak **CONSTRAINT** *kisit_ismi* **PRIMARY KEY** (*sutun1,sutun2,...*) şeklinde yazılmalıdır.

CREATE TABLE

- ❑ **FOREIGN KEY REFERENCES** *referans_edilen_tablo_adi (referans_edilen_sutun_adi)*
 - [**ON DELETE** { **NO ACTION** | **CASCADE** | **SET NULL** | **SET DEFAULT** }]
 - [**ON UPDATE** { **NO ACTION** | **CASCADE** | **SET NULL** | **SET DEFAULT** }]
 - ❖ Sütunu **FOREIGN KEY** yapar.
 - ❖ Varsayılan değer **NO ACTION**'dır (referential integrity (başvuru bütünlüğü)).
 - ❖ **CASCADE** yapılırsa referans verilen tabloda (parent) kayıt silinirse bu tablodaki (child) kaydı da sil demektir (weak entity).
 - ❖ **SET NULL** ise öyle bir durumda değerleri NULL ile değiştirir.
 - ❖ **SET DEFAULT** ise verilen varsayılan bir değer ile değiştirir
- ❑ **UNIQUE**
 - ❖ Sütundaki değer benzersiz olmasını gerektirir
- ❑ **CHECK (*mantıksal_ifade*)**
 - ❖ Sütundaki verinin verilen koşula uygun olmasını gerektirir

CREATE TABLE

- **IDENTITY (seed, increment)**

- **seed** değerinden başlayıp **increment** değeri kadar her defasında artan otomatik numaralandırılan bir sütun oluşturur.
- Bir tabloda bir tane tanımlanabilir.
- Tam sayı veri tipinde olmalıdır.
- Bu tarz sütunlarda insert işlemi sonrası satırın aldığı identity değerini **@@IDENTITY** sistem değişkeni verir.
 - ❑ **@@IDENTITY** değişkeni tablo ve scope'a bakmaksızın, connection'da üretilen son identity'yi verir
 - ❑ Eğer insert yaptığınız tabloda Trigger varsa, yanlış identity bilgisi alabilirsiniz
 - ❑ Açılmış olan bağlantıda ve sorgunun çalıştığı scope'ta son üretilen identity'ye **SCOPE_IDENTITY()** fonksiyonu ile erişilebilir. Trigger kullanılan tablolar için tavsiye edilen budur.
 - ❑ Bağlantı ve scope bakmaksızın, parametre olarak verilen tabloda üretilen son identity değeri ise **IDENT_CURRENT(tablo_adi)** ile elde edilebilir
- Elle IDENTITY sütuna giriş yapmak için => **SET IDENTITY_INSERT tablo_adi ON**
- Tekrar normal insert yapabilmek için OFF yapılmalıdır
- Sıfırlamak için => **DBCC CHECKIDENT ('tablo ismi', 'RESEED', başlaması_istenen_değer-1)**
 - ❑ Örneğin; DBCC CHECKIDENT ('deneme', 'RESEED', 0) -> deneme tablosunda identity olan alan için ilk insert işleminde 1 verilecektir ama o alan PK ise hata alınabilir
- **DBCC CHECKIDENT('tablo ismi', NORESEED)** ile tablodaki aktif seed değeri kontrol edilebilir

CREATE TABLE

- Hesaplamalı sütunlar *sutun_adi AS hesaplama_formulu* **[PERSISTED]** şeklinde yazılır.
 - Normalde satıra her erişimde formül yeniden hesaplanır. Yani gerçekte sütun yoktur sanal olarak vardır.
 - **PERSISTED** özellikte olan sütunların hesaplama sonucu, diğer sütunlar gibi diskte fiziksel olarak saklanır. Dolayısıyla tablonun boyutu artar

VERİ TİPLERİ

- **Tamsayı**

- "Kesin Sayısal" (Exact Numeric) kategorisinin temelini oluşturur.
- Bu veri tipleri, floating-point (kayan noktalı) aritmetiğinin aksine, ikilik (binary) sistemde tam karşılığı olan değerleri saklayarak, finansal ve bilimsel modellerde kritik öneme sahip olan "sıfır hata payı" ilkesiyle çalışır.
- Bir tablodaki birden fazla BIT kolonunu aynı byte bloğu içerisinde toplanır
 - Eğer bir tabloda 1 ile 8 arası BIT kolonu varsa, bunlar toplamda sadece 1 byte içinde saklanır.
 - 9 ile 16 arası BIT kolonu varsa 2 byte kaplar.
 - Yani SQL Server, bitleri yan yana paketleyerek alandan tasarruf sağlar.

Veri Tipi	Depolama (byte)	Alt Sınır	Üst Sınır
BIGINT	8	-9.22×10^{18}	9.22×10^{18}
INT	4	-2.14×10^9	2.14×10^9
SMALLINT	2	-32768	32767
TINYINT	1	0	255
BIT	1	0	1

VERİ TIPLERİ

- **Kayan Nokta (Floating Point, Approximate Numeric)**

- Bu veri tiplerinde hassasiyet, onluk sistemdeki sabit basamak pozisyonlarından ziyade, IEEE 754 standardı çerçevesinde ikilik tabanda ayrılan mantissa bit derinliği ile belirlenir
 - Bu durum, skaler büyüklükten bağımsız olarak sabit bir bağıl hata (relative error) oranı sunar
 - Bu nedenle de eşitlik kontrolü karşılaştırmalarında bu veri tiplerinin kullanılması tavsiye edilmez
- Bu veri tiplerinin skaler aralıkları (range) oldukça geniş bir spektrumu kapsasa da, anlamlı basamak sayıları (hassasiyet, precision) sınırlıdır.
 - Genellikle bilimsel ölçümler ve mühendislik hesaplamalarında tercih edilirler
- FLOAT veri tipin için bilimsel gösterimde mantis kısmında kullanılacak bit miktarını temsil eden (n) parametresi isteğe bağlı olarak verilebilir
 - Single precision (n=24, REAL veri tipi olur)
 - Double precision (n=53, varsayılan)

Veri Tipi	Depolama (byte)	Alt Sınır	Üst Sınır	Hassasiyet (p)
FLOAT [(n)]	4 / 8 (n=24/53)	-3.40×10^{38} (n=24) $-1.79 \dots \times 10^{308}$ (n=53)	3.40×10^{38} (n=24) $1.79 \dots \times 10^{308}$ (n=53)	7 (n=24) 15-17 (n=53)
REAL	4	-3.40×10^{38}	3.40×10^{38}	7

VERİ TİPLERİ

- **Sabit Nokta (Fixed Precision & Scale)**

- Kayan noktalı (floating-point) veri tiplerinin aksine, bu tiplerde hassasiyet derecesi (p) deklarasyon aşamasında belirlenir.
- Bu değer, şema tanımı (schema definition) süresince statik bir kısıt olarak kalır ve çalışma zamanında (runtime) dinamik bir değişim göstermez.
- Özellikle finansal hesaplamalar ve kesinlik gerektiren bilimsel veriler için kullanılırlar
- Sabit nokta veri tipleri tanımlanırken iki parametre kullanılır
 - **Precision (Hassasiyet - p)**
 - ❑ Sayının toplam basamak sayısını gösterir (ondalık noktanın hem solunun hem de sağının toplamı)
 - **Scale (Ölçek - s)**
 - ❑ Ondalık noktanın sağında saklanabilecek maksimum basamak sayısını gösterir
 - ❑ Bu sayı p değerinden çıkarılarak ondalık noktanın solundaki maksimum basamak sayısı belirlenmiş olur.
- Eğer tanımlanan Scale değerinden daha fazla ondalık basamağa sahip bir sayı girerseniz, SQL Server sayıyı yuvarlar.
- Ancak Precision sınırını aşan bir tam sayı girerseniz, "Arithmetic overflow" hatası alırsınız.
 - Örneğin DECIMAL(5,2) olan bir alana 1000.00 değerini yazmak istediğinizde (tam kısım 4 basamak)

VERİ TİPLERİ

- **Sabit Nokta (Fixed Precision & Scale)**

Veri Tipi	Hassasiyet (p)	Ölçek (s)	Depolama (byte)	Alt Sınır	Üst Sınır
DECIMAL/NUMERIC	$1 \leq p \leq 38$	$0 \leq s \leq p$	5(p=1-9), 9(p=10-19), 13(p=20-28), 17(p=29-38)	$-10^{(p-s)} + 10^{-s}$	$10^{(p-s)} - 10^{-s}$
MONEY	18-19	4	8	$-9.223 \dots \times 10^{14}$	$9.223 \dots \times 10^{14}$
SMALLMONEY	9-10	4	4	$-2.147 \dots \times 10^5$	$2.147 \dots \times 10^5$

- **DECIMAL(p, s)**

- Toplam basamak sayısı (precision) ve virgülden sonraki basamak sayısı (scale) isteğe bağlı olarak belirtilebilir
 - ❑ Varsayılan (p, s): 18, 0
- Hassas hesaplamalar için kullanılabilir
 - ❑ Kayan noktalı veri tiplerinden farklı olarak tam doğruluk (exact precision) sağlar
 - ❑ Bu sebepten "Kesin Sayısal" (Exact Numerics) kategorisinde yer alır
- Standart bir SQL veri tipi olduğu için veri tabanınızı SQL Server'dan PostgreSQL, Oracle veya MySQL gibi farklı bir platforma taşımanız gerekirse, DECIMAL sorunsuz çalışır.

VERİ TİPLERİ

- **Sabit Nokta (Fixed Precision & Scale)**

- **MONEY & SMALLMONEY**

- Sabit olarak virgülden sonra 4 basamak (fixed scale) saklarlar
 - ❑ Yuvarlama hatalarına meyillidirler
 - Eğer ara işlemlerde 5. veya 6. basamağa ihtiyaç varsa veri kaybı yaşanabilir
 - SQL Server'a özgüdür ve diğer sistemlerde karşılığı her zaman aynı olmayabilir
 - DECIMAL tipinin sunduğu mutlak kesinliği, bigint tipinin sunduğu işlem hızı ile birleştiren hibrid bir mimaridir
 - ❑ Bu sebepten "Tam Sayı Tabanlı Kesinlik" (Integer-Based Precision) kategorisinde yer alırlar

- SQL Server'da finansal verilerle çalışırken MONEY yerine DECIMAL (veya NUMERIC) kullanmak, sektördeki en iyi uygulama (best practice) olarak kabul edilir

VERİ TİPLERİ

- **Karakter ve Metin**

- **CHAR [(n)]**

- Sabit uzunluk
 - Metnin byte uzunluğu n ile belirtilebilir (karakter başına 1 bayt).
 - n değeri 1 ... 8000 arasında olabilir

- **VARCHAR [(n | max)]**

- Değişken uzunluk
 - Metin byte uzunluğu n ile belirtilebilir (karakter başına 1 bayt)
 - n değeri 1 ... 8000 arasında olabilir
 - Eğer max kullanılırsa 2GB veri saklanabilir (satır dışında saklanır, satırda 16-byte'lık bir işaretçi vardır)
 - Veri tabanının varsayılan kod sayfasını (Unicode olmayan) kullanır

VERİ TİPLERİ

- **Karakter ve Metin**

- **NCHAR [(n)] & NVARCHAR [(n | max)]**

- Unicode UTF-16 kodlamasını kullanır (karakter başına 2 bayt)
 - Çince, Arapça, vb. diller ile emoji gibi özel karakterleri saklamak için kullanılır.
 - n değeri 1 ... 4000 arasında olabilir
 - Eğer max kullanılırsa 2GB veri saklanabilir (~1M karakter)

- **TEXT**

- VARCHAR(MAX) kullanmak gibidir ancak en önemli farkı kaydedilen veri 8KB'dan küçük bile olsa veriler satır dışında saklanır.
 - Standart metin fonksiyonlarında (ör. LEN, LEFT, REPLACE, vb.) kullanılamaz.
 - ❑ Özel fonksiyonları vardır (ör. READTEXT, WRITETEXT, UPDATETEXT, vb.)
 - TEXT veri tipi deprecated (kullanımdan kaldırılmaya başlanmış) durumdadır; yerine VARCHAR(MAX) kullanılmalıdır.

- Sütun uzunlukları belirlenirken gereksiz alan verilmemelidir

- Örneğin RFC 2821'e göre bir e-mail adresi en çok 320 karakterdir.
 - VARCHAR tipinin max değeri 8KB olabilir ama bu değer mail adresi saklamak için çok fazla bir alan olacaktır.

VERİ TİPLERİ

- **Zamansal (Temporal)**

- Tarih veri tiplerinin formatı YYYY-MM-DD şeklindedir
- Saat veri tiplerinin formatı HH:MM:SS.nnnnnnnn
- Tarih ve saat tipindeki alanlara veri girişi yapılırken tek tırnak işaretleri arasında veriler girilir
- DATETIME2(n) ve TIME(n) tiplerindeki n parametresi (0-7), virgülden sonra kaç basamak saklanacağını belirler:
 - n = 0 - 2 ise: 6 Byte
 - n = 3 - 4 ise: 7 Byte
 - n = 5 - 7 ise: 8 Byte

VERİ TİPLERİ

- Zamansal (Temporal)

Veri Tipi	Depolama (Byte)	Hassasiyet (Precision)	Aralık (Range)	Önerilen Kullanım Alanı
DATE	3	1 Gün	0001-01-01 İle 9999-12-31	doğum tarihleri, tatiller
TIME(n)	3 - 5	100 Nanosaniye	00:00:00.0000000 İle 23:59:59.9999999	Mesai saatleri, Ders saatleri, kronometre verileri
DATETIME	8	3.33 Milisaniye (1/300 saniye)	1753-01-01 İle 9999-12-31	Eski sistemler (Legacy)

VERİ TİPLERİ

- Zamansal (Temporal)

Veri Tipi	Depolama (Byte)	Hassasiyet (Precision)	Aralık (Range)	Önerilen Kullanım Alanı
DATETIME2	6 - 8	100 Nanosaniye	0001-01-01 ile 9999-12-31	Modern Standart (Önerilen)
SMALLDATETIME	4	1 Dakika	1900-01-01 ile 2079-06-06	Randevu sistemleri (Basit)
DATETIMEOFFSET	8 - 10	100 Nanosaniye	Datetime2 + Zaman Dilimi	Global uygulamalar, log kayıtları

CREATE Örnek - Kütüphane DB

```
|IF DB_ID('kutuphane') IS NOT NULL
|  BEGIN
|      ALTER DATABASE [kutuphane] SET SINGLE_USER WITH ROLLBACK IMMEDIATE
|      USE master
|      DROP DATABASE kutuphane
|  END
|GO

|CREATE DATABASE kutuphane
|  ON PRIMARY (
|      NAME = 'kutuphancedb',
|      FILENAME = 'c:\database\kutuphane_db.mdf',
|      SIZE = 5MB,
|      MAXSIZE = 100MB,
|      FILEGROWTH = 5MB
|  )
|  LOG ON (
|      NAME = 'kutuphancedb_log',
|      FILENAME = 'c:\database\kutuphane_log.ldf',
|      SIZE = 2MB,
|      MAXSIZE = 50MB,
|      FILEGROWTH = 1MB
|  )
|GO

|USE kutuphane
```


USE kutuphane

-- Bu tabloda il isimleri tutulur.

```
CREATE TABLE tblIl
(
    Kod CHAR(2) PRIMARY KEY,
    Ad VARCHAR(20) NOT NULL
)
GO
```

-- Bu tabloda her bir ile ait ilçeler tutulur.

```
CREATE TABLE tblIlce
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Ad VARCHAR(20) NOT NULL,
    IlKodu CHAR(2) FOREIGN KEY REFERENCES tblIl(Kod)
        ON DELETE CASCADE ON UPDATE CASCADE NOT NULL
)
GO
```

-- Bu tabloda yazarlara ait bilgiler tutulur.

```
CREATE TABLE tblYazar
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Ad VARCHAR(80) UNIQUE NOT NULL,
    DTarihi DATE,
    IlKodu CHAR(2) FOREIGN KEY REFERENCES tblIl(Kod),
    IlceKodu INT FOREIGN KEY REFERENCES tblIlce(ID)
)
GO
```

-- Bu tabloda kitapları basan yayın evlerine ait bilgiler tutulur.

```
CREATE TABLE tblYayinEvi
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Ad VARCHAR(50) UNIQUE NOT NULL,
    Adres VARCHAR(100),
    IlKodu CHAR(2) FOREIGN KEY REFERENCES tblIl(Kod) NOT NULL,
    Tel CHAR(10),
    WebSite VARCHAR(100)
)
GO
```

-- Bu tabloda kitapların üst türleri tutulur. Örneğin Bilgisayar, Roman gibi

```
CREATE TABLE tblKitapTuru
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Ad VARCHAR(50) UNIQUE NOT NULL
)
```

-- Bu tabloda kitapların alt türleri tutulur. Örneğin Bilgisayar üst türündeki bir kitap için alt tür C programlama olabilir

```
CREATE TABLE tblKitapAltTuru
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Ad VARCHAR(50) UNIQUE NOT NULL,
    UstTurKodu INT FOREIGN KEY REFERENCES tblKitapTuru(ID) NOT NULL
)
GO
```

-- Bu tabloda dil listesi tutulur. Örneğin Türkçe, İngilizce gibi

```
CREATE TABLE tblDil
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Ad VARCHAR(50) UNIQUE NOT NULL
)
GO
```

-- Bu tabloda kitapların cilt tipleri tutulur. Örneğin Ciltli, Karton kapaklı gibi

```
CREATE TABLE tblCiltTipi
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Aciklama VARCHAR(50) UNIQUE NOT NULL
)
GO
```

-- Bu tabloda kitapların kağıt cinsleri tutulur. Örneğin 1.Hamur, 2.Hamur gibi

```
CREATE TABLE tblKagitCinsi
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Aciklama VARCHAR(50) UNIQUE NOT NULL
)
GO
```

-- Bu tabloda kitaplar hakkında bilgiler tutulur.

CREATE TABLE tblKitap

```
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    ISBN CHAR(13) UNIQUE NOT NULL,
    Ad VARCHAR(100) NOT NULL,
    Aciklama VARCHAR(100), |
    OduncVerilebilirMi SMALLINT DEFAULT 1 NOT NULL, -- 1 : Verilebilir, 0 : Verilemez
    SayfaSayisi INT NOT NULL,
    En SMALLINT NOT NULL,
    Boy SMALLINT NOT NULL,
    Yukseklik SMALLINT NOT NULL, -- Kitabın kalınlığı
    ToplamStok INT NOT NULL,
    OduncVerilmisMiktar INT DEFAULT 0 NOT NULL,
    YayinEviID INT FOREIGN KEY REFERENCES tblYayinEvi(ID) NOT NULL,
    BasimYeriKodu CHAR(2) FOREIGN KEY REFERENCES tblIl(Kod) NOT NULL,
    UstTurID INT FOREIGN KEY REFERENCES tblKitapTuru(ID) NOT NULL,
    AltTurID INT FOREIGN KEY REFERENCES tblKitapAltTuru(ID) NOT NULL,
    DilID INT FOREIGN KEY REFERENCES tblDil(ID) NOT NULL,
    CiltTipiID INT FOREIGN KEY REFERENCES tblCiltTipi(ID) NOT NULL,
    KagitCinsiID INT FOREIGN KEY REFERENCES tblKagitCinsi(ID) NOT NULL
)
```

)

GO

CREATE TABLE KitapYazarlari

```
(
    KitapID INT FOREIGN KEY REFERENCES tblKitap(ID) NOT NULL,
    YazarID INT FOREIGN KEY REFERENCES tblYazar(ID) NOT NULL,

    CONSTRAINT pktblKitapYazarlari PRIMARY KEY (KitapID, YazarID)
)
```

)

GO

```

CREATE TABLE tblUye
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    TCNO CHAR(11) UNIQUE NOT NULL,
    Ad VARCHAR(50) NOT NULL,
    Soyad VARCHAR(50) NOT NULL,
    Adres VARCHAR(100),
    IlKodu CHAR(2) FOREIGN KEY REFERENCES tblIl(Kod) NOT NULL,
    IlceKodu INT FOREIGN KEY REFERENCES tblIlce(ID) NOT NULL,
    Tel CHAR(10) CONSTRAINT chkTel CHECK (Tel LIKE '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'), -- Telefon numarası tamamen sayılardan oluşmalıdır
    Mail VARCHAR(50) -- Örnek olması amacıyla kısıtlar CONSTRAINT ifadesi ile yazılacaktır
        CONSTRAINT unqtblUyeMail UNIQUE -- E-Mail adresi benzersiz olmalıdır
        CONSTRAINT chktblUyeMail CHECK (Mail LIKE '%@%.%') -- E-Mail adresi içinde ilk önce @ ve sonrada . işareti gelmelidir
        CONSTRAINT notnulltblUyeMail NOT NULL, -- E-Mail adresi boş geçilemez
    DTarihi DATE NOT NULL,
    Yas AS DATEDIFF(yy, DTarihi, GETDATE()),
    UyeOlusTarihi DATE DEFAULT GETDATE() NOT NULL,
    AktifMi SMALLINT DEFAULT 1 NOT NULL -- Üyeliği kapatılırsa bu değer 0 yapılır
)
GO

CREATE TABLE tblOkumaListesi
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Aciklama VARCHAR(50),
    AktifMi SMALLINT DEFAULT 1 NOT NULL, -- Üye, okuma listesini artık kapatmak istiyorsa bu değer 0 yapılır
    UyeID INT FOREIGN KEY REFERENCES tblUye(ID) NOT NULL
)
GO

CREATE TABLE tblOkumaListesiKitaplari
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    OkumaListesiID INT FOREIGN KEY REFERENCES tblOkumaListesi(ID) NOT NULL,
    KitapID INT FOREIGN KEY REFERENCES tblKitap(ID) NOT NULL,
    OkumaDurumu SMALLINT NOT NULL, -- 0 : Okunuyor, 1 : Okunmuş, 2 : Okumak isteniyor
    SonOkunanSayfa INT
)
GO

```

```
CREATE TABLE tblYorumlar
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    UyeID INT FOREIGN KEY REFERENCES tblUye(ID) NOT NULL,
    KitapID INT FOREIGN KEY REFERENCES tblKitap(ID) NOT NULL,
    Yorum VARCHAR(1000),
    Tarih DATETIME DEFAULT GETDATE() NOT NULL
)
GO
```

```
CREATE TABLE tblOduncKitap
(
    ID INT IDENTITY(1,1) PRIMARY KEY,
    UyeID INT FOREIGN KEY REFERENCES tblUye(ID) NOT NULL,
    KitapID INT FOREIGN KEY REFERENCES tblKitap(ID) NOT NULL,
    AlinisTarihi DATE NOT NULL,
    GetirilecegiTarih DATE NOT NULL ,
    GetirildigiTarih DATE DEFAULT NULL,
    Durum SMALLINT -- 0 : Sorunsuz, 1 : Hasarlı, 2 : Kayıp
)
GO
```

-- Alter kullanıma örnek olması için yapılmıştır

```
ALTER TABLE tblOduncKitap ADD CONSTRAINT defOduncKitapAlinisTarihi DEFAULT GETDATE() FOR AlinisTarihi
```

ALTER

- Veri tabanında var olan nesnelerin yapısal özelliklerini değiştirmek için kullanılan bir DDL komutudur.
- Büyük tablolarda ALTER işlemi maliyetlidir ve şema değişiklik kilidi (schema modification lock, Sch-M) olarak eşzamanlı erişimi kısıtlayabilir.
- Canlı sistemlerde yani üretim ortamlarında (production) bu işlemler dikkatli yapılmalıdır.

- **Veri tabanı yapısı değiştirme (ALTER DATABASE)**

```
CREATE DATABASE ornek_db
ON PRIMARY (
    NAME = 'ornek1_db',
    FILENAME = 'c:\database\ornek1_db.mdf',
    SIZE = 5MB,
    MAXSIZE = 100MB,
    FILEGROWTH = 5MB
)
LOG ON (
    NAME = 'ornek1_log',
    FILENAME = 'c:\database\ornek1_log.ldf',
    SIZE = 2MB,
    MAXSIZE = 50MB,
    FILEGROWTH = 1MB
)

GO

ALTER DATABASE ornek_db MODIFY NAME=ornek1 -- DB ismini değiştir
GO

ALTER DATABASE ornek1 MODIFY FILE (NAME=ornek1_db, SIZE=200MB) -- DB boyutunu arttır
GO

USE ornek1
GO
```

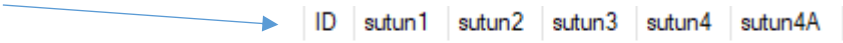
ALTER

• Tablo yapısı değiştirme (ALTER TABLE)

○ Yeni sütun eklemek (ADD)

```
ALTER TABLE tablo1 ADD sutun3 INT NOT NULL -- Yeni bir sütun ekle
GO
```

```
ALTER TABLE tablo1 ADD sutun4 FLOAT NOT NULL, sutun4A INT -- Yeni iki sütun ekle
SELECT * FROM tablo1
```



ID	sutun1	sutun2	sutun3	sutun4	sutun4A
----	--------	--------	--------	--------	---------

○ Sütun tipini değiştirmek (ALTER COLUMN)

```
-- Sütun tipini değiştir. Tip uyummasına dikkat edilmelidir
ALTER TABLE tablo1 ALTER COLUMN sutun2 MONEY NOT NULL
GO
```

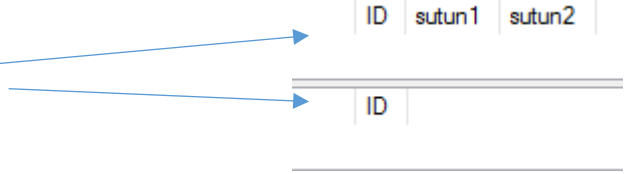
➤ Veri tipi değişikliğinde dikkat edilmesi gerekenler

- ❑ **Veri kaybı riski (data truncation):** Bir sütunun uzunluğu azaltılırken (ör. VARCHAR(100) → VARCHAR(50)), mevcut veriler yeni sınıra sığmıyorsa SQL Server hata vererek işlemi sonlandırır (The statement has been terminated).
- ❑ **Tip dönüşüm hataları (conversion errors):** VARCHAR'ı INT'e dönüştürürken veri tipi uyumsuzsa işlem başarısız olur.

```
CREATE TABLE tablo1
(
    ID INT IDENTITY(1,1),
    sutun1 VARCHAR(20),
    sutun2 INT NOT NULL
)
GO
```

```
CREATE TABLE tablo2
(
    ID INT IDENTITY(1,1) PRIMARY KEY
)
GO
```

```
SELECT * FROM tablo1
SELECT * FROM tablo2
```



ID	sutun1	sutun2
----	--------	--------

ID

ALTER

- Tablo yapısı değiştirme (ALTER TABLE)

- Sütun silmek (DROP COLUMN)

-- Sütunu sil

```
ALTER TABLE tablo1 DROP COLUMN sutun4, sutun4a
```

```
SELECT * FROM tablo1
```

```
GO
```

Results		Messages		
	ID	sutun1	sutun2	sutun3

ALTER

- **ALTER TABLE ile bütünlük kısıtlarının (integrity constraints) yönetimi**

- Birincil anahtar eklemek (PRIMARY KEY)

-- Her iki sütunda NOT NULL olmalıdır.

-- Eğer veri varsa PK çakışmasına neden olmamalıdır

```
ALTER TABLE tablo1 ADD CONSTRAINT pktablo1 PRIMARY KEY (sutun2, sutun3)
GO
```

```
INSERT INTO tablo1 VALUES('test1', 10, 100)
GO
```

```
INSERT INTO tablo1 VALUES('test2', 10, 100) -- PK hatası oluşacaktır
GO
```

```
SELECT * FROM tablo1
GO
```

	ID	sutun1	sutun2	sutun3
1	1	test1	10.00	100

ALTER

- **ALTER TABLE ile bütünlük kısıtlarının (integrity constraints) yönetimi**

- Koşul kısıtı tanımlamak (CHECK)

- Bir CHECK constraint eklerken WITH NOCHECK ifadesi kullanılırsa, SQL Server mevcut veriyi kontrol etmez, sadece yeni gelen veriyi kontrol eder.

```
-- sutun2'ye yazılacak değerler 10'dan büyük olmalıdır diye bir kısıt oluştur.  
-- Üstte sütun2 için 10 değeri girildiğinden burada WITH NOCHECK kullanılmazsa kısıt oluşturulamaz
```

```
ALTER TABLE tablo1 WITH NOCHECK ADD CONSTRAINT kontrol1 CHECK (sutun2 > 10)  
GO
```

```
-- sutun3'e yazılacak değerler 0'dan büyük olmalıdır diye bir kısıt oluştur  
-- Kayıtlı veride alttaki kısıtı delen bir değer olmadığından WITH NOCHECK gerekmiyor
```

```
ALTER TABLE tablo1 ADD CONSTRAINT kontrol2 CHECK (sutun3 > 0)  
GO
```

```
INSERT INTO tablo1 VALUES('test1', 1, -1) -- Üstte oluşturulan her iki kısıtı da deliyor  
GO
```

```
INSERT INTO tablo1 VALUES('test1', 1, 1) -- Üstte oluşturulan kontrol1 kısıtını deliyor  
GO
```

```
INSERT INTO tablo1 VALUES('test1', 50, -2) -- Üstte oluşturulan kontrol2 kısıtını deliyor  
GO
```

```
]INSERT INTO tablo1 VALUES('test1', 500, 2) -- Kısıtlara uyuyor. Kayıt başarıyla gerçekleştirilecek  
_SELECT * FROM tablo1  
GO
```

	ID	sutun1	sutun2	sutun3
1	1	test1	10.00	100
2	6	test1	500.00	2

ALTER

- **ALTER TABLE ile bütünlük kısıtlarının (integrity constraints) yönetimi**

- **Alan düzeyi kısıt eklemek (NOT NULL)**

- İçinde veri bulunan bir tabloya NOT NULL sütun eklenirken, ya tablo boş olmalı ya da o sütun için bir DEFAULT değer tanımlanmalıdır.
 - Aksi halde işlem başarısız olur.

```
-- Tabloda kayıt olduğundan oluşturulacak sütun NOT NULL olamaz.  
-- Eğer NOT NULL oluşturulacaksa DEFAULT parametresi kullanılmalıdır  
ALTER TABLE tablo1 ADD sutun9 INT NOT NULL  
GO
```



```
-- DEFAULT ve NULL kullanılarak başarılı bir şekilde sütunlar oluşturuluyor  
ALTER TABLE tablo1 ADD sutun4 INT NOT NULL DEFAULT 0 -- sutun4 için varolan veriler 0 olacak  
ALTER TABLE tablo1 ADD sutun5 INT DEFAULT -5 WITH VALUES -- sutun5 için varolan veriler NULL değil -5 olacak  
SELECT * FROM tablo1  
GO
```

	ID	sutun1	sutun2	sutun3	sutun4	sutun5
1	1	test1	10.00	100	0	-5
2	6	test1	500.00	2	0	-5

ALTER

- **ALTER TABLE ile bütünlük kısıtlarının (integrity constraints) yönetimi**

- Benzersizlik kısıtı eklemek (UNIQUE)

```
-- Varolan veriler kısıtı deldiğinden kısıt oluşturulamaz.
```

```
-- UNIQUE kısıtı WITH NOCHECK ifadesinden etkilenmez
```

```
ALTER TABLE tablo1 WITH NOCHECK ADD CONSTRAINT benzersizA UNIQUE (sutun5)  
GO
```

```
-- sutun3 için kısıtı delen değer olmadığından kısıt oluşturulur
```

```
ALTER TABLE tablo1 ADD CONSTRAINT benzersiz1 UNIQUE (sutun3)  
GO
```

```
INSERT INTO tablo1 VALUES('test1', 501, 2, 1, 1) -- Üstteki kısıta uymuyor  
GO
```

```
]INSERT INTO tablo1 VALUES('test1', 501, 3, 1, 1) -- Üstteki kısıta uyuyor
```

```
SELECT * FROM tablo1  
GO
```

	ID	sutun1	sutun2	sutun3	sutun4	sutun5
1	1	test1	10.00	100	0	-5
2	6	test1	500.00	2	0	-5
3	8	test1	501.00	3	1	1

ALTER

- **ALTER TABLE ile bütünlük kısıtlarının (integrity constraints) yönetimi**
 - Varsayılan değer tanımlamak (DEFAULT)

```
ALTER TABLE tablo1 ADD sutun6 INT  
GO
```

-- sutun6 için varsayılan değer -1 tanımı yap. Varolan değerler etkilenmez

```
ALTER TABLE tablo1 ADD CONSTRAINT varsayilan2 DEFAULT -1 FOR sutun6  
GO
```

```
INSERT INTO tablo1 VALUES('test1', 508, 88, 100, DEFAULT, DEFAULT) -- sutun6 -1 değerini alacaktır  
SELECT * FROM tablo1  
GO
```

Results		Messages					
	ID	sutun1	sutun2	sutun3	sutun4	sutun5	sutun6
1	1	test1	10.00	100	0	-5	NULL
2	6	test1	500.00	2	0	-5	NULL
3	8	test1	501.00	3	1	1	NULL
4	9	test1	508.00	88	100	-5	-1

ALTER

- **ALTER TABLE** ile bütünlük kısıtlarının (integrity constraints) yönetimi

- Kısıt silmek

```
ALTER TABLE tablo1 DROP CONSTRAINT varsayilan2 -- Kısıtı sil  
GO
```

ALTER

- **ALTER TABLE ile bütünlük kısıtlarının (integrity constraints) yönetimi**

- **Kısıtları devre dışı bırakmak**

- CHECK ve FOREIGN KEY kısıtları geçici olarak devre dışı bırakılabilir.
 - Diğer kısıt türleri (PRIMARY KEY, UNIQUE) doğrudan devre dışı bırakılamaz.
 - Kısıtlar düzenlenemez; silinip yeniden oluşturulmaları gerekir.

```
-- FK yada check constraint'leri tümünden yada isim verilerek devre dışı / devrede yapılabilir
ALTER TABLE tablo1 NOCHECK CONSTRAINT ALL -- Tablodaki tüm kısıtları devre dışı bırak
GO
```

```
ALTER TABLE tablo1 CHECK CONSTRAINT ALL -- Tablodaki tüm kısıtları devreye al
GO
```

```
ALTER TABLE tablo1 NOCHECK CONSTRAINT kontrol1 -- kontrol1 isimli kısıtı devre dışı bırak
GO
```

```
ALTER TABLE tablo1 CHECK CONSTRAINT kontrol1 -- kontrol1 isimli kısıtı devreye al
GO
```

```
-- sutun4 deki değerler refere edilen tabloda olmadığından kısıt kontrolünü geçici olarak kapatıyoruz
-- Eğer sutun4 teki değerler karşı tablonun sütununda da olsaydı yada zaten tablo1 tamamen boş olsaydı buna gerek kalmazdı
ALTER TABLE tablo1 WITH NOCHECK ADD CONSTRAINT fktablo1_tablo2 FOREIGN KEY (sutun4) REFERENCES tablo2(ID)
GO
```

DROP

- Veri tabanındaki bir nesnenin tanımını ve ilişkili yapıları kaldırarak nesneyi tamamen silen bir DDL komutudur.
- **DROP TABLE**
 - SQL Server'da bir tablo başka tablolar tarafından FOREIGN KEY ile referans ediliyorsa, DROP TABLE işlemi hata verir ve işlem iptal edilir.
- **DROP DATABASE**
 - Veri tabanındaki tüm tabloları, nesneleri ve verileri kalıcı olarak kaldırır.
 - İlgili veri tabanına ait aktif bağlantılar varsa DROP DATABASE işlemi başarısız olur.
- DROP işlemi öncesi ilgili nesnenin varlığının kontrol edilmesi tavsiye edilir.

```
DROP TABLE IF EXISTS deneme_tablosu
```

```
DROP DATABASE IF EXISTS gecici
```

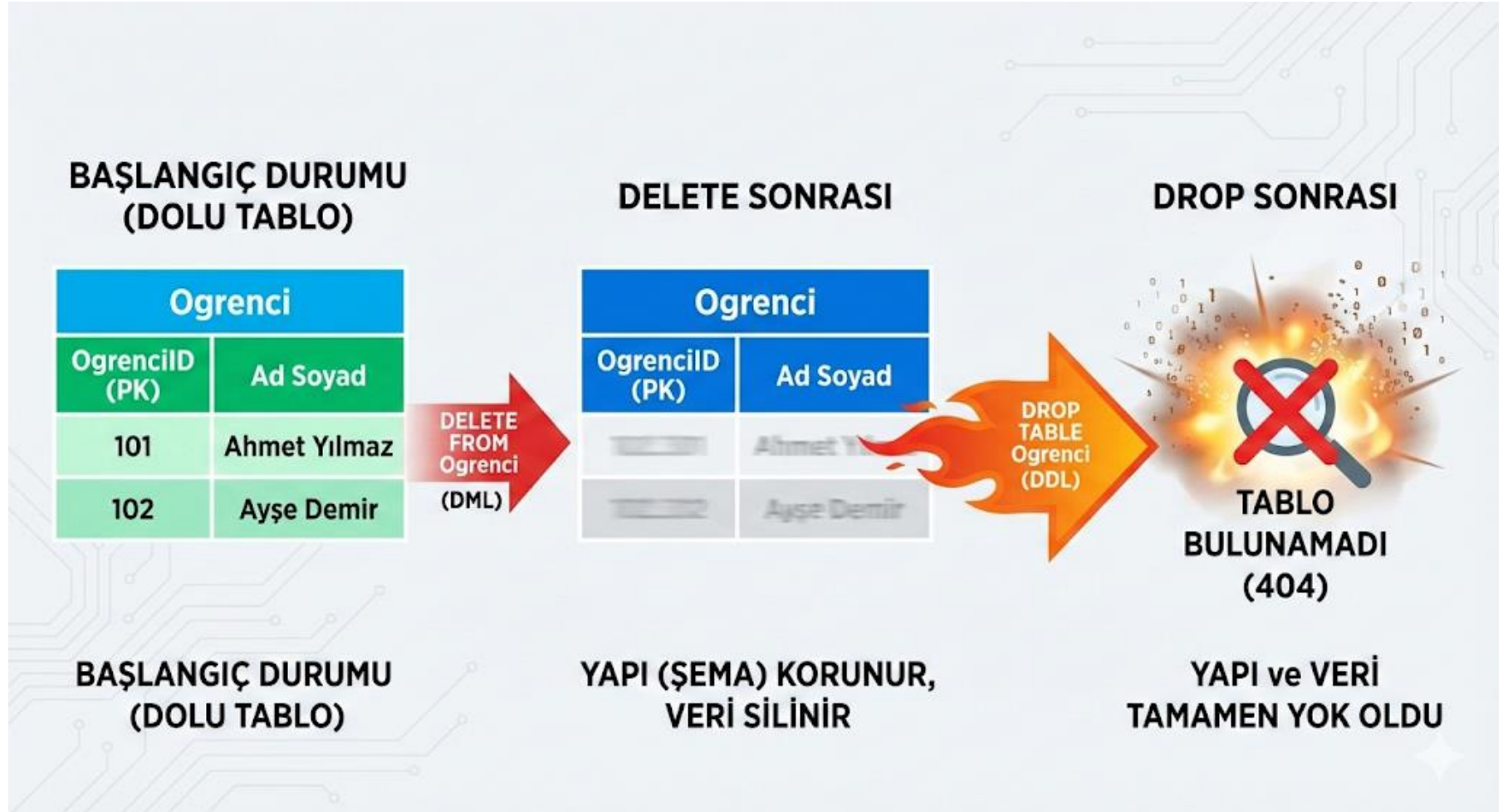

DROP

- Aşağıdaki örnekte;
 - Nesnenin var olma durumu DB_ID fonksiyonundan gelecek bilgiyle kontrol edilmiştir.
 - Diğer bağlantıların DROP işlemine engel olması sebebiyle
 - Öncelikle veri tabanı tek kullanıcıya getirilmiş
 - Ardından master veri tabanına geçiş yapılarak kalan son bağlantı da sonlandırılmış
 - Son olarak da sorunsuz bir şekilde DROP işlemi gerçekleştirilmiştir

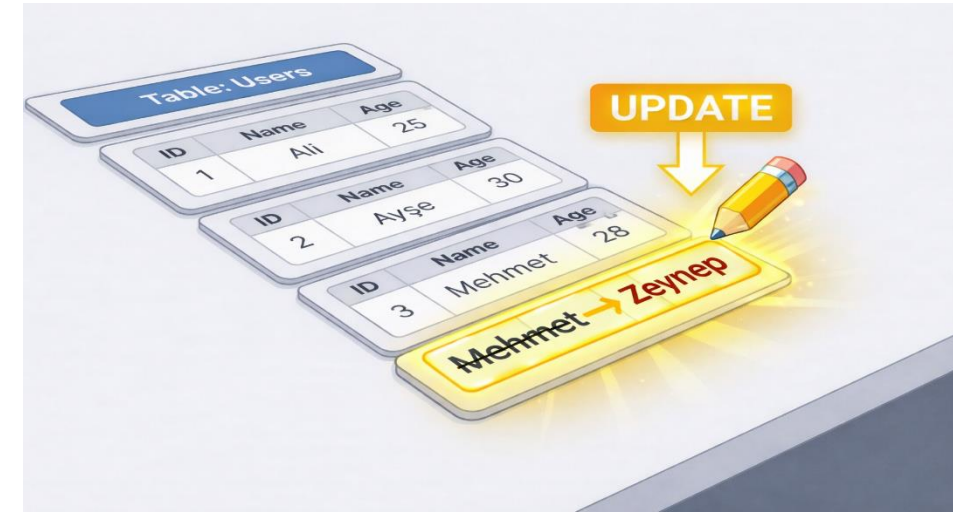
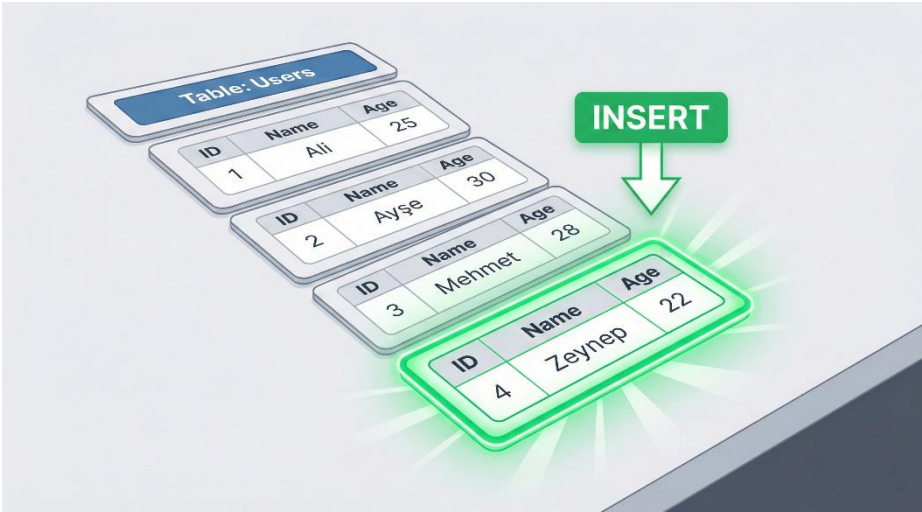
```
IF DB_ID('SST_DB') IS NOT NULL
    BEGIN
        ALTER DATABASE [SST_DB] SET SINGLE_USER WITH ROLLBACK IMMEDIATE
        USE master
        DROP DATABASE SST_DB
    END
GO
```

DROP vs. DELETE

- DELETE yalnızca tablodaki verileri siler (DML)
- DROP tablonun kendisini ve tanımını siler (DDL)
- DELETE sonrası tablo kullanılabilir, DROP sonrası tablo artık yoktur



Veri İşleme Dili (Data Manipulation Language – DML)



INSERT

- Bir tabloya yeni satır (row) eklemek için kullanılan bir DML komutudur.
- Tablonun yapısını değil, içeriğini (verisini) değiştirir.
- TCL komutları ile kontrol edilebilir.
- INSERT işlemi sırasında aşağıdaki kısıtlar kontrol edilir. Herhangi bir kısıt ihlal edilirse INSERT işlemi hata vererek iptal edilir.
 - PRIMARY KEY
 - FOREIGN KEY
 - UNIQUE
 - CHECK
 - NOT NULL
- INSERT edilen FOREIGN KEY değeri referans edilen tabloda mevcut olmalıdır.
 - Aksi halde işlem başarısız olur.
 - Bu davranış, referans bütünlüğü (referential integrity) ilkesinin bir sonucudur.
- Eğer bir INSERT işlemi herhangi bir kısıt nedeniyle hata verip iptal edilirse, söz konusu INSERT işlemi için üretilmiş otomatik anahtar değeri (IDENTITY/SEQUENCE) geri alınmaz.
 - Bu davranış, birçok veri tabanı sisteminde görülen ve otomatik anahtarların ardışık olmasının garanti edilmediğini gösteren bir tasarım tercihidir.

INSERT – Temel Sözdizimi (Syntax)

- Değerler, tablodaki sütun sırasına göre girilmelidir.
- Tüm sütunlar için değer verilmelidir.
- Bu kullanım, tablo yapısına sıkı bağımlı olduğundan önerilmez.

```
INSERT INTO tablo_adi VALUES (deger1, deger2, ...)
```

INSERT – Sütun Belirterek

- Sütun sırasından bağımsızdır.
- Okunabilirliği ve bakım kolaylığını artırdığı için önerilen yöntemdir.

```
INSERT INTO tablo_adi (sutun1, sutun2, sutun3) VALUES (deger1, deger2, deger3)
```

INSERT – NULL, DEFAULT ve IDENTITY Davranışı

- Değer girilmeyen sütunlar
 - NULL kabul ediyorsa → NULL atanır
 - DEFAULT tanımlıysa → varsayılan değer atanır
- IDENTITY sütunlara değer girilmez (aksi halde hata oluşur).

```
CREATE TABLE Ogrenci
(  
    ID INT IDENTITY(1,1),  
    Ad VARCHAR(50) NOT NULL,  
    Soyad VARCHAR(50) NOT NULL,  
    Tel CHAR(10) DEFAULT '0123456789',  
    GANO FLOAT DEFAULT 0,  
    KanGrubu VARCHAR(2)  
)
```

ID	Ad	Soyad	Tel	GANO	KanGrubu
1	Ali	Yılmaz	0123456789	0	NULL

```
INSERT INTO Ogrenci (Ad, Soyad, GANO)  
VALUES ('Ali', 'Yılmaz', DEFAULT)
```

```
INSERT INTO Ogrenci (Ad)  
VALUES ('Ayşe')
```

Cannot insert the value NULL into column 'Soyad', table 'gecici.dbo.Ogrenci'; column does not allow nulls. INSERT fails.
The statement has been terminated.

```
INSERT INTO Ogrenci (ID, Ad, Soyad)  
VALUES (2, 'Ayşe', 'Şahin')
```

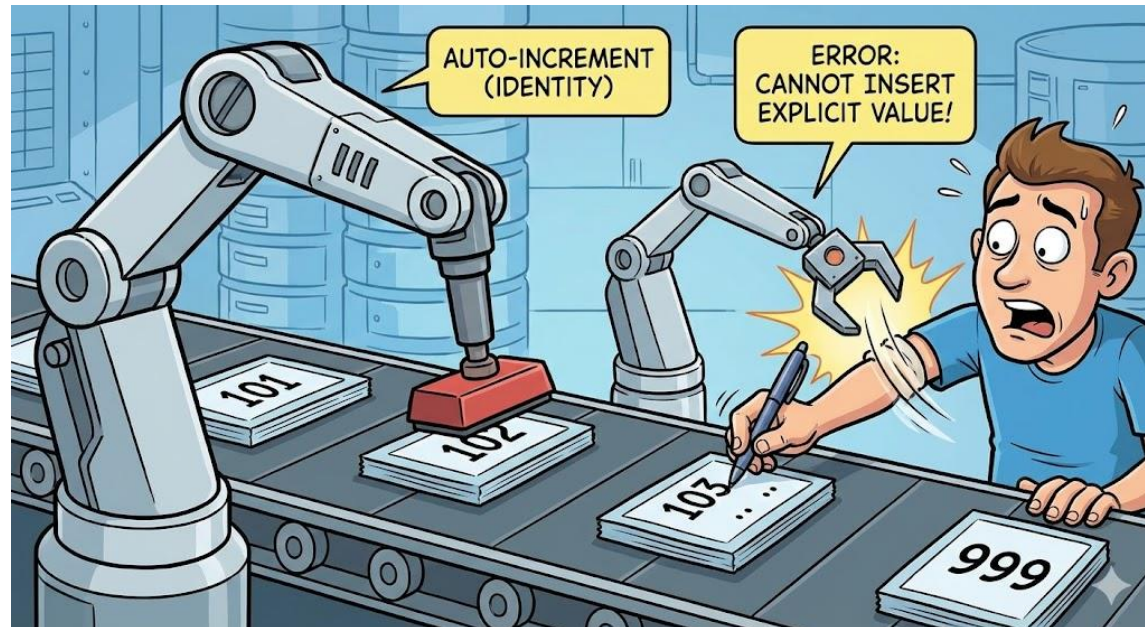
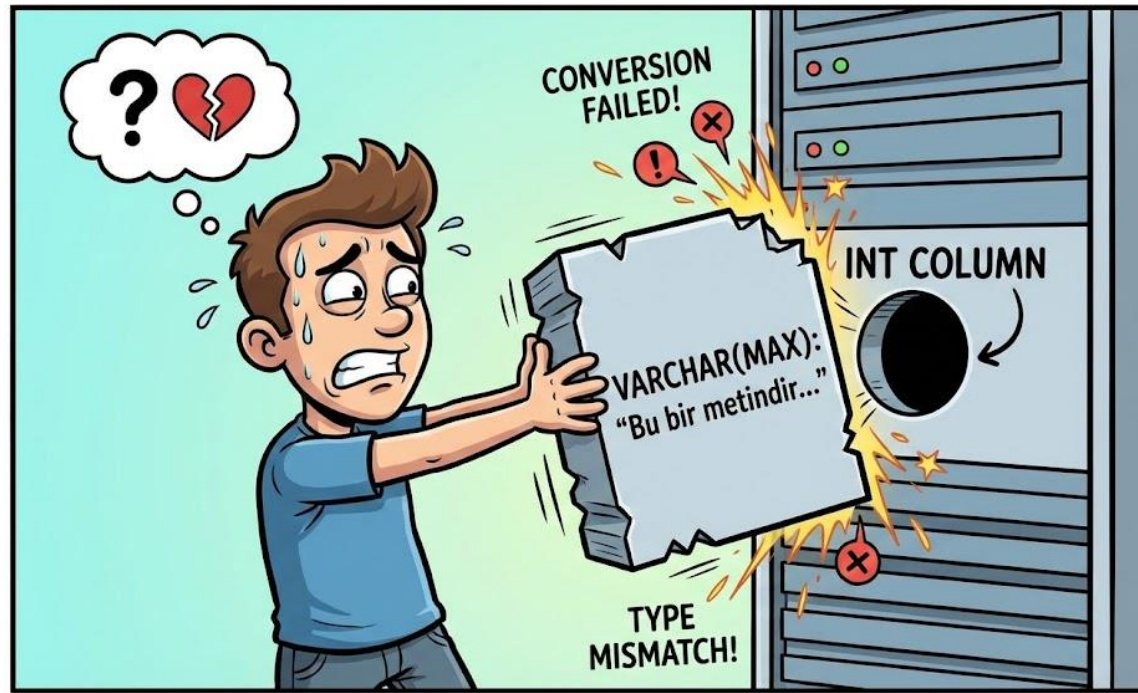
Cannot insert explicit value for identity column in table 'Ogrenci' when IDENTITY_INSERT is set to OFF.

INSERT – Çoklu Satır INSERT (Multiple Row Insert)

- Tek komutla birden fazla satır eklenebilir.
- SQL Server tarafından desteklenir.
- Performans açısından tek tek INSERT yapmaya göre daha verimlidir.
- Tek bir INSERT komutu ile gönderilen çoklu kayıtlarda, eğer satırlardan biri bile hata nedeniyle eklenemiyorsa, transaction mantığı (atomicity) gereği tüm işlem iptal edilir ve hiçbirisi eklenmez.

```
INSERT INTO Ogrenci (Ad, Soyad)  
VALUES ('Ayşe', 'Şahin'), ('Mehmet', 'Kaya')
```

ID	Ad	Soyad	Tel	GANO	KanGrubu
1	Ali	Yılmaz	0123456789	0	NULL
3	Ayşe	Şahin	0123456789	0	NULL
4	Mehmet	Kaya	0123456789	0	NULL





IDENTITY İÇİN ALTERNATİF - SEQUENCE

- Kullanıcı tanımlı, tablo bağımsız, nümerik artan değer nesnesi
- Insert yapılmadan da **NEXT VALUE FOR** ifadesi ile sıradaki değer alınabilir
- Her bir **SELECT NEXT VALUE FOR** ifadesi değerın değişmesine neden olur

CREATE SEQUENCE seq_ismi

[**AS** int_veri_tipleri] -- *tinyint, smallint, int, bigint, decimal & numeric scale=0. Default bigint*

[**START WITH** sabit_deger]

[**INCREMENT BY** sabit_deger] -- *Her bir **NEXT VALUE FOR** için poz ise arttırma, neg ise azaltma*

[**MINVALUE** sabit_değer | **NO MINVALUE**] -- *Default olarak verilen tipin min değeri*

[**MAXVALUE** sabit_değer | **NO MAXVALUE**] -- *Default olarak verilen tipin max değeri*

[**CYCLE** | **NO CYCLE**] -- *Varsayılan NO CYCLE*

[**CACHE** sabit_değer | **NO CACHE**] -- *Var sayılan CACHE'tir. Performans artışı sağlar*

- **ALTER SEQUENCE** seq_name **RESTART WITH** sonraki_deger
 - **RESTART WITH** değeri, bir sonraki üretilecek değerdir.

IDENTITY İÇİN ALTERNATİF - SEQUENCE

```
|CREATE SEQUENCE Id_Sequence
```

```
    START WITH 1
```

```
    INCREMENT BY 1
```

```
GO
```

```
create table deneme(a INT, b INT)
```

```
GO
```

```
|INSERT INTO deneme VALUES(NEXT VALUE FOR Id_Sequence, NEXT VALUE FOR Id_Sequence)
```

```
select* from deneme
```

```
INSERT INTO deneme VALUES(NEXT VALUE FOR Id_Sequence, NEXT VALUE FOR Id_Sequence)
```

```
select* from deneme
```

```
select NEXT VALUE FOR Id_Sequence
```

```
select NEXT VALUE FOR Id_Sequence, NEXT VALUE FOR Id_Sequence
```

```
|INSERT INTO deneme VALUES
```

```
    (NEXT VALUE FOR Id_Sequence, NEXT VALUE FOR Id_Sequence),
```

```
    (NEXT VALUE FOR Id_Sequence, NEXT VALUE FOR Id_Sequence)
```

```
select* from deneme
```

Results			Messages		
	a	b			
1	1	1			
1	1	1			
2	2	2			
1	(No column name)				
1	3				
1	(No column name)		(No column name)		
1	4	4			
1	1	1			
2	2	2			
3	5	5			
4	6	6			

IDENTITY vs. SEQUENCE

- **IDENTITY**

- Tabloya bağlıdır, tablo silinirse o da silinir
- Sadece tanımlandığı tabloda kullanılır
- IDENTITY değeri, doğrudan NEXT VALUE FOR benzeri bir ifadeyle INSERT öncesi alınamaz

- **SEQUENCE**

- Tablodan bağımsızdır
- Bir sequence numarasını birden fazla tablo kullanabilir
 - Örneğin Fatura ve Sipariş tabloları ortak tekil bir sequence numarası kullanabilir
- INSERT yapmadan önce değeri çekilip kullanılabilir (NEXT VALUE FOR)
- Tabloyu TRUNCATE etmeden sayaç başa döndürülebilir
 - **ALTER SEQUENCE** seq_ismi **RESTART WITH 1**
- CYCLE seçeneği aktif olduğunda, sequence MAXVALUE/ MINVALUE sınırına ulaştığında başa döner ve daha önce üretilmiş değerler tekrar üretilir.
 - Bu nedenle UNIQUE constraint gerektiren sütunlarda (özellikle PRIMARY KEY) CYCLE kullanımı Duplicate Key Error riski oluşturur.
 - PRIMARY KEY olarak kullanılan sequence'lar genellikle NO CYCLE olarak tanımlanır.
- NEXT VALUE FOR her çağrıldığında, işlem ROLLBACK edilse bile sequence değeri geri alınmaz

UPDATE

- Bir tabloda bulunan mevcut kayıtların bir veya daha fazla sütunundaki değerleri değiştirmek için kullanılan bir DML komutudur
- SQL Server'da UPDATE işlemi doğrudan yalnızca tek bir tabloyu günceller; ancak FROM ifadesi ile diğer tablolar filtreleme amacıyla kullanılabilir
- WHERE kısmı kullanılmazsa tüm tablodaki kayıtlar güncellenir.
 - **UYARI:** WHERE ifadesi kullanılmadan yapılan UPDATE işlemleri geri dönüşü zor hatalara yol açabilir
- Bir ya da birden çok koşul belirtilebilir
 - Birden fazla koşul olduğunda AND ya da OR ile bağlanabilir
 - Koşulların bağlanma önceliği parantezler ile kontrol edilmelidir
- Güncellenen sütun bir FOREIGN KEY tarafından referans ediliyorsa ve ON UPDATE CASCADE tanımı da yoksa UPDATE işlemi hata verilerek iptal edilir (varsayılan davranış: NO ACTION)
- PRIMARY KEY sütunlarının UPDATE edilmesi referans ilişkileri ve indeks yapısı nedeniyle genellikle önerilmez.

UPDATE

- Syntax

```
UPDATE tabloadi
SET alan1 = deger1,
    alan2 = deger2,
    .
    .
    .
    alanN = degerN
[FROM tablo(lar)]
[WHERE kosullar]
```

- Örnek

tcno	ad	soyad	maas	MedeniHali	IlceID
12345678901	ahmet	şahin	8500.00	NULL	1
98765432100	ayşe	kasap	18000.50	B	25

```
UPDATE tblPersonel
SET Maas=12345.67,
    MedeniHali='E',
    IlceID=5
WHERE TCNO='12345678901'
```

tcno	ad	soyad	maas	MedeniHali	IlceID
12345678901	ahmet	şahin	12345.67	E	5
98765432100	ayşe	kasap	18000.50	B	25

UPDATE

- WHERE ifadesinde kullanılabilecek operatörler

- **LIKE**

- Yüzde işareti (%)
 - 0-n adet karakter
 - Escape: \%
- Alt çizgi (_) işareti
 - Sadece tek bir karakter
 - Escape: _
- Köşeli parantez içinde karakterler
 - Ör. [arg] => a veya r veya g olanlar
- Köşeli parantez içinde şapka işareti
 - Ör. [^abc] => a, b, ve c den hiç biri olmasın
- Köşeli parantez içinde tire (-) işareti
 - [m-y] => m-y arasındaki harflerden birisi olsun
 - [^a-g] => a-g arasındaki harflerden olmasın
 - Escape: [[] veya []]

OPERATÖR	AÇIKLAMA	ÖRNEK
=	Eşittir	Ad='hüseyin'
<> !=	Eşit değildir	Kod<>5 AND Aktif!=0
> <	Büyüktür Küçüktür	Maas > 1000
>= <=	Büyük eşit Küçük eşit	Maas <= 2000
BETWEEN	Belirli bir aralıkta	puan BETWEEN 5 AND 12
LIKE NOT LIKE	Verilen örnek şablona benzeyenler / benzemeyenler	Soyad LIKE 'A%'

UPDATE

- LIKE

- Büyük/Küçük Harf Duyarlılığı

- Varsayılan olarak sorgunun çalıştığı veri tabanının Collation (karşılaştırma) ayarını izler

- ❑ **Turkish_CI_AS**: "Case Insensitive" (Büyük-küçük harf duyarsız) demektir. 'A' ile 'a'yı aynı görür

- ❑ **Turkish_CS_AS**: "Case Sensitive" (Büyük-küçük harf duyarlı) demektir. 'A' ile 'a'yı farklı görür

- Mevcut veri tabanınızın varsayılan dil ve duyarlılık ayarlarını görmek için en hızlı yol DATABASEPROPERTYEX fonksiyonunu kullanmaktır

```
SELECT DATABASEPROPERTYEX('Veritabani_Adiniz', 'Collation') AS DatabaseCollation
```

```
SELECT DATABASEPROPERTYEX('NORTHWND', 'Collation') AS Collation_NW,  
       DATABASEPROPERTYEX('kutuphane', 'Collation') AS Collation_LIB
```

Collation_NW	Collation_LIB
SQL_Latin1_General_CP1_CI_AS	Turkish_CI_AS

Kısaltma	Açılımı	Anlamı
CI	Case Insensitive	Büyük-küçük harf ayrımı yapmaz. (A = a)
CS	Case Sensitive	Büyük-küçük harf ayrımı yapar. (A ≠ a)
AI	Accent Insensitive	Aksan/İşaret ayrımı yapmaz. (ı = i veya e = é)
AS	Accent Sensitive	Aksan/İşaret ayrımı yapar. (ı ≠ i)

UPDATE

- WHERE ifadesinde kullanılabilecek operatörler

OPERATÖR	AÇIKLAMA	ÖRNEK
IN NOT IN	Verilen değerlerden her hangi birisi olanlar / olmayanlar Bir değeri birden fazla alanda aratma	Harf_notu IN ('A','B') Parantez içerisinde SELECT cümlesi gelebilir '1234567' IN (Tel1, Tel2) -> Tel1 ya da Tel2 sütununda verilen telefon numarası yazan satırlar
IS NULL IS NOT NULL	NULL mu değil mi?	Tel1 IS NOT NULL AND Tel2 IS NULL

DELETE & TRUNCATE

- **DELETE**

- Bir tabloda verilen bir koşula uyan verileri silmek için kullanılır
- Koşul belirtilmezse tüm kayıtlar silinir
- Tüm kayıtlar silinse bile IDENTITY değeri sıfırlanmaz
- Eğer silinmek istenen kayıtlar başka bir tablo tarafından FOREIGN KEY ile referans ediliyorsa ve ON DELETE CASCADE tanımı da yoksa DELETE işlemi hata verilerek iptal edilir (varsayılan davranış: NO ACTION)

- **TRUNCATE**

- Verileri koşulsuz olarak siler
- IDENTITY değeri sıfırlanır
- Daha az log alanı kullandığından DELETE komutuna göre daha hızlı çalışır
- Başka bir tablo (boş bile olsa) TRUNCATE edilecek tabloya FOREIGN KEY ile referans gösteriyorsa TRUNCATE çalışmaz.
 - Çalışması için FOREIGN KEY'yi kaldırmak veya disable etmek gerekir.
- Varsa TRIGGER nesneleri tetiklenmez
- Tablonun veriyi tuttuğu veri sayfalarını (data pages) serbest bıraktığı (page deallocation) yani satırları tek tek silerek değil de verinin bulunduğu alanı "boş" olarak işaretleyerek tabloyu boşalttığı için şema üzerinde etkisi olan bir işlem olarak değerlendirilir.
 - Bu açıdan DML değil DDL komutları içine dahil edilir.

DELETE & TRUNCATE

ÖZELLİK	DELETE	TRUNCATE
Komut türü	DML	DDL
WHERE	Var	Yok
Tetikleyici	Çalışır	Çalışmaz
Identity	Sıfırlanmaz	Sıfırlanır
FK varsa	CASCADE yoksa hata	Çalışmaz
Performans	Daha yavaş	Daha hızlı

DELETE & TRUNCATE

- Syntax

```
DELETE FROM tablo_adi [WHERE kosullar]
```

```
TRUNCATE TABLE tablo_adi
```

- Örnek

```
DELETE FROM tblPersonel WHERE TCNO='12345678901'
```

```
-- Bu kaydın silinebilmesi için, bu personele ait referanslanan tüm kayıtlar silinmiş olmalıdır.
```

```
TRUNCATE TABLE tblGeciciTablo
```

SELECT

- Veri tabanındaki tablo ya da tablolarda bulunan verilerin okunabilmesi amacıyla kullanılır

- Söz dizimi (syntax) şöyledir:

```
SELECT [TOP n [PERCENT] [WITH TIES]] [DISTINCT] sutun_isimleri  
[INTO yeni_tablo]  
FROM tablo  
[[INNER | LEFT | RIGHT | FULL] JOIN diger_tablolar ON join_kosullari]  
[WHERE arama_kosullari]  
[GROUP BY [ROLLUP | CUBE] gruplama_sutunlari [WITH ROLLUP | CUBE]  
[HAVING grup_kosullari]  
[ORDER BY siralama_sutunlari [ ASC | DESC ]]
```

SELECT – Mantıksal İşlem Sırası

- SQL'in söz dizimi sırası ile mantıksal işlem sırası farklıdır
 - 1) **FROM & JOIN**
 - Veri kaynağı ve tabloların birleştirildiği ilk aşamadır
 - 2) **WHERE**
 - Ham veriler üzerinde satır bazlı filtreleme yapılır
 - 3) **GROUP BY**
 - Veriler belirli kriterlere göre gruplara ayrılır
 - 4) **HAVING**
 - Gruplanmış veriler (aggregate sonuçları) filtrelenir
 - 5) **SELECT**
 - Sütunlar seçilir ve takma adlar (alias) tanımlanır
 - 6) **DISTINCT**
 - Tekrar eden satırlar sonuç kümesinden atılır
 - 7) **ORDER BY**
 - Sonuçlar sıralanır (burada SELECT takma adları geçerlidir)
 - 8) **TOP / OFFSET**
 - Kullanıcıya dönecek son satır sayısı belirlenir

SELECT

- **SELECT**

- Gösterilmek istenen sütunların isimleri virgülle ayrılarak yazılır.
- Eğer birden fazla tablodan okuma yapılıyor ve aynı isme sahip sütunlar da varsa bu durumda sütun isimlerinden önce tablo ismi (yada FROM cümlesinde verilen takma isim) yazılır.
 - Örneğin *tblCari.ad, tblUrun.ad, U.Stok*
- **AS**
 - İstenirse sütunları yeniden isimlendirmek için kullanılır.
 - Bu amaçla **AS** kullanılması zorunlu değildir.
 - Örneğin **SELECT** *tblCari.ad AS MusteriAdi, tblUrun.ad UrunAdi*
- Eğer tüm sütunların gösterilmesi isteniyorsa yıldız (*) karakteri kullanılır.
 - **SELECT *** kullanımı küçük örneklerde uygundur; ancak gerçek uygulamalarda bakım ve performans açısından önerilmez.

SELECT

- **SELECT**

- Sadece tablo sütunları değil sabit değerler, aritmetik ifadeler ya da fonksiyonlar da yazılabilir.

- Örneğin **SELECT** *tblCari.ad*, *2+3*, *'deneme'*, **LEFT**(*'yalova'*, 2)

- **DISTINCT**

- Bu ifadeden sonra yazılan sütun veya sütunlara göre veri tekrarları engellenir.
 - Kullanılmadığı takdirde veri tekrarı görülebilir.
 - DISTINCT, SELECT listesindeki sütunların kombinasyonuna uygulanır.

- **INTO**

- Sorgu sonucunun yeni bir tabloya yazılması amacıyla kullanılır

- **FROM**

- Bu ifadeden sonra bir ya da birden fazla tablo ismi yazılabilir.
- Tablo isimlerine takma isim de verilebilir.
- Eğer tablolar WHERE ifadesinden sonra belirli alanlara göre birleştirilmezse kartezyen çarpım yapılmış olur.

SELECT

- **[INNER | LEFT | RIGHT | FULL] JOIN**

- Birden fazla tablo ile çalışılacağı zaman tablolardan biri FROM ifadesinden sonra yazılıp diğerleri dört farklı JOIN işleminden uygun olanı ile birleştirilebilir.
- Böyle bir durumda bağlantı koşulu veya koşulları **ON** ifadesinden sonra yazılır.
- Örneğin **SELECT * FROM tblPersonel AS P INNER JOIN tblDepartman AS D ON D.ID=P.ID**

- **WHERE**

- Tablo ya da tabloların birleşmesinin sonucunda satırlara uygulanacak koşullar belirtilebilir.
- Burada daha önce UPDATE konusunda verilen tablodaki operatörler geçerlidir.
- Tablo JOIN koşulları burada da yazılabilir ancak genellikle JOIN koşulları ON ifadesinde, filtreleme koşulları ise WHERE ifadesinde yazılır.
- Dikkat! LEFT JOIN sonrası WHERE ile sağ tabloya filtre uygulanırsa LEFT JOIN davranışı INNER JOIN'e dönüşebilir. RIGHT ve FULL JOIN için de benzer durum yaşanabilir.

- **GROUP BY**

- Gruplama fonksiyonlarının kullanıldığı sorgularda söz konusu fonksiyonların dışında tablolardan bazı sütunlar da gösterilecekse **mutlaka** bu ifadenin kullanılması gerekir.
- Söz konusu sütunların hepsi bu kısımda belirtilmelidir.

SELECT

- **HAVING**

- Gruplama sonrası filtreleme yapmak amacıyla kullanılır.
- Burada genellikle gruplama fonksiyonlarına filtreleme yapılır.
 - GROUP BY ifadesinde yazılmış olmak şartıyla tablo sütunları da HAVING ifadesinde kullanılabilir ama çoğu kez bunun için WHERE ifadesi kullanılır.
- WHERE ifadesinden farkı belirtilen filtrelerin gruplama sonrası uygulanmasıdır
 - WHERE ifadesinde filtreler gruplama öncesi uygulanır (yani satırlara).
 - Dolayısıyla WHERE ifadesinde gruplama fonksiyonuna filtreleme **yapılamaz**.

- **ORDER BY**

- İstenen sütunlara göre artan (**ASC**) ya da azalan (**DESC**) sıralama yapılabilir.
- Varsayılan sıralama artan (**ASC**)'dır.
- Örneğin **ORDER BY TCNO, DogumTarihi DESC, Ad DESC**
- Belirtilen sütundaki NULL değerler varsayılan olarak en küçük değer olarak kabul edilir

SELECT

- **TOP**

- Sorgu sonucunda gösterilecek veri miktarını sınırlar
- Örneğin
 - **TOP 10** ifadesi kullanılırsa ilk 10 satır gösterilir
 - **TOP 10 PERCENT** denirse de sorgu sonucunun %10'luk kısmı gösterilir.
 - Örneğin sorgu sonucunda 47 kayıt dönüyorsa 5 satır gösterilecektir.
 - Sonuç yukarı yuvarlanır (ceiling mantığı)
- **WITH TIES**
 - Sıralama sonucu son satırda berabere kalan satırların gösterilmesini sağlar.
 - Bu ifade kullanılacaksa mutlaka ORDER BY ifadesi de kullanılmalıdır.
- Bu ifadeler SQL Server'a özgüdür; standart SQL'de FETCH yapısı kullanılır.
 - Bazı veri tabanı yönetim sistemlerinde LIMIT yapısı olarak geçer

SELECT – Basit

-- Tüm illeri listeleyelim

```
SELECT *  
FROM tblil
```

Results		Messages
	Kod	Ad
1	01	Adana
2	02	Adıyaman
3	03	Afyonkarahisar
4	04	Ağrı
5	05	Amasya
6	06	Ankara
7	07	Antalya
8	08	Artvin

SELECT – WHERE

```
-- Adı A ile başlayıp N ile biten ve  
-- arada mutlaka A ya da Y harfi geçen  
-- illeri listeleyelim  
SELECT *  
FROM tblil  
WHERE  
    Ad LIKE 'a%[ay]%n'
```

	Kod	Ad
1	02	Adıyaman
2	09	Aydın
3	75	Ardahan

SELECT – ORDER

```
-- Adı B ile başlayan illeri adına göre  
-- büyükten küçüğe listeleyelim  
SELECT *  
FROM tblil  
WHERE  
    Ad LIKE 'B%'  
ORDER BY  
    Ad DESC
```

	Kod	Ad
1	16	Bursa
2	15	Burdur
3	14	Bolu
4	13	Bitlis
5	12	Bingöl
6	11	Bilecik
7	69	Bayburt
8	72	Batman

SELECT – INNER JOIN

-- İl ve ilçe tablolarını birleştirelim

SELECT *

FROM tblIl AS A

INNER JOIN tblIlce AS B ON A.Kod = B.IlKodu

	Kod	Ad	ID	Ad	IlKodu
1	01	Adana	1	Seyhan	01
2	01	Adana	2	Ceyhan	01
3	01	Adana	3	Feke	01
4	01	Adana	4	Karaisalı	01
5	01	Adana	5	Karataş	01
6	01	Adana	6	Kozan	01
7	01	Adana	7	Pozantı	01
8	01	Adana	8	Saimbeyli	01

SELECT – Hepsi

- Önceki sorguda ID ve kod sütunları da gözüküyordu.
- Bunları göremeye gerek yok. Sadece il adı ve ilçe adı yeterli.
- Sütun isimlerini de buna göre ayarlayalım.
- Sıralamayı önce il adına göre büyükten küçüğe sonra
- ilçe adına göre küçükten büyüğe şeklinde yapalım.
- Filtreleme kriterimiz ise il kodu 34, 06, 35, 16 ve 77 olanlar olsun.

```
SELECT  A.Ad AS İlAdı,  
        B.Ad AS [İlçe Adı]  
FROM tblİl AS A  
      INNER JOIN tblİlce AS B ON A.Kod = B.İlKodu  
WHERE  
      A.Kod IN ('34', '06', '35', '16', '77')  
ORDER BY  
      A.Ad DESC,  
      B.Ad
```

	İlAdı	İlçe Adı
1	Yalova	Altınova
2	Yalova	Amutlu
3	Yalova	Çınarcık
4	Yalova	Çiftlikköy
5	Yalova	Temal
6	Yalova	Yalova...
7	İzmir	Aliağa
8	İzmir	Balçova

SELECT – LEFT JOIN

-- Tüm diller için (kitap basılmayanlar da dahil) kitap isimleri

-- yanına da hangi dilde oldukları gelsin

```
SELECT  K.Ad AS Kitap_Adi,  
        D.Ad AS Dili  
FROM    tblDil D  
        LEFT JOIN tblKitap K ON D.ID = K.DilID
```

tblDil

	ID	Ad
1	1	TÜRKÇE
2	2	İNGİLİZCE
3	3	ALMANCA
4	4	ÇİNCE

tblKitap

	Ad	DilID
1	KİTAP 1	1
2	KİTAP 2	3
3	KİTAP 3	1
4	KİTAP 4	2
5	KİTAP 5	1
6	KİTAP 6	1
7	KİTAP 7	3
8	KİTAP 8	2
9	KİTAP 9	2
10	KİTAP 10	1

	Ad	Ad
1	NULL	ÇİNCE
2	KİTAP 1	TÜRKÇE
3	KİTAP 2	ALMANCA
4	KİTAP 3	TÜRKÇE
5	KİTAP 4	İNGİLİZCE
6	KİTAP 5	TÜRKÇE
7	KİTAP 6	TÜRKÇE
8	KİTAP 7	ALMANCA
9	KİTAP 8	İNGİLİZCE
10	KİTAP 9	İNGİLİZCE
11	KİTAP 10	TÜRKÇE

SELECT – TOP

```
SELECT *  
FROM [kutuphane].[dbo].[tblUye]  
ORDER BY IlKodu DESC, IlceKodu
```

Results		Messages											
	ID	TCNO	Ad	Soyad	Adres	IlKodu	IlceKodu	Tel	Mail	DTarihi	Yas	UyeOlusTarihi	AktifMi
1	9	12345678909	ÜYE ADI 9	ÜYE SOYADI 9	ÜYE ADRES 9	77	13	0001234567	uye9@hotmail.com	1960-09-09	61	1975-12-28	0
2	10	12345678910	ÜYE ADI 10	ÜYE SOYADI 10	ÜYE ADRES 10	77	14	0001234567	uye10@hotmail.com	1999-10-10	22	2014-11-22	1
3	7	12345678907	ÜYE ADI 7	ÜYE SOYADI 7	ÜYE ADRES 7	35	8	0001234567	uye7@hotmail.com	1988-07-07	33	1998-03-09	1
4	1	12345678901	ÜYE ADI 1	ÜYE SOYADI 1	ÜYE ADRES 1	34	1	0001234567	uye1@hotmail.com	1996-01-01	25	2006-03-22	1
5	2	12345678902	ÜYE ADI 2	ÜYE SOYADI 2	ÜYE ADRES 2	34	2	0001234567	uye2@hotmail.com	1995-02-02	26	2005-12-03	1
6	3	12345678903	ÜYE ADI 3	ÜYE SOYADI 3	ÜYE ADRES 3	34	3	0001234567	uye3@hotmail.com	1997-03-03	24	2004-10-01	1
7	5	12345678905	ÜYE ADI 5	ÜYE SOYADI 5	NULL	34	4	NULL	uye5@hotmail.com	1998-05-05	23	2006-07-17	1
8	8	12345678908	ÜYE ADI 8	ÜYE SOYADI 8	ÜYE ADRES 8	16	11	0001234567	uye8@hotmail.com	1972-08-08	49	1985-01-14	0
9	4	12345678904	ÜYE ADI 4	ÜYE SOYADI 4	ÜYE ADRES 4	06	5	0001234567	uye4@hotmail.com	1994-04-04	27	2003-11-08	1
10	6	12345678906	ÜYE ADI 6	ÜYE SOYADI 6	ÜYE ADRES 6	06	6	0001234567	uye6@hotmail.com	1986-06-06	35	1996-05-11	1

```
SELECT TOP 5 *  
FROM [kutuphane].[dbo].[tblUye]  
ORDER BY IlKodu DESC, IlceKodu
```

Results		Messages											
	ID	TCNO	Ad	Soyad	Adres	IlKodu	IlceKodu	Tel	Mail	DTarihi	Yas	UyeOlusTarihi	AktifMi
1	9	12345678909	ÜYE ADI 9	ÜYE SOYADI 9	ÜYE ADRES 9	77	13	0001234567	uye9@hotmail.com	1960-09-09	61	1975-12-28	0
2	10	12345678910	ÜYE ADI 10	ÜYE SOYADI 10	ÜYE ADRES 10	77	14	0001234567	uye10@hotmail.com	1999-10-10	22	2014-11-22	1
3	7	12345678907	ÜYE ADI 7	ÜYE SOYADI 7	ÜYE ADRES 7	35	8	0001234567	uye7@hotmail.com	1988-07-07	33	1998-03-09	1
4	1	12345678901	ÜYE ADI 1	ÜYE SOYADI 1	ÜYE ADRES 1	34	1	0001234567	uye1@hotmail.com	1996-01-01	25	2006-03-22	1
5	2	12345678902	ÜYE ADI 2	ÜYE SOYADI 2	ÜYE ADRES 2	34	2	0001234567	uye2@hotmail.com	1995-02-02	26	2005-12-03	1

SELECT – TOP WITH TIES

```
SELECT *  
FROM [kutuphane].[dbo].[tblUye]  
ORDER BY IlKodu DESC, IlceKodu
```

Results		Messages											
	ID	TCNO	Ad	Soyad	Adres	IlKodu	IlceKodu	Tel	Mail	DTarihi	Yas	UyeOlusTarihi	AktifMi
1	9	12345678909	ÜYE ADI 9	ÜYE SOYADI 9	ÜYE ADRES 9	77	13	0001234567	uye9@hotmail.com	1960-09-09	61	1975-12-28	0
2	10	12345678910	ÜYE ADI 10	ÜYE SOYADI 10	ÜYE ADRES 10	77	14	0001234567	uye10@hotmail.com	1999-10-10	22	2014-11-22	1
3	7	12345678907	ÜYE ADI 7	ÜYE SOYADI 7	ÜYE ADRES 7	35	8	0001234567	uye7@hotmail.com	1988-07-07	33	1998-03-09	1
4	1	12345678901	ÜYE ADI 1	ÜYE SOYADI 1	ÜYE ADRES 1	34	1	0001234567	uye1@hotmail.com	1996-01-01	25	2006-03-22	1
5	2	12345678902	ÜYE ADI 2	ÜYE SOYADI 2	ÜYE ADRES 2	34	2	0001234567	uye2@hotmail.com	1995-02-02	26	2005-12-03	1
6	3	12345678903	ÜYE ADI 3	ÜYE SOYADI 3	ÜYE ADRES 3	34	3	0001234567	uye3@hotmail.com	1997-03-03	24	2004-10-01	1
7	5	12345678905	ÜYE ADI 5	ÜYE SOYADI 5	NULL	34	4	NULL	uye5@hotmail.com	1998-05-05	23	2006-07-17	1
8	8	12345678908	ÜYE ADI 8	ÜYE SOYADI 8	ÜYE ADRES 8	16	11	0001234567	uye8@hotmail.com	1972-08-08	49	1985-01-14	0
9	4	12345678904	ÜYE ADI 4	ÜYE SOYADI 4	ÜYE ADRES 4	06	5	0001234567	uye4@hotmail.com	1994-04-04	27	2003-11-08	1
10	6	12345678906	ÜYE ADI 6	ÜYE SOYADI 6	ÜYE ADRES 6	06	6	0001234567	uye6@hotmail.com	1986-06-06	35	1996-05-11	1

```
SELECT TOP 5 WITH TIES *  
FROM [kutuphane].[dbo].[tblUye]  
ORDER BY IlKodu DESC, IlceKodu
```

Results		Messages											
	ID	TCNO	Ad	Soyad	Adres	IlKodu	IlceKodu	Tel	Mail	DTarihi	Yas	UyeOlusTarihi	AktifMi
1	9	12345678909	ÜYE ADI 9	ÜYE SOYADI 9	ÜYE ADRES 9	77	13	0001234567	uye9@hotmail.com	1960-09-09	61	1975-12-28	0
2	10	12345678910	ÜYE ADI 10	ÜYE SOYADI 10	ÜYE ADRES 10	77	14	0001234567	uye10@hotmail.com	1999-10-10	22	2014-11-22	1
3	7	12345678907	ÜYE ADI 7	ÜYE SOYADI 7	ÜYE ADRES 7	35	8	0001234567	uye7@hotmail.com	1988-07-07	33	1998-03-09	1
4	1	12345678901	ÜYE ADI 1	ÜYE SOYADI 1	ÜYE ADRES 1	34	1	0001234567	uye1@hotmail.com	1996-01-01	25	2006-03-22	1
5	2	12345678902	ÜYE ADI 2	ÜYE SOYADI 2	ÜYE ADRES 2	34	2	0001234567	uye2@hotmail.com	1995-02-02	26	2005-12-03	1

SELECT – TOP WITH TIES

```
SELECT *  
FROM [kutuphane].[dbo].[tblUye]  
ORDER BY IlKodu DESC, IlceKodu
```

Results		Messages											
	ID	TCNO	Ad	Soyad	Adres	IlKodu	IlceKodu	Tel	Mail	DTarihi	Yas	UyeOlusTarihi	AktifMi
1	9	12345678909	ÜYE ADI 9	ÜYE SOYADI 9	ÜYE ADRES 9	77	13	0001234567	uye9@hotmail.com	1960-09-09	61	1975-12-28	0
2	10	12345678910	ÜYE ADI 10	ÜYE SOYADI 10	ÜYE ADRES 10	77	14	0001234567	uye10@hotmail.com	1999-10-10	22	2014-11-22	1
3	7	12345678907	ÜYE ADI 7	ÜYE SOYADI 7	ÜYE ADRES 7	35	8	0001234567	uye7@hotmail.com	1988-07-07	33	1998-03-09	1
4	1	12345678901	ÜYE ADI 1	ÜYE SOYADI 1	ÜYE ADRES 1	34	1	0001234567	uye1@hotmail.com	1996-01-01	25	2006-03-22	1
5	2	12345678902	ÜYE ADI 2	ÜYE SOYADI 2	ÜYE ADRES 2	34	2	0001234567	uye2@hotmail.com	1995-02-02	26	2005-12-03	1
6	3	12345678903	ÜYE ADI 3	ÜYE SOYADI 3	ÜYE ADRES 3	34	3	0001234567	uye3@hotmail.com	1997-03-03	24	2004-10-01	1
7	5	12345678905	ÜYE ADI 5	ÜYE SOYADI 5	NULL	34	4	NULL	uye5@hotmail.com	1998-05-05	23	2006-07-17	1
8	8	12345678908	ÜYE ADI 8	ÜYE SOYADI 8	ÜYE ADRES 8	16	11	0001234567	uye8@hotmail.com	1972-08-08	49	1985-01-14	0
9	4	12345678904	ÜYE ADI 4	ÜYE SOYADI 4	ÜYE ADRES 4	06	5	0001234567	uye4@hotmail.com	1994-04-04	27	2003-11-08	1
10	6	12345678906	ÜYE ADI 6	ÜYE SOYADI 6	ÜYE ADRES 6	06	6	0001234567	uye6@hotmail.com	1986-06-06	35	1996-05-11	1

```
SELECT TOP 5 WITH TIES *  
FROM [kutuphane].[dbo].[tblUye]  
ORDER BY IlKodu DESC
```

Results		Messages											
	ID	TCNO	Ad	Soyad	Adres	IlKodu	IlceKodu	Tel	Mail	DTarihi	Yas	UyeOlusTarihi	AktifMi
1	9	12345678909	ÜYE ADI 9	ÜYE SOYADI 9	ÜYE ADRES 9	77	13	0001234567	uye9@hotmail.com	1960-09-09	61	1975-12-28	0
2	10	12345678910	ÜYE ADI 10	ÜYE SOYADI 10	ÜYE ADRES 10	77	14	0001234567	uye10@hotmail.com	1999-10-10	22	2014-11-22	1
3	7	12345678907	ÜYE ADI 7	ÜYE SOYADI 7	ÜYE ADRES 7	35	8	0001234567	uye7@hotmail.com	1988-07-07	33	1998-03-09	1
4	5	12345678905	ÜYE ADI 5	ÜYE SOYADI 5	NULL	34	4	NULL	uye5@hotmail.com	1998-05-05	23	2006-07-17	1
5	1	12345678901	ÜYE ADI 1	ÜYE SOYADI 1	ÜYE ADRES 1	34	1	0001234567	uye1@hotmail.com	1996-01-01	25	2006-03-22	1
6	2	12345678902	ÜYE ADI 2	ÜYE SOYADI 2	ÜYE ADRES 2	34	2	0001234567	uye2@hotmail.com	1995-02-02	26	2005-12-03	1
7	3	12345678903	ÜYE ADI 3	ÜYE SOYADI 3	ÜYE ADRES 3	34	3	0001234567	uye3@hotmail.com	1997-03-03	24	2004-10-01	1

SELECT INTO

- Sorgu sonucunun yeni bir tabloya yazılması amacıyla kullanılır
- Belirtilen tablo veri tabanında var olmayan bir tablo ismi olmalıdır
 - SELECT INTO mevcut bir tabloya veri eklemek için kullanılamaz
- Oluşturulan tablo, kaynak tablodaki kısıtları (PK, FK, INDEX) otomatik olarak almaz

```
]SELECT TCNO, Ad, Soyad  
INTO tblUyeYalova  
FROM tblUye  
Where IlKodu = '77'
```

```
SELECT * FROM [kutuphane].[dbo].[tblUye]
```

```
SELECT * FROM [kutuphane].[dbo].[tblUyeYalova]
```

Results

Messages

	ID	TCNO	Ad	Soyad	Adres	IlKodu	IlceKodu	Tel	Mail	DTarihi	Yas	UyeOlusTarihi	AktifMi
1	1	12345678901	ÜYE ADI 1	ÜYE SOYADI 1	ÜYE ADRES 1	34	1	0001234567	uye1@hotmail.com	1996-01-01	22	2006-03-22	1
2	2	12345678902	ÜYE ADI 2	ÜYE SOYADI 2	ÜYE ADRES 2	34	2	0001234567	uye2@hotmail.com	1995-02-02	23	2005-12-03	1
3	3	12345678903	ÜYE ADI 3	ÜYE SOYADI 3	ÜYE ADRES 3	34	3	0001234567	uye3@hotmail.com	1997-03-03	21	2004-10-01	1
4	4	12345678904	ÜYE ADI 4	ÜYE SOYADI 4	ÜYE ADRES 4	06	5	0001234567	uye4@hotmail.com	1994-04-04	24	2003-11-08	1
5	5	12345678905	ÜYE ADI 5	ÜYE SOYADI 5	NULL	34	4	NULL	uye5@hotmail.com	1998-05-05	20	2006-07-17	1
6	6	12345678906	ÜYE ADI 6	ÜYE SOYADI 6	ÜYE ADRES 6	06	6	0001234567	uye6@hotmail.com	1986-06-06	32	1996-05-11	1
7	7	12345678907	ÜYE ADI 7	ÜYE SOYADI 7	ÜYE ADRES 7	35	8	0001234567	uye7@hotmail.com	1988-07-07	30	1998-03-09	1
8	8	12345678908	ÜYE ADI 8	ÜYE SOYADI 8	ÜYE ADRES 8	16	11	0001234567	uye8@hotmail.com	1972-08-08	46	1985-01-14	0
9	9	12345678909	ÜYE ADI 9	ÜYE SOYADI 9	ÜYE ADRES 9	77	13	0001234567	uye9@hotmail.com	1960-09-09	58	1975-12-28	0
10	10	12345678910	ÜYE ADI 10	ÜYE SOYADI 10	ÜYE ADRES 10	77	14	0001234567	uye10@hotmail.com	1999-10-10	19	2014-11-22	1

	TCNO	Ad	Soyad
1	12345678909	ÜYE ADI 9	ÜYE SOYADI 9
2	12345678910	ÜYE ADI 10	ÜYE SOYADI 10

SELECT INTO

- Bir tablonun şemasının kopyasını almak amacıyla da kullanılabilir

```
SELECT * INTO tblUye2 FROM tblUye WHERE 1 = 0
```

```
SELECT * FROM tblUye2
```

Results		Messages											
ID	TCNO	Ad	Soyad	Adres	IlKodu	IlceKodu	Tel	Mail	DTarihi	Yas	UyeOlusTarihi	AktifMi	

INSERT + SELECT

- Eğer sorgu sonucu zaten var olan bir tabloya kaydedilmek istenirse INSERT komutu ile beraber SELECT kullanılabilir.
- Özellikle kayıt işlemi için gerekli olan değerlerin belirli tablolardan okunarak alınması gerektiği durumlarda kullanılır
- Örneğin bir yazar kaydı silineceği zaman yedek bir tabloya tüm bilgilerinin aktarılması istenmiş olsun

```
-- INSERT INTO table SELECT
CREATE TABLE tblYazarSilinen
(
    ID INT PRIMARY KEY,
    Ad VARCHAR(80) UNIQUE NOT NULL,
    DTarihi DATE,
    IlKodu CHAR(2) FOREIGN KEY REFERENCES tblIl(Kod),
    IlceKodu INT FOREIGN KEY REFERENCES tblIlce(ID)
)
GO
```

```
INSERT INTO tblYazarSilinen SELECT * FROM tblYazar WHERE ID = 7
```

```
SELECT * FROM tblYazarSilinen
```

Results		Messages			
	ID	Ad	DTarihi	IlKodu	IlceKodu
1	7	Yazar 7	1979-07-31	77	13

SQL Server'da Gömülü (Built-in) Fonksiyonlar

- Belirli bir işlemi gerçekleştirmek üzere önceden yazılmış, parametre alabilen ve geriye tek bir değer (skaler) veya bir tablo seti döndüren alt programlara fonksiyon denir
- Temel Amaç
 - Kod tekrarını önlemek (Don't repeat yourself, DRY)
 - Karmaşık hesaplamaları sunucu tarafında (Server-side) yaparak ağ trafiğini azaltmak
 - Veriyi sorgulama anında formatlamak veya dönüştürmek
- Neden Fonksiyonlara İhtiyacımız Var?
 - Veri Temizliği (Data Cleansing)
 - NULL değerleri yönetmek veya hatalı girilen metinleri düzeltmek (örn: UPPER, TRIM)
 - Raporlama ve Analiz
 - Satış toplamaları, ortalamalar veya tarih bazlı gruplamalar yapmak (örn: SUM, AVG)
 - Dinamik Sorgular
 - O anki zamanı almak veya veri tiplerini birbirine dönüştürmek (örn: GETDATE, CAST).

SQL Server'da Gömülü (Built-in) Fonksiyonlar

- SQL Server Fonksiyon Kategorileri

- **Matematiksel (Mathematical) Fonksiyonlar**

- Sayısal işlemler ve hesaplamalar (ABS, POWER, ROUND...)

- **Karakter/Metin Fonksiyonları**

- Metin manipülasyonu ve biçimlendirme (SUBSTRING, LEN, REPLACE...)

- **Zaman (Date/Time) Fonksiyonları**

- Tarih ve saat üzerindeki aritmetik işlemler (DATEDIFF, DATEADD...)

- **Tip Dönüşüm (Conversion) Fonksiyonları**

- Veri tipleri arasında güvenli geçiş (CAST, CONVERT...)

- **NULL Değer Fonksiyonları**

- Eksik verilerin yönetimi ve varsayılan değer atama (ISNULL, COALESCE...)

- **Gruplama (Aggregate) Fonksiyonları**

- Çoklu satırları tek bir özet değere indirgemek (SUM, COUNT...)

SQL Server'da Gömülü (Built-in) Fonksiyonlar

- Skaler vs. Aggregate

- **Skaler (Scalar) Fonksiyonlar**

- Her bir satır için ayrı ayrı çalışır
 - Girdi: 1 Satır -> Çıktı: 1 Değer
 - Örnek: UPPER(Ad) fonksiyonu 100 satırlık bir tabloda 100 kez çalışır

- **Gruplama (Aggregate) Fonksiyonları**

- Birçok satırı alıp tek bir sonuç üretir
 - Girdi: N Satır -> Çıktı: 1 Değer
 - Örnek: AVG(Maas) fonksiyonu 100 çalışanın maaşını alır, tek bir ortalama değer döndürür

Matematiksel Fonksiyonlar – Trigonometri

- SQL Server'da trigonometrik fonksiyonlar matematiksel hesaplamalar, geometrik işlemler ve özellikle mekansal (spatial) veri analizlerinde sıkça kullanılır.
- Tüm trigonometrik fonksiyonlar varsayılan olarak radyan cinsinden değer kabul eder.
- Eldeki derece değerleri (30°, 90° vb.) hesaplamaya sokulmadan önce mutlaka radyana dönüştürülmelidir.
 - ***Radyan = Derece × PI() / 180***
- Hazır (built-in) derece/radyan dönüştürme fonksiyonları
 - ***RADIANS (numeric_expression)***: Derece → Radyan
 - ***DEGREES (numeric_expression)***: Radyan → Derece
 - Bu fonksiyonların dönüş tipi parametre olarak verilen sayısal ifadenin veri tipine bağlıdır.
 - Sonuç, dönüş türüne uymuyorsa, aritmetik taşma hatası oluşur.
- **PI()**
 - Matematiksel sabit π (pi) değerini döndüren bir fonksiyondur.
 - SQL Server, π 'yi 15 ondalık basamak hassasiyetle (float tipi) saklar

Input type	Return type
float, real	float
decimal(<i>p</i> , <i>s</i>)	decimal(38, <i>s</i>)
int, smallint, tinyint	int
bigint	bigint
money, smallmoney	money
bit	float

Matematiksel Fonksiyonlar – Trigonometri

- **Temel Fonksiyonlar (SIN, COS, TAN, COT)**

- Bu fonksiyonlar radyan cinsinden verilen açının (*float_expression*) ilgili trigonometrik fonksiyon değerini döndürürler

- *SIN (float_expression)*
- *COS (float_expression)*
- *TAN (float_expression)*
- *COT (float_expression)*

```
SELECT  SIN(RADIANS(30.0)) AS [Sinus_30],  
        COS(RADIANS(60.0)) AS [Cosinus_60],  
        TAN(RADIANS(45.0)) AS [Tanjant_45],  
        COT(RADIANS(45.0)) AS [Kotanjant_45]
```

Sinus_30	Cosinus_60	Tanjant_45	Kotanjant_45
0.5	0.5	1	1

Matematiksel Fonksiyonlar – Trigonometri

- **Ters (Arc) Fonksiyonlar (ASIN, ACOS, ATAN)**

- Bu fonksiyonlar bir sayı alıp bu sayıyı sağlayan açıyı radyan cinsinden döndürürler.
 - *ASIN (float_expression)*: Değeri -1 ile 1 arasında olmalıdır.
 - *ACOS (float_expression)*: Değeri -1 ile 1 arasında olmalıdır.
 - *ATAN (float_expression)*: Herhangi bir sayı alabilir.
 - *ATN2 (float_expression_y , float_expression_x)*: Kartezyen koordinat sisteminde (x, y) noktasının orijine (0,0) göre oluşturduğu açıyı hesaplar.

```
SELECT  DEGREES(ASIN(0.5))      AS [Sinusu_0.5_Olan_Aci],
        DEGREES(ACOS(0.5))      AS [Cosinusu_0.5_Olan_Aci],
        DEGREES(ATAN(1))        AS [Tanjanti_1_Olan_Aci],
        DEGREES(ATN2(1, -1))    AS [Koordinata_Gore_Yon]
```

Sinusu_0.5_Olan_Aci	Cosinusu_0.5_Olan_Aci	Tanjanti_1_Olan_Aci	Koordinata_Gore_Yon
30	60	45	135

Matematiksel Fonksiyonlar – Yuvarlama

• CEILING

- Sayıyı, kendisinden büyük veya eşit olan en küçük tam sayıya yuvarlar
 - Sayı doğrusunda her zaman sağa doğru hareket eder.
- **Sözdizimi (Syntax)**
CEILING (numeric_expression)
- **Argümanlar**
 - *numeric_expression*: Tam sayısal (exact numeric) veya yaklaşık sayısal (approximate numeric) veri türü kategorisinin bir ifadesi
- **Dönüş Tipi**
Dönüş veri tipi, parametre olarak verilen sayısal ifadenin veri tipine bağlıdır

C1	C2	C3	C4	C5	C6	C7
3	4	4	-3	-3	-3	2.00

Input type	Return type
float, real	float
decimal(<i>p</i> , <i>s</i>)	decimal(<i>p</i> , 0)
int, smallint, tinyint	int
bigint	bigint
money, smallmoney	money
bit	float

```
SELECT  CEILING(3.00)    AS C1,  
        CEILING(3.01)    AS C2,  
        CEILING(3.99)    AS C3,  
        CEILING(-3.00)   AS C4,  
        CEILING(-3.01)   AS C5,  
        CEILING(-3.99)   AS C6,  
        CEILING($1.23)   AS C7
```

Matematiksel Fonksiyonlar – Yuvarlama

• FLOOR

- Sayıyı, kendisinden küçük veya eşit olan en büyük tam sayıya yuvarlar
 - Sayı doğrusunda her zaman sola doğru hareket eder.
- **Sözdizimi (Syntax)**
FLOOR (numeric_expression)
- **Argümanlar**
 - *numeric_expression*: Tam sayısal (exact numeric) veya yaklaşık sayısal (approximate numeric) veri türü kategorisinin bir ifadesi
- **Dönüş Tipi**
Dönüş veri tipi, parametre olarak verilen sayısal ifadenin veri tipine bağlıdır

C1	C2	C3	C4	C5	C6	C7
3	3	3	-3	-4	-4	1.00

Input type	Return type
float, real	float
decimal(<i>p</i> , <i>s</i>)	decimal(<i>p</i> , 0)
int, smallint, tinyint	int
bigint	bigint
money, smallmoney	money
bit	float

```
SELECT  FLOOR(3.00)    AS C1,  
        FLOOR(3.01)    AS C2,  
        FLOOR(3.99)    AS C3,  
        FLOOR(-3.00)   AS C4,  
        FLOOR(-3.01)   AS C5,  
        FLOOR(-3.99)   AS C6,  
        FLOOR($1.23)   AS C7
```


Matematiksel Fonksiyonlar – Yuvarlama

- **ROUND**

- Sayıyı, belirtilen hassasiyete göre yuvarlar veya kırpar
- **Sözdizimi (Syntax)**
ROUND (numeric_expression, length [, function])
- **Argümanlar**
 - *numeric_expression*: Tam sayısal (exact numeric) veya yaklaşık sayısal (approximate numeric) veri türü kategorisinin bir ifadesi
 - *length*: Sayısal ifadenin yuvarlanacağı hassasiyet
 - ❑ Pozitif → virgülden sonra yuvarlama
 - ❑ 0 → Sayıyı birler basamağında yuvarlar (Ondalık kısmı ≥ 0.5 ise yukarı, < 0.5 ise aşağı yuvarlanır)
 - ❑ Negatif → virgülden önce (en yakın onluğa, en yakın yüzlüğe vb.) yuvarlama
 - *function*: Opsiyonel
 - ❑ 0 veya verilmezse → klasik yuvarlama
 - ❑ 1 → kesme/truncate (yuvarlamadan basamakları kırpar)
- **Dönüş Tipi**
Yandaki tabloda gösterilen veri tiplerini döner

Expression result	Return type
tinyint	int
smallint	int
int	int
bigint	bigint
decimal and numeric category (p, s)	decimal(p, s)
money and smallmoney category	money
float and real category	float

Matematiksel Fonksiyonlar – Yuvarlama

- ROUND

```
SELECT  ROUND(12.3456, 2)      AS R1,  
        ROUND(12.3456, 1)      AS R2,  
        ROUND(12.49, 0)        AS R3,  
        ROUND(12.50, 0)        AS R4,  
        ROUND(12.3456, 2, 1)   AS R5,  
        ROUND(1254.0, -1)      AS R6,  
        ROUND(1256.0, -1)      AS R7,  
        ROUND(1256.0, -2)      AS R8
```

R1	R2	R3	R4	R5	R6	R7	R8
12.3500	12.3000	12.00	13.00	12.3400	1250.0	1260.0	1300.0

Matematiksel Fonksiyonlar – Diğer

- **ABS**

- Verilen sayının mutlak değerini döndürür
- **Sözdizimi (Syntax)**
ABS (numeric_expression)
- **Argümanlar**
 - *numeric_expression*: Tam sayısal (exact numeric) veya yaklaşık sayısal (approximate numeric) veri türü kategorisinin bir ifadesi
- **Dönüş Tipi**
Dönüş veri tipi, parametre olarak verilen sayısal ifadenin veri tipine bağlıdır

```
SELECT  ABS(-12)  AS AbsNeg,  
        ABS(12.5) AS AbsPos,  
        ABS(0)    AS AbsZero,  
        ABS(NULL) AS Abs4
```

Input type	Return type
float, real	float
decimal(<i>p</i> , <i>s</i>)	decimal(38, <i>s</i>)
int, smallint, tinyint	int
bigint	bigint
money, smallmoney	money
bit	float

AbsNeg	AbsPos	AbsZero	Abs4
12	12.5	0	NULL

Matematiksel Fonksiyonlar – Diğer

- **EXP**

- Matematiksel bir sabit olan Euler ($e \cong 2.71828182845904$) sayısının belirtilen bir kuvvetini ($EXP(x) = e^x$) hesaplayan fonksiyondur.
- Bu işlem, doğal logaritmanın (LOG) tersidir.
- Üs alma işlemi yapıldığı için çok hızlı büyüyen sonuçlar üretir.
 - SQL Server'ın float veri tipi üst sınırı aşıldığında "arithmetic overflow error" (aritmetik taşma hatası) alınır

- **Sözdizimi (Syntax)**

EXP (float_expression)

- **Argümanlar**

- *float_expression*: float tipinde veya float tipine örtük (implicit) olarak dönüştürülebilen sayısal bir değer

- **Dönüş Tipi**

float

```
SELECT  EXP(0) AS Exp_0,  
        EXP(1) AS Exp_2,  
        EXP(2) AS Exp_2,  
        EXP(-1) AS Exp_Negatif1,  
        EXP(NULL) AS Exp_NULL
```

Exp_0	Exp_2	Exp_2	Exp_Negatif1	Exp_NULL
1	2.71828182845905	7.38905609893065	0.367879441171442	NULL

Matematiksel Fonksiyonlar – Diğer

- **LOG**

- Verilen sayının doğal logaritmasını (e tabanında) döndürür
- İki parametrelili kullanımda ise istenen tabanda logaritma alınabilir
- **Sözdizimi (Syntax)**
LOG (float_expression [, base])
- **Argümanlar**
 - *float_expression*: float tipinde veya float tipine örtük (implicit) olarak dönüştürülebilen sayısal bir değer. 0'dan büyük olmalıdır
 - *base*: Logaritmanın tabanını belirleyen isteğe bağlı tamsayı argüman. Pozitif ve 1'e eşit olmamalıdır

- **Dönüş Tipi**

float

```
SELECT  LOG(10.0)           AS Ln10,  
        LOG(8.0, 2.0)      AS Log2_8,  
        LOG(100.0, 10.0)   AS Log10_100
```

Ln10	Log2_8	Log10_100
2.30258509299405	3	2

Matematiksel Fonksiyonlar – Diğer

- **LOG10**

- Verilen sayının 10 tabanında (base-10) logaritmasını döndürür
- **Sözdizimi (Syntax)**
LOG10 (float_expression)
- **Argümanlar**
 - *float_expression*: float tipinde veya float tipine örtük (implicit) olarak dönüştürülebilen sayısal bir değer. 0'dan büyük olmalıdır
- **Dönüş Tipi**
float

```
SELECT  LOG10(10.0) AS Log10_10,  
        LOG10(100)  AS Log10_100,  
        LOG10(5)    AS Log10_5
```

Log10_10	Log10_100	Log10_5
1	2	0.698970004336019

Matematiksel Fonksiyonlar – Diğer

- **POWER**

- Verilen sayının belirtilen kuvvetini hesaplar
- **Sözdizimi (Syntax)**
POWER (float_expression , y)
- **Argümanlar**
 - *float_expression*: float tipinde veya float tipine örtük (implicit) olarak dönüştürülebilen sayısal bir değer
 - *y*: float ifadenin kaçınıcı kuvvetinin alınacağını belirten değer
- **Dönüş Tipi**
Dönüş veri tipi, parametre olarak verilen float ifadenin veri tipine bağlıdır

Input type	Return type
float, real	float
decimal(<i>p</i> , <i>s</i>)	decimal(38, <i>s</i>)
int, smallint, tinyint	int
bigint	bigint
money, smallmoney	money
bit, char, nchar, varchar, nvarchar	float

```
SELECT  POWER(2.0, 3.0)      AS P1,  
        POWER(9.0, 0.5)     AS P2,  
        POWER(5.0, 0.0)     AS P3,  
        POWER(0.123, -1.9)  AS P4
```

P1	P2	P3	P4
8.0	3.0	1.0	53.602

Matematiksel Fonksiyonlar – Diğer

- **RAND**

- 0 ile 1 arasında (0 dahil, 1 hariç) pseudo-rastgele bir float değer üretir
- $[a - b)$ aralığında ($x \in \mathbb{R} \mid a \leq x < b$) bir sayı üretilmek istenirse
$$a + rand() * (b - a)$$
- $[a - b]$ kapalı aralığında bir tamsayı ($x \in \mathbb{Z} \mid a \leq x \leq b$) üretilmek istenirse
$$\text{floor}(a + rand() * (b - a + 1))$$
- Sorgu başına bir kez çalışır, satır başına değil
 - Sorgu sonucunun rastgele sıralı olması için kullanılamaz
- **Sözdizimi (Syntax)**
$$RAND ([seed])$$
- **Argümanlar**
 - *seed*: Opsiyonel
 - ☐ Rastgele sayı üretimini başlatan tam sayı bir başlangıç değeridir
 - ☐ Aynı seed değeri için, fonksiyon her seferinde aynı sonucu döndürür (Deterministic Randomness)
 - ☐ Eğer seed belirtilmezse, SQL Server rastgele bir başlangıç değeri atar
- **Dönüş Tipi**
float

Matematiksel Fonksiyonlar – Diğer

- RAND

```
SELECT rand(77) seed77_1, rand(77) seed77_2, rand(77) seed77_3
```

```
SELECT rand(77) seed77_4, rand(77) seed77_5, rand(77) seed77_6
```

```
SELECT rand() noseed_1, rand() noseed_2, rand() noseed_3
```

```
SELECT rand() noseed_4, rand() noseed_5, rand() noseed_6
```

seed77_1	seed77_2	seed77_3
0.715008099028547	0.715008099028547	0.715008099028547

seed77_4	seed77_5	seed77_6
0.715008099028547	0.715008099028547	0.715008099028547

noseed_1	noseed_2	noseed_3
0.136300223869282	0.59300815300568	0.576451634402052

noseed_4	noseed_5	noseed_6
0.257955694151506	0.506087114858746	0.848246882506738

Matematiksel Fonksiyonlar – Diğer

- RAND

```
SELECT  RAND() * (10 - 5) + 5          AS RAND_5_10_R,  
        FLOOR(RAND() * (12 - 1 + 1) + 1) AS RAND_1_12_Z
```

RAND_5_10_R	RAND_1_12_Z
9.30141062325484	7

Matematiksel Fonksiyonlar – Diğer

- **SIGN**

- Verilen sayının aritmetik işaretini belirlemek için kullanılır

$$SIGN(x) = \begin{cases} -1 & x < 0 \\ 0 & x = 0 \\ 1 & x > 0 \end{cases}$$

- **Sözdizimi (Syntax)**

SIGN (numeric_expression)

- **Argümanlar**

➤ *numeric_expression*: Tam sayısal (exact numeric) veya yaklaşık sayısal (approximate numeric) veri türü kategorisinin bir ifadesi (bit veri tipi hariç)

- **Dönüş Tipi**

Dönüş veri tipi, parametre olarak verilen sayısal ifadenin veri tipine bağlıdır

Specified expression	Return type
bigint	bigint
int/smallint/tinyint	int
money/smallmoney	money
numeric/decimal	numeric/decimal
Other types	float

```
SELECT  SIGN(-25)    AS Neg,  
        SIGN(12.27) AS Poz,  
        SIGN(0)     AS Sifir
```

Neg	Poz	Sifir
-1	1.00	0

Matematiksel Fonksiyonlar – Diğer

- **SQUARE**

- Verilen sayının karesini döndürür
- **Sözdizimi (Syntax)**
SQUARE (float_expression)
- **Argümanlar**
 - *float_expression*: float tipinde veya float tipine örtük (implicit) olarak dönüştürülebilen sayısal bir değer
- **Dönüş Tipi**
float

```
SELECT  SQUARE(5.0)    AS S1,  
        SQUARE(-4.0)   AS S2
```

S1	S2
25	16

Matematiksel Fonksiyonlar – Diğer

- **SQRT**

- Verilen sayının karekökünü döndürür
- **Sözdizimi (Syntax)**
SQRT (float_expression)
- **Argümanlar**
 - *float_expression*: float tipinde veya float tipine örtük (implicit) olarak dönüştürülebilen sayısal bir değer. Negatif bir sayı girilirse SQL Server "Invalid floating point operation" hatası verir.
- **Dönüş Tipi**
float

```
SELECT  SQRT(16.0) AS Q1,  
        SQRT(2.0)  AS Q2
```

Q1	Q2
4	1.4142135623731

Karakter/Metin Fonksiyonları

- Metinsel veriler üzerinde işlem yapmak, biçimlendirmek, parçalamak ve analiz etmek için kullanılır
- Türleri
 - **Uzunluk**
 - LEN, DATALENGTH
 - **Büyük/Küçük Harf Dönüşümü**
 - UPPER, LOWER
 - **Birleştirme**
 - CONCAT, CONCAT_WS
 - **Metin Parçası Alma**
 - SUBSTRING, LEFT, RIGHT
 - **Metin Arama**
 - CHARINDEX, PATINDEX
 - **Metin Düzenleme ve Temizleme**
 - LTRIM, RTRIM, TRIM, STUFF, REPLACE, REVERSE, REPLICATE, QUOTENAME, STRING_ESCAPE, TRANSLATE
 - **Karakter Kodları**
 - ASCII, UNICODE, CHAR, NCHAR, SPACE
 - **Dönüştürme**
 - STR
 - **Biçimlendirme**
 - FORMAT

Karakter/Metin Fonksiyonları – Uzunluk

- **LEN**
 - Verilen metnin karakter sayısını döndürür (sondaki boşlukları saymaz)
 - **Sözdizimi (Syntax)**
LEN (string_expression)
 - **Argümanlar**
 - **string_expression**: Değerlendirilecek string ifade
 - **Dönüş Tipi**
Eğer ifade varchar(max), nvarchar(max) veya varbinary(max) veri tiplerinden biriye bigint; aksi takdirde int

```
SELECT LEN('SQL Server ') AS Uzunluk
```

Uzunluk
10

Karakter/Metin Fonksiyonları – Uzunluk

- **DATALENGTH**

- Verilen ifadeyi temsil etmek için kullanılan bayt sayısını döndürür
- LEN fonksiyonu sondaki boşlukları saymaz; DATALENGTH ise byte cinsinden gerçek uzunluğu döndürür
- **Sözdizimi (Syntax)**
DATALENGTH (expression)
- **Argümanlar**
 - **expression**: Herhangi bir veri türündeki bir ifade
- **Dönüş Tipi**
Eğer ifade nvarchar(max), varbinary(max) veya varchar(max) veri türüne sahipse bigint; aksi takdirde int

```
-- her ikisinde de sonda üç boşluk var
SELECT  LEN('  merhaba  yalova  ')      AS LEN_19,
        DATALENGTH('  merhaba  yalova  ') AS DL_22
```

LEN_19	DL_22
19	22

Karakter/Metin Fonksiyonları – Büyük/Küçük Harf Dönüşümü

- **LOWER**

- Verilen metnin tüm karakterlerini küçük harfe çevirir
- **Sözdizimi (Syntax)**
LOWER (character_expression)
- **Argümanlar**
 - *character_expression*: İşlem uygulanacak metin
- **Dönüş Tipi**
varchar

```
SELECT LOWER('SQL SERver')
```

(Sütun adı yok)

sql server

Karakter/Metin Fonksiyonları – Büyük/Küçük Harf Dönüşümü

- **UPPER**

- Verilen metnin tüm karakterlerini büyük harfe çevirir
- **Sözdizimi (Syntax)**
UPPER (character_expression)
- **Argümanlar**
 - *character_expression*: İşlem uygulanacak metin
- **Dönüş Tipi**
varchar

```
SELECT UPPER('sql serVER')
```

(Sütun adı yok)

SQL SERVER

Karakter/Metin Fonksiyonları – Birleştirme

- **CONCAT**

- Verilen metinleri birleştirir
- NULL değerleri boş string olarak kabul eder ki bu da + operatörüne göre daha güvenlidir
- Eğer sayısal değer verilirse otomatik string'e çevirir
- **Sözdizimi (Syntax)**
CONCAT (argument1 , argument2 [, argumentN] ...)
- **Argümanlar**
 - Herhangi bir string ifadesi
 - En az iki, en fazla 254 argüman gerektirir
- **Dönüş Tipi**
Uzunluğu ve türü girdiye bağlı olan bir string değeri

```
SELECT CONCAT('Ad: ', 'Ahmet', ' Soyad: ', 'Yılmaz', ' Bakiye:', 1234.56)
```

(Sütun adı yok)

Ad: Ahmet Soyad: Yılmaz Bakiye:1234.56

Karakter/Metin Fonksiyonları – Birleştirme

- **CONCAT_WS (SQL Server 2017+)**

- Verilen metinleri aralarına belirli bir ayırıcı koyarak birleştirir
- NULL değerleri boş string olarak kabul eder ki bu da + operatörüne göre daha güvenlidir
- **Sözdizimi (Syntax)**
CONCAT_WS (separator , argument1 , argument2 [, argumentN] ...)
- **Argümanlar**
 - **separator**: Herhangi bir karakter türünde (char, nchar, nvarchar veya varchar) bir ifade
 - **argument1, argument2 [, argumentN]**: Herhangi bir string ifade. En az iki, en fazla 254 argüman gerektirir
- **Dönüş Tipi**
Uzunluğu ve türü girdiye bağlı olan bir string değeri

```
SELECT CONCAT_WS('-', '2026', '02', '12')
```

(Sütun adı yok)

2026-02-12

Karakter/Metin Fonksiyonları – Metin Parçası Alma

• SUBSTRING

- Verilen metnin belirli bir konumundan başlayarak belirtilen uzunluk kadar kısmını döndürür.
- Filtreleme ve sıralama işlemlerinde INDEX kullanılmasına engel olabileceği için yerine LIKE operatörü tercih edilmelidir.
- **Sözdizimi (Syntax)**
SUBSTRING (expression , start , length)
- **Argümanlar**
 - **expression:** Metin olarak ifade
 - **start:** Başlangıç pozisyonu (1'den başlar)
 - **length:** Kaç karakter alınacağı (negatif indeks kullanılamaz)
- **Dönüş Tipi**
Uzunluğu ve türü girdiye bağlı

Specified expression	Return type
char / varchar / text	varchar
nchar / nvarchar / ntext	nvarchar
binary / varbinary / image	varbinary

```
SELECT SUBSTRING('xyzabc', 2, 3) AS Parca1,  
SUBSTRING('xyzabc', 2, 0) AS Parca2,  
SUBSTRING('xyzabc', 2, 9) AS Parca3,  
SUBSTRING('xyzabc', 8, 9) AS Parca4,  
SUBSTRING('xyzabc', 2, NULL) AS Parca5
```

INDEX – 1	INDEX – 2	INDEX – 3	INDEX – 4	INDEX – 5	INDEX – 6
x	y	z	a	b	c

Parca1	Parca2	Parca3	Parca4	Parca5
yza		yzabc		NULL

Karakter/Metin Fonksiyonları – Metin Parçası Alma

- **LEFT**

- Verilen metnin başlangıcından (solundan) itibaren belirtilen uzunluk kadar kısmını döndürür
- Metnin başındaki boşluklar karakter olarak sayılır
- Filtreleme ve sıralama işlemlerinde INDEX kullanılmasına engel olabileceği için yerine LIKE operatörü tercih edilmelidir.

- **Sözdizimi (Syntax)**

LEFT (character_expression , integer_expression)

- **Argümanlar**

- **character_expression**: Metin olarak ifade

- **integer**: Metnin solundan kaç karakter alınacağını belirten pozitif tam sayı.

- ☐ Metnin toplam uzunluğundan büyükse metnin tamamını döner

- **Dönüş Tipi**

Girişin veri tipine bağlıdır (varchar → varchar, nvarchar → nvarchar)

```
SELECT  LEFT('xyzabc', 2) AS Soldan1,  
        LEFT('xyzabc', 9) AS Soldan2,  
        LEFT('xyzabc', 0) AS Soldan3,  
        LEFT('xyzabc', NULL) AS Soldan4
```

Soldan1	Soldan2	Soldan3	Soldan4
xy	xyzabc		NULL

Karakter/Metin Fonksiyonları – Metin Parçası Alma

- **RIGHT**

- Verilen metnin sonundan (sağından) başa doğru belirtilen uzunluk kadar kısmını döndürür
- Metnin sonundaki boşluklar karakter olarak sayılır
- Filtreleme ve sıralama işlemlerinde INDEX kullanılmasına engel olabileceği için yerine LIKE operatörü tercih edilmelidir.

- **Sözdizimi (Syntax)**

RIGHT (character_expression , integer_expression)

- **Argümanlar**

- **character_expression**: Metin olarak ifade

- **integer**: Metnin sağından kaç karakter alınacağını belirten pozitif tam sayı.

- ☐ Metnin toplam uzunluğundan büyükse metnin tamamını döner.

- **Dönüş Tipi**

Girişin veri tipine bağlıdır (varchar → varchar, nvarchar → nvarchar)

```
SELECT RIGHT('xyzabc', 2) AS Sagdan1,  
       RIGHT('xyzabc', 9) AS Sagdan2,  
       RIGHT('xyzabc', 0) AS Sagdan3,  
       RIGHT('xyzabc', NULL) AS Sagdan4
```

Sagdan1	Sagdan2	Sagdan3	Sagdan4
bc	xyzabc		NULL

-- sayıların önüne sıfır ekleyerek her zaman

-- 5 haneli görünmesini sağlama

```
SELECT RIGHT('00000' + '123', 5) AS FormatliSayi
```

FormatliSayi

00123

Karakter/Metin Fonksiyonları – Metin Arama

- **CHARINDEX**

- Verilen metnin içerisinde belirli bir karakterin veya alt dizinin ilk geçtiği konumu (pozisyonunu) döndürür
- Eğer arama yaparken "%" veya "_" gibi joker karakterler (wildcards) kullanılması gerekiyorsa CHARINDEX yerine PATINDEX fonksiyonunu kullanmalıdır
- Büyük/Küçük Harf Duyarlılığı
 - Varsayılan olarak sorgunun çalıştığı veri tabanının Collation (karşılaştırma) ayarını izler
 - ❑ Turkish_CI_AS: "Case Insensitive" (Büyük-küçük harf duyarsız) demektir. 'A' ile 'a'yı aynı görür
 - ❑ Turkish_CS_AS: "Case Sensitive" (Büyük-küçük harf duyarlı) demektir. 'A' ile 'a'yı farklı görür
- **Sözdizimi (Syntax)**
CHARINDEX (expressionToFind , expressionToSearch [, start_location])
- **Argümanlar**
 - **expressionToFind:** Bulunacak karakter veya metin bloğu (maksimum 8000 karakter) ifadesi
 - **expressionToSearch:** İçinde arama yapılacak olan metin ifadesi
 - **start_location (isteğe bağlı):** Aramanın başlayacağı pozisyon
 - ❑ Belirtilmezse, sıfır (0) verilirse veya negatif bir sayı verilirse 1. karakterden başlar.
- **Dönüş Tipi**
Girişin veri tipine bağlıdır (nvarchar(max), varbinary(max), veya varchar(max) → bigint, diğer → int)
 - Eğer aranan ifade bulunamazsa 0 döner

Karakter/Metin Fonksiyonları – Metin Arama

- CHARINDEX

```
SELECT CHARINDEX('xy', 'abcxymna') AS CHARINDEX_01,  
       CHARINDEX('m', 'abcxymna') AS CHARINDEX_02,  
       CHARINDEX('a', 'abcxymna', 2) AS CHARINDEX_03,  
       CHARINDEX('a', 'abcxymna', -2) AS CHARINDEX_04,  
       CHARINDEX('z', 'abcxymna') AS CHARINDEX_05,  
       CHARINDEX(NULL, 'abcxymna') AS CHARINDEX_06
```

CHARINDEX_01	CHARINDEX_02	CHARINDEX_03	CHARINDEX_04	CHARINDEX_05	CHARINDEX_06
4	6	8	1	0	NULL

Karakter/Metin Fonksiyonları – Metin Arama

- **PATINDEX**

- Verilen metnin içerisinde belirli bir desene (pattern) uyan ilk eşleşmenin konumunu (pozisyonunu) döndürür
- Büyük/Küçük Harf Duyarlılığı
 - Varsayılan olarak sorgunun çalıştığı veri tabanının Collation (karşılaştırma) ayarını izler
 - ❑ Turkish_CI_AS: "Case Insensitive" (Büyük-küçük harf duyarsız) demektir. 'A' ile 'a'yı aynı görür
 - ❑ Turkish_CS_AS: "Case Sensitive" (Büyük-küçük harf duyarlı) demektir. 'A' ile 'a'yı farklı görür
- **Sözdizimi (Syntax)**
PATINDEX ('%pattern%' , expression)
- **Argümanlar**
 - **pattern**: Aranılan desen. *LIKE* operatöründe olduğu gibi % (herhangi bir karakter dizisi), _ (tek bir karakter), [] (karakter seti) gibi joker karakterler kullanılabilir
 - **expression**: İçinde arama yapılacak metin (başlangıç pozisyonu 1'dir)
- **Dönüş Tipi**
Girişin veri tipine bağlıdır (nvarchar(max) veya varchar(max) → bigint, diğer → int)
 - Eğer aranan ifade bulunamazsa 0 döner
 - Desen NULL ise NULL döner
 - İfade, string tipinde NULL ise sonuç NULL döner, türsüz NULL kullanılırsa hata verir

Karakter/Metin Fonksiyonları – Metin Arama

- PATINDEX

```
SELECT  PATINDEX('%[^A-Za-z]%', 'SQL#Server')    AS NonLetter,  
        PATINDEX('%xyz%', 'Database')           AS NotFound,  
        PATINDEX('%b%m%', 'abcxymn')            AS b_m,  
        PATINDEX (NULL, 'abcxymn')              AS null_  
-- PATINDEX ('%b%m%', NULL) -- hata verir
```

NonLetter	NotFound	b_m	null_
4	0	2	NULL

Karakter/Metin Fonksiyonları – Metin Düzenleme ve Temizleme

- **LTRIM**

- Verilen metnin başındaki (solundaki) boşluk karakterlerini temizler
- SQL Server 2022+ sürümlerinde silinecek karakter de belirtilebilir
- **Sözdizimi (Syntax)**
LTRIM (character_expression)
- **Argümanlar**
 - *character_expression*: char, varchar, nchar, nvarchar vb.
- **Dönüş Tipi**
Baştaki boşluk karakterleri (char(32)) kaldırılmış karakter ifadesini döner
 - İfade NULL ise geriye NULL döner.

```
SELECT '  MERHABA  YALOVA' AS ORJ_METIN,  
       LTRIM ('  MERHABA  YALOVA') AS LTRIM1,  
       LTRIM (NULL) AS LTRIM2
```

ORJ_METIN	LTRIM1	LTRIM2
MERHABA YALOVA	MERHABA YALOVA	NULL

Karakter/Metin Fonksiyonları – Metin Düzenleme ve Temizleme

- **RTRIM**

- Verilen metnin sonundaki (sağındaki) boşluk karakterlerini temizler
- SQL Server 2022+ sürümlerinde silinecek karakter de belirtilebilir
- **Sözdizimi (Syntax)**
RTRIM (character_expression)
- **Argümanlar**
 - *character_expression*: char, varchar, nchar, nvarchar vb.
- **Dönüş Tipi**
Sondaki boşluk karakterleri (char(32)) kaldırılmış karakter ifadesini döner
 - İfade NULL ise geriye NULL döner.

```
SELECT  RTRIM ('MERHABA  YALOVA  ') AS RTRIM1,  
        RTRIM (NULL) AS RTRIM2
```

RTRIM1	RTRIM2
MERHABA YALOVA	NULL

Karakter/Metin Fonksiyonları – Metin Düzenleme ve Temizleme

- **TRIM**

- Verilen metnin hem başındaki hem de sonundaki boşluk karakterlerini temizler
- SQL Server 2022+ sürümlerinde silinecek karakter de belirtilebilir
- **Sözdizimi (Syntax)**
TRIM ([characters FROM] string)
- **Argümanlar**
 - *character_expression*: char, varchar, nchar, nvarchar vb.
- **Dönüş Tipi**
Baştaki ve sondaki boşluk karakterleri (char(32)) kaldırılmış karakter ifadesini döner
 - İfade NULL ise geriye NULL döner.

```
SELECT  LTRIM(RTRIM ('  MERHABA  YALOVA  ')) AS L_R_TRIM,  
        TRIM('  MERHABA  YALOVA  ') AS TRIM1,  
        TRIM (NULL) AS TRIM2
```

L_R_TRIM	TRIM1	TRIM2
MERHABA YALOVA	MERHABA YALOVA	NULL

Karakter/Metin Fonksiyonları – Metin Düzenleme ve Temizleme

- **STUFF**

- Verilen metnin belirli bir bölümünü silip, yerine başka bir metin eklemek için kullanılır

- **Sözdizimi (Syntax)**

STUFF (character_expression , start , length , replace_with_expression)

- **Argümanlar**

- **character_expression**: İşlem yapılacak metin
- **start**: Başlangıç pozisyonu (1'den küçük değer verilirse sonuç NULL olur)
- **length**: Kaç karakter silinecek (0 verilirse silmeden araya ekleme yapılmış olur)
- **replace_with_expression**: Silinen yere yerleştirilecek olan yeni metin (NULL verilirse sadece silme yapılmış olur)

- **Dönüş Tipi**

Giriş ifadesinin veri tipine bağlıdır (varchar → varchar, nvarchar → nvarchar).

- start veya length parametrelerinden birisi NULL verilirse sonuç NULL olur

```
SELECT  STUFF('xyzabc', 2, 4, 'AB') AS STUFF_AB,  
        STUFF('SQL Server', 4, 0, ' 2022') AS STUFF_L0,  
        STUFF('SQL Server', 4, 1, '') AS STUFF_BOSLUK_SIL,  
        STUFF('SQL Server', 4, 1, NULL) AS REPLACE_WITH_NULL,  
        STUFF('SQL Server', 0, 3, 'DENEME') AS ZERO_START
```

STUFF_AB	STUFF_L0	STUFF_BOSLUK_SIL	REPLACE_WITH_NULL	ZERO_START
xABc	SQL 2022 Server	SQLServer	SQLServer	NULL

Karakter/Metin Fonksiyonları – Metin Düzenleme ve Temizleme

- **REPLACE**

- Verilen metin içerisindeki belirli bir karakter grubunu veya kelimeyi, başka bir karakter grubuyla değiştirmek için kullanılır
- Büyük/Küçük Harf Duyarlılığı
 - Varsayılan olarak sorgunun çalıştığı veri tabanının Collation (karşılaştırma) ayarını izler
 - ❑ Turkish_CI_AS: "Case Insensitive" (Büyük-küçük harf duyarsız) demektir. 'A' ile 'a'yı aynı görür
 - ❑ Turkish_CS_AS: "Case Sensitive" (Büyük-küçük harf duyarlı) demektir. 'A' ile 'a'yı farklı görür
- Sadece ilk değil, tüm eşleşmeler değiştirilir
- Pattern desteklemez ve tam eşleşme yapar
- **Sözdizimi (Syntax)**
REPLACE (string_expression , string_pattern , string_replacement)
- **Argümanlar**
 - **string_expression:** İşlem yapılacak metin
 - **string_pattern:** Değiştirilecek metin
 - **string_replacement:** Yerine yazılacak metin
- **Dönüş Tipi**
Giriş ifadesinin veri tipine bağlıdır (varchar → varchar, nvarchar → nvarchar)
 - Herhangi bir parametre NULL ise sonuç NULL olur

Karakter/Metin Fonksiyonları – Metin Düzenleme ve Temizleme

- REPLACE

```
SELECT REPLACE('SQL SERVER DERSİ T-SQL', 'SQL', 'MySQL') AS REPLACE1,  
       REPLACE('123 - 456 - 789', ' ', '-') AS REPLACE2,  
       REPLACE('(505) 123 45 67', ' ', '-') AS REPLACE3
```

REPLACE1	REPLACE2	REPLACE3
MySQL SERVER DERSİ T-MySQL	123-456-789	(505)-123-45-67

Karakter/Metin Fonksiyonları – Metin Düzenleme ve Temizleme

- **REVERSE**

- Verilen metnin tersini döner
- Metnin başındaki veya sonundaki boşlukları da birer karakter olarak kabul eder ve yerlerini değiştirir
- Sayısal bir değeri REVERSE içine yazarsanız, SQL Server önce onu metne dönüştürür, sonra ters çevirir (sonuç bir string (metin) olarak döner)
- Bir kelimenin düzden ve tersten okunuşunun aynı olup olmadığını kontrol etmek (palindrom kontrolü) veya bir dosyanın uzantısını almak için kullanılabilir
- **Sözdizimi (Syntax)**
REVERSE (string_expression)
- **Argümanlar**
 - *string_expression*: İşlem yapılacak metin
- **Dönüş Tipi**
varchar / nvarchar

```
SELECT REVERSE('Veritabanı') AS TersMetin1,  
       REVERSE(' Veritabanı Dersi ') AS TersMetin2,  
       REVERSE(123.98) AS TersMetinSayı,  
       RIGHT('deneme dosyasi.json', CHARINDEX('.', REVERSE('deneme dosyasi.json')) - 1) AS DosyaUzantisi
```

TersMetin1	TersMetin2	TersMetinSayı	DosyaUzantisi
ınabatireV	isreD ınabatireV	89.321	json

Karakter/Metin Fonksiyonları – Metin Düzenleme ve Temizleme

- **REPLICATE**

- Verilen metni belirtilen sayı kadar tekrar ettirir
- Genellikle verileri belirli bir uzunluğa tamamlamak (padding) veya görsel raporlar/ayraçlar oluşturmak amacıyla kullanılır.
- Eğer sonuç metni veri tipinin kapasitesini aşarsa (örneğin VARCHAR(MAX) kullanılmıyorsa), SQL Server sonucu kesebilir

- **Sözdizimi (Syntax)**

REPLICATE (string_expression , integer_expression)

- **Argümanlar**

- **string_expression**: İşlem yapılacak metin
- **integer_expression**: bigint dahil olmak üzere herhangi bir tamsayı türünde bir ifade

- **Dönüş Tipi**

string_expression ile aynı türü döndürür

- Integer ifade negatif ise NULL döndürülür
- Integer ifade 0 ise, sonuç boş bir string (") olur

```
SELECT REPLICATE('*', 10) AS Yildizlar,  
       REPLICATE('0', 9 - LEN('123456')) + '123456' AS Padding,  
       'MERHABA' + REPLICATE(' ', 20) + 'YALOVA' AS Bosluklar,  
       'YALOVA' + REPLICATE('*', 0) + 'ÜNİVERSİTESİ' AS BosString,  
       REPLICATE('*', -1) AS Negatif
```

Yildizlar	Padding	Bosluklar	BosString	Negatif
*****	000123456	MERHABA	YALOVA	YALOVAÜNİVERSİTESİ
				NULL

Karakter/Metin Fonksiyonları – Metin Düzenleme ve Temizleme

- **QUOTENAME**

- Verilen metni belirli bir sınırlayıcı karakter (delimiter) ile sarar
- Bu fonksiyonun temel amacı, nesne isimlerini köşeli ayraç ([]) içine alarak SQL komutlarını güvenli hale getirmek ve özellikle Dinamik SQL yazarken oluşabilecek hataları veya SQL Injection risklerini önlemektir

- **Sözdizimi (Syntax)**

`QUOTENAME ( 'character_string' [ , 'quote_character' ] )`

- **Argümanlar**

- **character_string**: Sınırlanacak olan nesne adı (en fazla 128 karakter)

- **quote_character**: Hangi karakterin kullanılacağı

- ☐ Varsayılan değer köşeli ayraçtır ([])

- ☐ Diğer seçenekler: tek tırnak ('), çift tırnak ("), süslü parantez ({}), veya normal parantez (())

- **Dönüş Tipi**

nvarchar

```
SELECT  QUOTENAME('Müşteri Siparişleri') AS KoseliParantez,  
        QUOTENAME('TabloAdi', '''') AS TekTirnak,  
        QUOTENAME('Sütun1', '"') AS Cifttirnak,  
        QUOTENAME('Test]Tablo') AS OtomatikEscape
```

KoseliParantez	TekTirnak	Cifttirnak	OtomatikEscape
[Müşteri Siparişleri]	'TabloAdi'	"Sütun1"	[Test]]Tablo]

Karakter/Metin Fonksiyonları – Metin Düzenleme ve Temizleme

- **STRING_ESCAPE (SQL Server 2016+)**

- Verilen metnin içerisindeki özel karakterleri belirli bir formata göre kaçış karakteri (escape) ile dönüştürür.
- SQL SERVER'da şu an desteklenen tip JSON'dır.
 - Eğer veri tabanındaki bir açıklama alanında ", \ veya tab karakteri varsa, bu veriyi doğrudan JSON içine koyarsanız veri yapısı bozulur (syntax error)
 - STRING_ESCAPE bu karakterlerin başına otomatik olarak \ ekler
- **Sözdizimi (Syntax)**
STRING_ESCAPE(text , type)
- **Argümanlar**
 - **text:** İşlem yapılacak metin
 - **type:** Uygulanacak kaçış kuralları.
 - ❑ Şu anda desteklenen değer json'dır
- **Dönüş Tipi**
NVARCHAR

Orijinal Karakter	Kaçış Hali (JSON)	Açıklama
"	\"	Çift Tırnak
\	\\	Ters Eğik Çizgi
/	\/	Bölü İşareti
b	\b	Backspace
f	\f	Form feed
n	\n	New line (Alt satır)
r	\r	Carriage return
t	\t	Tab

Karakter/Metin Fonksiyonları – Metin Düzenleme ve Temizleme

- **STRING_ESCAPE**

```
SELECT  STRING_ESCAPE('Monitör 24" \ Ultra', 'json') AS SE1,  
        STRING_ESCAPE('C:\Data\Files', 'json') AS SE2
```

SE1	SE2
Monitör 24\" \ \ Ultra	C:\\Data\\Files

Karakter/Metin Fonksiyonları – Metin Düzenleme ve Temizleme

- **TRANSLATE (SQL Server 2017+)**

- Verilen metin çerisindeki belirli karakterleri, hedefteki karşılıklarıyla tek tek değiştirmek için kullanılır
- REPLACE fonksiyonuna çok benzer ancak en büyük farkı, birden fazla tekil karakter değişimini tek bir hamlede yapabilmesidir
- *characters* ve *translations* parametrelerinin ikisi de aynı uzunlukta olmalıdır
 - Fonksiyon, birinci listedeki n. karakteri, ikinci listedeki n. karakterle değiştirir
- **Sözdizimi (Syntax)**
TRANSLATE (inputString, characters, translations)
- **Argümanlar**
 - **inputString**: İşlem yapılacak metin
 - **characters**: Değişmesi istenen karakterlerin listesi
 - **translations**: Karşılık gelen yeni karakterlerin listesi
- **Dönüş Tipi**
nvarchar

```
SELECT REPLACE(REPLACE('A-B.C', '-', '_'), '.', '_') AS ReplaceIle,  
TRANSLATE('A-B.C', '-.', '__') TranslateIle,  
TRANSLATE('abcç deę prö zyü ', 'çığışöü', 'cigsou') TR_karakter_donusumu
```

Replacelle	Translatelle	TR_karakter_donusumu
A_B_C	A_B_C	abcc deg pro zyu

Karakter/Metin Fonksiyonları – Karakter Kodları

- **ASCII**

- Verilen karakterin ASCII (American Standard Code for Information Interchange) kodunu döndürür
- Sadece standart Batı dillerini kapsayan 0-255 arası değerler için kullanılır
 - Unicode kodu vermez
 - Türkçe karakterlerde sistemin kod sayfasına göre beklenmedik sonuçlar olabilir
- **Sözdizimi (Syntax)**
ASCII (character_expression)
- **Argümanlar**
 - *character_expression*: char veya varchar tipinde bir ifade
- **Dönüş Tipi**
int

```
SELECT  ASCII('A') AS A_kodu,  
        ASCII('a') AS a_kodu,  
        ASCII(' ') AS bosluk_kodu,  
        ASCII(NULL) AS null_kodu
```

A_kodu	a_kodu	bosluk_kodu	null_kodu
65	97	32	NULL

Karakter/Metin Fonksiyonları – Karakter Kodları

- **UNICODE**

- Verilen karakterin Unicode standartlarındaki sayısal değerini döndürür
- ASCII'den çok daha geniştir; Çince, Arapça veya özel sembolleri kapsar
- **Sözdizimi (Syntax)**
UNICODE (ncharacter_expression)
- **Argümanlar**
 - *ncharacter_expression*: nchar veya nvarchar tipinde bir ifade
- **Dönüş Tipi**
int

```
SELECT  UNICODE('A') AS A_kodu,  
        ASCII('Ç') AS c_kodu,  
        UNICODE('Ç') AS C_kodu
```

A_kodu	c_kodu	C_kodu
65	199	199

Karakter/Metin Fonksiyonları – Karakter Kodları

• CHAR

- 0 ile 255 arasındaki bir tam sayıyı (ASCII kodu) bir karaktere dönüştürür
- Özellikle klavyeden girilemeyen kontrol karakterleri (satır başı gibi) için kullanılır
- **Sözdizimi (Syntax)**
CHAR (integer_expression)
- **Argümanlar**
 - *integer_expression*: 0 ile 255 arasında bir tamsayı
- **Dönüş Tipi**
char(1)

```
SELECT CHAR(65) AS CHAR_A,  
       CHAR(97) AS CHAR_a,  
       CHAR(NULL) AS CHAR_NULL,  
       'a' + CHAR(13) + CHAR(10) + 'b' AS CRLF
```

CHAR_A	CHAR_a	CHAR_NULL	CRLF
A	a	NULL	a b

Control character	Value
Tab	char(9)
Space	char(32)
Line feed (Bir alt satıra indirir, LF)	char(10)
Carriage return (İmleci satır başına götürür, CR)	char(13)

Not: Windows sistemlerinde satır sonu genelde CRLF (13 + 10) kombinasyonudur

Karakter/Metin Fonksiyonları – Karakter Kodları

- **NCHAR**

- Unicode standardında tanımlandığı şekilde, belirtilen tamsayı koduna karşılık gelen Unicode karakterini döndürür
- **Sözdizimi (Syntax)**
NCHAR (integer_expression)
- **Argümanlar**
 - *integer_expression*: Bir Unicode tam sayı değeri
- **Dönüş Tipi**
nchar(1)

```
SELECT  CHAR(350) AS [CHAR $],  
        NCHAR(350) AS [NCHAR $],  
        NCHAR(NULL) AS NCHAR_NULL
```

CHAR \$	NCHAR \$	NCHAR_NULL
NULL	\$	NULL

Karakter/Metin Fonksiyonları – Karakter Kodları

- **SPACE**

- Belirtilen sayıda boşluk karakteri oluşturur
- Genellikle raporlarda veya metin birleştirmelerinde hizalama yapmak için kullanılır
- **Sözdizimi (Syntax)**
SPACE (integer_expression)
- **Argümanlar**
 - *integer_expression*: Boşluk sayısını gösteren pozitif bir tamsayı
- **Dönüş Tipi**
varchar

```
SELECT  'Ali' + SPACE(3) + 'Veli' AS SPACE_3,  
        SPACE(NULL) AS SPACE_NULL,  
        SPACE(-5) AS SPACE_NEG
```

SPACE_3	SPACE_NULL	SPACE_NEG
Ali Veli	NULL	NULL

Karakter/Metin Fonksiyonları – Dönüştürme

- **STR**

- Sayısal verileri karakter verisine dönüştürür
 - Sayıyı karaktere dönüştürürken sağa yaslı bir format kullanır
- Daha çok sabit genişlikli çıktı üretmek için kullanılır
- Taşma (overflow) durumunda yıldız (*) döndürebilir
- **Sözdizimi (Syntax)**

STR (float_expression [, length [, decimal]])

- **Argümanlar**

- **float_expression:** float veri türünde bir ifade
- **length:** Toplam karakter uzunluğu (sola boşluk getirilebilir)
- **decimal:** Ondalık basamak sayısı

- **Dönüş Tipi**

varchar

```
SELECT  STR(123.456, 10, 2) AS STR_,  
        STR(12345.67, 5, 2) AS STR_Tamsayi,  
        STR(12345.67, 4, 2) AS STR_Tasma
```

STR_	STR_Tamsayi	STR_Tasma
123.46	12346	*****

Karakter/Metin Fonksiyonları – Biçimlendirme

- **FORMAT**

- Sayısal değerleri, tarihleri ve saatleri belirli bir biçime (format) ve yerel kültüre (culture) göre bir string (metin) olarak döndürmek için kullanılır
- Genel veri türü dönüşümleri için CAST veya CONVERT fonksiyonlarını kullanılmalıdır
- Genel kullanım alanları:
 - Verilen tarihi istenen metin formatına dönüştürmede
 - Parasal değerlerin gösteriminde
 - Sayısal değerlerin gösteriminde
- FORMAT fonksiyonu genellikle raporlama ve çıktı biçimlendirme amaçlıdır; yoğun sorgularda performans problemi yaratabilir
- **Sözdizimi (Syntax)**
FORMAT(value , format [, culture])
- **Argümanlar**
 - **value**: Biçimlendirmek istediğiniz veri ([desteklenen](#) veri türlerinden birisi olmalıdır)
 - **format**: Değerin nasıl görünmesini istediğinizi belirten desen ([Geçerli](#) bir .NET Framework format string'i içermelidir)
 - **culture**: Dil ve bölge ayarı ([Geçerli](#) bir .NET CultureInfo adı). Belirtilmezse oturumun varsayılan dili kullanılır.
- **Dönüş Tipi**
nvarchar veya null

Karakter/Metin Fonksiyonları – Biçimlendirme

- **FORMAT**

- **Culture (Kültür) Parametresi**

- Özellikle para birimi sembolleri, gün/ay adları ve ondalık ayırıcılar (nokta vs. virgül) için kritiktir
 - Aşağıdaki tabloda bazı kültür değerleri ile karşılık gelen dil ve ülke eşleşmeleri listelenmiştir

Culture	Açıklama
ar-SA	Arapça (Suudi Arabistan)
de-DE	Almanca (Almanya)
en-GB	İngilizce (BK)
en-US	İngilizce (ABD)
es-ES	İspanyolca (İspanya)
fr-FR	Fransızca (Fransa)
hi-IN	Hintçe (Hindistan)
it-IT	İtalyanca (İtalya)
ja-JP	Japonca (Japonya)
ru-RU	Rusça (Rusya)
tr-TR	Türkçe (Türkiye)
zh-CN	Çince (Çin)

Karakter/Metin Fonksiyonları – Biçimlendirme

- **FORMAT**

- **Sayısal Biçimlendirme (Numeric Formats)**

- Sayıları para birimi, yüzde veya özel basamak gruplarıyla göstermek için kullanılır
 - Aşağıdaki tabloda bazı parametre değerleri ve açıklamaları sunulmuştur

Parametre	Açıklama
C	Currency (Para Birimi)
N	Number (Binlik Ayracı ile)
P	Percent (Yüzde)
E	Exponential Notation (Üssel Gösterim)
X	Hexadecimal (onaltılık sayı sistemi)
0	Custom (Basamak tutucu)

Karakter/Metin Fonksiyonları – Biçimlendirme

- **FORMAT**

- **Sayısal Biçimlendirme (Numeric Formats)**

```
SELECT  FORMAT(1234.567, 'C', 'tr-TR') AS Currency_TR,  
        FORMAT(1234.567, 'C1', 'en-US') AS Currency_US,  
        FORMAT(1234.567, 'C2', 'en-GB') AS Currency_GB,  
        FORMAT(1234.567, 'C3', 'de-DE') AS Currency_DE,  
        FORMAT(1234.567, 'C4', 'fr-FR') AS Currency_FR,  
        FORMAT(1234.567, 'C5', 'it-IT') AS Currency_IT,  
        FORMAT(1234.567, 'C3', 'es-ES') AS Currency_ES,  
        FORMAT(1234.567, 'C3', 'ar-SA') AS Currency_SA,  
        FORMAT(1234.567, 'C3', 'ru-RU') AS Currency_RU,  
        FORMAT(1234.567, 'C3', 'zh-CN') AS Currency_CN,  
        FORMAT(1234.567, 'C3', 'ja-JP') AS Currency_JP,  
        FORMAT(1234.567, 'C3', 'hi-IN') AS Currency_IN
```

Currency_TR	Currency_US	Currency_GB	Currency_DE	Currency_FR	Currency_IT	Currency_ES	Currency_SA	Currency_RU	Currency_CN	Currency_JP	Currency_IN
₺1.234,57	\$1,234.6	£1,234.57	1.234,567 €	1 234,5670 €	1.234,56700 €	1.234,567 €	1,234.567 ر.س.	1 234,567 Р	¥1,234.567	¥1,234.567	₹1,234.567

Karakter/Metin Fonksiyonları – Biçimlendirme

- **FORMAT**

- **Sayısal Biçimlendirme (Numeric Formats)**

```
SELECT  FORMAT(1234.567, 'N')           AS Number_Default,  
        FORMAT(1234.567, 'N0')        AS Number_N0,  
        FORMAT(1234.567, 'N1')        AS Number_N1,  
        FORMAT(1234.567, 'N2')        AS Number_N2,  
        FORMAT(1234.567, 'N3')        AS Number_N3,  
        FORMAT(1234.567, 'N4')        AS Number_N4,  
        FORMAT(1234.567, 'N5')        AS Number_N5,  
        FORMAT(1234.567, 'N', 'tr-TR') AS Number_TR
```

Number_Default	Number_N0	Number_N1	Number_N2	Number_N3	Number_N4	Number_N5	Number_TR
1,234.57	1,235	1,234.6	1,234.57	1,234.567	1,234.5670	1,234.56700	1.234,57

Karakter/Metin Fonksiyonları – Biçimlendirme

- **FORMAT**

- **Sayısal Biçimlendirme (Numeric Formats)**

```
SELECT  FORMAT(0.1234, 'P')      AS Percent_Default,  
        FORMAT(0.1234, 'P0')    AS Percent_0,  
        FORMAT(0.1234, 'P1')    AS Percent_1,  
        FORMAT(0.1234, 'P2')    AS Percent_2,  
        FORMAT(0.1234, 'P3')    AS Percent_3,  
        FORMAT(-0.123, 'P')      AS Percent_Neg
```

Percent_Default	Percent_0	Percent_1	Percent_2	Percent_3	Percent_Neg
12.34%	12%	12.3%	12.34%	12.340%	-12.30%

Karakter/Metin Fonksiyonları – Biçimlendirme

- **FORMAT**

- **Sayısal Biçimlendirme (Numeric Formats)**

```
SELECT  FORMAT(123.456, 'E')           AS Exp_Def,  
        FORMAT(123.456, 'E0')         AS Exp_0,  
        FORMAT(123.456, 'E1')         AS Exp_1,  
        FORMAT(123.456, 'E2')         AS Exp_2,  
        FORMAT(123.456, 'E3')         AS Exp_3,  
        FORMAT(-123.456, 'E')         AS Exp_Neg
```

Exp_Def	Exp_0	Exp_1	Exp_2	Exp_3	Exp_Neg
1.234560E+002	1E+002	1.2E+002	1.23E+002	1.235E+002	-1.234560E+002

Karakter/Metin Fonksiyonları – Biçimlendirme

- **FORMAT**

- **Sayısal Biçimlendirme (Numeric Formats)**

```
SELECT  FORMAT(123.456, '00')           AS Custom1,  
        FORMAT(123.456, '0000')        AS Custom2,  
        FORMAT(123.456, '000000.0000') AS Custom3,  
        FORMAT(77, 'X')                 AS Hexa
```

Custom1	Custom2	Custom3	Hexa
123	0123	00123.4560	4D

Karakter/Metin Fonksiyonları – Biçimlendirme

- **FORMAT**

- **Tarih ve Saat Biçimlendirme (Date & Time Formats)**

- Tarihler, standart kalıplar veya özel karakterlerle özelleştirilebilir
- Bazı standart kalıplar:
 - ❑ 'd': Kısa tarih (12.02.2026)
 - ❑ 'D': Uzun tarih (12 Şubat 2026 Perşembe)
 - ❑ 'f': Tam tarih ve kısa saat
 - ❑ 'T': Uzun zaman formatı
- Bazı özel belirteçler:

Belirteç	Açıklama	Örnek
dd	Gün (Sayısal, iki hane)	01, 12
ddd	Gün adı (Kısa)	Per, Pzt
dddd	Gün adı (Tam)	Perşembe
MM	Ay (Sayısal)	02, 11
MMM	Ay adı (Kısa)	Şub, Ara
MMMM	Ay adı (Tam)	Şubat
yyyy	Yıl (4 hane)	2026

Karakter/Metin Fonksiyonları – Biçimlendirme

- **FORMAT**

- **Tarih ve Saat Biçimlendirme (Date & Time Formats)**

```
SELECT  FORMAT(GETDATE(), 'dd MMMM dddd yyyy')           AS Format_Date_01,
        FORMAT(GETDATE(), 'dd MMMM dddd yyyy', 'tr-TR')  AS Format_Date_02,
        FORMAT(GETDATE(), 'd', 'tr-TR')                  AS Format_Date_03,
        FORMAT(GETDATE(), 'dd', 'tr-TR')                  AS Format_Date_04,
        FORMAT(GETDATE(), 'ddd', 'tr-TR')                 AS Format_Date_05,
        FORMAT(GETDATE(), 'dddd', 'tr-TR')               AS Format_Date_06,
        FORMAT(GETDATE(), 'D', 'tr-TR')                   AS Format_Date_07,
        FORMAT(GETDATE(), 'M', 'tr-TR')                   AS Format_Date_08,
        FORMAT(GETDATE(), 'Y', 'tr-TR')                   AS Format_Date_09,
        FORMAT(GETDATE(), 'MM')                           AS Format_Date_10,
        FORMAT(GETDATE(), 'MMM')                          AS Format_Date_11,
        FORMAT(GETDATE(), 'MMMM')                         AS Format_Date_12
```

Format_Date_01	Format_Date_02	Format_Date_03	Format_Date_04	Format_Date_05	Format_Date_06	Format_Date_07	Format_Date_08	Format_Date_09	Format_Date_10	Format_Date_11	Format_Date_12
12 February Thursday 2026	12 Şubat Perşembe 2026	12.02.2026	12	Per	Perşembe	12 Şubat 2026 Perşembe	12 Şubat	Şubat 2026	02	Feb	February

Zaman (Date/Time) Fonksiyonları

- **DATEADD**

- Belirtilen tarihe belirli bir zaman aralığı (yıl, gün, saat vb.) ekler veya çıkarır
- Taşma (overflow) durumunda hata verir
- **Sözdizimi (Syntax)**

DATEADD (datepart , number , date)

- **Argümanlar**

- **datepart**: Eklenecek birim (year, month, day, hour vb.)
- **number**: Eklenecek miktar (negatif ise çıkarır)
- **date**: Başlangıç tarihi

- **Dönüş Tipi**

Girilen tarih tipine bağlı (datetime, date, datetime2 vb.)

```
SELECT DATEADD(day, 10, '2000-01-01') AS DATEADD_10Days,
       DATEADD(month, -5, '2000-01-01') AS DATEADD_Minus5Months,
       DATEADD(year, 10, '2000-01-01') AS DATEADD_10Years,
       DATEADD(week, 2, '2000-01-01') AS DATEADD_2Weeks,
       DATEADD(hour, 2, '2000-01-01') AS DATEADD_2Hours,
       DATEADD(minute, 2, '2000-01-01') AS DATEADD_2Minutes,
       DATEADD(second, 2, '2000-01-01') AS DATEADD_2Seconds
-- Hata: Adding a value to a 'datetime' column caused an overflow
--DATEADD(year, 10000, '2000-01-01') AS TasmaHatası
```

DATEADD_10Days	DATEADD_Minus5Months	DATEADD_10Years	DATEADD_2Weeks	DATEADD_2Hours	DATEADD_2Minutes	DATEADD_2Seconds
2000-01-11 00:00:00.000	1999-08-01 00:00:00.000	2010-01-01 00:00:00.000	2000-01-15 00:00:00.000	2000-01-01 02:00:00.000	2000-01-01 00:02:00.000	2000-01-01 00:00:02.000

Zaman (Date/Time) Fonksiyonları

- **DATEDIFF**

- İki tarih arasındaki farkı belirtilen zaman birimi cinsinden hesaplar
- Gerçek “geçen süreyi” değil, belirtilen datepart sınırlarının kaç kez geçildiğini hesaplar
- **Sözdizimi (Syntax)**

DATEDIFF (datepart , startdate , enddate)

- **Argümanlar**

- **datepart**: Fark alınacak zaman birimi (year, month, day, hour vb.)
- **number**: Başlangıç tarihi
- **date**: Bitiş tarihi

- **Dönüş Tipi**

int

```
SELECT DATEDIFF(year, '2000-01-01', '2005-01-01') AS DATEDIFF_YEAR,  
DATEDIFF(minute, '13:00', '14:30') AS DATEDIFF_MINUTE1,  
DATEDIFF(day, '2000-01-01 23:57', '2000-01-02 00:00') AS DATADD_DAY,  
DATEDIFF(minute, '2000-01-01 23:57', '2000-01-02 00:00') AS DATEDIFF_MINUTE2
```

DATEDIFF_YEAR	DATEDIFF_MINUTE1	DATADD_DAY	DATEDIFF_MINUTE2
5	90	1	3

Zaman (Date/Time) Fonksiyonları

- **DATENAME**

- Tarihin belirli bir bölümünü metin (string) olarak döndürür (Örn: Ay adı)
- Gün ve ay isimleri dil ayarına bağlıdır
- **Sözdizimi (Syntax)**
DATENAME (datepart , date)
- **Argümanlar**
 - **datepart**: Fark alınacak zaman birimi (year, month, day, hour vb.)
 - **date**: Zaman
- **Dönüş Tipi**
nvarchar

```
SELECT  DATENAME(MONTH, '2000-01-01')      AS DATENAME_MONTH,  
        DATENAME(DAYOFYEAR, '2000-12-01') AS DATENAME_DY,  
        DATENAME(WEEKDAY, '2000-01-01')   AS DATENAME_DW
```

DATENAME_MONTH	DATENAME_DY	DATENAME_DW
January	336	Saturday

Zaman (Date/Time) Fonksiyonları

- **DATEPART**

- Tarihin belirtilen kısmını sayısal olarak döndürür
- **Sözdizimi (Syntax)**
DATEPART (datepart , date)
- **Argümanlar**
 - **datepart**: Parçası alınacak zaman birimi (year, month, day, hour vb.)
 - **date**: Zaman
- **Dönüş Tipi**
int

```
SELECT  DATEPART(YEAR, '2000-01-01')      AS DATEPART_YEAR,  
        DATEPART(MONTH, '2000-12-01')    AS DATEPART_MONTH,  
        DATEPART(DAY, '2000-01-01')      AS DATEPART_DAY,  
        DATEPART(WEEK, '2000-12-01')     AS DATEPART_WEEK,  
        DATEPART(HOUR, '14:30')          AS DATEPART_HOUR,  
        DATEPART(MINUTE, '14:30')        AS DATEPART_MINUTE
```

DATEPART_YEAR	DATEPART_MONTH	DATEPART_DAY	DATEPART_WEEK	DATEPART_HOUR	DATEPART_MINUTE
2000	12	1	49	14	30

Zaman (Date/Time) Fonksiyonları

- **DAY / MONTH / YEAR**

- Tarihin gün, ay veya yıl kısmını tam sayı olarak döndüren kısa yol fonksiyonlar
 - DAY(date) = DATEPART(day, date)
 - MONTH(date) = DATEPART(month, date)
 - YEAR(date) = DATEPART(year, date)
- **Sözdizimi (Syntax)**
DAY (date), MONTH (date), YEAR (date)
- **Argümanlar**
 - *date*: Zaman
- **Dönüş Tipi**
int

```
SELECT DAY('2000-01-01') AS DAY_,  
       MONTH('2000-12-01') AS MONTH_,  
       YEAR('2000-01-01') AS YEAR_
```

DAY_	MONTH_	YEAR_
1	12	2000

Zaman (Date/Time) Fonksiyonları

- **GETDATE / CURRENT_TIMESTAMP / SYSDATETIME / SYSDATETIMEOFFSET / GETUTCDATE / SYSUTCDATETIME**
 - Genel olarak sunucunun geçerli tarih-saatini verirler

Söz Dizimi (Syntax)	Hassasiyet	Zaman Dilimi (Timezone)	Dönüş Tipi
GETDATE()	Düşük (.000, .003, .007 sn)	Sunucu Saati	datetime
CURRENT_TIMESTAMP (ANSI SQL)	Düşük (GETDATE ile aynı)	Sunucu Saati	datetime
SYSDATETIME()	Yüksek (7 basamak ms)	Sunucu Saati	datetime2
SYSDATETIMEOFFSET()	Yüksek	Sunucu Saati + Fark	datetimeoffset
GETUTCDATE()	Düşük	UTC (0) Saati	datetime
SYSUTCDATETIME()	Yüksek	UTC (0) Saati	datetime2

Zaman (Date/Time) Fonksiyonları

- **SWITCHOFFSET**

- datetimeoffset değerinin saat dilimini değiştirir

- **Sözdizimi (Syntax)**

SWITCHOFFSET (datetimeoffset_expression , timezoneoffset_expression)

- **Argümanlar**

- **datetimeoffset_expression:** Zaman

- **timezoneoffset_expression:** *[+|-]TZH:TZM biçiminde bir ifade veya zaman dilimi farkını temsil eden işaretli bir tamsayı (dakika cinsinden)*

- **Dönüş Tipi**

datetimeoffset

```
SELECT SWITCHOFFSET('2000-01-01 15:00 +03:00', '+05:00') AS OFFSET_1,  
       SWITCHOFFSET('2000-01-01 03:00 +03:00', '-05:00') AS OFFSET_2,  
       SWITCHOFFSET('2000-01-01 23:00 +00:00', '+07:00') AS OFFSET_3
```

OFFSET_1	OFFSET_2	OFFSET_3
2000-01-01 17:00:00.0000000 +05:00	1999-12-31 19:00:00.0000000 -05:00	2000-01-02 06:00:00.0000000 +07:00

Zaman (Date/Time) Fonksiyonları

- **TODATETIMEOFFSET**

- Bir datetime2 değerine zaman dilimi bilgisi ekleyerek onu datetimeoffset tipine dönüştürür
 - Mevcut tarih değerini dönüştürmez, sadece offset ekler
- **Sözdizimi (Syntax)**
TODATETIMEOFFSET (datetime_expression , timezoneoffset_expression)
- **Argümanlar**
 - **datetimeoffset_expression**: Zaman
 - **timezoneoffset_expression**: *[+|-]TZH:TZM biçiminde bir ifade veya zaman dilimi farkını temsil eden işaretli bir tamsayı (dakika cinsinden)*
- **Dönüş Tipi**
datetimeoffset

```
SELECT TODATETIMEOFFSET('2000-01-01 15:00', '+05:00') AS OFFSET_1,  
       TODATETIMEOFFSET('2000-01-01 03:00', '-05:00') AS OFFSET_2,  
       TODATETIMEOFFSET('2000-01-01 23:00', '+07:00') AS OFFSET_3
```

OFFSET_1	OFFSET_2	OFFSET_3
2000-01-01 15:00:00.0000000 +05:00	2000-01-01 03:00:00.0000000 -05:00	2000-01-01 23:00:00.0000000 +07:00

Zaman (Date/Time) Fonksiyonları

- **ISDATE**

- Verilen ifadenin geçerli bir tarih olup olmadığını kontrol eder
- **Sözdizimi (Syntax)**
ISDATE (expression)
- **Argümanlar**
 - *expression*: Zaman
- **Dönüş Tipi**
 - int (Geçerliyse 1, değilse 0)
 - İfade bir datetime2 değeri ise 0 döndürür

```
SELECT  ISDATE('2000-01-01') AS ISDATE_1,  
        ISDATE('2001-02-29') AS ISDATE_2,  
        ISDATE('2000-12-32') AS ISDATE_3,  
        ISDATE('abc')       AS ISDATE_4
```

ISDATE_1	ISDATE_2	ISDATE_3	ISDATE_4
1	0	0	0

Zaman (Date/Time) Fonksiyonları

- **DATEFROMPARTS / DATETIMEFROMPARTS**

- Verilen sayısal parçaları birleştirerek bir tarih/saat oluşturur
- **Sözdizimi (Syntax)**
 - DATEFROMPARTS (year, month, day)*
 - DATETIMEFROMPARTS (year , month , day , hour , minute , seconds , milliseconds)*
- **Argümanlar**
 - *year, month, day, ...: Oluşturulacak zaman bilgisinin parçaları*
- **Dönüş Tipi**
 - *DATEFROMPARTS -> date*
 - *DATETIMEFROMPARTS -> datetime*

```
SELECT DATETIMEFROMPARTS ( 2010, 12, 31, 23, 59, 59, 0 ) AS Result
```

Result

2010-12-31 23:59:59.000

Zaman (Date/Time) Fonksiyonları

- **EOMONTH**

- Verilen tarihin ay sonunu verir

- **Sözdizimi (Syntax)**

EOMONTH (start_date [, month_to_add])

- **Argümanlar**

- **start_date:** Ayın son günü döndürülecek tarih

- **month_to_add:** Tarihe eklenecek ay sayısını belirten isteğe bağlı bir tamsayı ifadesi

- **Dönüş Tipi**

date

```
SELECT  EOMONTH('2000-02-01') AS EOM_1,  
        EOMONTH('2001-02-01') AS EOM_2,  
        EOMONTH('2000-02-01', 3) AS EOM_3,  
        EOMONTH('2000-02-01', -6) AS EOM_4
```

EOM_1	EOM_2	EOM_3	EOM_4
2000-02-29	2001-02-28	2000-05-31	1999-08-31

Tip Dönüşüm (Conversion) Fonksiyonları

- **CAST**

- Bir veri tipindeki ifadeyi başka bir veri tipine dönüştürür
- ANSI SQL standartlarına uygundur
- Daralan dönüşümler veri kayıplarına neden olabileceği için dikkatli kullanılmalıdır
- Bu fonksiyon yerine hatalı dönüşümlerde sistemin kırılmasını engelleyen TRY_CAST fonksiyonu da kullanılabilir
- **Sözdizimi (Syntax)**

CAST (expression AS data_type [(length)])

- **Argümanlar**

- **expression:** Dönüştürülecek olan değer
- **data_type:** Hedef veri tipi (int, varchar, decimal vb.)
- **length:** (Opsiyonel) Hedef tipin uzunluğu

- **Dönüş Tipi**

Hedeflenen veri tipi

```
SELECT CAST(123 AS VARCHAR(10)) AS SayiyiMetne,  
       CAST('123' AS INT) + 5 AS MetniSayiya,  
       CAST(123.4567 AS DECIMAL(10,2)) AS OndalikHassasiyet,  
       CAST(15.95 AS INT) AS OndaliktanInte
```

SayiyiMetne	MetniSayiya	OndalikHassasiyet	OndaliktanInte
123	128	123.46	15

Tip Dönüşüm (Conversion) Fonksiyonları

- **CONVERT**

- Bir veri tipindeki ifadeyi başka bir veri tipine dönüştürür
- CAST'tan farklı olarak, özellikle tarih/saat ve MONEY ve FLOAT tipindeki verileri belirli bir formatta (stilde) görüntülemeyi sağlar
- Bazı tarih stilleri için stil kodundan 100 çıkarılırsa yıl bilgisi iki haneli gözükür
- Bu fonksiyon yerine hatalı dönüşümlerde sistemin kırılmasını engelleyen TRY_CONVERT fonksiyonu da kullanılabilir
- **Sözdizimi (Syntax)**
CONVERT (data_type [(length)] , expression [, style])
- **Argümanlar**
 - **data_type**: Hedef veri tipi (int, varchar, decimal vb.)
 - **length**: (Opsiyonel) Hedef tipin uzunluğu
 - **expression**: Dönüştürülecek olan değer
 - **style**: (Opsiyonel) Çıktının nasıl görüneceğini belirleyen tam sayı kodu
- **Dönüş Tipi**
Hedeflenen veri tipi

Tip Dönüşüm (Conversion) Fonksiyonları

- **CONVERT**

Stil Kodu	Açıklama (FLOAT / REAL)
0	Maksimum 6 basamak
1	Bilimsel gösterim (scientific)
2	16 basamak
3	17 basamak (hassas)

Stil Kodu	Açıklama (MONEY / SMALLMONEY)
0 (varsayılan)	Virgüllü binlik ayraç, 2 ondalık
1	Virgüllü binlik ayraç, 2 ondalık
2	Binlik ayraç yok, 4 ondalık

Stil Kodu	Format
103	dd/mm/yyyy
104	dd.mm.yyyy
105	dd-mm-yyyy
112 (ISO)	YYYYAAGG

Tip Dönüşüm (Conversion) Fonksiyonları

- CONVERT

```
SELECT  CONVERT(VARCHAR(10), CAST('2000-12-31' AS DATE), 103) AS CONVERT_DATE_103,  
        CONVERT(VARCHAR(10), CAST('2010-12-31' AS DATE), 3)   AS CONVERT_DATE_3,  
        CONVERT(VARCHAR(10), CAST('2000-12-31' AS DATE), 104) AS CONVERT_DATE_104,  
        CONVERT(VARCHAR(10), CAST('2000-12-31' AS DATE), 105) AS CONVERT_DATE_105,  
        CONVERT(VARCHAR(10), CAST('2000-12-31' AS DATE), 112) AS CONVERT_DATE_112
```

CONVERT_DATE_103	CONVERT_DATE_3	CONVERT_DATE_104	CONVERT_DATE_105	CONVERT_DATE_112
31/12/2000	31/12/10	31.12.2000	31-12-2000	20001231

```
SELECT  CONVERT(VARCHAR, CAST(123456.789 AS MONEY), 0) AS CONVERT_MONEY_0,  
        CONVERT(VARCHAR, CAST(123456.789 AS MONEY), 1) AS CONVERT_MONEY_1,  
        CONVERT(VARCHAR, CAST(123456.789 AS MONEY), 2) AS CONVERT_MONEY_2,  
        CONVERT(VARCHAR, CAST(123456.789 AS FLOAT), 0) AS CONVERT_FLOAT_0,  
        CONVERT(VARCHAR, CAST(123456.789 AS FLOAT), 1) AS CONVERT_FLOAT_1,  
        CONVERT(VARCHAR, CAST(123456.789 AS FLOAT), 2) AS CONVERT_FLOAT_2,  
        CONVERT(VARCHAR, CAST(123456.789 AS FLOAT), 3) AS CONVERT_FLOAT_3
```

CONVERT_MONEY_0	CONVERT_MONEY_1	CONVERT_MONEY_2	CONVERT_FLOAT_0	CONVERT_FLOAT_1	CONVERT_FLOAT_2	CONVERT_FLOAT_3
123456.79	123,456.79	123456.7890	123457	1.2345679e+005	1.2345678900000000e+005	1.2345678900000000e+005

NULL Değer Fonksiyonları

- ISNULL

- Belirtilen ifade NULL ise, onu başka bir değerle değiştirir
- **Sözdizimi (Syntax)**
ISNULL (check_expression , replacement_value)
- **Argümanlar**
 - *check_expression*: Kontrol edilecek ifade
 - *replacement_value*: Eğer ifade NULL ise yerine konacak değer
- **Dönüş Tipi**
Dönüş tipi her zaman ilk parametrenin tipidir

```
SELECT  ISNULL('MERHABA', 'YALOVA') AS ISNULL1,  
        ISNULL(NULL, 'YALOVA')      AS ISNULL2
```

ISNULL1	ISNULL2
MERHABA	YALOVA

NULL Değer Fonksiyonları

- **COALESCE**

- Parametre listesindeki ilk NULL olmayan değeri döndürür
- ANSI SQL standardıdır
- **Sözdizimi (Syntax)**
COALESCE (expression [, ...n])
- **Argümanlar**
 - ***expression***: Kontrol edilecek ifade
- **Dönüş Tipi**
En yüksek öncelikli veri tipi

```
SELECT  COALESCE(NULL, NULL, 10, 20, NULL, 30) AS C1,  
        COALESCE('MERHABA', NULL, 'YALOVA')   AS C2
```

C1	C2
10	MERHABA

NULL Değer Fonksiyonları

- **NULLIF**

- Verilen iki ifade birbirine eşitse NULL döndürür, değilse ilk ifadeyi döndürür
- **Sözdizimi (Syntax)**
NULLIF (expression , expression)
- **Argümanlar**
 - **expression:** Kontrol edilecek ifadeler
- **Dönüş Tipi**
expression'ın veri tipi

```
SELECT  NULLIF(5, 5)           AS NULLIF1,  
        NULLIF(5, 10)        AS NULLIF2,  
        100 / NULLIF(0, 0)    AS NULLIF3
```

NULLIF1	NULLIF2	NULLIF3
NULL	5	NULL

Gruplama (Aggregate) Fonksiyonları

- Veri tabanındaki birden fazla satırı analiz edip, bunlar hakkında tek bir özet sonuç üretmek için kullanılırlar.
- Özellikle:
 - İstatistiksel analiz
 - Veri özetleme
 - Karar destek sistemleri
 - Raporlama sorgularıiçin temel araçlardır.
- Genellikle GROUP BY ile birlikte veriyi kategorilere ayırıp her kategori için istatistiksel analiz yapmak amacıyla kullanılırlar.
 - SELECT içinde gruplama olmayan her sütun GROUP BY içerisinde yazılmak **zorundadır**.
- Gruplama sonucuna filtreleme yapılacaksa **HAVING** ifadesi kullanılır.

Gruplama (Aggregate) Fonksiyonları

- **COUNT**

- Bir sorgu sonucundaki satır (kayıt) sayısını verir
- **COUNT(*)**: Sorgu sonucundaki tüm satırları (NULL dahil) sayar
- **COUNT(sutun_adi)**: Sadece o sütundaki dolu (NULL olmayan) verileri sayar
- **COUNT(DISTINCT sutun_adi)**: Sütundaki benzersiz ve dolu verileri sayar
- Sorgu sonucunda 2,147,483,647 den fazla satır çıkarsa sonuç gösterilemez
 - Bu durumda **COUNT_BIG** fonksiyonu kullanılmalıdır

```
-- Şirkette toplam kaç çalışan var?
```

```
SELECT COUNT(*) AS [Toplam_Calisan] FROM Personel
```

```
-- Kan grubu tanımlı kaç personel var
```

```
SELECT COUNT(kan_grubu) AS [kan_grubu_olanlar] FROM Personel
```

```
-- Kaç farklı departman var? (DISTINCT tekrarı önler)
```

```
SELECT COUNT(DISTINCT Departman) AS [Farkli_Departman_Sayisi] FROM Personel
```

Gruplama (Aggregate) Fonksiyonları

- **SUM**

- Sayısal bir sütundaki değerlerin toplamını verir
- Yalnızca sayısal veri tipleri (INT, DECIMAL, FLOAT vb.) ile çalışır
- Sütundaki NULL değerler toplama dahil edilmez

```
-- Personellerin toplam maaşı ne kadar?  
SELECT SUM(Maas) AS [Toplam_Maas] FROM Personel
```

- **AVG**

- Sayısal bir sütunun aritmetik ortalamasını verir
- Sütundaki NULL değerler hesaplama dahil edilmez (ne toplama dahil edilir ne de paydaya (adet) eklenir)
 - Eğer NULL değer 0 kabul edilmesini isteniyorsa AVG(ISNULL(sutun, 0)) şeklinde yazılmalıdır
- Dönüş tipi, verilen sütunun veya değer verinin veri tipine bağlıdır
 - Eğer tam sayı (INT) olan bir sütunun ortalaması alınıyorsa, SQL Server sonucu yine bir tam sayı olarak döndürür ve ondalık kısmı atar (yuvarlamaz, direkt keser)
 - ❑ Tam sayı sütun ortalaması alınmada önce ondalıklı bir tipe dönüştürülürse (cast) sonuç o tipte olur
 - ❑ Ya da sütunu 1.0 çarpmak da implicit olarak decimal tipine çevirir
 - Eğer sütun DECIMAL, FLOAT veya MONEY tipindeyse, sonuç beklenildiği gibi ondalıklı döner

```
-- Şirket genelinde maaş ortalaması nedir?  
SELECT AVG(Maas) AS [Ortalama_Maas] FROM Personel
```

Gruplama (Aggregate) Fonksiyonları

- **MIN / MAX**

- Bir sütundaki minimum ve maksimum değerleri döndürür.
 - Sayı → küçük/büyük
 - Tarih → eski/yeni
 - Metin → alfabetik sıralama (collation'a bağlı)
- Sütundaki NULL değerler dikkate alınmaz

-- En düşük ve en yüksek maaşlar nedir?

SELECT

MIN(Maas) AS [En_Dusuk_Maas],

MAX(Maas) AS [En_Yuksek_Maas]

FROM Personel

-- Alfabetik olarak ilk isim kimin?

SELECT MIN(Ad) FROM Personel

Gruplama (Aggregate) Fonksiyonları

- **STDEV**

- Bir sayısal sütundaki değerlerin örnek standart sapmasını (sample standard deviation) hesaplar
- NULL değerler hesaba katılmaz

```
SELECT STDEV(Notu) AS StandartSapma  
FROM Notlar
```

- **VAR**

- Bir sayısal sütundaki değerlerin örnek varyansını (sample variance) hesaplar
- Standart sapmanın karesidir
- NULL değerler hesaba katılmaz

```
SELECT VAR(Notu) AS Varyans  
FROM Notlar
```

- Popülasyonun standart sapma ve varyansı için sırasıyla STDEVP ve VARP fonksiyonları kullanılabilir
- Boş bir tabloda COUNT fonksiyonu sıfır (0), diğerleri ise NULL döner
- Bir sütuna göre gruplama yapıldığında, SQL Server tüm NULL değerleri birbirine eşitmiş gibi kabul eder ve hepsini tek bir grupta toplar
 - Örneğin, eğer Sehir sütununa göre gruplama yapılırsa ve 10 satırda şehir bilgisi girilmemişse (NULL), sonuç kümesinde bu 10 satır tek bir satır olarak "NULL" başlığı altında görünür

Gruplama (Aggregate) Fonksiyonları – Örnekler

- Kütüphane tablosundaki tblKitap isimli tablo üzerinde gruplama fonksiyonları

```
SELECT COUNT(*) AS ToplamKitapSayisi,
COUNT(Aciklama) AS AciklamasiOlanKitapSayisi,
COUNT(DISTINCT BasimYeriKodu) AS BasimYerineGoreBenzersiz,
SUM(ToplamStok) AS ToplamKitapStogu,
AVG(En) AS OrtEn_Tam,
AVG(En * 1.0) AS OrtEn_Ondalik,
MIN(SayfaSayisi) AS EnAzSayfaSayisi,
MAX(SayfaSayisi) AS EnCokSayfaSayisi,
STDEV(SayfaSayisi) AS SayfaSayisiSTD,
VAR(SayfaSayisi) AS SayfaSayisiVAR
FROM tblKitap
```

ToplamKitapSayisi	AciklamasiOlanKitapSayisi	BasimYerineGoreBenzersiz	ToplamKitapStogu	OrtEn_Tam	OrtEn_Ondalik	EnAzSayfaSayisi	EnCokSayfaSayisi	SayfaSayisiSTD	SayfaSayisiVAR
10	8	5	31	17	17.200000	10	863	309.892902647207	96033.6111111111

```
SELECT ID, Ad, Aciklama, BasimYeriKodu, ToplamStok, En, SayfaSayisi FROM tblKitap
```

ID	Ad	Aciklama	BasimYeriKodu	ToplamStok	En	SayfaSayisi
1	KITAP 1	AÇIKLAMA 1	34	3	15	100
2	KITAP 2	AÇIKLAMA 2	06	1	17	250
3	KITAP 3	AÇIKLAMA 3	34	2	18	325
4	KITAP 4	AÇIKLAMA 4	34	5	23	681
5	KITAP 5	AÇIKLAMA 5	35	1	10	79
6	KITAP 6	NULL	16	1	5	10
7	KITAP 7	AÇIKLAMA 7	06	1	25	863
8	KITAP 8	AÇIKLAMA 8	06	8	22	647
9	KITAP 9	AÇIKLAMA 9	01	6	18	90
10	KITAP 10	NULL	34	3	19	50

Gruplama (Aggregate) Fonksiyonları – Örnekler

- Aktif personelleri bölümlerine göre grupta ve beşten fazla personeli olan bölümlerin ID'lerini ve personel sayılarını göster

```
SELECT  bolum_id,  
        COUNT(*)  
FROM personel  
WHERE  
    aktif = 1  
GROUP BY  
    bolum_id  
HAVING  
    COUNT(*) > 5
```

- Her bir bölümdeki kişilerin cinsiyetlerine göre kaçar kişi olduğunu listele

```
SELECT  bolum_id,  
        cinsiyet,  
        COUNT(*)  
FROM personel  
GROUP BY  
    bolum_id,  
    cinsiyet
```


Gruplama (Aggregate) Fonksiyonları – Örnekler

İlk sorgu da isimleri yazdıramayız. Neden?

```
-- Üyelerin en gencinin, en yaşlısının yaşları ve ortalama üye yaşı  
SELECT MIN(DATEDIFF(yy, DTarihi, GETDATE())) EnGencUyeninYasi,  
       MAX(DATEDIFF(yy, DTarihi, GETDATE())) EnYasliUyeninYasi,  
       AVG(DATEDIFF(yy, DTarihi, GETDATE())) OrtUyeYasi  
FROM tblUye
```

	EnGencUyeninYasi	EnYasliUyeninYasi	OrtUyeYasi
1	17	56	27

```
-- Ortalama üye yaşından 5 yaştan fazla büyük olmayan ve hiç kitap ödünç almamış üyeler  
SELECT TCNO, Ad, Soyad, DATEDIFF(yy, DTarihi, GETDATE()) Yasi  
FROM tblUye  
WHERE  
    ID NOT IN ( SELECT DISTINCT U.ID FROM tblUye U INNER JOIN tblOduncKitap O ON O.UyeID = U.ID ) AND  
    DATEDIFF(yy, DTarihi, GETDATE()) <= ( SELECT AVG(DATEDIFF(yy, DTarihi, GETDATE())) + 5 FROM tblUye )
```

	TCNO	Ad	Soyad	Yasi
1	12345678906	ÜYE ADI 6	ÜYE SOYADI 6	30
2	12345678907	ÜYE ADI 7	ÜYE SOYADI 7	28
3	12345678910	ÜYE ADI 10	ÜYE SOYADI 10	17

```
-- Her bir yayın evinin isminin yanına toplam çıkmış kitap sayısını yazan sorgu.
```

```
-- Sıralama büyükten küçüğe
```

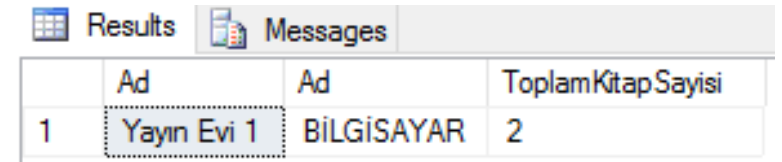
```
SELECT Y.Ad, COUNT(*) KitapSayisi  
FROM tblKitap K  
    INNER JOIN tblYayinEvi Y ON Y.ID = K.YayinEviID  
GROUP BY Y.Ad  
ORDER BY KitapSayisi DESC
```

	Ad	KitapSayisi
1	Yayın Evi 1	2
2	Yayın Evi 2	1
3	Yayın Evi 3	1
4	Yayın Evi 4	1
5	Yayın Evi 5	1
6	Yayın Evi 6	1
7	Yayın Evi 7	1
8	Yayın Evi 8	1
9	Yayın Evi 9	1

Gruplama (Aggregate) Fonksiyonları – Örnekler

-- Her bir yayın evinin kitabı olduğu her bir kategorideki toplam kitap sayısı
-- 0 kategoride 1 den fazla kitap yoksa gösterilmeyecektir

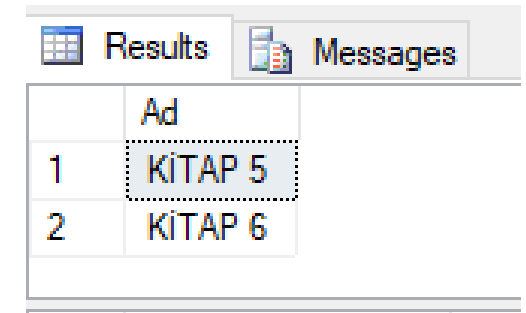
```
SELECT YE.Ad, KT.Ad, COUNT(*) ToplamKitapSayisi  
FROM tblKitap K  
    INNER JOIN tblKitapTuru KT ON KT.ID = K.UstTurID  
    INNER JOIN tblYayinEvi YE ON YE.ID = K.YayinEviID  
GROUP BY YE.Ad, KT.Ad  
HAVING COUNT(*) > 1  
ORDER BY YE.Ad, KT.Ad
```



	Ad	Ad	ToplamKitapSayisi
1	Yayın Evi 1	BİLGİSAYAR	2

-- 2015-2016 yılları arası ödünç alınan kitapların listesi

```
SELECT DISTINCT K.Ad  
FROM tblOduncKitap O  
    INNER JOIN tblKitap K ON K.ID = O.KitapID  
WHERE YEAR(O.AlinisTarihi) BETWEEN 2015 AND 2016
```



	Ad
1	KİTAP 5
2	KİTAP 6

Gruplama (Aggregate) Fonksiyonları – Örnekler

```
-- En az 2 üyenin olduğu illerde il bazında ort yaşlar
SELECT I.Ad, COUNT(*) UyeSayisi, AVG(DATEDIFF(yy, DTarihi, GETDATE())) OrtYas
FROM tblUye U
     INNER JOIN tblIl I ON I.Kod = U.IlKodu
GROUP BY I.Ad
HAVING COUNT(*) > 2
```

 Results  Messages			
	Ad	UyeSayisi	OrtYas
1	İstanbul	4	19

Gruplama (Aggregate) Fonksiyonları – Örnekler

```
-- Sipariş brüt toplamı (birim fiyat * miktar) 500'den fazla  
-- olan siparişlerin ID'lerini ve brüt toplam değerleri  
-- getirecek sorguyu yazınız
```

```
SELECT OrderID,  
       SUM(UnitPrice * Quantity) AS TotalGrossAmount  
FROM [Order Details] OD  
GROUP BY OrderID  
HAVING SUM(UnitPrice * Quantity) > 500  
ORDER BY TotalGrossAmount
```

OrderID	TotalGrossAmount
10489	502.20
11062	508.00
10730	509.75
11061	510.00
11071	510.00
10926	514.40
10320	516.00
10315	516.80
10534	517.40
10256	517.80
10000	518.00

SELECT – GROUP BY CUBE & ROLLUP

- Genellikle rapor sorgularında alt toplam ve genel toplam almak için kullanılırlar
- ROLLUP operatörü, GROUP BY içinde verilen sütunlar için sonuç üretir
- CUBE operatörü, GROUP BY içinde verilen sütunların tüm kombinasyonları için sonuç üretir

Results

Messages

	id	name	gender	salary	department
1	1	David	Male	5000	Sales
2	2	Jim	Female	6000	HR
3	3	Kate	Female	7500	IT
4	4	Will	Male	6500	Marketing
5	5	Shane	Female	5500	Finance
6	6	Shed	Male	8000	Sales
7	7	Vik	Male	7200	HR
8	8	Vince	Female	6600	IT
9	9	Jane	Female	5400	Marketing
10	10	Laura	Female	6300	Finance
11	11	Mac	Male	5700	Sales
12	12	Pat	Male	7000	HR
13	13	Julie	Female	7100	IT
14	14	Elice	Female	6800	Marketing
15	15	Wayne	Male	5000	Finance

```
SELECT Department, Sum(Salary) As Salary_Sum
FROM Employee
GROUP BY Department
```

Results	Messages
Department	Salary_Sum
1 Finance	16800
2 HR	20200
3 IT	21200
4 Marketing	18700
5 Sales	18700

```
SELECT Department AS Department,
Sum(Salary) As Salary_Sum
FROM Employee
GROUP BY ROLLUP (Department)
```

Results	Messages
Department	Salary_Sum
1 Finance	16800
2 HR	20200
3 IT	21200
4 Marketing	18700
5 Sales	18700
6 NULL	95600

```
SELECT Coalesce (Department, 'All Departments') AS Department,
Sum(Salary) As Salary_Sum
FROM Employee
GROUP BY ROLLUP (Department)
```

Results	Messages
Department	Salary_Sum
1 Finance	16800
2 HR	20200
3 IT	21200
4 Marketing	18700
5 Sales	18700
6 All Departments	95600

SELECT – GROUP BY CUBE & ROLLUP

	id	name	gender	salary	department
1	1	David	Male	5000	Sales
2	2	Jim	Female	6000	HR
3	3	Kate	Female	7500	IT
4	4	Will	Male	6500	Marketing
5	5	Shane	Female	5500	Finance
6	6	Shed	Male	8000	Sales
7	7	Vik	Male	7200	HR
8	8	Vince	Female	6600	IT
9	9	Jane	Female	5400	Marketing
10	10	Laura	Female	6300	Finance
11	11	Mac	Male	5700	Sales
12	12	Pat	Male	7000	HR
13	13	Julie	Female	7100	IT
14	14	Elice	Female	6800	Marketing
15	15	Wayne	Male	5000	Finance

```
SELECT  
  Coalesce (Department, 'All Departments') AS Department,  
  Coalesce (Gender, 'All Genders') AS Gender,  
  Sum(Salary) As Salary_Sum  
FROM Employee  
GROUP BY ROLLUP (Department, Gender)
```

- Şu grupları üretir:
 - (Department, Gender)
 - (Department)
 - ()

	Department	Gender	Salary_Sum
1	Finance	Female	11800
2	Finance	Male	5000
3	Finance	All Genders	16800
4	HR	Female	6000
5	HR	Male	14200
6	HR	All Genders	20200
7	IT	Female	21200
8	IT	All Genders	21200
9	Marketing	Female	12200
10	Marketing	Male	6500
11	Marketing	All Genders	18700
12	Sales	Male	18700
13	Sales	All Genders	18700
14	All Departments	All Genders	95600

SELECT – GROUP BY CUBE & ROLLUP

	id	name	gender	salary	department
1	1	David	Male	5000	Sales
2	2	Jim	Female	6000	HR
3	3	Kate	Female	7500	IT
4	4	Will	Male	6500	Marketing
5	5	Shane	Female	5500	Finance
6	6	Shed	Male	8000	Sales
7	7	Vik	Male	7200	HR
8	8	Vince	Female	6600	IT
9	9	Jane	Female	5400	Marketing
10	10	Laura	Female	6300	Finance
11	11	Mac	Male	5700	Sales
12	12	Pat	Male	7000	HR
13	13	Julie	Female	7100	IT
14	14	Elice	Female	6800	Marketing
15	15	Wayne	Male	5000	Finance

```
SELECT  
  Coalesce (Department, 'All Departments') AS Department,  
  Coalesce (Gender, 'All Genders') AS Gender,  
  Sum(Salary) As Salary_Sum  
FROM Employee  
GROUP BY CUBE (Department, Gender)
```

- Şu grupları üretir:
 - (Department, Gender)
 - (Department)
 - (Gender)
 - ()
- NOT: ROLLUP/CUBE ürettiği NULL gerçek NULL değildir, toplam satırı temsil eder.

	Department	Gender	Salary_Sum
1	Finance	Female	11800
2	HR	Female	6000
3	IT	Female	21200
4	Marketing	Female	12200
5	All Departments	Female	51200
6	Finance	Male	5000
7	HR	Male	14200
8	Marketing	Male	6500
9	Sales	Male	18700
10	All Departments	Male	44400
11	All Departments	All Genders	95600
12	Finance	All Genders	16800
13	HR	All Genders	20200
14	IT	All Genders	21200
15	Marketing	All Genders	18700
16	Sales	All Genders	18700

SELECT – LEFT JOIN KULLANIM ÖRNEĞİ

- Tüm üyelerin TCNO ve isimlerinin yanına KİTAP 2 isimli kitabı ödünç alış sayıları ve toplam kaç kitap ödünç aldıklarını getirecek sorguyu yazınız

tblUye

	ID	TCNO	Ad	Soyad
1	1	12345678901	ÜYE ADI 1	ÜYE SOY
2	2	12345678902	ÜYE ADI 2	ÜYE SOY
3	3	12345678903	ÜYE ADI 3	ÜYE SOY
4	4	12345678904	ÜYE ADI 4	ÜYE SOY
5	5	12345678905	ÜYE ADI 5	ÜYE SOY
6	6	12345678906	ÜYE ADI 6	ÜYE SOY
7	7	12345678907	ÜYE ADI 7	ÜYE SOY
8	8	12345678908	ÜYE ADI 8	ÜYE SOY
9	9	12345678909	ÜYE ADI 9	ÜYE SOY
10	10	12345678910	ÜYE ADI 10	ÜYE SOY

tblKitap

	ID	ISBN	Ad
1	1	1234567890123	KİTAP 1
2	2	1234567890124	KİTAP 2
3	3	1234567890125	KİTAP 3
4	4	1234567890126	KİTAP 4
5	5	1234567890127	KİTAP 5
6	6	1234567890128	KİTAP 6
7	7	1234567890129	KİTAP 7
8	8	1234567890130	KİTAP 8
9	9	1234567890131	KİTAP 9
10	10	1234567890132	KİTAP 10

tblOduncKitap

	ID	UyeID	KitapID	AlinisTarihi	GetirilecegiTarih	GetirildigiTarih	Durum
1	1	1	1	2019-07-31	2016-08-01	NULL	0
2	2	2	2	2019-07-31	2016-08-01	NULL	1
3	3	2	3	2019-07-31	2016-08-01	NULL	2
4	4	3	4	2019-07-31	2016-08-01	NULL	0
5	5	4	5	2016-01-10	2016-08-01	2019-07-31	0
6	6	5	6	2016-01-10	2016-08-01	NULL	0
7	7	3	1	2019-07-31	2016-08-01	NULL	0
8	8	3	9	2019-07-31	2016-08-01	NULL	1
9	9	3	2	2019-07-31	2016-08-01	NULL	0
10	10	3	6	2019-07-31	2016-08-01	NULL	2

SELECT – LEFT JOIN KULLANIM ÖRNEĞİ

```
SELECT  U.ID AS UyeID,  
        U.TCNO AS UyeTCNO,  
        U.Ad AS UyeAdi,  
        O.ID AS OduncKitapID,  
        O.UyeID AS OK_UyeID,  
        O.KitapID AS OK_KitapID  
FROM tblUye U  
LEFT JOIN tblOduncKitap O ON O.UyeID = U.ID
```

	UyeID	UyeTCNO	UyeAdi	OduncKitapID	OK_UyeID	OK_KitapID
1	1	12345678901	ÜYE ADI 1	1	1	1
2	2	12345678902	ÜYE ADI 2	2	2	2
3	2	12345678902	ÜYE ADI 2	3	2	3
4	3	12345678903	ÜYE ADI 3	4	3	4
5	3	12345678903	ÜYE ADI 3	7	3	1
6	3	12345678903	ÜYE ADI 3	8	3	9
7	3	12345678903	ÜYE ADI 3	9	3	2
8	3	12345678903	ÜYE ADI 3	10	3	6
9	4	12345678904	ÜYE ADI 4	5	4	5
10	5	12345678905	ÜYE ADI 5	6	5	6
11	6	12345678906	ÜYE ADI 6	NULL	NULL	NULL
12	7	12345678907	ÜYE ADI 7	NULL	NULL	NULL
13	8	12345678908	ÜYE ADI 8	NULL	NULL	NULL
14	9	12345678909	ÜYE ADI 9	NULL	NULL	NULL
15	10	12345678910	ÜYE ADI 10	NULL	NULL	NULL

SELECT – LEFT JOIN KULLANIM ÖRNEĞİ

```
SELECT  U.ID AS UyeID,  
        U.TCNO AS UyeTCNO,  
        U.Ad AS UyeAdi,  
        O.ID AS OduncKitapID,  
        O.UyeID AS OK_UyeID,  
        O.KitapID AS OK_KitapID,  
        K.ID AS KitapID,  
        K.Ad AS KitapAdi  
FROM    tblUye U  
        LEFT JOIN tblOduncKitap O ON O.UyeID = U.ID  
        INNER JOIN tblKitap K ON K.ID = O.KitapID
```

	UyeID	UyeTCNO	UyeAdi	OduncKitapID	OK_UyeID	OK_KitapID	KitapID	KitapAdi
1	1	12345678901	ÜYE ADI 1	1	1	1	1	KİTAP 1
2	2	12345678902	ÜYE ADI 2	2	2	2	2	KİTAP 2
3	2	12345678902	ÜYE ADI 2	3	2	3	3	KİTAP 3
4	3	12345678903	ÜYE ADI 3	4	3	4	4	KİTAP 4
5	4	12345678904	ÜYE ADI 4	5	4	5	5	KİTAP 5
6	5	12345678905	ÜYE ADI 5	6	5	6	6	KİTAP 6
7	3	12345678903	ÜYE ADI 3	7	3	1	1	KİTAP 1
8	3	12345678903	ÜYE ADI 3	8	3	9	9	KİTAP 9
9	3	12345678903	ÜYE ADI 3	9	3	2	2	KİTAP 2
10	3	12345678903	ÜYE ADI 3	10	3	6	6	KİTAP 6

SELECT – LEFT JOIN KULLANIM ÖRNEĞİ

```
SELECT  U.ID AS UyeID,  
        U.TCNO AS UyeTCNO,  
        U.Ad AS UyeAdi,  
        O.ID AS OduncKitapID,  
        O.UyeID AS OK_UyeID,  
        O.KitapID AS OK_KitapID,  
        K.ID AS KitapID,  
        K.Ad AS KitapAdi  
FROM tblUye U  
LEFT JOIN tblOduncKitap O ON O.UyeID = U.ID  
LEFT JOIN tblKitap K ON K.ID = O.KitapID
```

	UyeID	UyeTCNO	UyeAdi	OduncKitapID	OK_UyeID	OK_KitapID	KitapID	KitapAdi
1	1	12345678901	ÜYE ADI 1	1	1	1	1	KİTAP 1
2	2	12345678902	ÜYE ADI 2	2	2	2	2	KİTAP 2
3	2	12345678902	ÜYE ADI 2	3	2	3	3	KİTAP 3
4	3	12345678903	ÜYE ADI 3	4	3	4	4	KİTAP 4
5	3	12345678903	ÜYE ADI 3	7	3	1	1	KİTAP 1
6	3	12345678903	ÜYE ADI 3	8	3	9	9	KİTAP 9
7	3	12345678903	ÜYE ADI 3	9	3	2	2	KİTAP 2
8	3	12345678903	ÜYE ADI 3	10	3	6	6	KİTAP 6
9	4	12345678904	ÜYE ADI 4	5	4	5	5	KİTAP 5
10	5	12345678905	ÜYE ADI 5	6	5	6	6	KİTAP 6
11	6	12345678906	ÜYE ADI 6	NULL	NULL	NULL	NULL	NULL
12	7	12345678907	ÜYE ADI 7	NULL	NULL	NULL	NULL	NULL
13	8	12345678908	ÜYE ADI 8	NULL	NULL	NULL	NULL	NULL
14	9	12345678909	ÜYE ADI 9	NULL	NULL	NULL	NULL	NULL
15	10	12345678910	ÜYE ADI 10	NULL	NULL	NULL	NULL	NULL

SELECT – LEFT JOIN KULLANIM ÖRNEĞİ

```
SELECT  U.ID AS UyeID,  
        U.TCNO AS UyeTCNO,  
        U.Ad AS UyeAdi,  
        O.ID AS OduncKitapID,  
        O.UyeID AS OK_UyeID,  
        O.KitapID AS OK_KitapID,  
        K.ID AS KitapID,  
        K.Ad AS KitapAdi  
FROM tblUye U  
    LEFT JOIN tblOduncKitap O ON O.UyeID = U.ID  
    LEFT JOIN tblKitap K ON K.ID = O.KitapID  
WHERE K.Ad = 'KİTAP 2'
```



```
SELECT  U.ID AS UyeID,  
        U.TCNO AS UyeTCNO,  
        U.Ad AS UyeAdi,  
        O.ID AS OduncKitapID,  
        O.UyeID AS OK_UyeID,  
        O.KitapID AS OK_KitapID,  
        K.ID AS KitapID,  
        K.Ad AS KitapAdi  
FROM tblUye U  
    INNER JOIN tblOduncKitap O ON O.UyeID = U.ID  
    INNER JOIN tblKitap K ON K.ID = O.KitapID  
WHERE K.Ad = 'KİTAP 2'
```

	UyeID	UyeTCNO	UyeAdi	OduncKitapID	OK_UyeID	OK_KitapID	KitapID	KitapAdi
1	2	12345678902	ÜYE ADI 2	2	2	2	2	KİTAP 2
2	3	12345678903	ÜYE ADI 3	9	3	2	2	KİTAP 2

SELECT – LEFT JOIN KULLANIM ÖRNEĞİ

```
SELECT  U.ID AS UyeID,  
        U.TCNO AS UyeTCNO,  
        U.Ad AS UyeAdi,  
        O.ID AS OduncKitapID,  
        O.UyeID AS OK_UyeID,  
        O.KitapID AS OK_KitapID,  
        K.ID AS KitapID,  
        K.Ad AS KitapAdi  
FROM tblUye U  
LEFT JOIN tblOduncKitap O ON O.UyeID = U.ID  
LEFT JOIN tblKitap K ON K.ID = O.KitapID AND K.Ad = 'KİTAP 2'
```

	UyeID	UyeTCNO	UyeAdi	OduncKitapID	OK_UyeID	OK_KitapID	KitapID	KitapAdi
1	1	12345678901	ÜYE ADI 1	1	1	1	NULL	NULL
2	2	12345678902	ÜYE ADI 2	2	2	2	2	KİTAP 2
3	2	12345678902	ÜYE ADI 2	3	2	3	NULL	NULL
4	3	12345678903	ÜYE ADI 3	4	3	4	NULL	NULL
5	3	12345678903	ÜYE ADI 3	7	3	1	NULL	NULL
6	3	12345678903	ÜYE ADI 3	8	3	9	NULL	NULL
7	3	12345678903	ÜYE ADI 3	9	3	2	2	KİTAP 2
8	3	12345678903	ÜYE ADI 3	10	3	6	NULL	NULL
9	4	12345678904	ÜYE ADI 4	5	4	5	NULL	NULL
10	5	12345678905	ÜYE ADI 5	6	5	6	NULL	NULL
11	6	12345678906	ÜYE ADI 6	NULL	NULL	NULL	NULL	NULL
12	7	12345678907	ÜYE ADI 7	NULL	NULL	NULL	NULL	NULL
13	8	12345678908	ÜYE ADI 8	NULL	NULL	NULL	NULL	NULL
14	9	12345678909	ÜYE ADI 9	NULL	NULL	NULL	NULL	NULL
15	10	12345678910	ÜYE ADI 10	NULL	NULL	NULL	NULL	NULL

SELECT – LEFT JOIN KULLANIM ÖRNEĞİ

```
SELECT  U.TCNO AS UyeTCNO,  
        U.Ad AS UyeAdi,  
        COUNT(K.Ad) Kitap2,  
        COUNT(O.UyeID) Toplam,  
        COUNT(*) Hepsi  
FROM    tblUye U  
        LEFT JOIN tblOduncKitap O ON O.UyeID = U.ID  
        LEFT JOIN tblKitap K ON K.ID = O.KitapID AND K.Ad = 'KİTAP 2'  
GROUP BY U.TCNO, U.Ad
```

Results		Messages			
	UyeTCNO	UyeAdi	Kitap2	Toplam	Hepsi
1	12345678901	ÜYE ADI 1	0	1	1
2	12345678902	ÜYE ADI 2	1	2	2
3	12345678903	ÜYE ADI 3	1	5	5
4	12345678904	ÜYE ADI 4	0	1	1
5	12345678905	ÜYE ADI 5	0	1	1
6	12345678906	ÜYE ADI 6	0	0	1
7	12345678907	ÜYE ADI 7	0	0	1
8	12345678908	ÜYE ADI 8	0	0	1
9	12345678909	ÜYE ADI 9	0	0	1
10	12345678910	ÜYE ADI 10	0	0	1

Kitap adı yerine üye adı ya da TC'sini saydırmamamızın nedeni başka aldıkları kitapların sayılmaması için

SELECT – INNER JOIN (SORGU) ÖRNEĞİ

```
-- İngilizce kitap ödünç almış üyelerin yaşadıkları illerde toplam kaç üyemiz var
SELECT I.Ad, COUNT(*)
FROM tblIl I
    INNER JOIN tblUye U ON U.IlKodu = I.Kod
    INNER JOIN ( SELECT DISTINCT U.IlKodu FROM tblUye U
        INNER JOIN tblOduncKitap OK ON OK.UyeID = U.ID
        INNER JOIN tblKitap K ON K.ID = OK.KitapID
        INNER JOIN tblDil D ON D.ID = K.DilID WHERE D.Ad = 'İngilizce'
    ) X ON X.IlKodu = I.Kod
GROUP BY I.Ad
```

Results			Messages		
	Ad	(No column name)			
1	Ankara	2			
2	İstanbul	4			

SELECT – EXISTS & NOT EXISTS

- EXISTS, alt sorgunun en az bir satır döndürüp döndürmediğini kontrol eden mantıksal (boolean) bir operatördür.
 - EXISTS, mantıksal olarak “ $\exists$ (exists)” niceleyicisini temsil eder
- EXISTS değerle değil varlık (existence) ile ilgilenir
- EXISTS mantıksal olarak ilk eşleşmede TRUE döner; fiziksel yürütme planı optimizier’a bağlıdır
 - Bu yüzden IN operatörüne göre büyük veri setlerinde daha performanslı olabilir (short-circuit evaluation)
- **Syntax**

```
SELECT sütunlar
FROM tablo1 T1
WHERE
    EXISTS (SELECT 1
            FROM tablo2 T2
            WHERE
                T1.ID = T2.RefID
            )
```

- Alt sorguda SELECT * veya SELECT 1 kullanılmasının bir farkı olmaz çünkü EXISTS sadece "varlık" kontrolü yapmaktadır
- EXISTS genellikle dış sorgudaki bir sütuna referans verir (correlated subquery)
 - Her dış satır için iç sorgu tekrar değerlendirilir

SELECT – EXISTS & NOT EXISTS

- **EXISTS vs. JOIN vs. IN**

- **JOIN:** Eşleşen tüm satırları getirir (duplikasyon riski vardır, DISTINCT gerektirebilir)
- **EXISTS:** Sadece varlığı kontrol eder, duplikasyon yaratmaz (semi join, $R \bowtie S$, S'de eşleşmesi bulunan R satırlarını döndür)
 - NOT EXISTS -> Anti Semi Join ($R \not\bowtie S = R - R \bowtie S$, S'de eşleşmesi olmayan R satırlarını getir)
 - NULL değerlerden etkilenmez
- **IN:** Genellikle sabit listeler veya ilişkisiz alt sorgular için idealdir
 - Alt sorgu sonucu NULL içermiyorsa EXISTS ile mantıksal olarak eşdeğerdir
 - NULL değerlerde beklenmeyen sonuçlar doğabilir
 - Örneğin aşağıdaki ifade mantıksal olarak "T tablosundaki RefID değerleri arasında olmayan U.ID'leri getir" manasındadır

```
SELECT * FROM U
WHERE U.ID NOT IN (SELECT RefID FROM T)
```

- Eğer T tablosunun RefID sütununda sırasıyla 1, 2 ve NULL değerleri varsa bu durumda SQL şu karşılaştırmayı yapar

```
U.ID <> 1
AND U.ID <> 2
AND U.ID <> NULL
```

SELECT – EXISTS & NOT EXISTS

- Burada U.ID <> NULL ifadesinin sonucu ne TRUE ne de FALSE'dur
- SQL'de WHERE sadece TRUE olan satırları getirir, UNKNOWN elenir
- Dolayısıyla alt sorguda tek bir NULL varsa, NOT IN çoğu zaman hiçbir kayıt döndürmez
- Bu yüzden bu tarz senaryolarda tehlikelidir
- NOT EXISTS, alt sorguda eşleşme bulunmayan kayıtları döndürür
 - Önceki örnek NOT EXISTS ile tehlike oluşturmaz

```
SELECT * FROM U
WHERE NOT EXISTS (
  SELECT 1
  FROM T
  WHERE T.RefID = U.ID
)
```

- Çünkü EXISTS değer karşılaştırmaz; eşleşme var mı diye bakar
- NULL satırı, eşleşme üretmez
- Dolayısıyla NOT EXISTS, NULL'dan etkilenmez
- Eğer amaç yalnızca varlık kontrolü ise, EXISTS semantik olarak daha uygun bir ifadedir ve optimizer genellikle bunu semi-join planına dönüştürür

SELECT – EXISTS & NOT EXISTS

-- Şimdiye kadar en az bir defa KİTAP 9 isimli kitabı ödünç almış olan üyelerin TCNO, isim ve soyisimleri
-- Eğer INNER JOIN yaparsak DISTINCT kullanmamız gerekir çünkü bir üye birden fazla kez bu kitabı aldıysa
-- iç sorguda birden fazla satır döner ama EXIST ile buna gerek kalmayacaktır. Aynı sorgu IN ile de yazılabilir.

```
SELECT TCNO, Ad, Soyad
FROM tblUye U
WHERE EXISTS ( SELECT *
                FROM tblUye
                INNER JOIN tblOduncKitap ON tblOduncKitap.UyeID = tblUye.ID
                INNER JOIN tblKitap ON tblKitap.ID = tblOduncKitap.KitapID
                WHERE tblKitap.Ad = 'KİTAP 9' AND U.ID = tblUye.ID
              )
```

```
SELECT TCNO, Ad, Soyad
FROM tblUye U
WHERE U.ID IN ( SELECT tblUye.ID
                FROM tblUye
                INNER JOIN tblOduncKitap ON tblOduncKitap.UyeID = tblUye.ID
                INNER JOIN tblKitap ON tblKitap.ID = tblOduncKitap.KitapID
                WHERE tblKitap.Ad = 'KİTAP 9'
              )
```

```
SELECT DISTINCT tblUye.TCNO, tblUye.Ad, tblUye.Soyad
FROM tblUye
    INNER JOIN tblOduncKitap ON tblOduncKitap.UyeID = tblUye.ID
    INNER JOIN tblKitap ON tblKitap.ID = tblOduncKitap.KitapID
WHERE tblKitap.Ad = 'KİTAP 9'
```

Results		Messages	
	TCNO	Ad	Soyad
1	12345678901	ÜYE ADI 1	ÜYE SOYADI 1
2	12345678902	ÜYE ADI 2	ÜYE SOYADI 2

SELECT – ALL & ANY & SOME Operatörleri

- **ALL (evrensel niceleme, $\forall$)**

- Karşılaştırmanın, alt sorgudan dönen tüm değerler için doğru olmasını gerektirir
- Matematiksel olarak "Her biri için" ($\forall$) niceleyicisine karşılık gelir.
- Eğer alt sorgu boş küme dönerse ALL operatörü TRUE döner
 - Bu durumda aşağıda verilen MIN / MAX / NOT IN eşitlikleri geçerli olmaz
- " $A < ALL(S)$ " = " $A < MIN(S)$ " = " $NOT EXISTS (SELECT * FROM S WHERE A \geq S.x)$ "
 - En küçüğünden daha küçük
- " $A > ALL(S)$ " = " $A > MAX(S)$ " = " $NOT EXISTS (SELECT * FROM S WHERE A \leq S.x)$ "
 - En büyüğünden daha büyük
- " $A <> ALL(S)$ " = " $A NOT IN (S)$ " = " $NOT EXISTS (SELECT * FROM S WHERE A = S.x)$ " (NULL olmadığı varsayımıyla)
 - Hiç birine eşit değil

SELECT – ALL & ANY & SOME Operatörleri

- **ANY (varoluş niceleme, $\exists$)**

- Karşılaştırmanın, alt sorgudan dönen değerlerden en az biri için doğru olması yeterlidir
- Matematiksel olarak "Vardır" ($\exists$) niceleyicisine karşılık gelir
- Eğer alt sorgu boş küme dönerse ANY operatörü FALSE döner
 - Bu durumda aşağıda verilen MIN / MAX / IN eşitlikleri geçerli olmaz
- " $A < ANY(S)$ " = " $A < MAX(S)$ " = " $EXISTS (SELECT * FROM S WHERE A < S.x)$ "
 - En büyüğünden daha küçük
- " $A > ANY(S)$ " = " $A > MIN(S)$ " = " $EXISTS (SELECT * FROM S WHERE A > S.x)$ "
 - En küçüğünden daha büyük
- " $A = ANY(S)$ " = " $A IN (S)$ " = " $EXISTS (SELECT * FROM S WHERE A = S.x)$ "
 - En az birine eşit

- **SOME**

- ANY ile tamamen aynıdır
- SQL standartlarında ANY operatörünün eş anlamlısı olarak kabul edilir
- Aralarında işlevsel hiçbir fark yoktur

SELECT – ALL & ANY & SOME Operatörleri

- NULL nedeniyle sonuç UNKNOWN olabilir
 - Örneğin: $5 > \text{ALL} (1, 2, \text{NULL})$
 - Burada:
 - $5 > 1 \rightarrow \text{TRUE}$
 - $5 > 2 \rightarrow \text{TRUE}$
 - $5 > \text{NULL} \rightarrow \text{UNKNOWN}$
 - Sonuç: UNKNOWN
- ALL ve ANY çoğu zaman optimizer tarafından EXISTS / MIN / MAX şeklinde yeniden yazılır
- **Sözdizimi (Syntax)**
column_name operator { ALL | ANY | SOME } (subquery)

SELECT – ALL & ANY & SOME Operatörleri

-- ALL ve ANY örneği

-- Tüm yazarlardan daha genç ve yaşlı olan üyelerin isimleri ayrı ayrı listelensin

```
SELECT 'GENÇLER', U.Ad, U.Soyad, DATEDIFF(yy, DTarihi, GETDATE()) Yas
FROM tblUye U
WHERE DATEDIFF(yy, DTarihi, GETDATE()) < ALL(SELECT DATEDIFF(yy, DTarihi, GETDATE()) YazarYasi FROM tblYazar)
UNION ALL
SELECT 'YAŞLILAR', U.Ad, U.Soyad, DATEDIFF(yy, DTarihi, GETDATE()) Yas
FROM tblUye U
WHERE DATEDIFF(yy, DTarihi, GETDATE()) > ALL(SELECT DATEDIFF(yy, DTarihi, GETDATE()) YazarYasi FROM tblYazar)
```

	YazarYasi
1	41
2	47
3	27
4	34
5	61
6	54
7	37

	(No column name)	Ad	Soyad	Yas
1	GENÇLER	ÜYE ADI 1	ÜYE SOYADI 1	20
2	GENÇLER	ÜYE ADI 2	ÜYE SOYADI 2	21
3	GENÇLER	ÜYE ADI 3	ÜYE SOYADI 3	19
4	GENÇLER	ÜYE ADI 4	ÜYE SOYADI 4	22
5	GENÇLER	ÜYE ADI 5	ÜYE SOYADI 5	18
6	GENÇLER	ÜYE ADI 10	ÜYE SOYADI 10	17
7	YAŞLILAR	ÜYE ADI 9	ÜYE SOYADI 9	76

SELECT – ALL & ANY & SOME Operatörleri

--Önceki örnek MIN ve MAX fonksiyonları ile de yazılabilir

```
SELECT 'GENÇLER', U.Ad, U.Soyad, DATEDIFF(yy, DTarihi, GETDATE()) Yas  
FROM tblUye U  
WHERE DATEDIFF(yy, DTarihi, GETDATE()) < (SELECT MIN(DATEDIFF(yy, DTarihi, GETDATE())) YazarYasi FROM tblYazar)  
UNION ALL  
SELECT 'YAŞLILAR', U.Ad, U.Soyad, DATEDIFF(yy, DTarihi, GETDATE()) Yas  
FROM tblUye U  
WHERE DATEDIFF(yy, DTarihi, GETDATE()) > (SELECT MAX(DATEDIFF(yy, DTarihi, GETDATE())) YazarYasi FROM tblYazar)
```


SELECT – ALL & ANY & SOME Operatörleri

-- ÜYE ADI 2 isimli üyenin aldığı kitaplardan her hangi birini alan diğer üyelerin TCNO, ad ve soyad bilgileri

```
SELECT DISTINCT U.TCNO, U.Ad, U.Soyad
FROM tblUye U
    INNER JOIN tblOduncKitap OK ON OK.UyeID = U.ID
    INNER JOIN tblKitap K ON K.ID = OK.KitapID
WHERE K.ID = ANY( SELECT DISTINCT K.ID FROM tblUye U INNER JOIN tblOduncKitap OK ON OK.UyeID = U.ID
    INNER JOIN tblKitap K ON K.ID = OK.KitapID WHERE U.Ad = 'ÜYE ADI 2'
)
AND U.Ad <> 'ÜYE ADI 2'
```

```
SELECT U.Ad, K.Ad FROM tblUye U
LEFT JOIN tblOduncKitap OK ON OK.UyeID = U.ID
LEFT JOIN tblKitap K ON K.ID = OK.KitapID
ORDER BY U.ID, K.Ad
```

	Ad	Ad
1	ÜYE ADI 1	KİTAP 2
2	ÜYE ADI 2	KİTAP 2
3	ÜYE ADI 2	KİTAP 3
4	ÜYE ADI 3	KİTAP 3
5	ÜYE ADI 3	KİTAP 4
6	ÜYE ADI 3	KİTAP 4
7	ÜYE ADI 3	KİTAP 4
8	ÜYE ADI 3	KİTAP 4
9	ÜYE ADI 4	KİTAP 3
10	ÜYE ADI 5	KİTAP 2
11	ÜYE ADI 5	KİTAP 3
12	ÜYE ADI 6	NULL
13	ÜYE ADI 7	NULL
14	ÜYE ADI 8	KİTAP 8
15	ÜYE ADI 9	NULL
16	ÜYE ADI...	NULL

	TCNO	Ad	Soyad
1	12345678901	ÜYE ADI 1	ÜYE SOYADI 1
2	12345678903	ÜYE ADI 3	ÜYE SOYADI 3
3	12345678904	ÜYE ADI 4	ÜYE SOYADI 4
4	12345678905	ÜYE ADI 5	ÜYE SOYADI 5

SELECT - UNION, INTERSECT, EXCEPT

- İlişkisel cebirdeki karşılıklarının aynıdır
- **Şemaları aynı olan tablolarda kullanılabilir**
 - Sütun sayısı aynı olmalı
 - Veri tipleri uyumlu olmalı
 - Sütun sırası önemlidir
- Sonuç kümesi set semantiği ile çalışır (varsayılan DISTINCT)
 - UNION ALL → bag semantiği
- Set operatörlerinde ORDER BY yalnızca en sonda yazılabilir
- İlişkisel cebirdeki bölme ($\div$) işleminin SQL karşılığı şöyle yazılabilir
 - NOT EXISTS (SELECT ... EXCEPT ...)
- Küme operatörlerinde NULL değerler "eş değer" (equal) kabul edilir
 - Yani iki NULL'ın birbiriyle INTERSECT olabilmesi mümkündür
- Set operatörlerinde öncelik
 - INTERSECT > UNION = EXCEPT
 - Karma set operatörlerinde parantez kullanımı önerilir

SQL	Relational Algebra
UNION	$\cup$
INTERSECT	$\cap$
EXCEPT	$-$ (set difference)

SELECT - UNION, INTERSECT, EXCEPT

-- KİTAP 2 yada KİTAP 3 isimli kitaplardan birini ödünç almış olan üyelerin isim listesi

```
]SELECT DISTINCT U.Ad  
FROM tblUye U  
    INNER JOIN tblOduncKitap O ON O.UyeID = U.ID  
    INNER JOIN tblKitap K ON K.ID = O.KitapID  
WHERE K.Ad = 'KİTAP 2' OR K.Ad = 'KİTAP 3'
```

	Ad
1	UYE ADI 1
2	UYE ADI 2
3	UYE ADI 3
4	UYE ADI 4
5	UYE ADI 5

-- Hem KİTAP 2 hem de KİTAP 3 isimli kitapları ödünç almış olan üyelerin isim listesi
-- Üstteki OR ifadesi AND ile değiştirilemez bunun yerine INTERSECT kullanılır

```
SELECT DISTINCT U.Ad  
FROM tblUye U  
    INNER JOIN tblOduncKitap O ON O.UyeID = U.ID  
    INNER JOIN tblKitap K ON K.ID = O.KitapID  
WHERE K.Ad = 'KİTAP 2'  
INTERSECT  
SELECT DISTINCT U.Ad  
FROM tblUye U  
    INNER JOIN tblOduncKitap O ON O.UyeID = U.ID  
    INNER JOIN tblKitap K ON K.ID = O.KitapID  
WHERE K.Ad = 'KİTAP 3'
```

	Ad
1	UYE ADI 2

SELECT - UNION, INTERSECT, EXCEPT

```
-- Üstteki örneğin aynısı IN operatörü içinde SELECT yazılarakta çözülebilir
]SELECT DISTINCT U.Ad
FROM tblUye U
    INNER JOIN tblOduncKitap O ON O.UyeID = U.ID
    INNER JOIN tblKitap K ON K.ID = O.KitapID
WHERE
    K.Ad = 'KİTAP 2' AND
    U.Ad IN (SELECT DISTINCT U.Ad FROM tblUye U
        INNER JOIN tblOduncKitap O ON O.UyeID = U.ID
        INNER JOIN tblKitap K ON K.ID = O.KitapID
        WHERE K.Ad = 'KİTAP 3'
    )
```

SELECT - UNION, INTERSECT, EXCEPT

-- KİTAP 2 isimli kitabı almış ama KİTAP 3 isimli kitabı almamış olan üyelerin isim listesi

```
SELECT DISTINCT U.Ad
FROM tblUye U
    INNER JOIN tblOduncKitap O ON O.UyeID = U.ID
    INNER JOIN tblKitap K ON K.ID = O.KitapID
WHERE K.Ad = 'KİTAP 2'
EXCEPT
SELECT DISTINCT U.Ad
FROM tblUye U
    INNER JOIN tblOduncKitap O ON O.UyeID = U.ID
    INNER JOIN tblKitap K ON K.ID = O.KitapID
WHERE K.Ad = 'KİTAP 3'
```

	Ad
1	ÜYE ADI 1

- Bir önceki örnek gibi bu da IN kullanılarak yazılabilir ancak bu sefer NOT IN kullanılması gerekecektir
- Ancak NOT IN NULL içerirse beklenmeyen sonuçlar doğabilir

SELECT - UNION, INTERSECT, EXCEPT

-- Hiç ödünç alınmamış yada kütüphaneye bağış yoluyla gelmiş kitapların isimleri

```
SELECT K.Ad
FROM tblKitap K
    LEFT JOIN tblOduncKitap O ON O.KitapID = K.ID
WHERE O.KitapID IS NULL
UNION -- ALL denirse otomatik yapılan DISTINCT iptal edilir
SELECT K.Ad
FROM tblKitap K
    INNER JOIN tblBasim B ON B.KitapID = K.ID
    INNER JOIN tblNusha N ON N.BasimID = B.ID
    INNER JOIN tblGelisSekli G ON G.ID = N.GelisSekliID
WHERE G.Aciklama = 'BAĞIŞ'
```

	Ad
1	KİTAP 1
2	KİTAP 5
3	KİTAP 6
4	KİTAP 7
5	KİTAP 8
6	KİTAP 9
7	KİTAP 10
8	KİTAP 1
9	KİTAP 1
10	KİTAP 1

	Ad
1	KİTAP 1
2	KİTAP 10
3	KİTAP 5
4	KİTAP 6
5	KİTAP 7
6	KİTAP 8
7	KİTAP 9

SELECT - UNION, INTERSECT, EXCEPT

```
-- EN genç ve yaşlı üye  
SELECT 'EN GENÇ ÜYE', * FROM tblUye WHERE DATEDIFF(yy, DTarihi, GETDATE()) = (SELECT MIN(DATEDIFF(yy, DTarihi, GETDATE())) FROM tblUye)  
UNION ALL  
SELECT 'EN YAŞLI ÜYE', * FROM tblUye WHERE DATEDIFF(yy, DTarihi, GETDATE()) = (SELECT MAX(DATEDIFF(yy, DTarihi, GETDATE())) FROM tblUye)
```

	(No column name)	ID	TCNO	Ad	Soyad	Adres	IlKodu	IlceKodu	Tel	Mail	DTarihi	Yas	UyeOlusTarihi	AktifMi
1	EN GENÇ ÜYE	10	12345678910	ÜYE ADI 10	ÜYE SOYADI 10	ÜYE ADRES 10	77	14	0001234567	uye10@hotmail.com	1999-10-10	17	2014-11-22	1
2	EN YAŞLI ÜYE	9	12345678909	ÜYE ADI 9	ÜYE SOYADI 9	ÜYE ADRES 9	77	13	0001234567	uye9@hotmail.com	1960-09-09	56	1975-12-28	0

SELECT - UNION, INTERSECT, EXCEPT

-- Kütüphane bulunan tüm kitapları en az bir defa ödünç alan üyeler

```
SELECT * FROM tblUye U
```

```
WHERE NOT EXISTS (
```

```
    (SELECT ID FROM tblKitap) -- Tüm kitaplar
```

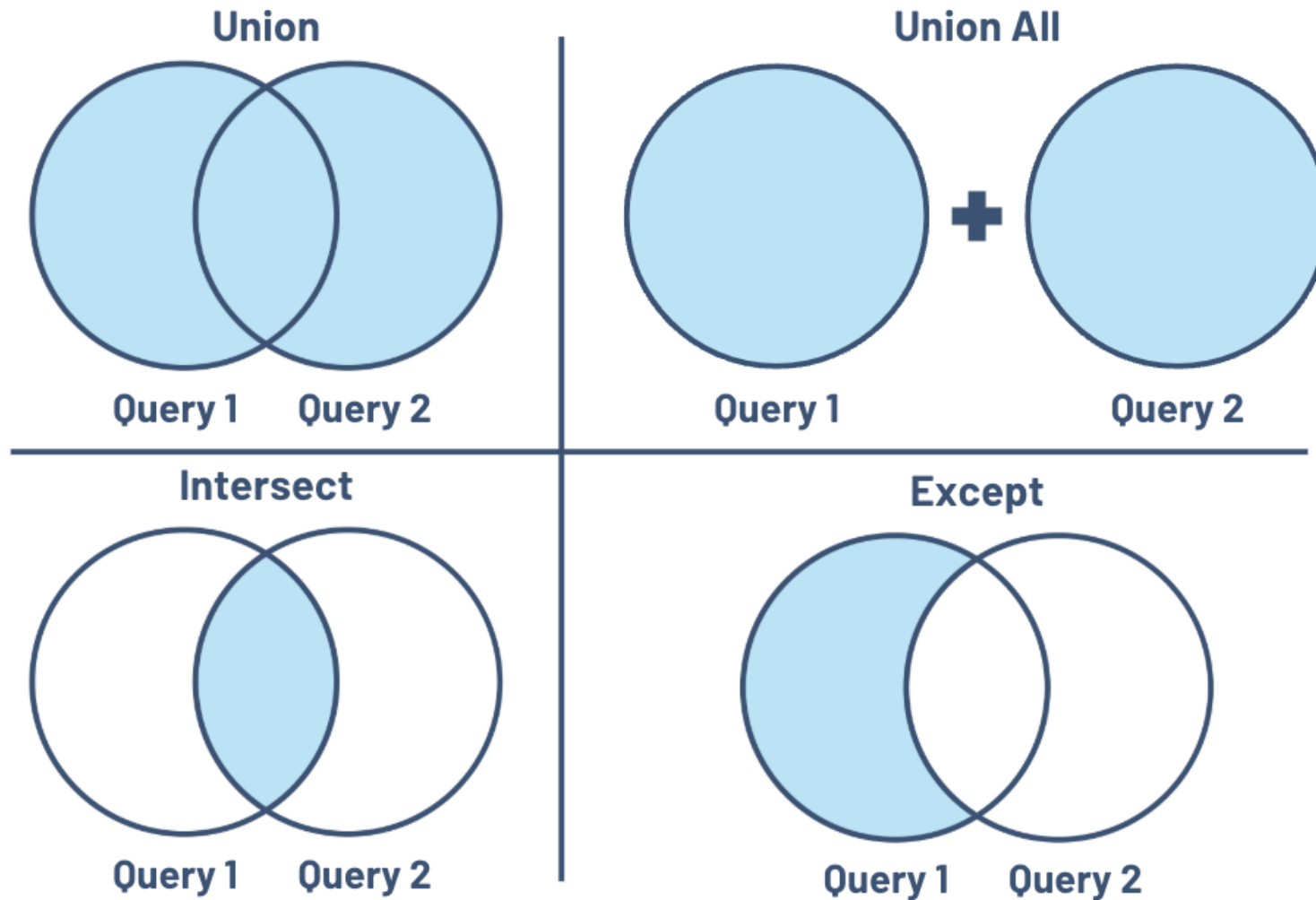
```
EXCEPT -- Çıkartma yap
```

```
(SELECT K.ID FROM tblOduncKitap O INNER JOIN tblKitap K ON K.ID = O.KitapID WHERE O.UyeID = U.ID) -- O anki üyenin aldığı kitapları çıkararak  
-- Eğer üye tüm kitapları aldıysa bu çıkarma işleminin sonucu boş kümedir ki bu da dıştaki NOT EXISTS ifadesini doğrulayacaktır
```

```
)
```

Results		Messages											
	ID	TCNO	Ad	Soyad	Adres	IlKodu	IlceKodu	Tel	Mail	DTarihi	Yas	UyeOlusTarihi	AktifMi
1	2	12345678902	ÜYE ADI 2	ÜYE SOYADI 2	ÜYE ADRES 2	34	2	0001234567	uye2@hotmail.com	1995-02-02	21	2005-12-03	1

SELECT - JOINS



CASE

- Bir grup koşulu kontrol edip olası birden fazla sonuçtan birine döner
 - IF gibi akış kontrolü yapmaz
 - CASE bir ifadedir (expression), değer döndürür
- DML/Sorgu ifadeleri içinde expression olarak kullanılabilir
- Mutlaka END ifadesi ile sonlandırılmalıdır.
- WHEN koşulları yukarıdan aşağıya değerlendirilir
 - İlk eşleşmede (ilk TRUE sonuçta) durulur
- CASE'in tüm THEN/ELSE kolları aynı veri tipinde dönüş vermelidir
- Basit (simple) ve aramalı (searched) CASE olarak iki türü vardır
 - Basit
 - Tek bir ifadenin (sabit, skalar fonksiyon, sütun adı, değişken) alabileceği mutlak değerleri kontrol etmek için kullanılır.
 - Programlama dillerindeki SWITCH yapısına benzer şekilde çalışır.
 - Yani sadece eşitlik kontrolü vardır.
 - Ancak burada break komutu yoktur onun yerine bir eşitlik bulunduğunda ona ait bilgi gösterilir ve otomatik olarak CASE biter yani diğer WHEN ifadelerine devam etmez.
 - Hiçbir WHEN ifadesi doğru olmadıysa NULL döner.

CASE

➤ Yazım şekli

CASE *girdi_ifadesi*

WHEN *when_ifadesi_1* **THEN** *sonuc_ifadesi_1*

WHEN *when_ifadesi_2* **THEN** *sonuc_ifadesi_2*

.....

WHEN *when_ifadesi_N* **THEN** *sonuc_ifadesi_N*

[**ELSE** *else_sonuc_ifadesi*]

END

CASE

- **Aramalı**

- Verilen mantıksal ifadeler kontrol edilerek doğru olanı döner
- Diğerleri gibi sadece tek bir değere ait eşitlik yerine daha karmaşık kontroller yapılabilir (AND ve OR kullanarak).
- Yazım şekli

CASE

WHEN *Boolean_ifade_1* **THEN** *sonuc_ifadesi_1*

WHEN *Boolean_ifade_2* **THEN** *sonuc_ifadesi_2*

.....

WHEN *Boolean_ifade_N* **THEN** *sonuc_ifadesi_N*

[**ELSE** *else_sonuc_ifadesi*]

END

CASE – SELECT İÇERİSİNDE KULLANIMI

-- Sayfa sayısı 100-200 arası olan kitapların adını, ödünç verilebilirliğini (yazı ile), yayın evi adını, yazar adını, basım yeri adını listeleyelim.

```
SELECT K.Ad KitapAdi,  
       CASE WHEN OduncVerilebilirMi = 0 THEN 'Ödünç Verilemez' ELSE 'Ödünç Verilebilir' END OduncDurumu,  
       YE.Ad YayinEvi,  
       Y.Ad Yazar,  
       I.Ad BasimYeri  
FROM tblKitap K  
     INNER JOIN tblYayinEvi YE ON YE.ID = K.YayinEviID  
     INNER JOIN tblYazar Y ON Y.ID = K.YazarID  
     INNER JOIN tblIl I ON I.Kod = K.BasimYeriKodu  
WHERE  
       K.SayfaSayisi BETWEEN 100 AND 200
```

	KitapAdi	OduncDurumu	YayinEvi	Yazar	BasimYeri
1	KİTAP 1	Ödünç Verilemez	Yayın Evi 1	Yazar 1	İstanbul
2	KİTAP 2	Ödünç Verilebilir	Yayın Evi 1	Yazar 2	Ankara
3	KİTAP 3	Ödünç Verilebilir	Yayın Evi 2	Yazar 3	İstanbul
4	KİTAP 4	Ödünç Verilebilir	Yayın Evi 3	Yazar 4	İstanbul
5	KİTAP 5	Ödünç Verilebilir	Yayın Evi 4	Yazar 5	İzmir
6	KİTAP 6	Ödünç Verilebilir	Yayın Evi 5	Yazar 6	İzmir

CASE – SELECT İÇERİSİNDE KULLANIMI

- Örtüşen (Overlapping) Koşullar

```
SELECT Ad,  
       Soyad,  
       Yas,  
       -- overlapping koşullar  
       CASE  
         WHEN Yas > 18 THEN 'Yetişkin'  
         WHEN Yas > 30 THEN 'Olgun'  
         Else 'Genç'  
       END AS YasGrubu  
FROM tblUye  
ORDER BY Yas
```

Ad	Soyad	Yas	YasGrubu
ÜYE ADI 10	ÜYE SOYADI 10	15	Genç
ÜYE ADI 5	ÜYE SOYADI 5	28	Yetişkin
ÜYE ADI 3	ÜYE SOYADI 3	29	Yetişkin
ÜYE ADI 1	ÜYE SOYADI 1	30	Yetişkin
ÜYE ADI 2	ÜYE SOYADI 2	31	Yetişkin
ÜYE ADI 4	ÜYE SOYADI 4	32	Yetişkin
ÜYE ADI 7	ÜYE SOYADI 7	38	Yetişkin
ÜYE ADI 6	ÜYE SOYADI 6	40	Yetişkin
ÜYE ADI 8	ÜYE SOYADI 8	54	Yetişkin
ÜYE ADI 9	ÜYE SOYADI 9	66	Yetişkin

CASE – SELECT İÇERİSİNDE KULLANIMI

- **Veri Tipi Uyumu**

```
SELECT
    CASE
        WHEN Yas < 18 THEN 'Genç'
        ELSE 0
    END AS YasGrubu
    -- Veri Tipi Uyumluluğu Hatası
FROM tblUye
```

Conversion failed when converting the varchar value 'Genç' to data type int.

```
SELECT
    CASE
        WHEN Yas < 18 THEN 1
        ELSE 0.5
    END
    -- implicit dönüşüm
    -- hata oluşmaz
FROM tbluye
```

[illegible]

CASE – GROUP BY İÇERİSİNDE KULLANIMI

- NORTHWND veri tabanındaki Orders tablosu üzerinde çalışacak ve aşağıdaki şekilde çıktı verebilecek sorguyu yazınız

$$\mathbf{FreightGroup} = \begin{cases} \mathbf{Freight} < 50 & \mathbf{A} \\ \mathbf{Else} & \mathbf{B} \end{cases}$$

CustomerID	FreightGroup	FreightSum
ALFKI	A	95.03
ALFKI	B	130.55
ANTON	A	125.25
ANTON	B	143.27

CASE – GROUP BY İÇERİSİNDE KULLANIMI

```
SELECT CustomerID,  
       Freight  
FROM Orders  
WHERE CustomerID IN ('ALFKI', 'ANTON')  
ORDER BY CustomerID, Freight
```

CustomerID	Freight
ALFKI	1.21
ALFKI	23.94
ALFKI	29.46
ALFKI	40.42
ALFKI	61.02
ALFKI	69.53
ANTON	4.03
ANTON	15.64
ANTON	22.00
ANTON	36.13
ANTON	47.45
ANTON	58.43
ANTON	84.84

```
SELECT CustomerID,  
       SUM(Freight) AS FreightSum  
FROM Orders  
WHERE CustomerID IN ('ALFKI', 'ANTON')  
GROUP BY CustomerID
```

CustomerID	FreightSum
ALFKI	225.58
ANTON	268.52

```
SELECT CustomerID,  
       CASE  
         WHEN Freight < 50 THEN 'A'  
         ELSE 'B'  
       END AS FreightGroup,  
       SUM(Freight) AS FreightSum  
FROM Orders  
WHERE CustomerID IN ('ALFKI', 'ANTON')  
GROUP BY CustomerID, Freight  
ORDER BY CustomerID, FreightGroup
```

CustomerID	FreightGroup	FreightSum
ALFKI	A	1.21
ALFKI	A	23.94
ALFKI	A	29.46
ALFKI	A	40.42
ALFKI	B	61.02
ALFKI	B	69.53
ANTON	A	47.45
ANTON	A	36.13
ANTON	A	4.03
ANTON	A	15.64
ANTON	A	22.00
ANTON	B	58.43
ANTON	B	84.84

- GROUP BY CustomerID, FreightGroup yazılamaz çünkü mantıksal işlem sırasına göre GROUP BY işlemi SELECT'ten daha öncedir
- Dolayısıyla henüz öyle bir sütun oluşmamıştır.

CASE – GROUP BY İÇERİSİNDE KULLANIMI

```
SELECT CustomerID,  
       Freight  
FROM Orders  
WHERE  
       CustomerID IN ('ALFKI', 'ANTON')  
ORDER BY CustomerID, Freight
```

CustomerID	Freight
ALFKI	1.21
ALFKI	23.94
ALFKI	29.46
ALFKI	40.42
ALFKI	61.02
ALFKI	69.53
ANTON	4.03
ANTON	15.64
ANTON	22.00
ANTON	36.13
ANTON	47.45
ANTON	58.43
ANTON	84.84

```
SELECT CustomerID,  
       CASE  
           WHEN Freight < 50 THEN 'A'  
           ELSE 'B'  
       END AS FreightGroup,  
       SUM(Freight) AS FreightSum  
FROM Orders  
WHERE  
       CustomerID IN ('ALFKI', 'ANTON')  
GROUP BY  
       CustomerID,  
       CASE WHEN Freight < 50 THEN 'A' ELSE 'B' END  
ORDER BY CustomerID, FreightGroup
```

CustomerID	FreightGroup	FreightSum
ALFKI	A	95.03
ALFKI	B	130.55
ANTON	A	125.25
ANTON	B	143.27

CASE – SELECT İÇERİSİNDE KULLANIMI

- Aynı veriler üzerinde aşağıdaki şekilde çıktı verebilecek sorguyu yazınız

CustomerID	FreightGroup	FreightSum_Before_1998	FreightSum_In_1998
ALFKI	A	53.4	41.63
ALFKI	B	61.02	69.53
ANTON	A	125.25	0
ANTON	B	84.84	58.43

```
SELECT CustomerID,  
        YEAR(OrderDate) AS OrderDateYear,  
        Freight  
FROM Orders  
WHERE  
        CustomerID IN ('ALFKI', 'ANTON')  
ORDER BY CustomerID,  
        OrderDateYear,  
        Freight
```

CustomerID	OrderDateYear	Freight
ALFKI	1997	23.94
ALFKI	1997	29.46
ALFKI	1997	61.02
ALFKI	1998	1.21
ALFKI	1998	40.42
ALFKI	1998	69.53
ANTON	1996	22.00
ANTON	1997	4.03
ANTON	1997	15.64
ANTON	1997	36.13
ANTON	1997	47.45
ANTON	1997	84.84
ANTON	1998	58.43

CASE – SELECT İÇERİSİNDE KULLANIMI

```
SELECT CustomerID,
       CASE
         WHEN Freight < 50 THEN 'A'
         ELSE 'B'
       END AS FreightGroup,
       SUM(CASE
         WHEN YEAR(OrderDate) < 1998 THEN Freight
         ELSE 0
       END
      ) AS FreightSum_Before_1998,
       SUM(CASE
         WHEN YEAR(OrderDate) = 1998 THEN Freight
         ELSE 0
       END
      ) AS FreightSum_In_1998
FROM Orders
WHERE CustomerID IN ('ALFKI', 'ANTON')
GROUP BY CustomerID,
         CASE WHEN Freight < 50 THEN 'A' ELSE 'B' END
ORDER BY CustomerID, FreightGroup
```

**Koşullu Toplulaştırma
(Conditional Aggregation)**

CustomerID	FreightGroup	FreightSum_Before_1998	FreightSum_In_1998
ALFKI	A	53.40	41.63
ALFKI	B	61.02	69.53
ANTON	A	125.25	0.00
ANTON	B	84.84	58.43

CASE – HAVING İÇERİSİNDE KULLANIMI

- Toplam harcaması 30000'i geçen müşterilerden, 1998 yılındaki harcaması toplamın en az %50'si olanları bulan sorguyu yazınız

```
SELECT O.CustomerID,  
       SUM(UnitPrice * Quantity) AS TotalGrossAmount,  
       SUM(UnitPrice * Quantity) * 0.5 AS HalfTotalGrossAmount,  
       SUM(CASE  
           WHEN YEAR(OrderDate) = 1998 THEN UnitPrice * Quantity  
           ELSE 0  
       END  
       ) AS TotalGrossAmount98  
FROM [Order Details] AS OD  
     INNER JOIN Orders AS O ON O.OrderID = OD.OrderID  
GROUP BY O.CustomerID  
HAVING  
       SUM(UnitPrice * Quantity) > 30000  
ORDER BY TotalGrossAmount
```

	CustomerID	TotalGrossAmount	HalfTotalGrossAmount	TotalGrossAmount98
1	QUEEN	30226.10	15113.05000	7007.65
2	KOENE	31745.75	15872.87500	20204.95
3	MEREP	32203.90	16101.95000	0.00
4	FOLKO	32555.55	16277.77500	15973.85
5	HANAR	34101.15	17050.57500	24238.05
6	RATTC	52245.90	26122.95000	21725.60
7	HUNGO	57317.39	28658.69500	22796.34
8	ERNSH	113236.68	56618.34000	42598.90
9	SAVEA	115673.39	57836.69500	42806.25
10	QUICK	117483.39	58741.69500	40526.99

	CustomerID	TotalGrossAmount	HalfTotalGrossAmount	TotalGrossAmount98
1	KOENE	31745.75	15872.87500	20204.95
2	HANAR	34101.15	17050.57500	24238.05

CASE – HAVING İÇERİSİNDE KULLANIMI

- Toplam harcaması 30000'i geçen müşterilerden, 1998 yılındaki harcaması toplamın en az %50'si olanları bulan sorguyu yazınız

```
SELECT O.CustomerID,
       SUM(UnitPrice * Quantity) AS TotalGrossAmount,
       SUM(UnitPrice * Quantity) * 0.5 AS HalfTotalGrossAmount,
       SUM(CASE
            WHEN YEAR(OrderDate) = 1998 THEN UnitPrice * Quantity
            ELSE 0
           ) AS TotalGrossAmount98
FROM [Order Details] AS OD
     INNER JOIN Orders AS O ON O.OrderID = OD.OrderID
GROUP BY O.CustomerID
HAVING SUM(UnitPrice * Quantity) > 30000
       AND
       SUM(CASE
            WHEN YEAR(O.OrderDate) = 1998 THEN UnitPrice * Quantity
            ELSE 0
           ) >= 0.5 * SUM(UnitPrice * Quantity)
ORDER BY TotalGrossAmount
```

	CustomerID	TotalGrossAmount	HalfTotalGrossAmount	TotalGrossAmount98
1	KOENE	31745.75	15872.87500	20204.95
2	HANAR	34101.15	17050.57500	24238.05

CASE – ORDER BY İLE KULLANIMI

```
-- Ülkesi USA olan müşterileri firma ismine göre tersten, diğerlerini de ilgili kişi ismine göre normal sıralayan sorguyu yazınız
} SELECT *
FROM [NORTHWND].[dbo].[Customers]
ORDER BY
CASE Country WHEN 'USA' THEN CompanyName END DESC,
CASE WHEN Country <> 'USA' THEN ContactName END ASC
```

	CustomerID	CompanyName	ContactName	ContactTitle	Address	City	Region	PostalCode	Country	Phone	Fax
1	WHITC	White Clover Markets	Karl Jablonski	Owner	305 - 14th Ave. S. Suite 3B	Seattle	WA	98128	USA	(206) 555-4112	(206) 555-4115
2	TRAIH	Trail's Head Gourmet Provisioners	Helvetius Nagy	Sales Associate	722 DaVinci Blvd.	Kirkland	WA	98034	USA	(206) 555-8257	(206) 555-2174
3	THECR	The Cracker Box	Liu Wong	Marketing Assistant	55 Grizzly Peak Rd.	Butte	MT	59801	USA	(406) 555-5834	(406) 555-8083
4	THEBI	The Big Cheese	Liz Nixon	Marketing Manager	89 Jefferson Way Suite 2	Portland	OR	97201	USA	(503) 555-3612	NULL
5	SPLIR	Split Rail Beer & Ale	Art Braunschweiger	Sales Manager	P.O. Box 555	Lander	WY	82520	USA	(307) 555-4680	(307) 555-6525
6	SAVEA	Save-a-lot Markets	Jose Pavarotti	Sales Representative	187 Suffolk Ln.	Boise	ID	83720	USA	(208) 555-8097	NULL
7	RATTC	Rattlesnake Canyon Grocery	Paula Wilson	Assistant Sales Representative	2817 Milton Dr.	Albuquerque	NM	87110	USA	(505) 555-5939	(505) 555-3620
8	OLDWO	Old World Delicatessen	Rene Phillips	Sales Representative	2743 Bering St.	Anchorage	AK	99508	USA	(907) 555-7584	(907) 555-2880
9	LONEP	Lonesome Pine Restaurant	Fran Wilson	Sales Manager	89 Chiaroscuro Rd.	Portland	OR	97219	USA	(503) 555-9573	(503) 555-9646
10	LETSS	Let's Stop N Shop	Jaime Yonnes	Owner	87 Polk St. Suite 5	San Francisco	CA	94117	USA	(415) 555-5938	NULL
11	LAZYK	Lazy K Kountry Store	John Steel	Marketing Manager	12 Orchestra Terrace	Walla Walla	WA	99362	USA	(509) 555-7969	(509) 555-6221
12	HUNGC	Hungry Coyote Import Store	Yoshi Latimer	Sales Representative	City Center Plaza 516 Main St.	Elgin	OR	97827	USA	(503) 555-6874	(503) 555-2376
13	GREAL	Great Lakes Food Market	Howard Snyder	Marketing Manager	2732 Baker Blvd.	Eugene	OR	97403	USA	(503) 555-7555	NULL
14	ROMEY	Romero y tomitillo	Alejandra Camino	Accounting Manager	Gran Vía, 1	Madrid	NULL	28001	Spain	(91) 745 6200	(91) 745 6210
15	MORGK	Morgenstem Gesundkost	Alexander Feuer	Marketing Assistant	Heerstr. 22	Leipzig	NULL	04179	Germany	0342-023176	NULL
16	ANATR	Ana Trujillo Emparedados y helados	Ana Trujillo	Owner	Avda. de la Constitución 2222	México D.F.	NULL	05021	Mexico	(5) 555-4729	(5) 555-3745
17	TRADH	Tradição Hipermarcados	Anabela Domingues	Sales Representative	Av. Inês de Castro, 414	Sao Paulo	SP	05634-030	Brazil	(11) 555-2167	(11) 555-2168
18	GOURL	Gourmet Lanchonetes	André Fonseca	Sales Associate	Av. Brasil, 442	Campinas	SP	04876-786	Brazil	(11) 555-9482	NULL
19	EASTC	Eastern Connection	Ann Devon	Sales Agent	35 King George	London	NULL	WX3 6FW	UK	(171) 555-0297	(171) 555-3373
20	LAMAI	La maison d'Asie	Annette Roulet	Sales Manager	1 rue Alsace-Lorraine	Toulouse	NULL	31000	France	61.77.61.10	61.77.61.11
21	ANTON	Antonio Moreno Taquería	Antonio Moreno	Owner	Mataderos 2312	México D.F.	NULL	05023	Mexico	(5) 555-3932	NULL
22	FAMIA	Familia Arquibaldo	Ana Cruz	Marketing Assistant	Rua Orós, 92	Sao Paulo	SP	05442-030	Brazil	(11) 555-9857	NULL
23	QUEDE	Que Delícia	Bernardo Batista	Accounting Manager	Rua da Panificadora, 12	Rio de Janeiro	RJ	02389-673	Brazil	(21) 555-4252	(21) 555-4545
24	FRANR	France restauration	Carine Schmitt	Marketing Manager	54, rue Royale	Nantes	NULL	44000	France	40.32.21.21	40.32.21.20
25	LILAS	LILA-Supermercado	Carlos González	Accounting Manager	Carrera 52 con Ave. Bolívar ...	Barquisimeto	Lara	3508	Venez...	(9) 331-6954	(9) 331-7256
26	HILAA	HILARION-Abastos	Carlos Hernández	Sales Representative	Carrera 22 con Ave. Carlos ...	San Cristóbal	Tách...	5022	Venez...	(5) 555-1340	(5) 555-1948
27	MAISD	Maison Dewey	Catherine Dewey	Sales Agent	Rue Joseph-Bens 532	Bruxelles	NULL	B-1180	Belgium	(02) 201 24 67	(02) 201 24 68

CASE – ORDER BY İLE KULLANIMI

-- Ülkesi USA olanları Region değerine göre diğerlerini de City değerine göre sırala

```
SELECT CompanyName, City, Region, Country  
FROM [NORTHWND].[dbo].[Customers]  
ORDER BY CASE Country WHEN 'USA' THEN Region ELSE City END
```

Results		Messages		
	CompanyName	City	Region	Country
1	Drachenblut Delikatessen	Aachen	NULL	Germany
2	Old World Delicatessen	Anchorage	AK	USA
3	Vaffeljemet	Århus	NULL	Denmark
4	Galería del gastrónomo	Barcelona	NULL	Spain
5	LILA-Supernmercado	Barquisimeto	Lara	Venezuela
6	Magazzini Alimentari Riuniti	Bergamo	NULL	Italy
7	Alfreds Futterkiste	Berlin	NULL	Germany
8	Chop-suey Chinese	Bern	NULL	Switzerland
9	Folk och fä HB	Bräcke	NULL	Sweden
10	Königlich Essen	Brandenburg	NULL	Germany
11	Maison Dewey	Bruxelles	NULL	Belgium
12	Océano Atlántico Ltda.	Buenos Aires	NULL	Argentina
13	Rancho grande	Buenos Aires	NULL	Argentina
14	Cactus Comidas para llevar	Buenos Aires	NULL	Argentina
15	Let's Stop N Shop	San Francisco	CA	USA
16	Gourmet Lanchonetes	Campinas	SP	Brazil
17	GROSELLA-Restaurante	Caracas	DF	Venezuela
18	Suprêmes délices	Charleroi	NULL	Belgium
19	Hungry Owl All-Night Grocers	Cork	Co. Cork	Ireland
20	Island Trading	Cowes	Isle of Wight	UK
21	QUICK-Stop	Cunewalde	NULL	Germany
22	Lehmanns Marktstand	Frankfurt a.M.	NULL	Germany
23	Richter Supermarkt	Genève	NULL	Switzerland
24	Ernst Handel	Graz	NULL	Austria
25	Wilman Kala	Helsinki	NULL	Finland
26	LINO-Delicateses	I. de Margarita	Nueva Es...	Venezuela
27	Save-a-lot Markets	Boise	ID	USA
28	Simons bistro	Kobenhavn	NULL	Denmark

CASE – UPDATE İLE KULLANIMI

```
-- Birim fiyatı 20'nin altında olan ürünlere %5 zam, diğerlerinden kategorisi 2 olanların fiyatını 50 yapan kod  
-- Her iki koşula uymayanların fiyatı aynen kalmalıdır (else kullanılması gerekiyor aksi halde NULL değer alırlar)  
SELECT ProductName, CategoryID, UnitPrice FROM [NORTHWND].[dbo].[Products]
```

```
UPDATE [NORTHWND].[dbo].[Products]  
SET UnitPrice = (CASE WHEN UnitPrice < 20 THEN UnitPrice * 1.05  
                      WHEN CategoryID = 2 THEN 50  
                      ELSE UnitPrice  
                  END)
```

```
SELECT ProductName, CategoryID, UnitPrice FROM [NORTHWND].[dbo].[Products]
```

Results Messages			
	ProductName	CategoryID	UnitPrice
1	Chai	1	18.00
2	Chang	1	19.00
3	Aniseed Syrup	2	10.00
4	Chef Anton's Cajun Seasoning	2	22.00
5	Chef Anton's Gumbo Mix	2	21.35
6	Grandma's Boysenberry Spread	2	25.00
7	Uncle Bob's Organic Dried Pears	7	30.00
8	Northwoods Cranberry Sauce	2	40.00
9	Mishi Kobe Niku	6	97.00
10	Ikura	8	31.00
11	Queso Cabrales	4	21.00
12	Queso Manchego La Pastora	4	38.00

	ProductName	CategoryID	UnitPrice
1	Chai	1	18.90
2	Chang	1	19.95
3	Aniseed Syrup	2	10.50
4	Chef Anton's Cajun Seasoning	2	50.00
5	Chef Anton's Gumbo Mix	2	50.00
6	Grandma's Boysenberry Spread	2	50.00
7	Uncle Bob's Organic Dried Pears	7	30.00
8	Northwoods Cranberry Sauce	2	50.00
9	Mishi Kobe Niku	6	97.00
10	Ikura	8	31.00
11	Queso Cabrales	4	21.00
12	Queso Manchego La Pastora	4	38.00

CASE – DELETE İLE KULLANIMI

- CASE'in DELETE ile kullanımı doğrudan "DELETE CASE ..." şeklinde olmaz
 - Çünkü CASE bir statement değil, expression'dır
- Ancak WHERE içinde bir koşul üretme amaçlı dolaylı biçimde kullanılabilir
 - En anlamlı kullanım şekli, "karşılaştırma operatörü sabitse ama eşik değeri dinamikse" senaryolarındadır
- Örneğin NORTHWND veri tabanının Products tablosundaki ürünlerden
 - Üretimi durdurulanların (Discontinued=1) stoğu (UnitsInStock) 25'den az olanlarını
 - Üretimi devam edenlerin ise stoğu 10'dan az olanlarınısilecek sorguyu yazınız

```
SELECT ProductID,  
       ProductName,  
       UnitsInStock,  
       Discontinued  
FROM Products  
WHERE  
    (Discontinued = 1 AND UnitsInStock < 25)  
    OR (Discontinued = 0 AND UnitsInStock < 10)  
ORDER BY Discontinued, UnitsInStock
```

	ProductID	ProductName	UnitsInStock	Discontinued
1	31	Gorgonzola Telino	0	0
2	21	Sir Rodney's Scones	3	0
3	66	Louisiana Hot Spiced Okra	4	0
4	74	Longlife Tofu	4	0
5	45	Rogede sild	5	0
6	68	Scottish Longbreads	6	0
7	8	Northwoods Cranberry Sauce	6	0
8	32	Mascarpone Fabioli	9	0
9	5	Chef Anton's Gumbo Mix	0	1
10	29	Thüringer Rostbratwurst	0	1
11	17	Alice Mutton	0	1
12	53	Perth Pasties	0	1
13	24	Guaraná Fantástica	20	1

```
DELETE FROM Products  
WHERE UnitsInStock <  
    CASE  
        WHEN Discontinued = 1 THEN 25  
        ELSE 10  
    END
```

IIF Cümlesi

- SQL Server 2012 ve sonraki sürümlerinde koşul kontrolü için IIF cümlesi kullanılabilir
- C dilindeki ? operatörüne benzetilebilir
- SELECT içerisinde IIF kullanıldığında o kısım yeni bir sütun olur
- IIF, SQL Server'a özgü bir kısayoldur
- ANSI SQL standardının parçası değildir
- Mantıksal olarak CASE ifadesinin kısaltılmış halidir
 - IIF(condition, a, b) = CASE WHEN condition THEN a ELSE b END
- IIF her zaman CASE kadar güvenli değildir
 - CASE teorik olarak ilk TRUE'da durur
 - IIF ise fonksiyon gibi davrandığından optimizer bazen her iki ifadeyi de değerlendirebilir
- Sözdizimi (Syantax)
 - IIF(boolean_ifade, doğru_değer, yanlış_değer)**

IIF Cümlesi

- SQL Server 2012 ve sonraki sürümlerinde koşul kontrolü için IIF cümlesi kullanılabilir
- C dilindeki ? operatörüne benzetilebilir
- SELECT içerisinde IIF kullanıldığında o kısım yeni bir sütun olur
- IIF, SQL Server'a özgü bir kısayoldur
- ANSI SQL standardının parçası değildir
- Mantıksal olarak CASE ifadesinin kısaltılmış halidir

IIF(condition, a, b) = CASE WHEN condition THEN a ELSE b END

- CASE teorik olarak ilk TRUE'da durur.

IIF ise fonksiyon gibi davrandığından optimizer bazen her iki ifadeyi de değerlendirebilir

- Sözdizimi (Syantax)

IIF(boolean_ifade, doğru_değer, yanlış_değer)

```
SELECT Freight, IIF(Freight>50, 1 ,0) FROM [NORTHWND].[dbo].[Orders]
```

```
SELECT DISTINCT EmployeeID, IIF(EmployeeID IN(1,2,3), 'A', 'B') FROM [NORTHWND].[dbo].[Orders] ORDER BY EmployeeID
```

Results			Messages		
	Freight	(No column name)			
1	32.38	0			
2	11.61	0			
3	65.83	1			
4	41.34	0			
5	51.30	1			
6	58.17	1			
7	22.98	0			
8	148.33	1			
9	13.97	0			
10	81.91	1			
11	140.51	1			
12	3.25	0			
13	55.09	1			
14	3.05	0			

	EmployeeID	(No column name)
1	1	A
2	2	A
3	3	A
4	4	B
5	5	B
6	6	B
7	7	B
8	8	B
9	9	B

IIF Cümlesi

```
SELECT OrderID,  
       SUM(IIF(Quantity > 25 AND ProductID = 11, 1, 0)) Onbir25tenCok,  
       SUM(IIF(ProductID = 42, Quantity * 10, 0)) OnPuan42ye  
FROM [NORTHWND].[dbo].[Order Details]  
WHERE ProductID IN ('11', '42')  
GROUP BY OrderID  
ORDER BY OrderID
```

```
SELECT OrderID, ProductID, Quantity  
FROM [NORTHWND].[dbo].[Order Details]  
WHERE ProductID IN ('11', '42')  
ORDER BY OrderID
```

	OrderID	ProductID	Quantity
1	10248	11	12
2	10248	42	10
3	10296	11	12
4	10309	42	2
5	10311	42	6
6	10327	11	50
7	10332	42	10
8	10345	42	9
9	10353	11	12
10	10365	11	24
11	10404	42	40
12	10407	11	30
13	10434	11	6
14	10442	11	30

	OrderID	Onbir25tenCok	OnPuan42ye
1	10248	0	100
2	10296	0	0
3	10309	0	20
4	10311	0	60
5	10327	1	0
6	10332	0	100
7	10345	0	90
8	10353	0	0
9	10365	0	0
10	10404	0	400
11	10407	1	0
12	10434	0	0
13	10442	1	0
14	10443	0	0

ORDER BY ile OFFSET ve FETCH NEXT

- SQL Server 2012 ve sonrası sürümlerde ORDER BY cümlesi içerisinde OFFSET ve FETCH NEXT ifadeleri kullanılabilir
 - OFFSET/FETCH, ANSI SQL:2008 standardının parçasıdır
- OFFSET/FETCH mutlaka ORDER BY ile birlikte kullanılmalıdır
- ORDER BY benzersiz değilse sayfalama tutarsız olabilir
 - ORDER BY CompanyName → aynı isimler varsa sonuç kayabilir
- **OFFSET**
 - Gelen sonuç kümesindeki ilk n satırın gösterilmemesini sağlar.
 - Örneğin ilk 5 satır silinir ve 6. satırdan itibaren gösterilir.
- **FETCH NEXT**
 - TOP ifadesi gibi belirli sayıda sonuç döndürülmesini sağlar

ORDER BY ile OFFSET ve FETCH NEXT

```
SELECT *  
FROM [NORTHWND].[dbo].[Customers]  
ORDER BY CompanyName DESC
```

```
SELECT *  
FROM [NORTHWND].[dbo].[Customers]  
ORDER BY CompanyName DESC  
OFFSET 10 ROWS  
FETCH NEXT 3 ROWS ONLY
```

Results		Messages	
	CustomerID	CompanyName	ContactName
1	WOLZA	Wolski Zajazd	Zbyszek
2	WILMK	Wilman Kala	Matti Ka
3	WHITC	White Clover Markets	Karl Jabl
4	WELLI	Wellington Importadora	Paula Pa
5	WARTH	Wartian Herkku	Pirkko K
6	VINET	Vins et alcools Chevalier	Paul Hei
7	VICTE	Victuailles en stock	Mary Sa
8	VAFFE	Vaffeljemet	Palle lbs
9	TRAIH	Trail's Head Gourmet Provisioners	Helvetiu
10	TRADH	Tradição Hipercardos	Anabela
11	TORTU	Tortuga Restaurante	Miguel A
12	TOMSP	Toms Spezialitäten	Karin Jo
13	THECR	The Cracker Box	Liu Won
<			
	CustomerID	CompanyName	ContactName
1	TORTU	Tortuga Restaurante	Miguel Angel Paolino
2	TOMSP	Toms Spezialitäten	Karin Josephs
3	THECR	The Cracker Box	Liu Wong

UPDATE ve DELETE içinde SELECT kullanımı

-- Şimdiye kadar hiç ödünç alınmamış kitapların ödünç verilebilirliğini 0 (verilemez) yap

```
SELECT * FROM tblKitap
```

```
UPDATE tblKitap SET OduncVerilebilirMi = 0 WHERE ID IN (SELECT K.ID FROM tblKitap K WHERE NOT EXISTS (SELECT OK.KitapID FROM tblOduncKitap OK WHERE OK.KitapID = K.ID))
```

```
SELECT * FROM tblKitap
```

	ID	ISBN	Ad	Aciklama	OduncVerilebilirMi	SayfaSayisi
1	1	1234567890123	KİTAP 1	AÇIKLAMA 1	0	100
2	2	1234567890124	KİTAP 2	AÇIKLAMA 2	1	120
3	3	1234567890125	KİTAP 3	AÇIKLAMA 3	1	140
4	4	1234567890126	KİTAP 4	AÇIKLAMA 4	1	160
5	5	1234567890127	KİTAP 5	AÇIKLAMA 5	1	180
6	6	1234567890128	KİTAP 6	NULL	1	200
7	7	1234567890129	KİTAP 7	AÇIKLAMA 7	1	220
8	8	1234567890130	KİTAP 8	AÇIKLAMA 8	1	240
9	9	1234567890131	KİTAP 9	AÇIKLAMA 9	1	260
10	10	1234567890132	KİTAP 10	NULL	1	280

	ID	ISBN	Ad	Aciklama	OduncVerilebilirMi	SayfaSayisi
1	1	1234567890123	KİTAP 1	AÇIKLAMA 1	0	100
2	2	1234567890124	KİTAP 2	AÇIKLAMA 2	1	120
3	3	1234567890125	KİTAP 3	AÇIKLAMA 3	1	140
4	4	1234567890126	KİTAP 4	AÇIKLAMA 4	1	160
5	5	1234567890127	KİTAP 5	AÇIKLAMA 5	0	180
6	6	1234567890128	KİTAP 6	NULL	0	200
7	7	1234567890129	KİTAP 7	AÇIKLAMA 7	0	220
8	8	1234567890130	KİTAP 8	AÇIKLAMA 8	0	240
9	9	1234567890131	KİTAP 9	AÇIKLAMA 9	0	260
10	10	1234567890132	KİTAP 10	NULL	0	280

UPDATE ve DELETE içinde SELECT kullanımı

```
-- İstanbuldaki her bir üyenin en son ki yorumunu silen sorgu
DELETE FROM tblYorumlar
WHERE ID IN (SELECT Y.ID
             FROM tblYorumlar Y
             WHERE
               EXISTS (SELECT * FROM tblYorumlar X
                      INNER JOIN tblUye U ON U.ID = X.UyeID
                      INNER JOIN tblIl I ON I.Kod = U.IlKodu
                      WHERE I.Ad = 'İstanbul' AND X.UyeID = Y.UyeID
                      GROUP BY UyeID
                      HAVING Y.Tarih = MAX(Tarih)
                     )
            )
```

Alternatif Yöntem

```
DELETE FROM tblYorumlar
WHERE
  ID IN (
    SELECT ID FROM tblYorumlar Y1
    INNER JOIN
      (
        SELECT UyeID, MAX(Tarih) SonYorumTarihi
        FROM tblYorumlar Y
        INNER JOIN tblUye U ON U.ID = Y.UyeID
        INNER JOIN tblIl I ON I.Kod = U.IlKodu
        WHERE I.Ad = 'İstanbul'
        GROUP BY UyeID
      ) Y2 ON Y1.UyeID = Y2.UyeID AND Y1.Tarih = Y2.SonYorumTarihi
  )
```